

INFORMATION-THEORETIC NECESSARY AND
SUFFICIENT CONDITIONS FOR THE TASK-DRIVEN
CONTROL OF ROBOTS

VINCENT PACELLI

A DISSERTATION
PRESENTED TO THE FACULTY
OF PRINCETON UNIVERSITY
IN CANDIDACY FOR THE DEGREE
OF DOCTOR OF PHILOSOPHY

RECOMMENDED FOR ACCEPTANCE BY
THE DEPARTMENT OF
MECHANICAL AND AEROSPACE ENGINEERING
ADVISER: ANIRUDHA MAJUMDAR

MAY 2023

© Copyright by Vincent Pacelli, 2023.

All Rights Reserved

Abstract

The development of modern robotic systems has significantly benefited from the availability of high-fidelity sensors and efficient computational resources. Together, advances in these areas have led to a significant commercial interest in the applications of autonomous robots for complex tasks. However, while powerful, the high-dimensional sensors now common in robotics (e.g., cameras and LIDAR) pose several challenges for which traditional, general-purpose control and estimation are not well-suited. Specifically, these sensors lack tractable analytical models, and their high-dimensional output spaces are challenging to explore fully empirically or in simulation. Moreover, traditional approaches often result in an unnecessarily tight coupling between the sensing, estimation, and control components of the robot’s feedback loop, causing performance degradation when deploying the robot in a new environment due to a shift in the distribution of sensor outputs — even if the shift only modifies the outputs in a manner that is *irrelevant* to the robot’s task.

This dissertation addresses traditional control methods’ limitations by establishing *task-driven* necessary and sufficient conditions for performant feedback control. Necessary conditions quantify how much information a sensor must provide the robot to achieve a performance criterion independent of the controller employed by the robot — thereby providing a principled method to determine the necessary sensing capabilities of the robot for the task. Sufficient conditions guarantee that a robot will achieve a specific level of performance in new environments by limiting the controller to only depend on *task-relevant* information. Newly developed algorithms for their implementation accompany these conditions. Also included are demonstrations of the efficacy of these methods on problems on common robotics problems featuring high-dimensional sensors.

Acknowledgements

To begin, I would like to thank my advisor, Anirudha Majumdar for his constant guidance and support throughout my time at Princeton. As his student, Ani taught me how to identify, pose, and develop a research question so that it captures the fundamental character of the problem and how to address it fully through patient refinement. I also greatly appreciated the freedom he allowed me to wander and explore many different topics so that I could develop the understanding needed to find answers to my questions that are both impactful and acceptable to my intellectual tastes and sensibilities. Finally, as the first student of a then-new professor, I appreciate the trust Ani placed in me to both succeed academically and help shape a new research group at its outset.

Moreover, I would also like to thank my readers and committee members: Naomi Leonard and Clancy Rowley. They have contributed to this work throughout my time at Princeton through both their valuable feedback and more broadly by guiding my education as a young researcher.

I am very fortunate to have been surrounded by an excellent group of lab mates throughout my degree. Alec Farid, Meghan Booker, Allen Ren, David Snyder, Nate Simon, Sasha Bodrova, May Mei, and Sushant Veer all made our research group a fun and engaging place to work, and have contributed in innumerable ways to my research through suggestions, feedback, and encouragement when results seemed hard to come by. I'd especially like to thank Alec and Meghan for being particularly supportive and sharing their animals during the pandemic when we all felt alone and isolated with our work.

This dissertation would not exist without the guidance of the teachers and mentors from whom I benefited long before my time at graduate school began. Most importantly, I thank my parents for their love, support, encouragement of my curiosity from a young age, and instilling in me a commitment to pursue my interests with the total commitment of my heart and mind. I thank my high school physics teacher, Gregory Devine, for engendering my interest in understanding the physical world and showing me that, with an appropriate effort, even the most difficult material can be mastered. I thank Matthew O'Kelly for the incredible guidance he provided me as a lost and aimless undergraduate and for serving

as the ideal model of a graduate student and engineer that I strive to emulate. Finally, I am grateful for all the helpful souls on the old IRC channels who endured my many novice questions about programming and taught me in my adolescence more about computers than I can articulate.

I am fortunate to have been surrounded by many supportive friends during my pursuit of my degree. While there are too many to thank individually, I would like to express my gratitude explicitly to: Robin Ortelere for being ubiquitous in my daily life, Kerry Klemmer for being my partner in crime, Sebastián Rojas Mata for understanding my obscure popular culture references, Tasman Powis for being the perfect roommate with whom to watch a medical drama, Adam Efrus for his indomitable double bass pedal, Sam Saskin for teaching me how to love the game of baseball, Aaron Goodman for being the best softball coach I could ask for, Alyssa Cady for her sage life advice, Kelly Wecker for giving me the best pair of slippers I have ever owned, and every member of the Mechanical and Aerospace Engineering department lunch table and softball team for their company throughout the years.

Finally, scientific and engineering work is a labor not done for its own sake but to understand and shape the world for the benefit of humanity as a whole. The same applies to the arts and other pursuits that provide us with meaning and seek to cleanly articulate the many messy and complicated facets of the human condition. To this end, I would like to thank a number of artists and performers who have inspired me throughout my time as a graduate student and kept me company during many late nights in the laboratory. Included are musicians and bands like The Gaslight Anthem, The Hold Steady (along with all the “clever kids” at their shows, most notably Brendan Hilliard), Titus Andronicus, The Mountain Goats, Strand of Oaks, Bright Eyes, The Grateful Dead, Bob Dylan, Tom Waits, and Bruce Springsteen, the humorous critics of the Bombcast (Jeff, Ryan, Brad, and Vinny, and the many other members who appeared through the years) to which I listened weekly, the athletes of the New York Football Giants and the Los Angeles Dodgers, and the long list of science fiction writers whose visions of the future have inspired me since my youth.

This dissertation carries **T#3450** in the records of the Department of Mechanical and Aerospace Engineering.

Contents

Abstract	iii
Acknowledgements	iv
1 Introduction	1
1.1 Organization and Summary of Contributions	4
1.2 Notation and Preliminary Definitions	6
I Sufficient Conditions	8
2 Introduction	9
2.1 Motivation	10
2.2 Related Work	11
2.2.1 Differential Privacy and Applications in Control Theory	11
2.2.2 Bounded Rationality and Robust Decision-Making	12
2.2.3 Optimal Control as Bayesian Inference	13
3 Information-Theoretic Sufficient Conditions for Performant Control	14
3.1 Problem Formulation	14
3.1.1 System Dynamics	15
3.1.2 Online and Offline Optimal Control Problems	17
3.2 Bounded-Rationality Control	18
3.3 Bounded Rationality Implies Differential Privacy	21
3.4 Bounded-Rationality Control provides Robustness to Estimation Error . . .	24
3.5 Extension to Controllers Dependent on Reduced Order Task-Relevant Variables	26

3.5.1	Synthesis of TRV-Based Controllers	28
3.5.2	Efficient Online Estimation of TRVs for Control	30
3.6	Task-Relevance and the Choice of Metric	31
4	Algorithms for Implementing Bounded-Rationality Control	34
4.1	Local Solutions from Linear-Quadratic Approximation	34
4.1.1	Iterative Local Dynamic Programming Algorithm	35
4.2	Reinforcement Learning of Task-Relevant Controllers	39
4.3	Stein-Variation Gradient Descent	41
5	Empirical Studies	43
5.1	Spring-Loaded Inverted Pendulum	44
5.2	When the Separation Principle Fails: The Lava Problem	44
5.3	Vision-Based Ball Catching	47
5.4	Quadrotor Stabilization	48
5.4.1	Linearized Results	49
5.4.2	Nonlinear Results	49
5.5	Motion Planning in Cluttered Spaces	50
II	Necessary Conditions	57
6	Introduction	58
6.1	Motivation	58
6.2	Related Work	59
6.2.1	Performance Limits of Sensors for Feedback Control	59
6.2.2	Application-Specific Performance Limits in Robotics	60
6.2.3	Fano’s Inequality: An Information Theoretic Performance Limit	61
6.2.4	Landauer’s Principle and the Landauer Limit	61
7	A Necessary Condition from Generalized Fano-Type Inequalities	63
7.1	Problem Formulation	63

7.2	The Inverse of the Relative Entropy	65
7.3	Necessary Conditions in Terms of the Task-Relevant Information Potential .	66
7.3.1	A Necessary Condition for Single-Step Problems	67
7.3.2	Extension for Multi-Step Problems	68
7.4	Algorithms for Consistent Approximation of the TRIP Condition	69
7.5	Numerical Experiments	71
7.5.1	The Lava Problem	71
7.5.2	Ball-Catching	72
7.5.3	Obstacle Avoidance with a Depth Sensor	73
7.6	Tighter Bounds Using the Relative f -Entropy	74
8	A Necessary Condition from Non-Equilibrium Thermodynamics	79
8.1	Background: Jaynes' Entropy	80
8.2	Optimal Control Problem Formulation	81
8.3	A Necessary Condition from the Irreversibility of Feedback	81
8.4	Recovering the Landauer Limit	84
9	Conclusion	87
9.1	Summary	87
9.2	Limitations and Future Work	89
A	Overview of Peer Reviewed Publications	92
B	Task-Driven Estimation and Control via Information Bottlenecks	95
B.1	Introduction	95
B.1.1	Related Work	97
B.2	Task-Driven Representations and Controllers	99
B.3	Algorithms for Synthesizing Representations	102
B.3.1	Discrete Systems	103
B.3.2	Linear-Gaussian Systems with Quadratic Costs	104
B.3.3	Nonlinear-Gaussian Systems	106
B.4	Online Estimation and Control	107

B.5	Examples	108
B.5.1	Lava Problem	109
B.5.2	SLIP Model Problems	110
B.6	Conclusion	111
B.7	Proofs	113
B.7.1	Proof of Theorem 7	113
B.7.2	Proof of Theorem 8	114
B.7.3	Proof of Theorem 9	115
C	Learning Task-Driven Control Policies via Information Bottlenecks	118
C.1	Introduction	118
C.1.1	Related Work	120
C.2	Defining Task-Driven Policies	123
C.3	Learning Task-Driven Policies	125
C.3.1	Mutual Information Neural Estimator (MINE)	126
C.3.2	Task-Driven Policy Gradient Algorithm	127
C.4	Examples	130
C.4.1	Lava Problem	131
C.4.2	Vision-Based Ball Catching	133
C.4.3	Grasping using a Depth Camera	136
C.5	Discussion and Conclusion	139
D	Robust Control Under Uncertainty via Bounded Rationality and Differential Privacy	141
D.1	Introduction	141
D.2	Related Work	144
D.2.1	Differential Privacy and Applications in Control Theory	144
D.2.2	Bounded Rationality and Robust Decision-Making	145
D.2.3	Optimal Control as Bayesian Inference	146
D.3	Notation	146
D.4	Robust Single-Step Decision-Making	147

D.4.1	Problem Statement	147
D.4.2	Differential Privacy	148
D.4.3	The Exponential Mechanism and Bounded Rationality	149
D.4.4	Quantifying the Robustness of Bounded Rationality	151
D.5	Extension to Multi-Step Problems	152
D.6	Numerical Examples	154
D.6.1	Motion Planning	155
D.6.2	Planar Quadrotor Stabilization Problems	156
D.7	Conclusion	157
D.8	Proofs	157
D.9	Explicit LQG Solution of the Bounded-Rationality Policy	164

E Fundamental Performance Limits for Sensor-Based Robot Control and Policy Learning 167

E.1	Introduction	167
E.1.1	Related Work	170
E.2	Problem Formulation	171
E.2.1	Notation	171
E.2.2	Problem Statement	171
E.3	Background	172
E.3.1	KL Divergence and Mutual Information	172
E.3.2	Inverting Bounds on the KL Divergence	173
E.4	Performance Bound for Single-Step Problems	173
E.5	Performance Bound for Multi-Step Problems: Fano's Inequality with Feedback	175
E.6	Numerical Implementation	179
E.6.1	Analytic Computation	179
E.6.2	Computation via Sampling and Concentration Inequalities	180
E.6.3	Tightening the Bound with Multiple Horizons	182
E.7	Examples	182
E.7.1	Lava Problem	182

E.7.2	Catching a Falling Object	184
E.7.3	Obstacle Avoidance with a Depth Sensor	187
E.8	Discussion and Conclusions	189
E.9	Proofs	191
E.10	Chernoff-Hoeffding Bound	200

Chapter 1

Introduction

Over the last two decades, robotics as a field has undergone incredible growth. Commercial applications like autonomous vehicles and legged platforms have transitioned from the hypothetical to products nearing commercial release. Central to these advances is the development of high-resolution sensors available to roboticists (e.g., cameras and LIDAR) and expanding computational resources used to process sensor data. Together, these advances allow the robot to create rich, high-dimensional representations of their environment (either explicitly or implicitly within a deep learning framework), and use general-purpose algorithms to determine which control input to apply for the current task.

While successful, this task-agnostic approach to feedback control suffers from two critical shortcomings which require significant engineering effort to ameliorate. First, processing sensor data into a rich representation often requires the majority of computation available to the robot, both “online” during real-time execution when deployed or “offline” during the design phase to ensure the space of sensor observations is fully explored. As a result, limited computational resources are available for other components of the feedback loop, e.g., planning, control, or other processes. Second, this methodology is often brittle when deployed beyond the lab due to the tight coupling between sensing and control. This fragility occurs in two common ways: the *separation principle* (that performant controllers and state estimators can be designed independently of one another [1]) failing to hold or when not enough data is available for the robot’s control system to generalize to new situations it encounters in the wild. The separation principle is a common assumption used

by engineers to simplify designing the complete control loop into designing the control and estimation algorithms separately. While the assumption is a good approximation given a sufficiently accurate state estimator, it rarely holds in practice due to the presence of nonlinear dynamics and sensors or non-Gaussian noise. As a result, if the robot’s sensors or estimators begin to return spurious results in a new environment, the robot may behave erratically and perform poorly. The second difficulty occurs when collected data used to synthesize (if using a learning framework) and evaluate the control loop’s performance does not capture the true distribution of values seen when the robot is deployed. Even under the generous assumption that data is collected from the true distribution of situations the robot will encounter, high-dimensional sensors and state representations require a very large data set to represent it properly, and its compilation and analysis is a significant practical challenge.

Behavioral science suggests an alternative, biologically-inspired approach to the task of optimally designing a feedback loop. While the common engineering method uses task-agnostic algorithms, empirically, humans (and other animals) are able to make robust decisions in new environments for dynamical tasks by combining low-information task-relevant rules to process high-dimensional sensory information (e.g., visual and auditory systems). Instead of compiling all available sensor information to make a robust decision, they rely on *task-driven* heuristics that are simple to compute in terms of a small number of *task-relevant variable* (TRV). This information-constrained decision-making process is known as *bounded-rationality*, and the heuristics used easily adapt to new situations as long as the task-relevant variables can be estimated [2–4].

A canonical example of bounded rationality is the *gaze heuristic* employed by humans in control problems ranging from ball catching to landing aircraft [3–5]. Experimental studies have shown that, if a human (e.g., Mookie Betts) is running to catch a fly ball, they will follow a path that results in the ball remaining at a fixed point in their visual field. That is, all visual information is discarded except for the *gaze angle* between the player’s eye line and the ball. This strategy only depends on a single task-relevant variable, is formally proven to drive the player to the point where the ball reaches eye level [5, 6], and naturally adapts to new environments (e.g., different visual backdrops or balls with drag coefficients)

and unmodified spontaneous disturbances (e.g., wind gusts).

In comparison, the standard approaches for designing a vision-based ball-catching robot would be either a model-predictive control approach, where, to account for uncertainty, each planning cycle uses a physical model based on the current estimate of the robot and ball states to compute a trajectory for the robot, or to use an end-to-end deep learning approach directly mapping sequences of observed sensor readings to robot motor torques. In both cases, if the robot is tasked with catching the ball in a new environment (e.g., the images contain unexpected irrelevant distractors or the drag coefficient is different than used in the physical model), the tight coupling between sensors, modeling assumptions, and control can cause errors to propagate and the robot to fail.

Inspired by the capability of *bounded-rationality* (BR) agents, like humans, to arrive at robust control strategies using high-dimensional sensors, this dissertation focuses on studying theoretically and empirically both necessary and sufficient conditions on the amount of *task-relevant* state and sensor information required to achieve performant control.

Necessary conditions take the form of a lower bound on the best achievable control cost by *any* controller given a sensor and problem description — thereby establishing a necessary amount of task-relevant information a sensor must provide. Not many such bounds are available in the robotics literature, although such fundamental limits are an important component in the natural sciences (e.g. Landauer’s principle described in Chapter 6 and its refinements are important tools in thermodynamics). This dissertation presents newly developed necessary conditions in terms of quantifications of the robot’s dynamics and sensors to determine a fundamental limit on the performance achievable on a task. They are derived using intuition from fundamental limits that appear in information theory and statistical thermodynamics, yet are tailored to control problems faced by roboticists, are shown to be approximately computable using a novel dynamic programming algorithm, and their practical use is demonstrated by their efficacy on different control tasks.

Sufficient conditions instead take the form of a robustness statement that establishes whether enough sensor information is sufficient to achieve a level of performance in the presence of disturbances (either random or antagonistic) to the sensing and estimation process. The resulting performance bounds are established primarily using information theory

to quantify task-relevant information. This model requires that the optimal controller takes the form of a Gibbs measure, which is a well-studied probability distribution in a number of fields, the most pertinent being computational privacy and large deviation theory. The Gibbs measure serves as a *lingua franca* allowing for the novel combination of techniques to establish the desired sufficient condition. These results apply to both a class of algorithms already developed with the goal of efficiently generating approximate solutions to optimal control problems (detailed in Section 2.2) and to newly proposed algorithms which arrive at BR by synthesizing a controller dependent on only a small number of task-relevant variables. Empirical studies included show that, even for algorithms included in the framework that were not designed for robust control, they exhibit a level of robustness in the manner characterized by the sufficient conditions derived.

1.1 Organization and Summary of Contributions

This dissertation is divided into three parts. The first two are development and analysis of the necessary and sufficient conditions described in the preceding section. While key results have appeared in peer reviewed publications produced during the course of the author’s research, the treatment in these sections is a unified presentation of these results with augmented discussion and analysis. The third is an appendix containing reproductions of these publications. As such, proofs and implementation details will largely be deferred from the main body to the appendix. These reproductions have been minimally edited to account for changes in format from their original presentation. An outline of the body of this work is as follows:

- Part I treats the sufficient conditions for task-driven control, since the bulk of the work in this dissertation pertains to these conditions.
 - Chapter 2 contains motivation and a comprehensive description of related literature. Much of the related work is dispersed across disciplines, and the synthesis here is a unifying overview.
 - Chapter 3 establishes the theoretical framework — most critically the framework

of bounded-rationality used and distinction between “offline” performance, where the robot is assumed access to the full state of the dynamics, and “online” performance, where the same controller is applied in a partial-observability setting. A formal notion of TRVs is introduced, and two formal information-theoretic sufficient conditions are established. In comparison to the original publication of these results [7], additional discussion includes the design of Bayesian filters over the reduced-order system of TRV and how the framework is applicable to output-feedback control.

- Chapter 4 provides a brief overview of algorithmic implementations of task-driven bounded-rationality controllers. Algorithms described include analytic dynamical programming solutions for linear systems [7, 8], iterative local approximation algorithms [8], deep reinforcement learning methods [9], and a previously extant inference-as-control algorithm. .
- Chapter 5 details a series of simulations to verify the efficacy of the task-driven approach to control. These experiments include nonlinear dynamical systems, systems featuring high-dimensional camera sensors, and motion planning experiments.
- Part II describes two distinct necessary conditions, one of which appears for the first time in this dissertation.
 - Chapter 6 describes the motivation for developing necessary conditions, which are not commonly studied in robotics, but have significant practical application. This chapter also contains the related work, of which there is relatively little extant in robotics. New to this dissertation is related work relevant to Chapter 8, which provides intuition for the pertinence of Landauer’s principle to establishing a task-relevant necessary condition in robotics.
 - Chapter 7 contains theory, algorithms, and empirical studies conducted *in silico* for a necessary condition derived using recent generalizations of *Fano’s inequality*. These results have appeared in prior publications [10, 11].

- Chapter 8 describes new, unpublished results that derive a necessary condition and algorithm for its evaluation through generalizing *Landauer’s principle* [12] from nonequilibrium thermodynamics. Despite the differences in conceptual framework used to arrive at the necessary condition, it is, in fact, a generalization of the one derived in Chapter 7.
- Chapter 9 concludes this dissertation with a summary of both parts and a discussion of limitations and future work.

Additionally, reproductions of four publications in which much of the material in this dissertation first appeared are included as appendices [7–10]. Full citations and a summary of author contributions are described in Appendix A.

1.2 Notation and Preliminary Definitions

In the bulk of this dissertation, the following conventions are used. Sets are denoted with boldface, e.g., $\mathbf{X}$, operators are denoted with calligraphy with optional around the operand, e.g., $\mathcal{L}[\cdot]$, and functionals are double-struck, e.g., $\mathbb{E}[\cdot]$. The indicator function for a set $\mathbf{A}$ is denoted $\mathbb{1}_{\mathbf{A}}(\cdot)$. A sequence $x_0, \dots, x_n$ is denoted $x_{0:N}$ for short, and, when it is clear in context that the whole sequence is being referred to, just a single variable subscript may be used, e.g., x_i .

This dissertation uses a somewhat unique notation for probabilistic quantities to bridge the differential privacy literature with information theory and stochastic control. Differential privacy, being an algorithmic framework, conceptualizes conditional probability distributions as stochastic algorithms or functions, and express them in a similar notation to easily express operations like composition. This approach is equivalent to the traditional method of representing transition probabilities (or densities / kernels), but is more compact when the value of a measure does not need to be written explicitly frequently.

The set of probability measures on $\mathbf{X}$ is denoted $\Delta(\mathbf{X})$. To simplify exposition, measures will be treated as continuous probability density, however most results admit generalization to a proper measure-theoretic framework. Random variables are capitalized with distri-

butions in calligraphy, e.g., $X \sim \mathcal{N}(\bar{x}, \Sigma)$ denotes a (multivariate) Gaussian with mean $\bar{x}$ and covariance Σ . To highlight their common interpretation in this work as a stochastic algorithm, conditional distributions are treated as operators overloaded in the following manner. The distribution $\mathcal{F} : \mathbf{X} \rightarrow \Delta(\mathbf{Y})$ denotes a distribution with support $\mathbf{Y}$ conditioned on members of $\mathbf{X}$, i.e., $\forall x \in \mathbf{X}, \mathcal{F}(x) \in \Delta(\mathbf{Y})$. If $\mathcal{X} \in \Delta(\mathbf{X})$, then $\mathcal{F}[\mathcal{X}]$ and $(\mathcal{F} \circ \mathcal{X})$ are both used to denote the marginal $\mathcal{F}(X) \in \Delta(\mathbf{Y})$ where $X \sim \mathcal{X}$. In both cases, it may be treated as a random variable conditioned on its argument. Finally, braces are appended to distributions to denote their probability density function, i.e., $\mathcal{N}(\bar{x}, \Sigma)\{x\}$ and $\mathcal{F}[\mathcal{X}]\{y\}$ denote the Gaussian density evaluated at x and the aforementioned marginal evaluated at y respectively. Finally, expectations are denoted using the functional $\mathbb{E}[\cdot]$, and integration is performed over all random variables present in the integrand, i.e., if $X \sim \mathcal{X}$, the expected value of $f(X, Y)$ conditioned on $Y = y$:

$$\mathbb{E}[f(X, y)] = \int_{\mathbf{X}} dx f(x, y) \cdot \mathcal{X}\{x\}. \quad (1.1)$$

If the conditioning variables do not appear explicitly in an expectation, they may be included after a $|$, i.e. $\mathbb{E}[Z|Y = y]$ for $Z = f(X, Y)$.

Since they are minimally edited, appendices introduce and use their own conventions that may vary from the body of this document.

Part I

Sufficient Conditions

Chapter 2

Introduction

Work on establishing sufficient conditions for task-driven control of robotic systems comprises the bulk of this dissertation. The *bounded-rationality* (BR) control problem is formulated as what is known as a rate-distortion problem in information theory [13] and a maximum entropy problem in statistical mechanics [14, 15]. Specifically, the expected cost incurred by the controller is minimized subject to a constraint on state information used by the controller as measured using the relative entropy (also known as the Kullback-Liebler divergence) [13]. Two such sufficient conditions are present both of which hinge on an analysis using the variational representation of the relative entropy known as the *Donsker-Varadhan representation* [16] and Jaynes’ *principle of maximum entropy* [14]. Critically, the optimal control policy follows a *Gibbs measure*,¹ which satisfies a novel variant of *differential privacy* (DP) — a concept first introduced in the computational privacy literature to quantify and limit the capabilities of an antagonist given access to noisy statistics computed on an anonymous database [17, 18] — formulated for optimal control applications. This property allows for expectations of quantities in the partially-observable “online” setting, corresponding to when the robot is deployed, to be evaluated with data collected “offline” in a fully-observable setting corresponding to simulation or the laboratory where the robot’s estimation algorithms are well tuned. Additionally, it is discussed how this framework can

¹The conditional form of the Gibbs measure used is also called the exponential mechanism in the privacy literature. In statistical physics, when it is most commonly supported on a finite set of states, it is known as the Boltzmann distribution. In statistics and machine learning, it goes by both the latter as well as the “SoftArgmin” due to its behavior of selecting an approximate minimizer.

be applied to output-feedback problems common in the deep learning literature (the most famous example being playing Atari by learning a feedback controller mapping screen data to controls [19, 20]).

This chapter also includes two new topics of discussion. The first explains how this framework also applies to algorithms described in Chapter 4 that simultaneously synthesize an optimal control policy that depends on a reduced-order representation of the state known as the *task-relevant variable* (TRV). The second discusses the importance of the choice of metric used in the definition of DP and how the choice pertains to capturing task-relevance.

2.1 Motivation

As described in Chapter 1, the chief motivation of these sufficient conditions is twofold. The first is to provide a kind of information-theoretic *separation principle* [1] for situations beyond the typically linear cases in which the typical separation principle holds. That is, the condition will quantify, in terms of information used by the controller and accuracy of the state estimator, an upper bound on online control cost. Restated, it is a condition that the designed estimator and controller are sufficient for a particular control performance even when certain levels of stochastic or antagonistic noise is present.

Second, similarly to the gaze heuristic discussed in Chapter 1, bounded-rationality controllers do not require the estimation of the complete, high-dimensional state of the robot and its environment, but only require the estimation of a few variables. Therefore, a bounded-rationality controller can be limited to depend on a low-dimensional latent state representation referred to as the problem’s TRV, and only these variables need to be estimated — resulting in a simplified and more efficient estimation process for the robot to run online. Therefore, the bounded-rationality controllers formulated in this chapter will benefit from both the robustness and computational advantages identified in cognitive systems described in Chapter 1.

2.2 Related Work

This section briefly reviews three areas of the literature that are typically considered independently from one another. However, the unifying theme is that they exploit various properties of the Gibbs measure.

2.2.1 Differential Privacy and Applications in Control Theory

Differential privacy (DP) [17, 18, 21, 22] is an algorithmic framework that arose to meet the conflicting needs of statisticians (who ultimately want to publish analyses of sensitive data sets) and participants (who want to keep their data private). DP offers an elegant solution based on the intuition that if the results published from a study are insensitive to the substitution of data from any given individual, then this offers privacy. The literature on DP is vast and many differentially-private *mechanisms* (i.e., stochastic algorithms) have been proposed. The most relevant for our purpose is the *exponential mechanism* [18] defined by the Gibbs measure; this mechanism provides a method for releasing the results of an optimal decision-making procedure based on data (e.g., Bayesian inference [23]). Additionally, DP enjoys several appealing theoretical properties, namely the preservation of privacy under composition of mechanisms, and the ability to rigorously analyze the trade-off between privacy and accuracy [18].

In the context of control theory, application of the DP formalism is largely limited to networked control and distributed systems [24–29]. These include privately solving distributed optimization problems, modifying control inputs to keep the system state private from an external observer, and aggregating measurements from privacy-concerned agents for filtering performed by a central entity. In contrast, we *do not* use DP for the sake of privacy; instead, DP formalizes the intuition described in Chapter 1: limiting the sensitivity of control inputs to state estimates affords robustness to estimation error. To our knowledge, our work is the first to utilize DP for this purpose.

2.2.2 Bounded Rationality and Robust Decision-Making

Bounded rationality is a model of decision-making first introduced to address misalignment between economic theory based on rational agents and the reality of sub-optimal human decision-making [2]. In short, while agents often have an objective they are trying to optimize, their ability to optimize this objective is bounded by informational and computational constraints. The cognitive science literature has identified several *heuristics* used by humans to deal with such constraints [3, 4, 30, 31]. Empirical work in cognitive science and robotics suggests that in addition to efficiency, such heuristics can provide generalization to new environments [3–5, 8, 9].

While bounded rationality is formalized in a number of ways for artificial agents, the most relevant framework adds an *information-theoretic constraint* to an otherwise rational agent. More precisely, a bounded-rational agent is modeled as solving an *entropy-maximization* or *rate-distortion* problem [32–37]. Despite empirical evidence that such a bounded-rational agent can be robust to uncertainty or noise in sensor measurements and dynamics [8, 9, 38–40], formal analyses justifying such robustness benefits are limited. One approach is to use a variational representation of the (relative) entropy (e.g., the Donsker-Varadhan representation [41]) to derive a zero-sum game that the bounded-rational agent is implicitly playing against an adversary that chooses a cost function [35, 36, 42]. This approach is closely related to the *maximum-entropy principle* from Bayesian statistics [43] and leads to generalization results for some estimation problems. However, the payoff of the game is difficult to interpret (especially for the adversary) in the context of most control problems, which limits the usefulness of the analysis. Alternatively, an upper bound on the performance degradation of a bounded-rational agent under measurement error exists [8] using variational representations of the entropy, but its assumptions make the bound difficult to interpret practically. Instead, this chapter demonstrates that the connection between DP and bounded rationality through the Gibbs measure allows for the derivation of a bound on control performance in the presence of estimation error through a large-deviation analysis. The trade-offs presented by this bound are easily understood and, importantly, the bound only depends on access to simulation or lab data without actually deploying the

robot in the target environment.

2.2.3 Optimal Control as Bayesian Inference

Another relevant vein of research at the intersection of information theory, Bayesian inference, and control theory is *linearly-solvable optimal control* (LSOC) [44–53]. These techniques exploit an identified equivalence between optimal control and Bayesian inference: an approximately optimal control sequence is found by sampling from a Gibbs measure over control inputs conditioned on the current state and the event that the sequence results in an optimal trajectory [47]. The result is that the nonlinear stochastic optimal control problem is solved as a linear, albeit infinite-dimensional, differential equation using a transformed cost function, and the solution is approximately computed using inference algorithms that include importance sampling [54], Stein-variational gradient descent (SVGD) [55], and Q -learning [56]. Despite the growing popularity of these algorithms for optimal control and empirical evidence for the generalization benefits they confer [57], theoretical knowledge of their robustness properties is limited [58]. This paper aims to fill this gap by providing a new, concrete analysis that these algorithms are provably robust to estimation error.

Chapter 3

Information-Theoretic Sufficient Conditions for Performant Control

This chapter establishes the problem formulation and establishes formal sufficient conditions for performant, task-relevant optimal control. Two *mechanisms* (stochastic functions) that encode how information flows from states to TRV to inputs in a manner that minimizes a control cost over a finite time horizon: one mechanism defining the feedback controller and one representing the change in sensing capabilities between the “offline” design and “online” evaluation environments. To simplify their introduction, results will first be established for the special case where the task-relevant variables are assumed to be the same as the full system state, meaning only a feedback policy must be synthesized. This chapter will conclude with discussion on how the results can then be used to synthesize both mechanisms.

3.1 Problem Formulation

To begin, this section formulates the two optimal control problems studied, the first being a fully-observable “offline” problem describing the control loop in simulation or the laboratory where the robot has a well-tuned state estimator and thus approximately full access to its state, and the second being the “online” setting in which the robot is deployed and only receives noisy state estimates.

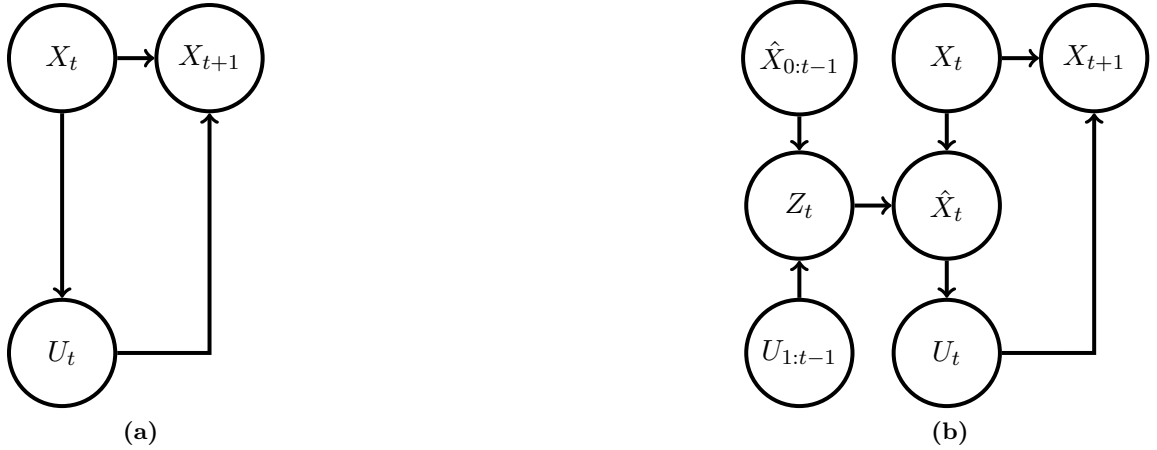


Figure 3.1: A graphical comparison of the offline and online settings in this chapter. In the offline setting, TRVs are computed directly from the current state and TRVs at the current time step, and the controller is computed using the current TRVs. In the online setting, TRVs are estimated from sensor readings using information from past estimates and the current sensor output.

3.1.1 System Dynamics

Formally, let $\mathbf{X} \subseteq \mathbf{R}^n$, $\mathbf{U} \subseteq \mathbf{R}^m$ be the state space, output space, and input space respectively. Interpretations of these spaces in the context of robotics problems will be described in the next paragraph. The initial state is randomly selected according, i.e., $X_0 \sim \mathcal{X}_0$, the Markovian stochastic dynamics are denoted by the mechanism $\mathcal{F} : \mathbf{X} \times \mathbf{U} \rightarrow \Delta(\mathbf{X})$. Feedback is provided by the mechanism specified at each time t , $\mathcal{U}_t : \mathbf{X} \rightarrow \mathbf{U}$. The dynamics for the “offline” setting are then given by a fully-observable, discrete-time Markov process:

$$X_{t+1} \sim \mathcal{F}(X_t, U_t), \quad U_t \sim \mathcal{U}_t(X_t). \quad (\text{OFF})$$

In the “online” setting *the same controller* does not receive the state directly, but instead receives an approximate estimate of the state given by the mechanism $\hat{\mathcal{X}}_t : \mathbf{X} \times \mathbf{Z}_t \rightarrow \Delta(\mathbf{X})$, where $\mathbf{Z}_t$ is the set supporting some random variable, Z_t , that is measurable given $(X_{0:t}, U_{0:t-1})$. leading to the partially-observable process:

$$X_{t+1} \sim \mathcal{F}(X_t, U_t), \quad \hat{X}_t \sim \hat{\mathcal{X}}_t(X_t; Z_t), \quad U_t \sim \mathcal{U}_t(X_t), \quad (\text{ON})$$

These two dynamical settings are graphically summarized in Figure 3.1. There are two interpretations of this dynamical setup that are relevant:

Feedback with State Estimation. In the most straight-forward interpretation, the states are the robot’s coordinates and momenta, and the control inputs are motor torques. The mechanism $\hat{\mathcal{X}}_t(x; z)$ is formed by the composition of a sensing mechanism and a (possibly recursive) estimation mechanism. Specifically, let $\mathbf{Y} \subseteq \mathbf{R}^p$ be the set of sensor outputs (e.g., camera images), $\mathcal{Y}_t : \mathbf{Y} \rightarrow \Delta(\mathbf{Y})$, be the sensing mechanism, $\hat{\mathcal{X}}_t : \mathbf{Y} \times \mathbf{Z} \rightarrow \Delta(\mathbf{Y})$ be a state-estimation mechanism, and $Z_t = (Y_{0:t}, U_{0:t-1})$. Then $\hat{\mathcal{X}}_t(x_t; z_t) = \hat{\mathcal{X}}_t(\mathcal{Y}_t(x_t); z_t)$ is a mechanism that outputs an estimate of the current state given the history of sensor outputs and control inputs. The estimator can be Bayesian, e.g., a Kalman filter [59], or another method.

Output Feedback. Alternatively, a common way to handle output feedback in deep learning approaches to control is to treat the state X_t as either the current sensor observation (again, e.g., camera images) or a recent history of sensor observations [19], i.e., $X_t = (Y_{t-k}, \dots, Y_t)$ for $k \in \mathbf{N}$.¹ Then, $\hat{X}_t$ represents the outputs of a sensor that is perturbed in some way. No specific assumptions are made about the nature of perturbation. A simple example is the perturbation being an increase in sensor noise. A more interesting example is when the sensor is very high-dimensional, like a camera, and its measurement distribution is modified in a largely task-irrelevant way. In Section 5.3, both situations are explored empirically for a ball-catching task, where the latter is instantiated as a change in the background of the scene where the ball is thrown. Under this interpretation, $\hat{X}_t$ is an estimate of X_t in the sense that it contains approximately as much task-relevant information as X_t .

Remark 1. Deterministic dynamics are considered a special case of the stochastic dynamics and are handled similarly. Equivalent results for time-varying dynamics also proceed in the

¹It is worth noting that the dynamics of the system output are, in general, not Markovian [15]. The impact of approximating them as Markovian is not considered here. In deep learning approaches to output feedback, they are often assumed to be Markovian [19], which is a good approximation if the dependence between the initial and current underlying state of the system decays quickly compared to the timescale of the planning horizon. Moreover, the study of the dynamics of observables of a system is a common element of statistical mechanics, where non-Markovian the non-Markovian aspects of their evolution are treated as process noise driving the system [15].

same manner as the following settings, but are not shown explicitly to reduce subscripts.

3.1.2 Online and Offline Optimal Control Problems

The control objective is specified by an instantaneous cost function, denoted $c_t : \mathbf{X} \times \mathbf{U} \rightarrow \mathbf{R}_+$ and a terminal cost function denoted $c_{t_f} : \mathbf{X} \rightarrow \mathbf{R}_+$. $\hat{X}$ (to simplify notation in sums, $c_{t_f}(x, u) := c_{t_f}(x)$ may be used). The primary measure of performance of interest is the performance of a controller as a solution to the *online optimal control problem*:

$$\underset{\mathcal{U}_t(\cdot)}{\text{minimize}} \quad \mathbb{E} \left[\sum_{t=0}^{t_f} c_t(X_t, U_t) \right] \quad \text{subject to} \quad (\text{ON}). \quad (3.1)$$

The value of this optimization problem for a particular (not necessarily optimal) controller will be denoted $\mathbb{J}^{\text{on}}[\mathcal{U}_t]$.

While the goal is to solve the optimization problem described in (3.1), there are two critical challenges to solving this problem directly in practice. First, as with most partially-observable optimal control problems, the optimal solution requires computing the *belief state* at each time step, which is, in general, infinite-dimensional. Second, robotics applications often feature high-dimensional sensors, i.e., $p \gg n, m$, and sensors may not even have a tractable analytical model. For example, even if a pinhole model is used to represent an camera, determining the color of each pixel requires modeling detailed reflective properties of each object the robot may encounter, knowledge of light sources, etc.

To eschew these challenges, an approximate solution will be considered to the more tractable, fully-observable *offline* problem:

$$\underset{\mathcal{U}_t(\cdot)}{\text{minimize}} \quad \mathbb{E} \left[\sum_{t=0}^{t_f} c_t(X_t, U_t) \right] \quad \text{subject to} \quad (\text{OFF}). \quad (3.2)$$

The value of this optimization problem for a particular (again, not necessarily optimal) controller will be denoted $\mathbb{J}^{\text{off}}[\mathcal{U}_t]$. The approximate solution will be a BR controller, and it will be shown that this controller allows us to bound, with high probability, with the gap

in its performance between these two problems, i.e.,

$$\mathbb{J}^{\text{on}}[\mathcal{U}_t] - \mathbb{J}^{\text{off}}[\mathcal{U}_t], \quad (3.3)$$

as well as the gap between its online performance and the best achievable control:

$$\mathbb{J}^{\text{on}}[\mathcal{U}_t] - \inf_{\mathcal{U}_t} \mathbb{J}^{\text{off}}[\mathcal{U}_t]. \quad (3.4)$$

3.2 Bounded-Rationality Control

To achieve this guarantee, the offline problem is augmented with a BR constraint. That is, a constraint on how much the controller is allowed to depend on the present state when computing the present input is added to (3.2). The intuition for doing so is that the less information the controller uses, the less the estimation error introduced to $\hat{X}$ can impact its performance. This line of thought is particularly clear in the zero-information limit: the controller becomes an open-loop sequence of inputs that are independent of estimation noise. However, given a perfect state estimate, the best feedback controller will always perform at least as well as the best open-loop controller, and in general, perform better. Thus, there is a trade-off between online and offline performance when using a BR controller.

There are multiple information-theoretic models of BR. The ones of interest use a constraint on the *relative entropy* between $\mathcal{U}_t(X)$ and a chosen time-varying open-loop controller $\mathcal{U}_t^\perp \in \mathbf{\Delta}(\mathbf{U})$ satisfying the absolute continuity assumption: $\forall x \in \mathbf{X}, 0 \leq t < t_f, \mathcal{U}(X) \ll \mathcal{U}^\perp$. The relative entropy is defined [13]:

$$\mathbb{D}[\mathcal{U}_t(x); \mathcal{U}_t^\perp] := \mathbb{E} \left[\log \frac{\mathcal{U}_t(x)\{U\}}{\mathcal{U}_t^\perp\{U\}} \right] \quad (3.5)$$

The relative entropy is a means of quantifying how different two distributions are, and, importantly, $\mathbb{D}[\mathcal{U}(X); \mathcal{U}^\perp] = 0$ if and only if for almost all $x \in \mathbf{X}$, $\mathcal{U}_t^\perp = \mathcal{U}_t(x)$ almost surely. Therefore, constraining the relative entropy corresponds to constraining how different the distribution of the closed-loop controller can be from the open-loop controller and, as a result, how “closed-loop” can therefore be. The relative entropy constraint is added to (3.2)

to create the BR problem, which is most conveniently presented in the form of a Lagrangian function:

$$\underset{\mathcal{U}_t(\cdot)}{\text{minimize}} \quad \mathbb{E} \left[\sum_{t=0}^{t_f} c_t(X_t, U_t) + \frac{1}{\beta} \mathbb{D}[\mathcal{U}_t(X_t); \mathcal{U}_t^\perp] \right] \quad \text{subject to} \quad (\text{OFF}). \quad (\text{BR})$$

Here $\beta^{-1} \geq 0$ is the Lagrange multiplier corresponding to the constraint:²

$$\mathbb{E} \left[\sum_{t=0}^{t_f} \mathbb{D}[\mathcal{U}_t(X_t); \mathcal{U}_t^\perp] \right] \leq t_f d^{\max}. \quad (3.6)$$

Larger values of β correspond to the controller using more state information, i.e., there is a monotonic relationship between β and $d_{\max}$.

This optimization is well-studied beyond the BR literature and is known as a (multi-step) rate-distortion problem in information theory [13, 61] and a maximum-entropy problem in statistical mechanics [14]. As a necessary condition following from a first-order variational analysis, the optimal solution is a distribution as the (conditional) *Gibbs measure*, or, in the DP literature, the *exponential mechanism*. The specific form of this distribution for (BR) is given in the following proposition:

Proposition 1. Let $U_t^\perp \sim \mathcal{U}_t^\perp$. The optimal solution to the problem (BR), $\mathcal{U}_{0:t_f}^\beta[\cdot]$, satisfies the following dynamic programming equations for all $t = 0, \dots, t_f$ and almost all $x \in \mathbf{X}, u \in \mathbf{U}$:

$$\begin{aligned} \mathcal{U}_t^\beta(x)\{u\} &= \frac{\mathcal{U}_t^\perp\{u\} \exp(-\beta H_t(x, u))}{Z_t^\beta(x)}, \\ H_t(x, u) &= c(x, u) + \mathbb{E}[V_{t+1} \circ \mathcal{F}(x, u)], \\ V_t(x) &= -\frac{1}{\beta} \log Z_t^\beta(x), \\ Z_t^\beta(x) &= \mathbb{E} \left[\exp \left(-\beta H_t(x, U_t^\perp) \right) \right], \end{aligned} \quad (3.7)$$

with $V_{t_f}(x) = c_{t_f}(x)$.

²Technically, $\beta = 0^{-1}$ is undefined, but the optimal solution converges to $\mathcal{U}_t^\perp$ and optimal value converges to $\mathbb{J}^{\text{off}}[\mathcal{U}^\perp]$ as $\beta \rightarrow 0$. Therefore it is standard to define this behavior for the problem as a special case [60].

Proof. The proof of this is standard in the literature and arises from the Euler-Lagrange equations for the variational problem at hand [13, 61]. See Appendix B for a derivation. $\square$

The Gibbs measure has two notable interpretations. In an optimal control context, $V_t : \mathbf{X} \rightarrow \mathbf{R}_+$ is interpreted as the value function and $H_t : \mathbf{X} \times \mathbf{U} \rightarrow \mathbf{R}_+$ is the expected value conditioned on a state-input pair at time t conditioned on $X_t = x, U_t = u$ (i.e., the Q -function in reinforcement learning). The Gibbs measure selects an approximate minimizing input of $H_t(x, u)$ for the current state, where the accuracy of the minimizer is controlled by β . As $\beta \rightarrow \infty$, the closed-loop solution to (3.2) is recovered, and as $\beta \rightarrow 0$, the open-loop control distribution $\mathcal{U}_t^\perp$ is recovered. The normalizing factor $Z_t^\beta(x)$ is called the *partition function*.

In addition to being the value function, $V_t(x)$ is referred to as the *equilibrium free energy* in thermodynamics, which is the amount of energy available to perform work in a thermal system. Used in the derivation of Proposition 1 is the fact that $V_t(x)$ provides a variational lower bound on (BR) for each time step, i.e.,

$$V_t(x) = \inf_{\mathcal{U}_t} \mathbb{E}[H_t(x, U_t)] + \frac{1}{\beta} \mathbb{D}[\mathcal{U}_t(x); \mathcal{U}_t^\perp], \quad \text{subject to} \quad U_t \sim \mathcal{U}_t(x). \quad (3.8)$$

This relationship is known as the *Donsker-Varadhan* representation. In physics, the objective being optimized is the *non-equilibrium free energy* and therefore it is also called the *minimum free entropy principle*. While not pertinent to the present conversation, the relative entropy can be written re-expressed the same variational principle, and this form is employed in Chapter 7.

A common variant of (BR) is to replace the relative entropy with the *mutual information* between X_t and U_t . The mutual information is an information-theoretic measure of statistical dependence between two random variables interpreted as how much information one variable shares as the other [13]. Specifically, the mutual information is defined as the relative entropy used in (BR) where the open-loop controller taken to be the marginal

distribution of U_t corresponding to the closed-loop policy, specifically,

$$\mathbb{I}[X_t; U_t] := \mathbb{E} \left[\mathbb{D} \left[\mathcal{U}_t(X_t); \mathcal{U}^\dagger \right] \right], \quad \mathcal{U}_t^\dagger\{u\} := \mathbb{E}[\mathcal{U}_t(X_t)\{u\}]. \quad (3.9)$$

The problem (BR) is then:

$$\underset{\mathcal{U}_t(\cdot)}{\text{minimize}} \quad \mathbb{E} \left[\sum_{t=0}^{t_f} c_t(X_t, U_t) + \frac{1}{\beta} \mathbb{I}[U_t; X_t] \right] \quad \text{subject to} \quad (\text{OFF}). \quad (\text{MI-BR})$$

All subsequent analysis in this chapter applies equally to this model as the marginal is a specific choice of open-loop controller. This relationship is made clear by the variational relationship [62]:

$$\mathbb{I}[X_t; U_t] = \inf_{\mathcal{U}_t^\perp} \mathbb{D} \left[\mathcal{U}_t(X); \mathcal{U}_t^\perp \right]. \quad (3.10)$$

Thus, the mutual information bounded rationality models can be interpreted as jointly minimizing the objective of (BR) over both $\mathcal{U}_t(X)$ and $\mathcal{U}_t^\perp$. Bounds derived in the sequel will be tighter for this model of BR. However, this model is more difficult to solve numerically. Parametric methods require some choice of parameterization of the dagger $\mathcal{U}_t^\dagger$, which may be difficult to represent if analytic forms are not available (in most cases they are not), and the mutual information is similarly difficult to compute analytically [62]. Moreover, Monte Carlo methods that approximate implementing the solution to (BR) via sampling require access to the density describing $\mathcal{U}_t^\perp$, which is not known *a priori* for (MI-BR). In Chapter 4, an alternative method using recent deep learning methods to compute and optimize over the mutual information is described.

3.3 Bounded Rationality Implies Differential Privacy

Differential privacy is a notion of smoothness of stochastic mechanisms that is similar to Lipschitz continuity with a specific choice of metric on the mechanism's range,³ but weaker in that it can be relaxed to only hold with some probability. Moreover, DP features a

³The metric on the space of distributions used is $\sup_{X \subseteq \mathbf{X}} |\log \mathcal{M}_1\{\mathbf{X}\} - \log \mathcal{M}_2\{\mathbf{X}\}|$, where $\mathcal{M}_1, \mathcal{M}_2 \in \Delta(\mathbf{X})$ are two distributions.

number of composition properties that allow differentially private algorithms to be built by combining mechanisms.

The original motivation for DP is that it limits the the impact substitution a database for a “nearby” database has on statistics computed on the data base, thereby limiting how much information an antagonist can gain from knowing the computed statistic and how much an individual’s utility (for any reasonable utility function) decreases by being present in the database. The intuition to applying this concept to the present optimal control framework is that, similarly to exchanging two “nearby” databases has a limited impact an individual’s utility, exchanging a state estimate for the true state will have a limited impact on the cost for employing a differentially private mechanism. The Gibbs measure, whose form the BR controller takes, has appeared in past privacy literature. The focus of this section is a new generalized definition (published by the author in [7]) of DP suitable for common control problems and establishing its useful properties for proving the main sufficient condition (Theorem 1). Specifically, the definition of DP used is a relaxed version of metric DP [22]:

Definition 1 (Differential Privacy [7]). Let $(\mathbf{X}, \rho)$ be a pseudometric space and $X, \hat{X}$ be two random variables supported on $\mathbf{X}$. A mechanism $\mathcal{U} : \mathbf{X} \rightarrow \Delta(\mathbf{U})$ is said to have (ρ, γ) -random differential privacy $((\rho, \gamma)$ -DP) if, with probability $1 - \gamma$:

$$\forall u \in \mathbf{U}, \quad \log \mathcal{U}(X)\{u\} - \log \mathcal{U}(\hat{X})\{u\} \leq \rho(X, \hat{X}). \quad (3.11)$$

The primary use of DP in establishing robustness to estimation error is that it implies a method for estimating statistics on the output of a mechanism under one distribution over the mechanism’s argument using another distribution that is potentially more accessible.

Lemma 1. Let $\mathcal{M} : \mathbf{X} \rightarrow \Delta(\mathbf{Y})$ be a (ρ, γ) -DP mechanism. Let $g : \mathbf{X} \times \mathbf{Y} \rightarrow \mathbf{R}$ be a suitably integrable function, and let $\mathcal{X}^1, \mathcal{X}^2 \in \Delta(\mathbf{X})$ be two distributions on $\mathbf{X}$. Then, with probability $1 - \gamma$, the following bound holds for $X^1 \sim \mathcal{X}^1, X^2 \sim \mathcal{X}^2$:

$$\mathbb{E}[g(X^1, \mathcal{M}(X^1))] \leq \mathbb{E}[\exp(\rho(X^1, X^2))g(X^2, \mathcal{M}(X^2))]. \quad (3.12)$$

Proof. This inequality follows directly from substituting Definition 1 into the definition of the expectation on the left-hand side. $\square$

If the distance between the two random variables is uniformly bounded $\rho(X^1, X^2) \leq r$, the right-hand side above can be further bounded and the result is independent of $\mathcal{X}^2$. This bound may be further relaxed if uniformity only holds with high probability. In the sequel, this property is used to bound expectations for the online cost using expectations computed using the fully-observable, offline setting.

If $\gamma = 0$, this definition of DP reduces to Lipschitz continuity. However, this definition is more flexible, since it applies to mechanisms whose range has certain low-probability regions with large local Lipschitz constants. As a result, it applies to bounded rationality controller (3.7) even when the typical requirement of $x \mapsto H_t(x, u)$ being uniformly Lipschitz continuous for all $u \in \mathbf{U}$ fails:

Proposition 2 ([7]). Let $X^1 \sim \mathcal{X}^1, X^2 \sim \mathcal{X}^2$ be two random variables taking values in $\mathbf{X}$ and let $\rho : \mathbf{X} \times \mathbf{X} \rightarrow \mathbf{R}_+$ be a metric on $\mathbf{X}$. Consider the set of all $u \in \mathbf{U}$ for which $H_t(x, u)$ is l -Lipschitz continuous in x with respect to ρ , i.e. $\mathbf{U}(l) := \{u \in \mathbf{U} \mid \mathbb{E}[x \mapsto H_t(x, u); \rho] \leq l\}$. Then, $\mathcal{U}_t^\beta(x)$ is $(2\beta l\rho, \gamma(l))$ -DP, where:

$$\gamma(l) := 1 - \mathbb{E} \left[\mathbf{1}_{\mathbf{U}(l)}(U) \exp \left(-2\beta l\rho(X^1, X^2) \right) \right]. \quad (3.13)$$

Proof. See Appendix D. $\square$

Note that $1 - \gamma(l)$ is interpreted as the probability of selecting an input for which $H(x, u)$ is l -Lipschitz continuous re-weighted by the inverse-exponential distance.

Importantly, this modified definition maintains the composition properties of other common definitions of DP [18].

Proposition 3 ([7]). Mechanisms satisfying Definition 1 hold the following composition properties:

Post-Processing. If $\mathcal{M}_1 : \mathbf{X} \rightarrow \Delta(\mathbf{U})$ is (ρ, γ) -DP, then for any $\mathcal{M}_2$, the sequential composition $\mathcal{M}_2 \circ \mathcal{M}_1$ is (ρ, γ) -DP.

Parallel Composition. Let $\mathcal{M}_1, \mathcal{M}_2 : \mathbf{X} \rightarrow \Delta(\mathbf{U})$ be (ρ_1, γ_1) -DP and (ρ_2, γ_2) -DP mechanisms respectively. Then $\mathcal{M} = (\mathcal{M}_1, \mathcal{M}_2)$ is (ρ, γ) -DP where $\rho(x, \hat{x}) := \rho_1(x, \hat{x}) + \rho_2(x, \hat{x})$ and $\gamma := \gamma_1 + \gamma_2$.

Proof. See Appendix D. □

These properties are useful for two reasons. First, these properties are used to prove the main robustness result in the next section by applying Proposition 2 in a backward-recursive manner. Moreover, if the BR controller is used as part of a greater control system, other elements can inherit its privacy benefits.

3.4 Bounded-Rationality Control provides Robustness to Estimation Error

This section describes the main sufficient condition establishing the robustness properties of the BR controller defined in (3.7). A brief summary of proof steps is provided here, with a complete proof included in the original publication reproduced in Appendix D.

Theorem 1 ([7]). Let $l_t = \mathbb{E}[x \mapsto H_t(x, u); \rho]$. Define,

$$\begin{aligned} \rho_\beta(x_{0:t_f}, \hat{x}_{0:t_f}) &:= \sum_{t=0}^{t_f-1} 2\beta l_t \rho(x_t, \hat{x}_t), & \gamma &:= \sum_{t=0}^{t_f-1} \gamma_t, \\ \gamma_t &:= 1 - \mathbb{E} \left[1_{\mathbf{U}_t(l_t)}(U_t) \exp \left(-2\beta l_t \rho(X_t, \hat{X}_t(X_t)) \right) \right], \end{aligned} \quad (3.14)$$

Additionally, denote $\mathcal{T}^\beta$ the distribution of state-input generated by employing $\mathcal{U}_t^\beta$ with (OFF) and similarly for $\mathcal{T}^\perp$ and $\mathcal{U}_t^\perp$. Then, with probability at least $1 - \gamma(l_t)$, the following two bounds on performance gaps hold:

$$\begin{aligned} \mathbb{J}^{\text{on}}[\mathcal{U}_t] - \inf_{\mathcal{U}_t} \mathbb{J}^{\text{off}}[\mathcal{U}_t] &\leq \frac{1}{\beta} \mathbb{E} \left[\exp \left(\rho_\beta(X_{0:t_f}, \hat{X}_{0:t_f}) + \mathbb{D}[\mathcal{T}^\beta; \mathcal{T}^\perp] \right) \right], \\ \mathbb{J}^{\text{on}}[\mathcal{U}_t^\beta] - \mathbb{J}^{\text{off}}[\mathcal{U}_t^\beta] &\leq \frac{1}{\beta} \mathbb{E} \left[\exp \left(\rho_\beta(X_{0:t_f}, \hat{X}_{0:t_f}) + \mathbb{J}^{\text{off}}[\mathcal{U}_t^\beta] + \mathbb{D}[\mathcal{T}^\beta; \mathcal{T}^\perp] \right) \right]. \end{aligned} \quad (3.15)$$

Proof. The complete proof of the second inequality is presented in [7] and reproduced in Appendix D. The proof of both bounds only differs in choice of a free parameter describing

certain limits of integration. There are three main steps to the argument, which are briefly described here:

1. Differential privacy of the mechanism stochastically mapping the initial state to future state-input sequences is established in a recursive manner using Proposition 3.
2. The Chernoff bound [63] is used to establish an upper bound on the survival function⁴ of the gap between online cost incurred and either $\inf_{\mathcal{U}_t} \mathbb{J}^{\text{off}}[\mathcal{U}_t]$ or $\mathbb{J}^{\text{off}}[\mathcal{U}_t^\beta]$. This is a common approach in establishing bounds in large deviation theory [64], information theory [65], and statistical mechanics [60]. The DP result established in the first step is used to convert the expectation present in the Chernoff bound from the online setting (ON) to the offline setting (OFF). That is, DP allows this expectation to be approximated assuming that the state is fully-observable as described in Lemma 1.
3. The Chernoff method in the previous step introduces a free parameter $\gamma \in \mathbf{R}_+$ which may be optimized to achieve the tightest possible bound. However, doing so is difficult in this instance both analytically and numerically. Instead, the choice $\gamma = \beta$ is made, which simplifies the upper bound and allows the survival function to be integrated, leading to a bound on the expected online cost. The relative entropy term appears from manipulating terms in the exponent using certain variational relationships in a manner similar to those used in statistical mechanics analyses [60].

□

This bound relates the information usage of the controller, in terms of both β and the relative entropy, and the error of the state estimator as measured by $\rho(x, \hat{x})$. Importantly, the expectation does not require the state estimator to evaluate. The state estimator is only required in terms of quantifying its estimation error computed. Alternatively, a uniform bound on estimation error, e.g., $\rho(x, \hat{x}) \leq r$ for all $x, \hat{x} \in \mathbf{X}$ to achieve a looser estimator independent bound.

⁴The survival function is the complement of the cumulative density function, i.e., it is the probability that a random variable is greater than a specific value.

Remark 2. At larger values of β , this bound suggests that a better guaranteed performance can be guaranteed for large estimation error by reducing β . This trade-off applies to the regime where the exponent dominates β appearing in the denominator. At small values of β , the inverse of β dominates. In this regime, a tighter bound can be guaranteed by taking the minimum of the bound in Theorem 1 with the one achieved by applying Lemma 1 directly to the cost function. The latter bound grows very quickly in β , however, due to the exponential dependence on β , and is not as effective for large β in comparison to the result in Theorem 1.

Remark 3. As alluded to earlier, if the mutual information formulation of BR described in (MI-BR) is used, Theorem 1 will be tighter due to the equality,

$$\mathbb{D}[\mathcal{T}^\beta; \mathcal{T}^\perp] = \mathbb{E} \left[\sum_{t=0}^{t_f-1} \mathbb{D}[\mathcal{U}(X_t); \mathcal{U}^\perp] \right], \quad (3.16)$$

and the variational relationship (3.10).

3.5 Extension to Controllers Dependent on Reduced Order Task-Relevant Variables

This chapter concludes by describing how the sufficient description applies to a more general setting, where the controller is separated into a possibly recursive mapping over a small set of TRVs and a mapping from TRVs to control inputs. This approach is used in the earlier works of which this dissertation is comprised [8, 9], which are attempts to algorithmically search for task-relevant heuristics, like the gaze heuristic described in Chapter 1, for general optimal control formulations of a robot tasks.

The intuition for this reduced-model approach is that typically, only a subset of the system dynamics are relevant for optimal or near-optimal control and that these variables tend to be easy to estimate. A common example in robotics, beyond ball catching, is navigation in a space with many independent agents. If a robot is trying to navigate a hallway or side walk with pedestrians, the complete state of the robot and its environment includes the positions and momenta of all pedestrians present. However, the state of pedestrians far

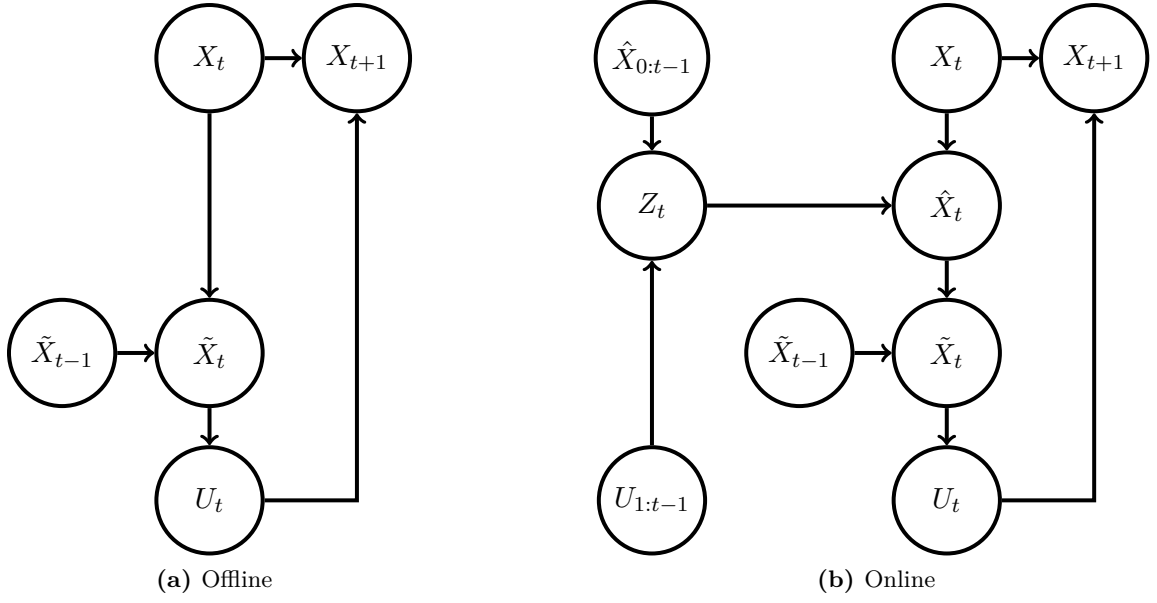


Figure 3.2: A graphical comparison of the offline and online settings in this chapter. In the offline setting, TRVs are computed directly from the current state and TRVs at the current time step, and the controller is computed using the current TRVs. In the online setting, TRVs are estimated from sensor readings using information from past estimates and the current sensor output.

from the robot is not relevant to charting the robot’s path. By the time the robot reaches their location, they may be moving in a manner totally different from what the robot anticipated and therefore their present state is not relevant to planning at the present time step. Thus, sufficiently far away individuals can be removed from the complete state with little impact on planning performance. Moreover, the position and velocity of the relevant nearby pedestrians tends to be easier to estimate, both due to the fact that the relevant system state is of a smaller dimensionality than the complete one and that their variables are more prominently captured by the robots sensors: e.g., they are less likely to be occluded by other pedestrians, they will be larger in a camera image, or they are in a distance for which a LIDAR provides more accurate depth information. This dimensionality reduction principle has a rich history in statistical physics [15, Chapter 8], where projection operators are used to create low-order models of observable dynamics from very high-dimensional systems.

3.5.1 Synthesis of TRV-Based Controllers

Formally, let $\tilde{\mathbf{X}} \subseteq \mathbf{R}^q$ denote what will be the set of TRV, where $p \leq q \leq n$. Let $\tilde{\mathcal{X}}_t : \mathbf{X} \times \tilde{\mathbf{X}} \rightarrow \Delta(\tilde{\mathbf{X}})$ denote a recursive mechanism mapping to the current TRV from the current state and past TRV. Thus, the TRV are defined implicitly via this mechanism. The control mechanism $\mathcal{U}_t : \tilde{\mathbf{X}} \rightarrow \Delta(\mathbf{U})$ is now restricted to depend only on the present TRV, which is denoted $\tilde{X}_t \sim \tilde{\mathcal{X}}(X_t, \tilde{X}_{t-1})$. Thus, the dynamics of the offline are now,

$$X_{t+1} \sim \mathcal{F}(X_t, U_t), \quad \tilde{X}_t \sim \tilde{\mathcal{X}}_t(X_t, \tilde{X}_{t-1}) \quad U_t \sim \mathcal{U}_t(\tilde{X}_t). \quad (\text{TRV-OFF})$$

and the online dynamics are now:

$$\begin{aligned} X_{t+1} &\sim \mathcal{F}(X_t, U_t), & \hat{X}_t &\sim \hat{\mathcal{X}}_t(X_t; Z_t), & (\text{TRV-ON}) \\ \tilde{X}_t &\sim \tilde{\mathcal{X}}_t(\hat{X}_t, \tilde{X}_{t-1}) & U_t &\sim \mathcal{U}_t(X_t). \end{aligned}$$

The initial TRV are initialized randomly: $\tilde{X}_0 \sim \tilde{\mathcal{X}}_0$. These modified offline and online settings are graphically summarized in Figure 3.2. The bounded-rationality control problem (BR) is modified accordingly:

$$\begin{aligned} \underset{\tilde{\mathcal{X}}_t(\cdot, \cdot), \mathcal{U}_t(\cdot)}{\text{minimize}} \quad & \mathbb{E} \left[\sum_{t=0}^{t_f} c_t(X_t, U_t) + \frac{1}{\beta} \mathbb{D}[\tilde{\mathcal{X}}_t(X_t, \tilde{X}_{t-1}); \tilde{\mathcal{X}}_t^\perp] \right] & (\text{TRV-BR}) \\ \text{subject to} \quad & (\text{TRV-OFF}). \end{aligned}$$

The following proposition shows that a necessary condition for optimality is that the TRV mechanism is a Gibbs measure, similarly to the optimal controller for (BR). Unfortunately, a similar first-order analysis no longer elucidates the structure of $\mathcal{U}_t(\tilde{x})$ except for special cases (e.g., see Section 4.1). Therefore, it typically needs to be found numerically.

Proposition 4. For $\beta \in \mathbf{R}_+$ and any control mechanism $\mathcal{U}_t : \tilde{\mathbf{X}} \rightarrow \Delta(\mathbf{U})$, the optimal TRV mechanism for (TRV-BR), $\tilde{\mathcal{X}}_t^\beta[\cdot, \cdot]$, satisfies the following dynamic programming equations

for all $t = 0, \dots, t_f$ and almost all $x \in \mathbf{X}, u \in \mathbf{U}$:

$$\begin{aligned}\tilde{\mathcal{X}}_t^\beta(x_t, \tilde{x}_{t-1})\{\tilde{x}_t\} &= \frac{\tilde{\mathcal{X}}_t^\perp\{\tilde{x}_t\} \exp(-\beta H_t(\tilde{x}_t; x_t, \tilde{x}_{t-1}))}{Z_t^\beta(x_t, \tilde{x}_{t-1})}, \\ H_t(\tilde{x}_t; x_t, \tilde{x}_{t-1}) &= \mathbb{E} \left[\sum_{s=t-1}^{t+1} c_t(X_t, U_t) \middle| X_t = x, \tilde{X}_t = \tilde{x}_t, \tilde{X}_{t-1} = \tilde{x}_{t-1} \right], \\ V_t(x, \tilde{x}) &= -\frac{1}{\beta} \log Z_t^\beta(x, \tilde{x}), \\ Z_t^\beta(x, \tilde{x}) &= \mathbb{E} \left[\exp \left(-\beta H_t(\tilde{X}_t^\perp; x, \tilde{x}) \right) \right], \quad \tilde{X}_t^\perp \sim \tilde{\mathcal{X}}_t,\end{aligned}\tag{3.17}$$

and $V_{t_f}(x, \tilde{x}) = \mathbb{E} \left[c_{t_f}(x) + c_{t_f-1}(X_{t_f}, U_{t_f}) \mid \tilde{X}_{t_f-1} = \tilde{x} \right]$.

Proof. Follows from an identical first-order analysis to Proposition 1. $\square$

The TRV mechanism $\tilde{\mathcal{X}}_t^\beta$ is DP by the same argument as $\mathcal{U}_t^\beta$ described in Proposition 2, *mutatis mutandis*. Moreover, due to the post-processing property in Proposition 3, $(\mathcal{U}_t \circ \tilde{\mathcal{X}})(x, \tilde{x})$ is DP. The robustness of BR control using TRV, as quantified by the bounds Theorem 1, therefore carries over as well *mutatis mutandis*.

Remark 4. The equations in Proposition 4 have the same form as Proposition 1 if the state is augmented to be the pair $(X_t, \tilde{X}_{t-1})$. The recursive dependence of $\tilde{X}_t$ on $\tilde{X}_{t-1}$ presented here is somewhat arbitrary and selected to align with Algorithm 2 described in Chapter 4. An equivalent Gibbs measure will result if $\tilde{X}_t$ depends on any information available to the mechanism from a finite number of previous time steps. The draw back for allowing the current TRV to depend on more information is primarily the increase in the size of the augmented state space for dynamic programming purposes and the impact this has on the Lipschitz constants for the purpose of DP.

Remark 5. It is not explicitly stated how to select the dimensionality of the TRV. In Chapter 5, two methods are used. The first is to pick $q = n$, solve (TRV-BR), compute the rank of $\tilde{\mathcal{X}}_t^\beta$, and then solve (TRV-BR) with q set to that rank. The second, when the rank is not easy to compute, is to find the smallest q for which there exists β such that a performance criteria is satisfied, e.g., using Theorem 1 or simply the offline expected cost being below some threshold.

3.5.2 Efficient Online Estimation of TRVs for Control

This subsection briefly describes how a Bayesian filter for the current TRV is designed. Such an estimator is useful in the state feedback interpretation of the offline and online dynamics described in Section 3.1.1. Offline, the full state is available and the TRV mechanism is designed as a function of the full state. However, online, only the output of a sensor mechanism $\mathcal{Y}_t(x)$ is available and the state needs to be inferred from a history of sensor outputs and control inputs to determine the current state estimate for computation of the control input. However, instead of computing the full state online, appropriate use of Bayes' rule, a TRV estimator can be designed directly if offline computation is cheap. The benefit of doing so is that the estimator only needs to infer a small number of variables online, and will thus do so more efficiently.

Specifically, the estimator will be a Bayesian filter [66] consisting of two mechanism:

Process Update. The process update, represented by the mechanism $\mathcal{B}_t^{\text{proc}} : \tilde{\mathbf{X}} \times \mathbf{U} \rightarrow \Delta(\tilde{\mathbf{X}})$, computes the belief over TRVs at the next time step given the current TRVs and control input.

Measurement Update. The measurement update, represented by the mechanism $\mathcal{B}_t^{\text{meas}} : \tilde{\mathbf{X}} \times \mathbf{Y} \rightarrow \Delta(\tilde{\mathbf{X}})$, integrates knowledge gained by receiving an output $Y_t \sim \mathcal{Y}_t(x_t)$ into the filter's belief over TRVs.

These mechanisms are applied in an alternating fashion,

$$\hat{X}_t^{\text{proc}} \sim \mathcal{B}_t^{\text{proc}}(\hat{X}_t^{\text{meas}}, U_t), \quad \hat{X}_t^{\text{meas}} \sim \mathcal{B}_t^{\text{meas}}(\hat{X}_t^{\text{proc}}, Y_t), \quad (3.18)$$

where $\hat{X}_0^{\text{proc}} \sim \tilde{X}_0$. The two update mechanisms are defined similarly to the standard Bayesian filter [66], but the state dynamics are marginalized out, leaving updates only dependent on the present TRVs and control inputs.

Equations for the specific case where $\tilde{\mathcal{X}}_t(X_t, \tilde{X}_{t-1}) = \tilde{\mathcal{X}}_t(X_t)$ will be given since they are simpler and consistent with the author's prior work [8] referenced later in Section 4.1. For recursive TRV mechanisms, the same method is used, but the state must be augmented as

described in Remark 4. For the non-recursive case, $\tilde{\mathcal{X}}_t$ may be inverted via Bayes rule:

$$\tilde{\mathcal{X}}_t^{-1}(\tilde{x})\{x\} := \frac{\tilde{\mathcal{X}}(x_t)\{\tilde{x}_t\}\mathcal{X}_t^\dagger\{x_t\}}{\tilde{\mathcal{X}}_t^\dagger\{\tilde{x}_t\}}. \quad (3.19)$$

Then, the process update mechanism is defined via its density:

$$\begin{aligned} \mathcal{B}_{t+1}^{\text{proc}}(\tilde{x}_t, u_t)\{\tilde{x}_{t+1}\} &:= \mathbb{E}\left[\tilde{\mathcal{X}}_{t+1} \circ \mathcal{F}(X_t, U_t)\{\tilde{x}_{t+1}\} \mid U_t = u_t, \tilde{X}_t = \tilde{x}_t\right], \\ &= \int d(x_t, x_{t+1}) \left[\tilde{\mathcal{X}}_t(x_{t+1})\{\tilde{x}_{t+1}\} \mathcal{F}(x_t, u_t)\{x_{t+1}\} \tilde{\mathcal{X}}_t^{-1}(\tilde{x})\{x\} \right], \end{aligned} \quad (3.20)$$

where $\tilde{\mathcal{X}}_t^\dagger$, $\tilde{\mathcal{X}}_t$ are the marginal distributions for $X_t, \tilde{X}_t$ at time t and the fraction is due to using Bayes' rule to invert the mechanism $\tilde{\mathcal{X}}_t$. Similarly, the measurement update in its unnormalized form is:

$$\mathcal{B}_t^{\text{meas}}(\hat{x}_t, y_t)\{\hat{x}'_t\} \propto \left[y_t \circ \tilde{\mathcal{X}}_t^{-1} \right] (\tilde{x})\{y_t\}. \quad (3.21)$$

In practice, normalization of the measurement update is difficult. Instead, either the maximum likelihood estimate is used, which is given by the maximizer of the right-hand side, or $\mathcal{B}_t^{\text{meas}}(\hat{x}_t, y_t)$ can be sampled from using inference algorithms like importance sampling [67, 68] that can sample from unnormalized distributions.

3.6 Task-Relevance and the Choice of Metric

In the output feedback interpretation of the present framework described in Section 3.1.1, where X_t is interpreted as the current (typically high-dimensional) sensor output, it is important to choose an appropriate metric ρ to acquire a tight bounds in Theorem 1. As an example, return to the ball-catching example, where a robot is tasked with catching a ball using visual feedback from the camera. In this case, X_t is very high-dimensional due to the resolution of modern camera sensors.⁵ The dynamics of the image are the evolution of the cameras pixels according to the motion of the ball and controlled motion of the robot. The gaze heuristic demonstrates that only a single pixel corresponding to the center of the

⁵Even at the relatively low resolution 128×128 , the output space dimension is $p = 49152$ for RGB images and $m = 16384$ for grayscale.

ball, is relevant to performing the task of catching the ball.

In this example, the offline setting, X_t corresponds to images from the location(s) used for synthesizing the controller. In a learning approach, the images $X_{0:t_f}$ might be random variables distributed according to data collected from the robot trying to catch the ball in the outfield of a local ballpark (e.g., Dodger Stadium). The online setting then corresponds to the robot tasked with catching the same ball in a new ball park (e.g., Yankee Stadium), where the controller receives a sequence of images from $\tilde{X}_{0:t_f}$ from a different distribution. The two sequences can contain the same task-relevant information, specifically, where the ball is in the frame, but will differ significantly in irrelevant ways due to the background of the image being a different stadium. For the same realization of the underlying dynamics, X_t and $\hat{X}_t$ will appear very different measured by the Euclidean metric when they are “close” in a task-relevant sense.

This issue is compounded by the high-dimensionality of the camera sensor. Formally, consider $\hat{X}_t$ as a perturbation of X_t satisfying,

$$\rho(X_t, \hat{X}_t) \leq \delta, \quad \delta \in \mathbf{R}_+, \quad (3.22)$$

where δ is typically small. If $\rho(x, \hat{x})$ is the Euclidean metric, the volume of a ball with radius δ decreases at higher dimensions.⁶ The result is that, in changing between the offline and online setting, Theorem 1 provides very loose bounds even though the settings do not differ significantly in a task-relevant manner.

Since the definition of DP in Definition 1 does not stipulate the Euclidean metric be used, and the choice of a task-relevant metric will yield better results in high dimensional settings. In this case, a task-relevant metric would discard information not pertinent to the trajectory of the ball, like the image background. Algorithmically determining such a metric is left to future work. However, to conclude this discussion, demonstration of the existence of a task-relevant metric under some conditions is described.

Assume that there indeed exists a set of TRVs whose information is embedded in the each sensor output. Consider a mechanism, $\mathfrak{X}_t : \tilde{\mathbf{X}} \rightarrow \Delta(\mathbf{X})$ that generates a sensor

⁶Specifically, the volume of the n -ball centered at $x \in \mathbf{R}^n$ with respect to the induced Euclidean metric, $\rho(x, x') = \|x - x'\|_2$, i.e., $\mathbf{B}_\delta(x) = \{x' \mid \rho(x, x') \leq \delta\}$, is $\pi^{\frac{n}{2}}/\Gamma(n/2+1)$ where $\Gamma(\cdot)$ is Euler’s gamma function.

output for which the TRVs are specifically identifiable, i.e., denoting the support of the mechanism this mechanism for a given $\tilde{x}$ by $\text{supp } \mathcal{X}_t(\tilde{x}) := \{x \mid \mathcal{X}_t(\tilde{x}) > 0\}$, the collection $\{\text{supp } \mathcal{X}_t(\tilde{x})\}_{\tilde{x} \in \tilde{\mathbf{X}}}$ is a partition a partition of $\tilde{\mathbf{X}}$. Then, a one-sided inverse can be defined $\mathcal{X}^{-1} : \mathbf{X} \rightarrow \tilde{\mathbf{X}}$,

$$\mathcal{X}_t^{-1}(x) = \tilde{x} \iff x \in \text{supp } \mathcal{X}_t(\tilde{x}). \quad (3.23)$$

If $(\tilde{\rho}, \tilde{\mathbf{X}})$ is a metric space, a task-relevant metric on $\mathbf{X}$ is defined using the pushforward of $\mathcal{X}_t(\tilde{x})$:

$$\rho_t^{\text{trv}}(x, \hat{x}) := \tilde{\rho}(\mathcal{X}_t^{-1}(x), \mathcal{X}_t^{-1}(\hat{x})). \quad (3.24)$$

This metric corresponds to the distance between only the task-relevant information contained in the sensor observations and discards all task-irrelevant information. For the ball-catching example, the distance between two images may only be the distance between the ball location in the image plane and perturbations to the background are discarded. Following this reasoning, it is expected that $\rho_t^{\text{trv}}(x, \hat{x})$ will typically be smaller than the Euclidean distance between x and $\hat{x}$.

This metric may be difficult to use in practice due to the strong partitioning assumption, but it serves as an example of how a task-relevant choice of metric for DP can ameliorate the difficulties of working with high-dimensional sensors and lead to a stronger performance guarantee applying Theorem 1.

Chapter 4

Algorithms for Implementing Bounded-Rationality Control

This chapter describes algorithms implementing the BR controller solving (BR) and (TRV-BR). Often, exact solutions are difficult to compute numerically due to nonlinear dynamics and costs preventing closed-form representations of many of the distributions involved. Even for (TRV-BR), linear dynamics, quadratic costs, and Gaussian noise are not sufficient to guarantee exact solutions due to the bilinearity of the objective function. Therefore, sufficiently accurate approximate solutions are sought that maintain the Gibbs measure form described in (3.7) and (3.17). As long as the controller is implemented as sampling from this measure, the sufficient condition for robustness described in Theorem 1 holds.

Many algorithms already used in robotics implement sampling from a Gibbs measure controller, with the motivation of approximately solving nonlinear optimal control problems in a numerically efficient manner. This chapter focuses on algorithms, novel or existing, that are used in Chapter 5. Other extant algorithms that fit into the framework described in Chapter 3 are summarized in Section 2.2.

4.1 Local Solutions from Linear-Quadratic Approximation

The first set of algorithms is the most straight-forward to describe numerically and a familiar numerical method in control theory. Non-linear dynamics and cost functions are locally

approximated about a reference trajectory as a linear system with Gaussian noise and a quadratic cost function. A local solution (or, in some cases discussed, approximately locally optimal) is computed for these dynamics. The reference trajectory is then updated using the nonlinear model with the identified input sequence to find a new reference trajectory, and the process is repeated until convergence. This iterative process is a common way to solve nonlinear optimal model-predictive control (MPC) problems, with the most common algorithm in the family being iLQR [69]. This algorithm, as well as an algorithm for estimating TRVs online, were previously published in research done during the course of research conducted by the author during the course of the Ph.D. program [8], although it builds on existing solutions to multi-step rate distortion problems [33].

4.1.1 Iterative Local Dynamic Programming Algorithm

The specific dynamics for which this algorithm are given by a nonlinear model with additive Gaussian noise:

$$\mathcal{F}(x, u) := f_t(x, u) + D_t, \quad D_t \sim \mathcal{N}(0, \Sigma^D). \quad (4.1)$$

The nonlinear term $f_t(x, u)$ is assumed to be differentiable. The cost function at each time step is assumed to be quadratic:

$$c_t(x, u) = \frac{1}{2}x^\top Q_t x + \frac{1}{2}u^\top R_t u \quad (4.2)$$

It is assumed that $Q_{0:t_f}, R_{0:t_f} \in \mathbf{S}_{++}$. Under these assumptions, the dynamics are locally described about a reference state-input trajectory $(x_{0:t_f}^{\text{ref}}, u_{0:t_f-1}^{\text{ref}})$ by studying its effect on small perturbations (denoted by a δ superscript) of the reference via linearization. The perturbed system's dynamics, $X_{t+1}^\delta \sim \mathcal{F}_t^\delta(X_t^\delta, U_t^\delta)$, and the cost function are then expressed,

$$\begin{aligned} X_t^\delta &:= X_t - x_t^{\text{ref}} & U_t^\delta &:= U_t - u_t^{\text{ref}}, \\ \mathcal{F}_{t+1}^\delta(x^\delta, u^\delta) &:= A_t x^\delta + B_t u^\delta + D_t, & c_t^\delta(x^\delta, u^\delta) &:= c_t(x^\delta + x_t^{\text{ref}}, u^\delta + u_t^{\text{ref}}), \end{aligned} \quad (4.3)$$

where $A_t := \partial_{x_t} f_t(x_t^{\text{ref}}, u_t^{\text{ref}})$, $B_t := \partial_{u_t} f_t(x_t^{\text{ref}}, u_t^{\text{ref}})$. Finally, the open-loop TRV mechanism $\mathcal{X}_t^\perp$ used in the relative constraint of (BR) is assumed to be Gaussian is assumed to be Gaussian: $\mathcal{X}_t^\perp = \mathcal{N}(x_t^\perp, \Sigma_t^\perp)$.

Remark 6. Note that the formulation here is a simplification of Section 3.5 in that the TRV mechanism only depends on the present state. This simplification is primarily to keep in accordance with the algorithm's original presentation [8] and limit the quantity of notation. It can be modified to the more general recurrent TRV mechanism described in Section 3.5 by adding a linear mapping from the previous TRVs to the present TRVs. Moreover, $c_t(x, u)$ can be any locally convex twice-differentiable function and locally approximated by a quadratic function, but slight generalization is not done explicitly for similar reasons.

Under these structural assumptions, local solutions about the reference trajectory can be computed in a dynamic programming algorithm if $\mathcal{X}^\perp$ is a fixed. Moreover, for solving (MI-BR), a forward iteration is added to the forward step to compute the uncertainty of the TRVs at each time step. The derivation of this algorithm relies on the fact that Gaussian random variables remain Gaussian under affine transformations, allowing all pertinent quantities in (3.17) can be computed in closed form.

Proposition 5 ([8]). For the following variant of (TRV-BR) using the dynamics linearized about the trajectory $(x_{0:t_f}^{\text{ref}}, u_{0:t_f-1}^{\text{ref}})$,

$$\begin{aligned}
& \underset{\tilde{\mathcal{X}}_t[\cdot], \mathcal{U}_t[\cdot]}{\text{minimize}} && \mathbb{E} \left[\sum_{t=0}^{t_f-1} c_t^\delta(X_t^\delta, U_t^\delta) + c_{t_f}(X_{t_f}^\delta) \right] \\
& \text{subject to} && X_{t+1}^\delta \sim \mathcal{F}[X_t^\delta, U_t^\delta] && \forall t = 0, \dots, t_f, \\
& && \tilde{X}_t^\delta \sim \tilde{\mathcal{X}}_t[X_t^\delta], && \mathbb{D}[\tilde{\mathcal{X}}_t^\delta(X_t^\delta); \tilde{\mathcal{X}}_t^\perp] \leq d_{\max}, \\
& && U_t^\delta \sim \mathcal{U}_t[\tilde{X}_t^\delta].
\end{aligned} \tag{4.4}$$

there exists $\beta \in \mathbf{R}_+$ such that optimal mechanisms satisfy,

$$\begin{aligned}
\tilde{\mathcal{X}}_t(x) &= C_t \tilde{x} + G_t, && G_t \sim \mathcal{N}(g_t, \Sigma_t^G), \\
\mathcal{U}_t(\tilde{x}) &= K_t \tilde{x} + h_t,
\end{aligned} \tag{4.5}$$

where the terms of these mechanisms satisfy the backward equations,

$$\begin{aligned}
M_t &:= A_t + B_t K_t C_t, \\
C_t &= -\beta \Sigma_t^G K_t^T B_t^T P_{t+1} A_t, \\
\Sigma_t^{G^{-1}} &= \Sigma_t^{\perp^{-1}} + \beta K_t^T (B_t^T P_{t+1} B_t + R_t) K_t, \\
g_t &= -\beta \Sigma_t^G (K_t^T B_t^T (b_{t+1} + P_{t+1} B_t h_t) + K_t^T R_t h_t) \\
&\quad - \Sigma_t^G \Sigma_t^{\perp} x_t^{\perp}, \\
P_t &= Q_t + \beta^{-1} G_t + C_t^T K_t^T R_t K_t C_t + M_t^T P_{t+1} M_t, \\
b_t &= M_t^T P_{t+1} B_t (h_t + K_t g_t) - \beta^{-1} G_t x_t^{\delta} + C_t^T K_t^T R_t (K_t g_t + h_t) + M_t^T b_{t+1},
\end{aligned} \tag{4.6}$$

with $P_{t_f} = Q_{t_f}$, $b_{t_f} = 0$, as well as the forward equations:

$$\begin{aligned}
N_t &:= C_t^T (C_t \Sigma_t^X C_t^T + \Sigma_t^G)^{-1} C_t, \\
\tilde{x}_t^{\delta} &= C_t \tilde{x}_t^{\delta} + g_t, & \Sigma_{t+1}^{\tilde{X}} &= C_t \Sigma_t^X C_t^T + \Sigma_t^G, \\
u_t^{\delta} &= K_t \tilde{x}_t^{\delta} + h_t, & \Sigma_{t+1}^U &= K_t \Sigma_t^X K_t^T, \\
x_{t+1}^{\delta} &= A x_t^{\delta} + B u_t^{\delta}, & \Sigma_{t+1}^{\tilde{X}} &= M_t \Sigma_t^X M_t^T + B_t K_t \Sigma_t^G K_t^T B_t^T + \Sigma_t^D.
\end{aligned} \tag{4.7}$$

Finally, if the constraint $\mathbb{D}[\tilde{\mathcal{X}}_t^{\delta}(X_t^{\delta}); \tilde{\mathcal{X}}_t^{\perp}] \leq d_{\max}$ is replaced with $\mathbb{I}[\tilde{X}_t; X_t] \leq d_{\max}$, then two additional equations must hold:

$$\tilde{x}_t^{\perp} = \tilde{x}_t, \quad \Sigma_t^{\perp} = \Sigma_t^{\perp}. \tag{4.8}$$

Proof. See [8], reproduced in Appendix B, for the mutual information case. The relative entropy case without TRVs is derived in [7], reproduced in Appendix D. With TRVs, derivation for the TRV framework is the same where, for a fixed $\mathcal{U}_t(\tilde{x}_t)$ of the form (4.5), the quantity $\tilde{x}_t$ can be interpreted as the control input to an affine-Gaussian system created by composing $\mathcal{U}_t(\tilde{x}_t)$ with the linear-Gaussian dynamics. $\square$

The forward equations in this proposition dictate that, at each time step, the random variables for the state, TRVs, and input are all Gaussian: $X_t \sim \mathcal{N}(x_t, \Sigma_t^X)$, $\tilde{X}_t \sim$

Algorithm 1 Bounded-Rationality Control via Iterative Linearization

Require: Initial reference trajectory $(x_{0:t_f}^{\text{ref}}, u_{0:t_f-1}^{\text{ref}})$
Require: Initial open-loop distribution $\tilde{x}_{0:t_f-1}^\perp, \Sigma_{0:t_f-1}^\perp$.
Require: Initial Guess $C_{0:t_f-1}, g_{0:t_f-1}, \Sigma_{0:t_f-1}^G, K_{0:t_f-1}, h_{0:t_f-1}$.

- 1: **repeat**
- 2: $A_t, B_t \leftarrow \partial_x f(x_t^{\text{ref}}, u_t^{\text{ref}}), \partial_u f(x_t^\delta, u_t^\delta)$ for $t = 0, \dots, t_f - 1$
- 3: **repeat**
- 4: $K_t, h_t \leftarrow \text{Solve Quadratic Program for } t = 0, \dots, t_f - 1.$
- 5: $x_t^\delta, \tilde{x}_t^\delta, u_t^\delta, \Sigma_t^X, \Sigma_t^{\tilde{X}}, \Sigma_t^U \leftarrow \text{Solve (4.7) for } t = 0, \dots, t_f - 1$
- 6: $M_t, C_t, \Sigma_t^G, g_t, P_t, b_t \leftarrow \text{Solve (4.6) for } t = 0, \dots, t_f - 1.$
- 7: $\tilde{x}_t^\perp, \Sigma_t^\perp \leftarrow \tilde{x}_t, \Sigma_t^{\tilde{X}}$ for $t = 0, \dots, t_f - 1.$ ▷ Optional
- 8: **until** Convergence
- 9: $x_t^{\text{ref}}, u_t^{\text{ref}} \leftarrow x_t^{\text{ref}} + x_t^\delta, u_t^{\text{ref}} + u_t^\delta$ for $t = 0, \dots, t_f - 1.$
- 10: **until** Convergence

$\mathcal{N}(\tilde{x}_t, \Sigma_t^{\tilde{X}}), X_t \sim \mathcal{N}(u_t, \Sigma_t^U)$. Moreover, it is worth noting that the value function for this optimal control problem is a quadratic function with P_t and b_t defining the quadratic and linear terms at each time step.

The result described in Proposition 5 suggests a method to locally solve (TRV-BR) with nonlinear dynamics iteratively. This method is described in Algorithm 1. The algorithm alternates between enforcing the forward and backward equations in Proposition 5 for the majority of decision variables. With these variables fixed, K_t, h_t are able to be found as the solutions to an unconstrained quadratic program (whose solution is computed in any usual manner but is omitted due to the size of the resulting equations). Once this process has converged, the reference trajectory is updated using the means of the perturbed trajectory variables and this is repeated once again. In practice, it is often the case that convergence is not exact, but that the decision variables rapidly collapse to a stable limit cycle, so the best set of variables as measured by the objective is used after a fix number of iterations or when improvements have not been found after many iterations steps.

Moreover, in this setting, the Bayesian filter described in (3.20) and (3.21) can be computed in closed-form using the locally linearized dynamics.

Proposition 6 ([8]). The Bayesian filter $(\mathcal{B}_t^{\text{proc}}, \mathcal{B}_t^{\text{meas}})$ described in Section 3.5.2 is equiva-

lent to the Kalman filter for a linear-Gaussian system describing the evolution of the TRVs:

$$\tilde{X}_{t+1} = \tilde{A}_t \tilde{X}_t + \tilde{B}_t U_t + \tilde{D}_t, \quad \tilde{Y}_t = \tilde{F}_t \tilde{X}_t + \tilde{S}_t, \quad (4.9)$$

where $\tilde{D}_t \sim \mathcal{N}(\tilde{d}_t, \Sigma_t^{\tilde{D}})$. The complete equations for the filter updates of this system are given in Appendix B.

Proof. See [8] reproduced in Appendix B. □

4.2 Reinforcement Learning of Task-Relevant Controllers

Local solutions to (TRV-BR) are simple and efficient to compute, but only admit limited performance on tasks asked of modern robots. Modern high-dimensional sensors, like cameras, are difficult to characterize analytically due to their high-dimensionality, dependence on their ambient environment (e.g., lighting conditions and background scene), and interaction with material properties (e.g., specularity of surfaces observed), and local analyses based around linearization are rarely effective. Nevertheless, deep learning approaches to controller synthesis demonstrate that effective mappings from the output of such sensors to control inputs can achieve powerful results. This section details a novel algorithm for solving (TRV-BR) using *reinforcement learning* (RL) to synthesize TRV-based controllers suitable for robots equipped with sensors or other high-dimensional sensors.

The principle challenge that needs to be overcome is that, unlike in the linear-Gaussian case described in Section 4.1, nonlinear dynamics, controllers, and sensors make computing and optimizing the state information quantified by $\mathbb{I}[X_t; \tilde{X}_t]$. Therefore, instead, a recently proposed technique to learn a differentiable estimate of this quantity, known as the *Mutual Information Neural Estimator* (MINE) [70], use this estimator to solve (TRV-BR).

The MINE is based on a variational representation of the relative entropy due to Donsker and Varadhan described briefly in Chapter 3. The MINE uses a duel formulation where the decision variable is the statistic whose expectation is computed, not the measure used to compute that expectation. Stated in the following lemma is a duel formulation for general random variables.

Lemma 2 (Donsker-Varadhan [41]). Let $Z \sim \mathcal{Z}$ and $Z' \sim \mathcal{Z}'$ be two random variables taking values in $\mathbf{Z}$ satisfying the absolute continuity condition $\mathcal{Z} \ll \mathcal{Z}'$. Then, the relative entropy between these two is given by,

$$\mathbb{D}[Z; Z'] = \sup_{f: \mathbf{Z} \rightarrow \mathbf{R}} \mathbb{E}[f(Z)] - \log \mathbb{E}[\exp(f(Z'))], \quad (4.10)$$

where the supremum is taken over all functions for which the expectations are finite.

The MINE estimator replaces optimization over functions with the optimization over neural network parameters. Let $f: \mathbf{Z} \times \Theta \rightarrow \mathbf{R}$ is a neural network with parameter space Θ , and let $\mathbb{E}^n[\cdot]$ denote the empirical expectation computed with n i.i.d. samples. The MINE estimator $\hat{\mathbb{D}}^n[Z; Z']$ then serves as an approximation of $\mathbb{D}[Z; Z']$:

$$\hat{\mathbb{D}}^n[Z; Z'] := \sup_{\theta \in \Theta} \mathbb{E}^n[f(Z; \theta)] - \log \mathbb{E}^n[\exp(f(Z'; \theta))]. \quad (4.11)$$

This estimator converges to $\mathbb{D}[Z; Z']$ in probability and has sample complexity bounds available to ascertain the quality of approximation [70].

Solving (TRV-BR) using the MINE is then fairly straight-forward via policy gradient methods. This method is an output-feedback approach where the TRV mechanism receives, along with the previous time step's TRVs, the current sensor output directly instead of a state estimate. The TRV $\tilde{\mathcal{X}}_t^\beta: \mathbf{Y} \times \tilde{\mathbf{X}} \times \Phi \rightarrow \Delta(\tilde{\mathbf{X}})$ and control mechanism $\mathcal{U}_t: \tilde{\mathbf{X}} \times \Psi \rightarrow \Delta(\mathbf{U})$ are represented by neural networks with parameters $\phi \in \Phi$ and $\psi \in \Psi$ respectively. After each policy gradient step, taken to optimize $\phi \in \Phi, \psi \in \Psi$, the parameters of the MINE are optimized to convergence. Once its parameters are fixed, the MINE can be backpropogated through along with the cost function to optimize the TRV and controller mechanism parameters using policy gradient style algorithms.

The algorithm used in the examples included in Chapter 5 is described in Algorithm 2 and referred to as *task-relevant policy gradient* (TDPG). Further implementation details regarding this algorithm are present in [9], which is reproduced in Appendix C.

Algorithm 2 Task-Driven Policy Gradient [9]

Require: Initial guess θ, ϕ, ψ .

- 1: **repeat**
 - 2: Roll out N trajectories: $\{x_t^i, \tilde{x}_t^i, u_t^i, y_t^i\}_{n=1}^N$.
 - 3: **repeat**
 - 4: $\theta_t \leftarrow \text{MINE-GRAD-STEP}(\theta_t, \{(y_t^i, \tilde{x}_t^i)\}_{n=1}^N)$.
 - 5: **until** Convergence for all $t = 0, \dots, t_f$.
 - 6: $\phi_t, \psi_t \leftarrow \text{POLICY-GRAD-STEP}(\theta_t^s, \{(\tilde{x}_t^i)\}_{n=1}^N)$.
 - 7: **until** Convergence for all $t = 0, \dots, t_f$.
-

Algorithm 3 Stein-Variational Gradient Descent [55]

Require: Initial samples $\{x_i\}_{i=1}^n \subset \mathbf{X}$, initial parameters $\theta^0 \in \Theta$, step size $\epsilon \in \mathbf{R}_{++}$.

- 1: **repeat**
 - 2: $\theta \leftarrow \text{UPDATEPARAMS}(x_{1:n}, \theta)$
 - 3: **for** $i = 1, \dots, n$ **do**
 - 4: $d_i \leftarrow \frac{1}{n} \sum_{j=1}^n (k(x_j, x_i; \theta) \partial_x [\log \mathcal{T}]\{x_j\} + \partial_x k(x_j, x_i; \theta))$ ▷ Compute gradient.
 - 5: **end for**
 - 6: $x_i \leftarrow x_i + \epsilon d_i$ for $i = 1, \dots, n$.
 - 7: **until** Convergence.
-

4.3 Stein-Variation Gradient Descent

The final algorithm described in this chapter is recent variational inference algorithm, not due to the author, known as *Stein-Variational Gradient Descent* (SVGD) [55]. Variational inference algorithms attempt to approximate sampling from a *target distribution*, which cannot be directly sampled from directly, finding the closest distribution to the target in a family of tractable distributions as measured by the relative entropy. SVGD is a non-parametric, kernel-based approach to variational inference where randomly initialized particles are evolved via deterministic gradient descent regularized by a kernel.¹ In this dissertation, SVGD is used to sample from the Gibbs measure (3.7) that the optimal BR controller must satisfy. Separately, SVGD has been used for MPC from the standpoint that optimal control can be framed as an inference problem by introducing a dummy variable indicating whether a sequence of inputs is optimal and sampling from the distribution conditioned on this indicator [52] The result is the same Gibbs measure described in (3.7), however the motivation for the inference-as-control algorithms traditionally is numerical efficiency, not robustness.

¹Recall, a kernel is a symmetric function describing how similar two inputs are to each other.

The SVGD algorithm is succinctly stated. Let $\mathcal{T} \in \Delta(\mathbf{X})$ be the target distribution with density $\mathcal{T}\{x\}$ supported on some set $\mathbf{X}$ from which samples are desired. Let $k(x, x'; \theta)$ be a symmetric, positive-definite kernel on $\mathbf{X}$ with parameters $\theta \in \Theta$. Then, gradient ascent on the log-probability of $\mathcal{T}$ is performed for each sample with the addition of a repulsive force generated by the gradients of the kernel evaluated pairwise between samples. This algorithm is shown to converge to sampling from $\mathcal{T}$ in the infinite-sample limit. Moreover, the kernel parameters are permitted to be updated at the start of each algorithm iteration. Doing so has been shown to improve convergence by, e.g., modifying them based on local curvature of the target density and median distance between samples [55, 71].

It is worth noting that, in the case of sampling from the Gibbs measure in (3.7), the samples are done over sequences of control inputs. Moreover, as $\beta \rightarrow \infty$, MPC with multiple random initializations is recovered where as in the limit $\beta \rightarrow 0$, SVGD simply samples from the open-loop policy. In practice, SVGD is a more practical method than other gradient-based sampling methods like Hamiltonian Monte Carlo or Stochastic Gradient Langevin Dynamics, since its deterministic updates may be done in parallel, lead to rapid convergence, and convergence is easily identified, unlike other stochastic methods.

Chapter 5

Empirical Studies

This chapter summarizes empirical studies of the robustness bound derived in Chapter 3. These studies include both simulation and experiments with hardware robots. All examples feature implementations of the algorithms in Chapter 4. For experiments also present in papers reproduced in the appendix, only key implementation details are described in this chapter while complete discussion is deferred to the appropriate appendix. The experiments described in this chapter are the most prominent ones demonstrating the efficacy of the BR approach to robust control, however additional experiments are present in the appendices.

Remark 7. Experiments using TDPG (Algorithm 2) was done prior to the development of the theory in Chapter 3. In these experiments, the information-theoretic quantity optimized using the MINE is not $\mathbb{I}[\tilde{X}_t; Y_t]$ but $\mathbb{I}[\tilde{X}_t; X_t]$. The TRV mechanism is, as in the description of Algorithm 2, a function of the current sensor output Y_t and previous TRVs, $\tilde{X}_{t-1}$. This formulation of BR is not directly consistent with Chapter 3. The robustness guarantees of Theorem 1 are conferred if implicitly the learned TRV mechanism operates by attempting to estimate the state X_t from the output Y_t and then compute the TRVs $\hat{X}_t$ from there. This structure is not enforced explicitly in the network parameterization, but the included results demonstrate that this altered formulation of bounded rationality leads to robustness.

5.1 Spring-Loaded Inverted Pendulum

This experiment, previously published by the author [8], demonstrates the efficacy of the local solution to (TRV-BR) described in Algorithm 1 on a simple nonlinear walking model, known as the *Spring-Loaded Inverted Pendulum* (SLIP) model [72–74]. This model is a staple of robotics literature as a simple nonlinear model of legged locomotion, and was chosen as an experimental model here due to inspiration from cognitive science experiments identifying human heuristics foot placement while running [30, 75]. While the complete model is a hybrid system with two modes (one in air and one in contact with the ground), the approach employed here is to treat it as a discrete-time system via the Poincaré defined at the moment the leg contacts the ground. The dynamics of the SLIP model are described in Figure 5.1.

The efficacy Algorithm 1 is demonstrated in Fig. 5.2. The TRV-based controller was selected by running Algorithm 1 for a range of values of β and taking the smallest value of β (corresponding to lowest state information contained in the TRVs) satisfying an offline cost constraint. Online, the sensor used returns the state of the robot corrupted by Gaussian noise generated by a randomly selected covariance matrix. This process is repeated for each choice of dimension for $\tilde{\mathbf{X}}$, and it was found that in all cases, $\text{rank } C_t = 1$, implying only a single TRV was needed for control. The performance of Algorithm 1 is compared a separation principle approach where an iLQG controller is used in conjunction with a Kalman filter. Both the Kalman filter with the correct estimation error covariance and a fixed nominal covariance were used, where the former acts as a baseline and only the latter was used to design the estimator described in Proposition 6 for the TRV-based controller. As anticipated, the TRV-based controller outperforms the iLQG controller when both use estimators designed with the wrong estimation error covariance.

5.2 When the Separation Principle Fails: The Lava Problem

This empirical study was done in the author’s prior work [9] and is summarized here. The lava problem is a simple example where the separation principle does not apply [76, 77]. Depicted visually in Figure 5.3, the robot is modeled as a double integrator on positive half

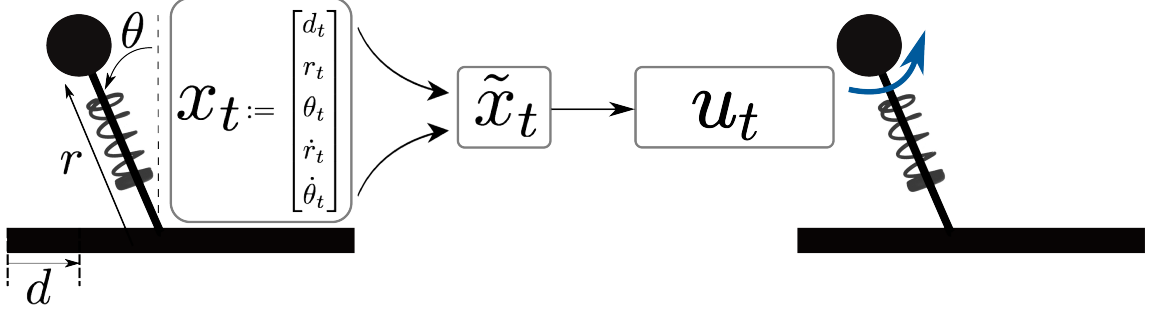


Figure 5.1: The SLIP model is common in robotics for prototyping legged locomotion controllers. It consists of a single leg whose lateral motion is derived from a spring/piston combination. At touchdown, the state of the robot is given by $x = (d, \theta, \dot{r}, \dot{\theta})$ where d is displacement of the head from the origin, θ is the touchdown angle, and $\dot{r}, \dot{\theta}$ are the radial and angular velocities. The system input is $u_t = \theta_{t+1} - \theta_t$, i.e., the change in the next touchdown angle. Despite the model’s simplicity, the touchdown return map eludes a closed-form description, therefore the time-discretized dynamics are found by numerically computing the Poincaré map at touchdown.

of the real line. The position of the robot is given by $d \geq 0$ and the robot is tasked with reaching a goal position d_{goal} at the end of the scenario. The robot freely moves within the interval $[0, d_{\text{lava}}]$. At $d = 0$, there is a wall with which the robot collides with inelastically — that is, if the robot were to move into a position $d < 0$, its position and velocity are both set to zero. The set of states with $d \in (d_{\text{lava}}, \infty)$ are absorbing and represent the robot falling into a pit of lava, which yields a high cost in addition to preventing the robot from reaching its goal. The robot is initialized randomly within $[0, d_{\text{lava}}]$ uniformly with zero velocity.

This scenario is interesting because the asymmetry of the configuration space boundaries presents a good open-loop policy: if the robot is unsure of its position due to sensor noise, it can choose an open-loop control that is guaranteed to drive it into the wall regardless of its initial configuration, thereby localizing itself at $d = 0$, and then proceed to the goal without need for any sensor information. Meanwhile, if the robot employs the optimal offline policy that drives it directly to the goal online with a noisy sensor, it is liable to drive the robot into the lava instead of toward the goal.

This scenario is used to study the robustness benefits of Algorithm 2 versus an assumed separation approach using standard policy gradient (PG). Offline, sensor measurements are the robot’s current state corrupted by a small amount of isotropic Gaussian noise with vari-

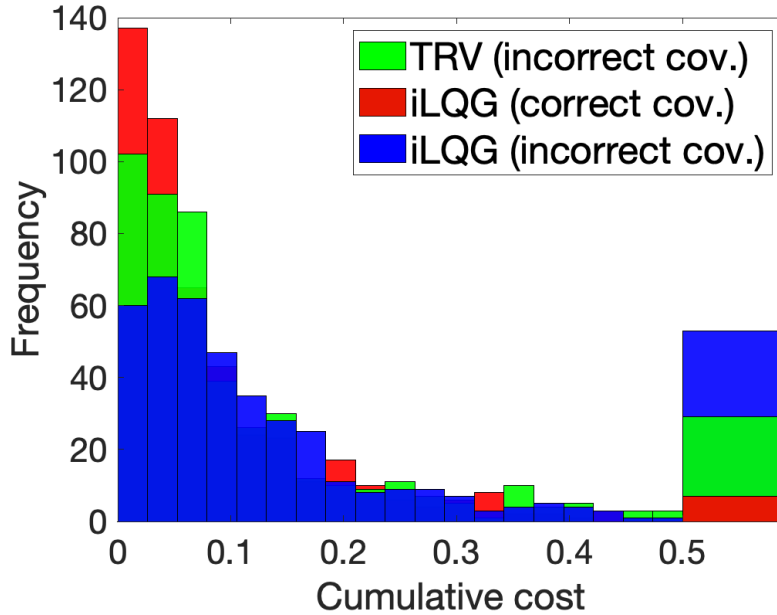


Figure 5.2: This plot summarizes SLIP Problem with different control strategies. Measurement covariance matrices were randomly sampled. For the iLQG control policy, a Kalman filter tracked the current state estimate, and the corresponding control was applied. For the TRV policy, a Kalman filter was maintained on the TRVs, and the control input corresponding to the MLE TRV was applied, and $\beta = 23.11$ was used. The right most histogram bar represents the frequency of all costs greater than 0.5.

ances $\sigma_{\text{off}}^2 = 0.0001$ representing an accurate state estimator. Online, the estimation noise variances, σ_{on}^2 , are varied and distribution of costs recorded. The results of this experiment conducted using are shown in Figure 5.4. Complete experimental details, including the cost function structure and neural network architectures used are included in Appendix C.

Due to the presence of low sensor noise in the training environment, the PG controller learns the optimal, fully-observable, strategy to drive the robot directly towards the goal state. Interestingly, the TDPG algorithm finds a *qualitatively different* approach. In particular, TDPG recovers the robust open-loop behavior described in [76, 77]: regardless of initial position, the robot moves left until it collides with the wall, then moves right to the goal state. As the sensor noise σ_{on}^2 is increased, the performance of the PG policy degrades rapidly. Largely, this is due to the case where the sensor reports the estimated position is $\hat{d} \leq d_{\text{goal}}$ when in truth it is on the opposite side, i.e., $d \geq d_{\text{goal}}$, and drives it into the lava. In contrast, the TDPG is unaffected by the increased sensor noise. Complete experimental

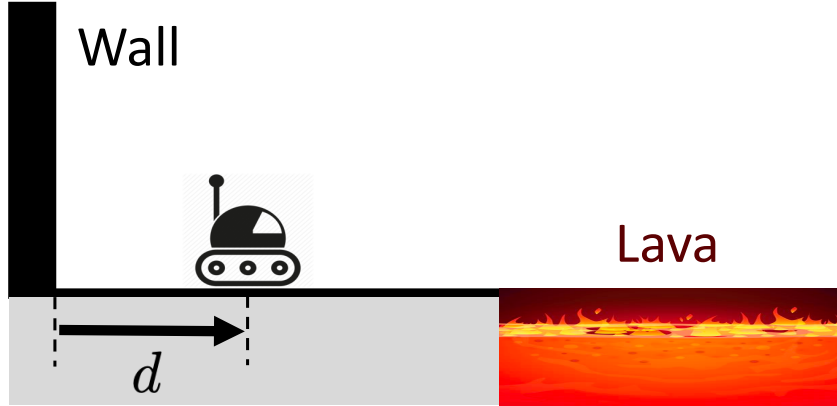


Figure 5.3: An illustration of the Lava Problem described in Section 5.2. The robot (a double integrator) needs to navigate to a target state without moving so far right that the robot falls into the lava.

details are available in the published description of this experiment [9], which is reproduced in Appendix C.

5.3 Vision-Based Ball Catching

This experiment is an instantiation of the ball-catching experiment inspired by the gaze heuristic described in Chapter 1. This experiment previously appeared in published work by the author [9], which is reproduced in Appendix C. The robot, modeled as a double integrator, is tasked with moving from a randomized starting location randomly distributed between on the interval $[-2, 2]$ to catch a ball (interpreted as a cost penalizing distance from where the landing location of the ball) using an RGB camera sensor. The state thus contains the positions and velocities of the ball and the robot and the sensor output is an RGB image. The ball is launched from a height $h = 1$ with a fixed velocity. Good performance requires more than a purely open-loop strategy due to the uninformative initial state distribution given the scale of the experiment.

Offline, both PG and TDPG are trained using a fixed background texture shown in Figure 5.6. Standard PG outperforms TDPG on this training setting. Online, the sensor outputs fed to the two learned controllers are altered in two different ways. In the first set of trials, the image the camera receives is corrupted by random Gaussian noise added to

each pixel value (pixel values are constrained to be valid RGB images after noise is added). This experiment is repeated for multiple scales of added noise. In the second set of trials, no noise is added, but the brick background texture is replaced with different backgrounds for evaluation, listed in Figure 5.6. Neural network architectures and training details for both policies are available in Appendix C. For TDPG, process for determining the value of β used is similar as in Section 5.2, where both were varied and the one smallest. The number of dimensions of TRVs was fixed at 8.

The results of these experiments are presented in Table 5.1. both sets of studies, PG outperforms TDPG during training. However, PG is outperformed by TDPG in the majority of online scenarios. In the first set of experiments, PG rapidly begins to fail even at the lowest level of added noise, and, even though it marginally outperforms TDPG, its average cost roughly doubles while TDPG’s average cost only increases slightly. In the second set of trials, the only background where PG outperforms TDPG is on a red brick background which is somewhat qualitatively similar to the one used during training.

Scenario	Policy Gradient		Task-Driven PG	
	Cost	Dist. (m)	Cost	Dist. (m)
Training	8.702	0.085	18.577	0.184
Sensor Noise $\sigma = 0.10$	19.12	0.189	19.48	0.193
Sensor Noise $\sigma = 0.15$	46.68	0.464	20.12	0.199
Sensor Noise $\sigma = 0.20$	124.63	1.244	21.59	0.214
Sensor Noise $\sigma = 0.25$	187.53	1.874	27.11	0.274
Test Background 1	182.1	1.828	122.7	1.225
Test Background 2	214.3	2.141	79.90	0.797
Test Background 3	138.7	1.385	26.62	0.264
Test Background 4	95.38	0.952	28.11	0.279
Test Background 5	82.01	0.818	36.95	0.367
Test Background 6	208.5	2.083	166.4	0.166
Test Background 7	9.180	0.090	14.92	0.147

Table 5.1: Mean cost and final distance to the ball of PG and TDPG policies in training and testing ball-catching experiments from 1000 trials.

5.4 Quadrotor Stabilization

These examples focus on the stabilization of a planar quadrotor (constrained to the Y-Z plane and rotating about the X axis). They first appeared in the author’s past work, [7],

reproduced in Appendix D. The TRV formalism is not used, so the controller depends directly on the state offline and the state with additive Gaussian measurement noise online. The system state $x \in \mathbf{R}^6$ consists of the position and rotation of the robot and the time derivatives of these variables, i.e. $x = (y, z, \theta, \dot{y}, \dot{z}, \dot{\theta})$, while the control input $u \in \mathbf{R}^2$ is the thrust exerted by two opposing pairs of rotors (see [78, Section 6.6] for dynamics). The cost functions are the linear-quadratic regulator (LQR) cost functions [1].

The system parameters are chosen to align with the Crazyflie 2.0 quadrotors [79]. The system is temporally discretized and the initial condition is sampled from small Gaussian perturbations about the hover state. The open-loop controller $\mathcal{U}^\perp$ is chosen by projecting the initial distribution through the LQR solution to find the marginal input. The estimation error is drawn from a stationary Gaussian distribution. Specifically, $\hat{\mathcal{X}}(x) = \mathcal{N}(x, \sigma_x^2 \Sigma)$, where $\sigma_x^2 \in \mathbf{R}_+$ is a scale parameter and $\Sigma = \text{Diag}(0.25, 0.25, 0.1, 0.25, 0.25, 0.1)$.

5.4.1 Linearized Results

The system is linearized and the dynamic programming equations specifying the control policy are solved exactly for a feedback policy that is linear in the state with additive Gaussian noise. Details are included in Appendix D. The performance of the bounded-rationality controller is evaluated and compared with a controller that is optimal assuming perfect state estimation (LQR) paired with a sub-optimal state estimator. As shown in Fig. 5.7, the bounded-rationality controller is more robust to estimation error than the LQR controller. The optimal information usage β^* decreases with the σ_x^2 .

5.4.2 Nonlinear Results

The experiments were then repeated using the nonlinear planar quadrotor dynamics. There is no closed-form solution to (BR), but numerical methods for sampling from $\mathcal{U}_t^\beta(x)$ are available (see Section 2.2.3). An MPC controller implemented using SVGD [39, 52], as described in Algorithm 3 is used. The results are similar to the linear case: there is a prominent local minimum for β that outperforms MPC (which is optimal assuming perfect estimation) in the presence of estimation error when averaged over 100 trials. In this case, the monotonic relationship between β^* and σ_x^2 is not seen — possibly due to SVGD

only approximating the controller’s distribution. Theoretical investigation of whether this relationship is truly monotonic is reserved for future work.

5.5 Motion Planning in Cluttered Spaces

This example is a qualitative study of a bounded-rationality motion planning problem for a robot in a cluttered workspace. This is a refinement of an experiment present in Appendix D [7]. The robot’s workspace is depicted in Figure 5.8. Planning occurs in a two-dimensional space defining the positional coordinates of the robot. Trajectories in this space can be tracked arbitrarily closely by robots which are suitably differentially flat [80, 81] which applies to the dynamics exhibited by many robots [82], including quadrotors [83]. The goal for this task is to determine a continuous trajectory $q : [0, t_f] \rightarrow \mathbf{X}$ that avoids obstacles with a high probability and satisfies $q(0) = x_0$, where x_0 is a deterministic initial condition, and $q(t_f) = x_g$, which is a goal position while taking the shortest path possible.

The SVGD algorithm (Algorithm 3) is used to solve this task. The main challenge involved is handling non-convexity of the workspace and incorporating uncertainty into the planner such that a high-information controller does not drive the robot arbitrarily close to the boundary of an obstacle. To formulate the problem in a manner that addresses these difficulties, candidate trajectories will be sampled by a Gaussian process with mean $q(t)$ and kernel $k : \mathbf{X} \times \mathbf{X} \times \Theta \rightarrow \mathbf{R}_+$ [84]. Denote the random trajectory and Gaussian process then by $Q(t) \sim \mathcal{G}(q, k; \theta)$ for kernel parameters $\theta \in \Theta$. A control input u is s coordinates that define the mean of $Q(t)$ at a fixed number of times $0 < t_1 < \dots < t_s < t_f$, i.e., $u = (q_1, \dots, q_s)$ specifies $q(t_i) = q_i$ for $i = 1, \dots, s$. Denote the predictive posterior distribution of the Gaussian process conditioned on,

$$q(0) = x_0, \quad q(t_1) = q_1, \quad \dots \quad q(t_s) = q_s, \quad q(t_f) = x_g, \quad (5.1)$$

using $k(q, q'; \theta)$ by the mechanism $\mathcal{G}^{\text{pr}}(u)$. The cost function to be minimized is chosen to

be:

$$c(x_0, u) = \mathbb{E} \left[w_l \sum_{i=1}^s \|Q(t_i) - Q(t_{i-1})\|_2^2 + w_o \mathbb{C}^{\text{ob}}[Q] \right] \quad \text{where} \quad Q \sim \mathcal{G}^{\text{ob}}(u). \quad (5.2)$$

Here, the first term is discrete approximation of the length of $Q(t)$ and $\mathbb{C}^{\text{ob}}$ is a function denoting a penalty for trajectories that intersect an obstacle. The coefficients $w_l = 1$ and $w_o = 100$ are weighting parameters. Spherical obstacles are treated so obstacle i is defined by a ball with centroid c_i and radius r_i , i.e., $\mathbf{B}_{r_i}(c_i) \subset \mathbf{X}$, and $\mathbb{C}^{\text{ob}}[q]$ is:

$$\mathbb{C}^{\text{ob}}[q] := \sum_{i=1}^6 \int_0^1 dt b_i(q(t)), \quad b_i(q) = \begin{cases} 1 + \exp\left(-\frac{\|c_i - q\|_2^2}{2}\right) & \|c_i - q\|_2 \leq r_i \\ 0 & \|c_i - q\|_2 > r_i \end{cases}. \quad (5.3)$$

Formulating the motion planning problem in this way allows a nominal level of uncertainty about the initial configuration and obstacle locations to be encoded via the Gaussian process kernel. It also allows for an efficient implementation of SVGD, where k is chosen to be a relatively small number (in Figure 5.8, $s = 7$), and in $\mathbb{C}^{\text{ob}}[q]$ can be computed via sampling T trajectories at $t'_{1:s'}$ times from $\mathcal{G}^{\text{pr}}(u)$, where $s' > s$. In this experiment values were $T = 10, s = 50$. Algorithm 3 was run for 10,000 which showed good convergence when parameters were varied. The large number of iterations justifies the small number of sample trajectories for each iteration of the algorithm since it is approximately the same as using a larger batch of samples to estimate a larger gradient step. The open-loop control $\mathcal{U}^\perp$ is a Gaussian distribution $\mathcal{N}(u^\perp, 2I)$, where $u^\perp$ is a linear interpolation between x_0 and x_g . Both sequences $t_{1:s}$ and $t_{1:s'}$ were evenly spaced between 0 and 1. The kernel chosen is the RBF kernel with bandwidth $1/4$ the ADAM optimizer with a learning rate of 0.01 is used [85] in this example, and Algorithm 3 is implemented with the median bandwidth heuristic [55]. Trajectories are sampled from $\mathcal{U}^\beta(x)$ by evaluating $\exp(-\beta c(x_0, u))$ using $T = 1000$ samples, sampling a SVGD particle u' proportionally to $\exp(-\beta c(x_0, u))$, and then sampling from $\mathcal{G}^{\text{pr}}(u')$.¹ Finally, while the example here is a single-step decision-making problem, this formulation is easily adapted to online implementation through warm-starting SVGD.

¹Note that the sampling of $Q(t)$ is DP via Proposition 3.

The results in Figure 5.8 demonstrate that varying β qualitatively changes the behavior of the motion planner that induces robustness. When the controller uses high enough levels of state information, it selects an approximately optimal path that navigates through the narrow gaps of the central cluster of obstacles. However, at lower values of β , which corresponds to less information use, the planner elects to take a more circuitous route around that avoids these gaps. The latter path is more robust to unmodeled uncertainty about the robot's position on workspace obstacles as a result.

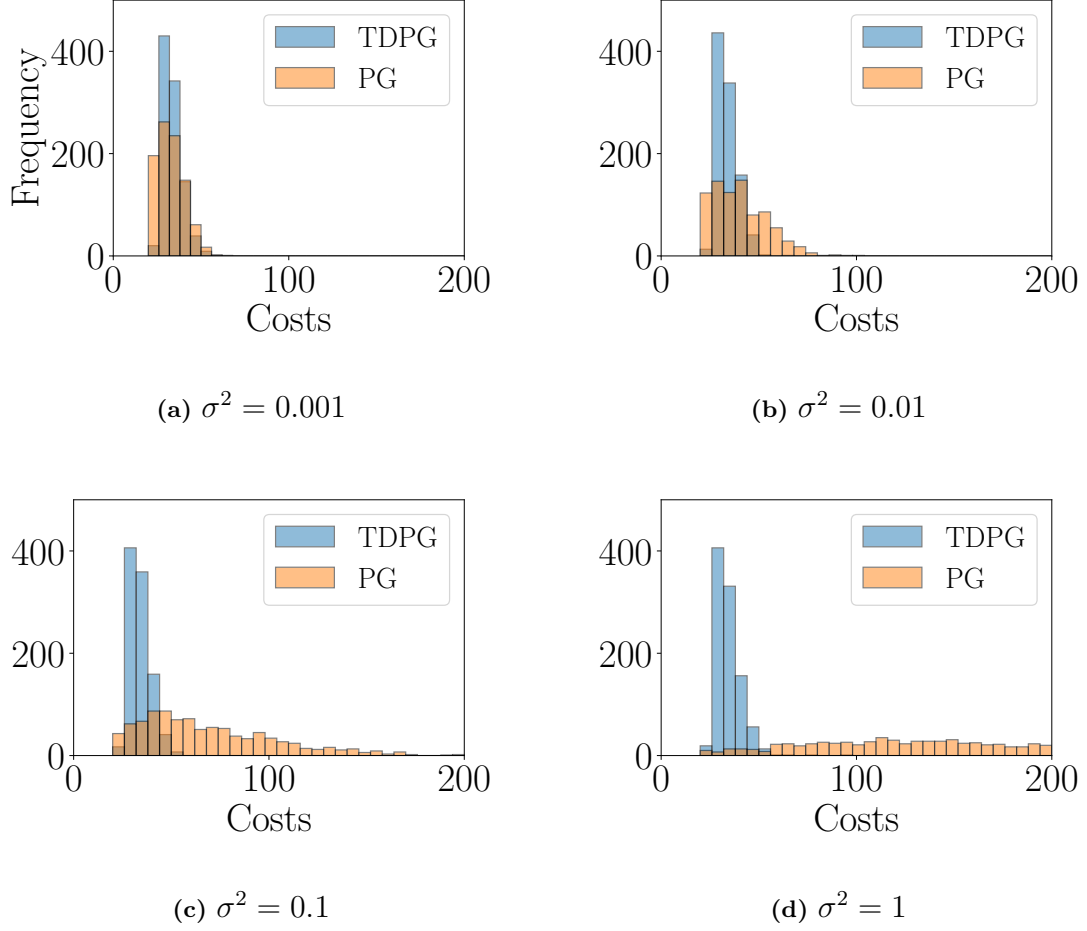
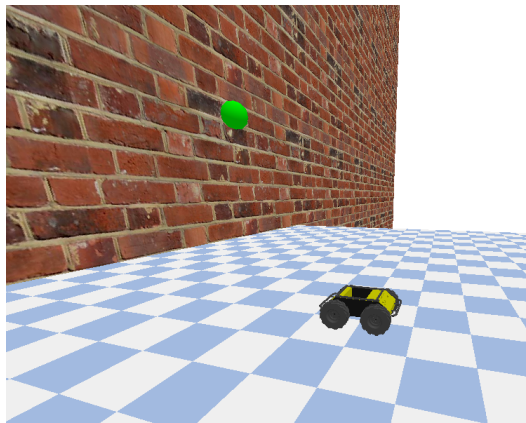
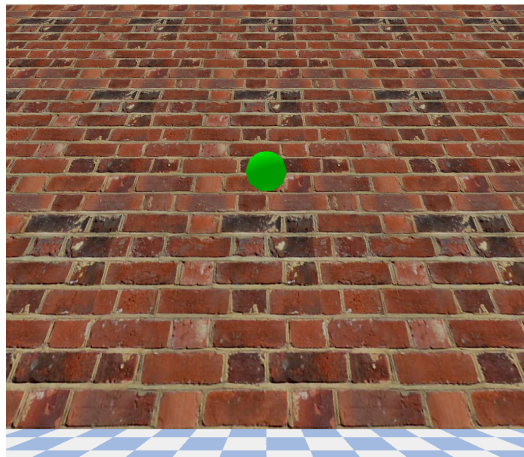


Figure 5.4: These histograms compare our policy (TDPG) to the policy found by policy gradient (PG) in the Lava Problem described in Section 5.2 by showing the frequencies at which each policy incurs different costs. The sensor noise of the test environment is increased from left to right. As the noise increases, the PG policy performance degrades, while the TDPG policy performance remains almost constant. This is because the TDPG algorithm found a task-relevant open-loop policy that is robust to this kind of disturbance.

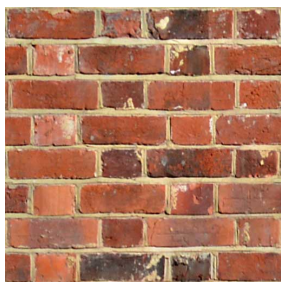


(a) Third-person depiction of the ball-catching scenario.



(b) First-person depiction of the ball-catching scenario.

Figure 5.5



(a) Training



(b) Test 1



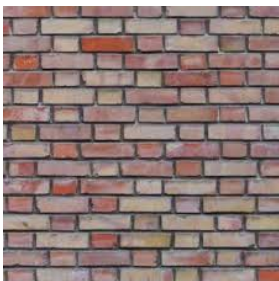
(c) Test 2



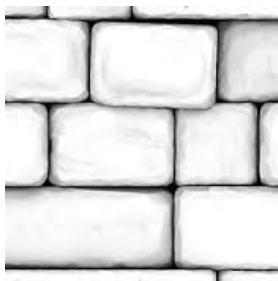
(d) Test 3



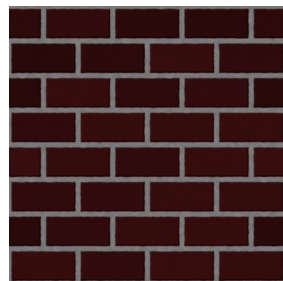
(e) Test 4



(f) Test 5



(g) Test 6



(h) Test 7

Figure 5.6: Textures used as backdrops in different ball-catching environments as part of the example described in Section 5.3.

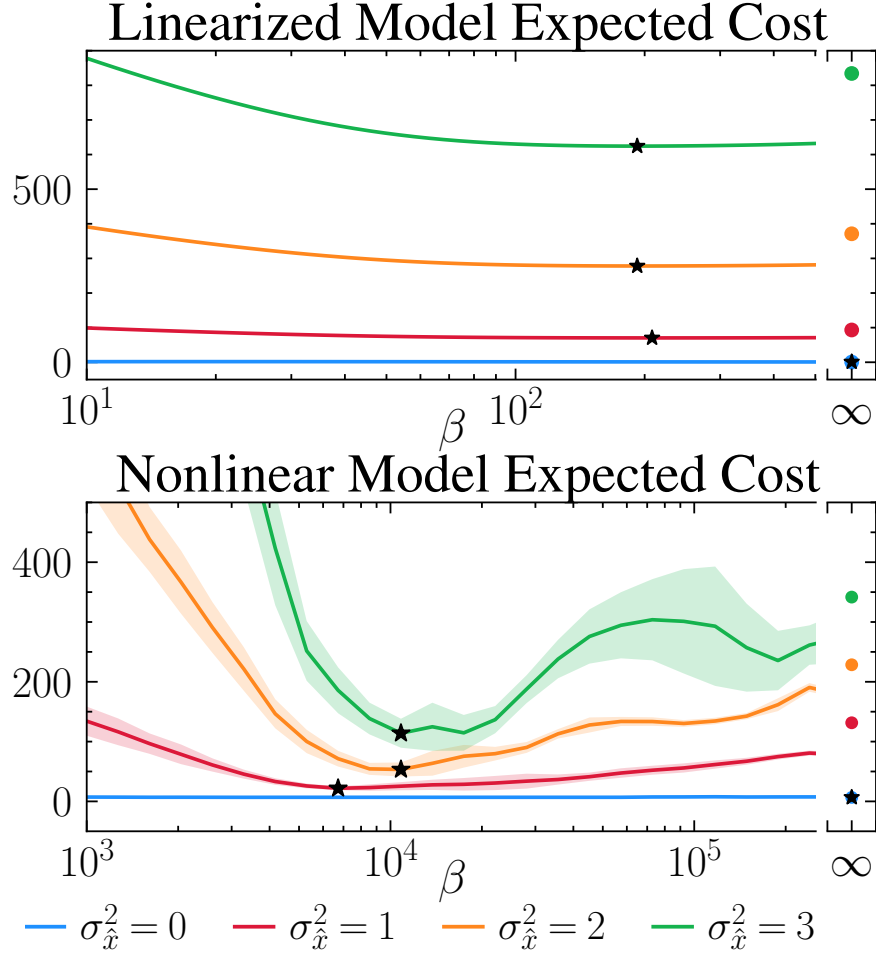


Figure 5.7: Results of the numerical experiments described in Section 5.4. Lines indicate the expected cost of stabilizing a linearized / nonlinear quadrotor model for a range of β and $\beta = \infty$. The latter case corresponds to the performance of the LQR / MPC controllers, which are performant given perfect state information. The $\star$ marker indicates $\beta^\star$ found for each value of $\sigma_{\hat{x}}^2$. For the nonlinear example, shading indicates standard deviations for 100 trials of SV-MPC. The bounded-rationality controllers outperform LQR / MPC in the presence of estimation error.

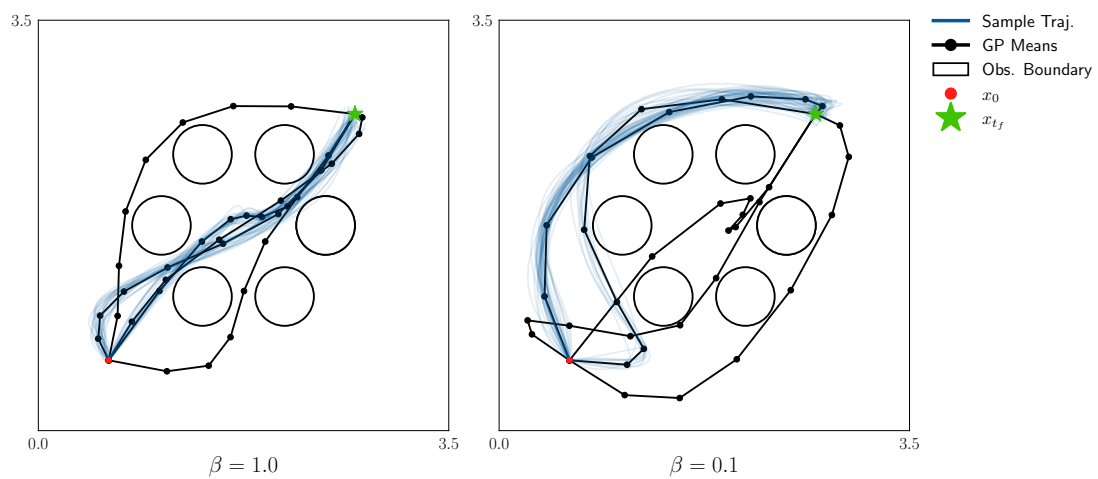


Figure 5.8: Results of the motion planning experiment described in Section 5.5

Part II

Necessary Conditions

Chapter 6

Introduction

The next two chapters focus on deriving *necessary conditions* for task-driven control. Each chapter will present a separate necessary condition derived using independent methods, but both will establish, for a given robot and task encoded via a cost function, a *lower bound on the cost achievable by any controller regardless of its representation*. Thereby, establishing a fundamental performance limit for the robot on a given task. Equivalently, such a bound establishes what kind of dynamical and sensing capabilities are necessary for a robot to achieve a particular level of performance for the task.

6.1 Motivation

The primary motivation for investigating such necessary conditions is to provide a means of determining whether a robot’s particular hardware design, comprised of its sensors, actuators, and dynamics, limits its performance on a task. As a concrete example is the question of whether or not an autonomous automobile requires sensors in addition to cameras to achieve performant driving on public roads. At the time of writing, companies working on self-driving cars would provide different answers: Tesla is notably pursuing a camera-only approach to sensing for autonomy while Waymo equips their vehicles with sensors of multiple modalities [86]. It is unclear whether Waymo gains any fundamental performance advantages from their approach or if multiple modalities simply provide more engineering flexibility. Intuition states that only cameras should be necessary, as humans primarily drive

using their vision, however, roboticists presently lack the tools to determine the validity of this hypothesis.

Historically, robotics as a field has focused on understanding sufficient conditions in the form of *proofs of robustness*, like those described in Part I. Necessary conditions, however, play an important role in the natural sciences, often in the form of fundamental limits. A pertinent example (discussed in Section 6.2.4 and in greater detail in Chapter 8) of such a limit is *Landauer’s limit* — the minimum amount of work necessary to erase one bit of information [12]. Such conditions provide a principled understanding of the boundaries of what is achievable and provide insight on what is achievable during the design of new systems (e.g., whether a robot capable of performing all the tasks required of it or if a robot’s performance is limited by its sensing capabilities or the controller used). Developing such limits will help grow robotics from a primarily *empirically-driven* engineering practice into a more rigorous *synthetic science* [87, 88].

6.2 Related Work

While the kind of necessary conditions stated in the following chapters are unique in robotics literature, they are drawn upon techniques used to establish necessary conditions in information theory, statistics, and thermodynamics. The pertinent work from these fields as well as some of the limited work on necessary conditions in robotics are laid out in this section.

6.2.1 Performance Limits of Sensors for Feedback Control

There is some extant work on information-theoretic performance limits of general dynamical systems using sensor-based feedback. The frameworks in this area take a topological approach that produces necessary conditions that are broadly applicable but difficult to evaluate or apply for non-trivial examples. They generally have roots in Adler’s definition of *topological entropy* of a dynamical system, which is informally the number of bits needed to discretize an invariant set such that the discretization can differentiate all trajectories within the set given enough time, and interprets the members sets of the discretization as pre-images of a continuous, deterministic sensor. For example, Nair et al. generalized this

framework to include feedback considerations, thereby arriving at necessary conditions for a controller to render a set invariant or stabilize the system [89]. In robotics, a similar topological approach based on sensor pre-images is present in generalized filtering models, e.g., *nondeterministic filter* and *combinatorial filter* [90].

Another application of similar topological methods based on sensor pre-images is the *sensor lattice*, which equates the power of a sensor (and, in generalizations, a dynamical system or actuator) with its capability to simulate other sensors, establishing a partial-ordering on families of sensors [90–92]. However, not all sensors are directly comparable with this framework, and it suffers from the same computational challenges when sensors are either continuous or high-dimensional. Moreover, the sensor lattice is a task-independent characterization of a sensor’s capabilities and does not provide information on its performance for a specific task in absolute terms or relative to a specific sensors.

In contrast to these approaches, the necessary conditions derived in the following chapters eschew the generality of these topological frameworks in favor of establishing a computable bound for a specific task, dynamical system, and sensor. Moreover, an ordering for the specific task can be established by identifying a controller for one pair of system dynamics and controller that outperforms the computed limit for another pair.

6.2.2 Application-Specific Performance Limits in Robotics

Work on task-specific performance limits in robotics has thus far been limited to case-by-case analyses for specific problems. For example, the maximum speed a drone can fly through an “ergodic forest” of obstacles without crashing was established using a problem specific approach based in percolation theory [93–95]. In another case, classical control techniques have established the existence of performance limits for linear output-feedback problems on problems like pole-balancing in the presence of sensor noise or delay [96]. While, in the latter case, the limits established empirically correlate with learned vision-based controllers for the same tasks [97], the theoretical performance limits are constrained to specific problem domains (i.e., ergodic forests and linear output-feedback systems) and do not apply to broader robotics settings. In contrast, the results stated in the following chapters apply to general optimal control problems.

6.2.3 Fano’s Inequality: An Information Theoretic Performance Limit

Fano’s inequality is an early result in information theory that establishes a performance limit on the error rate of a decoder receiving messages across a noisy channel quantified by [13, 98]. Beyond information theory itself, Fano’s inequality has primarily been applied in mathematical statistics to provide performance limits in different settings, e.g., for optimal estimators under the Bayes or minimax risk [99, 100]. Fano’s inequality has recently been generalized to be written in terms of an inverse of the relative entropy defined using its variational representation [99, 100]. In Chapter 7, this technique will be further generalized yet remain computationally tractable via convex programming. It will then be applied as part of a dynamic programming algorithm to compute a fundamental limit on achievable closed-loop control cost as a function of open-loop control cost and task-relevant information provided by a specific sensor.

6.2.4 Landauer’s Principle and the Landauer Limit

Landauer’s principle is a refinement of the Second Law of Thermodynamics that connects the computational process of dissipating information with the amount of physical work necessary to do so [12]. Specifically, Landauer identified that a computational process, when implemented in actuality, is also a physical process and therefore it is constrained by physical laws. Using this intuition, he was able to establish his eponymous principle, which is informally stated as follows:

For a physical system encoding n bits of data residing in an environment at temperature T , any process that erases a bit of information from the system satisfies on average:

$$\text{Work Done to Erase a Bit} \geq K_B T \log 2, \quad (6.1)$$

where K_B is Boltzmann’s constant [12].

The constant forming the lower bound is known as *Landauer’s limit*, which is only achievable by a quasistatic process, i.e., a non-physical process that transfers the system between states

infinitely slowly.

Landauer’s original observations led to the development of number of results collectively known as *fluctuation theorems* [101] pioneered by Evans, Jarzynski, and Crooks [102–105]. More recent iterations include the ability to account for feedback control of the process and, importantly, how much information the feedback control is provided through sensing [106–108]. These theorems relate the entropy production of a system to the relative probabilities of witnessing forward and reverse trajectories and imply the Second Law of Thermodynamics as a corollary. Pertinent to the content of Part I, fluctuation theorems have begun to receive attention in beyond statistical physics, in the realm of such as bounded-rationality decision making [109].

In Chapter 8, Landauer’s principle will provide the main intuition for the derived necessary condition. Informally, if our robot is randomly initialized in a random state, and the goal is to drive the robot to a small set of low-cost states, then the initial distribution must be concentrated via feedback, which corresponds to a reduction in entropy of the terminal distribution of the robot, and reducing the entropy requires a minimum amount of control cost, i.e., “work.”

This intuition bears similarity to optimal transport theory [110], and there has been recent work by Chen et al. [111] taking an optimal control perspective that connects minimum-work transitions between thermodynamic equilibria, the Second Law of Thermodynamics, to optimal transport problems via the Wasserstein Distance. However, optimal transport problems are typically formulated as a boundary-value problem, where there is a specific target distribution to which the system must be driven. The problem in Chapter 8 is a relaxation of standard optimal control problems — there is no fixed terminal distribution, the objective is to reach *any* distribution of sufficiently low energy. The proposed algorithm for solving this problem is based on methods in optimal transport theory, specifically, the Cole-Hopf transformation [103, 110] will be used to convert nonlinear PDEs to linear PDEs, but this relaxed optimal control problem and its solution method is novel to the author’s knowledge.

Chapter 7

A Necessary Condition from Generalized Fano-Type Inequalities

This chapter describes a necessary condition for a robot achieving a particular level of performance on a specified task in terms of how suited its dynamical and sensing capabilities are for the task, as well as the best achievable *open-loop* performance for the robot on the task. This condition is achieved using a new generalization of Fano's inequality [13, 98]. The tightness of this performance gap is demonstrated on a variety of control problems for which extant methods are not well suited, including continuous state spaces, high-dimension nonlinear sensors.

7.1 Problem Formulation

The dynamics considered in this section are very similar to (ON), where the sensor will be explicitly described. Let $\mathbf{X} \subseteq \mathbf{R}^n$, $\mathbf{U} \subseteq \mathbf{R}^m$, $\mathbf{Y} \subseteq \mathbf{R}^p$ be the state space, input space, and output space respectively. The initial state is randomly selected according, i.e., $X_0 \sim \mathcal{X}_0$, the Markovian stochastic dynamics are denoted by the mechanism $\mathcal{F} : \mathbf{X} \times \mathbf{U} \rightarrow \Delta(\mathbf{X})$. The sensor model is given by a mechanism $\mathcal{Y} : \mathbf{X} \rightarrow \Delta(\mathbf{Y})$. Feedback is provided by a mechanism specified at each time step based on all available sensor outputs: $\mathcal{U}_t : \mathbf{Y}^{t+1} \rightarrow \mathbf{U}$. Thus,

the dynamics are:

$$X_{t+1} \sim \mathcal{F}(X_t, U_t), \quad Y_t \sim \mathcal{Y}(X_t), \quad U_t \sim \mathcal{U}_t(Y_{0:t}). \quad (\text{CL})$$

Not all tasks require the same amount of sensor information to achieve good performance. For example, a good open-loop control sequence exists for Appendix E.7.1. Therefore, as a baseline, the best performance achievable by an open-loop control sequence $u_{0:t_f}^\perp$, whose dynamics will be denoted:

$$X_{t+1} \sim \mathcal{F}(X_t, u_t^\perp), \quad Y_t \sim \mathcal{Y}(X_t). \quad (\text{OL})$$

For consistency with the paper originally proposing this methodology, reproduced in Appendix C, the formalism of reward (the negative of cost) will be used to specify the task instead of cost, as in the other chapters. Reward at each time step is specified via a reward function: $r_t : \mathbf{X} \times \mathbf{U} \rightarrow \mathbf{R}$. The following assumption is made to aid in deriving a dynamic programming algorithm in the sequel.

Assumption 1 (Bounded Rewards). considered are bounded rewards given by a time-varying reward function, and, without loss of generality it is assumed these rewards are normalized such that, $0 \leq r_t(x, u) \leq 1$.

The optimization problem for which a necessary condition for performant control is sought is then,

$$R^*[\mathcal{Y}; r_t] := \underset{\mathcal{U}_t(\cdot)}{\text{minimize}} \quad \mathbb{E} \left[\sum_{t=0}^{t_f} r_t(X_t, U_t) \right] \quad \text{subject to} \quad (\text{CL}). \quad (\text{CL-OPT})$$

and this condition will take the form of an *upper bound* on the reward function in terms of the task-relevant capabilities of the robot's dynamics, actuators, and sensors. The explicit dependence of the optimal reward on $\mathcal{Y}$ and $r_t(x, u)$ is written to highlight the quantity of primary interest in this section: the task-relevant information provided by the sensor $\mathcal{Y}(x)$ about the task encoded by $r_t(x, u)$. Moreover, *this necessary condition will hold regardless of the parameterization of $\mathcal{U}_t(\cdot)$* , i.e., the condition will hold for controllers implemented via

MPC, analytical solutions, learning-based approaches, etc. In part, the task-relevance of the dynamics and actuators will be captured by the optimal open-loop reward,

$$\underset{u_{0:t_f}^\perp}{\text{minimize}} \quad \mathbb{E} \left[\sum_{t=0}^{t_f} r_t(X_t, u_t^\perp) \right] \quad \text{subject to} \quad (\text{OL}). \quad (\text{OL-OPT})$$

Remark 8. Numerical examples described in Section 7.5 will also feature mixtures of continuous and discrete state, input, and output spaces. All theory described in this and subsequent sections maps equivalently to these settings with probability density functions replaced by appropriate probability mass functions. Similarly, generalization to a measure-theoretic framework is possible with appropriate assumptions and considerations.

7.2 The Inverse of the Relative Entropy

Let $\mathcal{B}(p) \in \Delta(\{0, 1\})$ denote the Bernoulli distribution with parameter p . The key component used to formulate the recent branch of generalizations of Fano's inequality used to derive an upper bound on (CL-OPT) is based on the definition of a (one-sided) inverse for relative entropy. In general, the relative entropy is not invertible. As a brief example, since $\mathbb{D}[\mathcal{B}(\frac{1}{4}); \mathcal{B}(\frac{1}{2})] = \mathbb{D}[\mathcal{B}(\frac{3}{4}); \mathcal{B}(\frac{1}{2})]$, there are two values of p that map to the same value of $\mathbb{D}[\mathcal{B}(p); \mathcal{B}(\frac{1}{2})]$, and a rule is needed to specify which value is selected as the inverse.

Definition 2 (Relative Entropy Inverse [100]). Let $q \in [0, 1]$. The inverse of the relative entropy for Bernoulli distributions is defined:

$$\mathbb{D}_{\mathcal{B}}^{-1}[q; c] := \sup \{p \in [0, 1] \mid \mathbb{D}[\mathcal{B}(p); \mathcal{B}(q)] \leq c\}. \quad (7.1)$$

For a distribution supported on the interval, $[0, 1]$, denoted, $\mathcal{D} \in \Delta([0, 1])$, the inverse of the relative entropy is defined:

$$\mathbb{D}_{\mathcal{B}}^{-1}[\mathcal{D}; c] := \mathbb{D}_{\mathcal{B}}^{-1}[\mathbb{E}[X]; c], \quad X \sim \mathcal{D}. \quad (7.2)$$

Remark 9. For Bernoulli random variables, this definition is a one-sided inverse in the

following sense. Assume let $p' \in [0, 1]$ such that $\mathbb{D}[\mathcal{B}(p'); \mathcal{B}(q)] = c$

$$\mathbb{D}_{\mathcal{B}}^{-1}[q; c] = p \implies \mathbb{D}[\mathcal{B}(p); \mathcal{B}(q)] = c. \quad (7.3)$$

So, while the inverse does not necessarily return the parameter p' , it returns *a parameter* for specifying a distribution yielding the same relative entropy. The definition for general distributions $\mathcal{D}^1, \mathcal{D}^2 \in \mathbf{\Delta}([0, 1])$, the definition is interpreted in the following sense. By the data processing inequality [13, 100],

$$\mathbb{D}[\mathcal{D}^1; \mathcal{D}^2] \geq \mathbb{D}_{\mathcal{B}}[\mathcal{B}(\mathbb{E}[X^1]); \mathcal{B}(\mathbb{E}[X^2])], \quad X^1 \sim \mathcal{D}^1, X^2 \sim \mathcal{D}^2. \quad (7.4)$$

Setting $c = \mathbb{D}[\mathcal{D}^1; \mathcal{D}^2]$, the definition (7.2) seeks to make this inequality tight and returns the mean of a distribution $\mathcal{D} \in \mathbf{\Delta}([0, 1])$ that yields the best approximation, i.e., $p = \mathbb{E}[X]$, $X \sim \mathcal{D}$.

Finally, relative entropy inverse has two monotonicity properties. These properties are important for establishing how the established necessary conditions depend on the robot's task, dynamics, and sensor.

Proposition 7 ([10, 11]). The inverse of the relative entropy for Bernoulli random variables, $\mathbb{D}_{\mathcal{B}}^{-1}[q; c]$ is:

1. monotonically non-decreasing in $c \geq 0$ for fixed $q \in [0, 1]$.
2. monotonically non-decreasing in $q \in [0, 1]$ for fixed $c \geq 0$.

Proof. See Appendix C [10]. The proof is largely an application of the Envelope Theorem from the parametric optimization literature [112]. $\square$

7.3 Necessary Conditions in Terms of the Task-Relevant Information Potential

This section will define a new quantity, the *Task-Relevant Information Potential* (TRIP), which is a specific instance of Definition 2. For single-step optimal control problems, the

TRIP yields a necessary condition immediately, and, in fact, this necessary condition is a new extension (published by the author in [10, 11]) of the generalized Fano-type inequality in [100]. For multi-step problems, this result is used in a dynamic programming algorithm that computes an upper bound on the best reward achievable by any feedback controller.

7.3.1 A Necessary Condition for Single-Step Problems

First, the TRIP and the necessary condition in which it is used are presented for the single-step case. This case is more simply stated and interpreted. Formally, the TRIP is defined:

Definition 3 (Single-Step Task-Relevant Information Potential [10, 11]). The *task-relevant information potential* for single-step problems, i.e., $t_f = 0$, is defined:

$$\mathbb{T}[\mathcal{Y}; r_0] := \mathbb{D}^{-1}[R^\perp; \mathbb{I}[Y_0; X_0]]. \quad (7.5)$$

For single step control problems, the TRIP directly provides a necessary condition for performant control:

Theorem 2. For single step problems (when $t_f = 0$), the best achievable reward using a feedback controller, i.e., the value of (CL-OPT), is upper-bounded by the TRIP:

$$\mathbb{T}[\mathcal{Y}; r_0] \geq R^*[\mathcal{Y}; r_0]. \quad (7.6)$$

Proof. The proof is provided in [10], which is reproduced in Appendix C. $\square$

By Proposition 7, the TRIP is increased by either modifying the dynamics to admit a better open-loop control sequence or by equipping the robot with a more informative sensor. The Lava Problem discussed in Appendix E.7.1, for example admits a good open-loop control sequence that allows the robot to exploit the state space geometry to localize itself without explicit sensing. However, performance on the task can be improved using a feedback controller in conjunction with a capable sensor (or state estimator).

It is worth noting that Theorem 2 is an instance of a generalized Fano-type inequality. It is similar to the generalized Fano-type inequalities derived in [99, 100], but is tighter since

their methods result in an indirect bound on the relative entropy inverse. The intuition for employing Fano's inequality to arrive at a necessary condition is that the sensor output Y_t can be interpreted as an encoding X_t and transmitting it through a noisy channel, namely $\mathcal{Y}(X_t)$. The controller needs to decode this transmission into the correct control input in a way that minimizes the distortion of the message as measured by $-r_t(X_t, U_t)$.

7.3.2 Extension for Multi-Step Problems

To extend this necessary condition to multiple step optimal control problems, dynamic programming is used. The notation,

$$R_t[\mathcal{U}_t](u_{0:t-1}) := \mathbb{E} \left[\sum_{s=t}^{t_f} r_s(X_s, U_s) \mid U_{0:t-1} = u_{0:t-1} \right], \quad 0 \leq t \leq t_f, \quad (7.7)$$

$$R_{t_f+1}[\mathcal{U}_t](u_{0:t_f}) := 0.$$

is introduced to denote the expected future reward at a given for a given closed-loop controller conditioned on a sequence of previous inputs. This expectation is taken using the distribution given by (OL) with the inputs $u_{0:t-1}$ until time t , then propagating the distribution over states and actions using $\mathcal{U}_t$ and (CL) for the remaining time steps. Also, define the expected future reward given a history of inputs and a feedback controller if the input at t is selected independently from observations,

$$R_t^\perp[\mathcal{U}_t](u_{0:t-1}) := \sup_{u_t \in \mathbf{U}} \mathbb{E}[r_t(X_t, u_t) + R_t[\mathcal{U}_t](u_{0:t-1}, u_t) \mid U_{0:t} = u_{0:t-1}], \quad (7.8)$$

and this reward optimized over future feedback controllers:

$$R_t^{\perp*}[\mathcal{U}_t](u_{0:t-1}) := \sup_{\mathcal{U}_{t:t_f}(\cdot)} R_t^\perp[\mathcal{U}_t](u_{0:t-1}). \quad (7.9)$$

The multi-step TRIP and it's necessary condition are now stated using these quantities:

Definition 4 (Multi-step Task-Relevant Information Potential [10, 11]). For any $t =$

$0, \dots, t_f$, the TRIP at time t is defined:

$$\mathbb{T}_t[\mathcal{Y}; r_{0:t_f}] := (t_f - t + 1) \mathbb{D}^{-1} \left[\frac{R_t^{\perp \star}[\mathcal{U}_t](u_{0:t-1})}{(t_f - t + 1)} ; \mathbb{I}[Y_t; X_t] \right] \quad (7.10)$$

Theorem 3. The multi-step trip at $t = 0$ provides an upper-bound on the cost:

$$R^{\star} \leq \mathbb{T}_0[\mathcal{Y}; r_{0:t_f}]. \quad (7.11)$$

Proof. The proof is provided in [10], which is reproduced in Appendix C. The main step is determining the recursive structure for dynamic programming, in this case, $R_t^{\perp \star}[\mathcal{U}_t](u_{0:t-1})$. Surprisingly, the objective of (CL-OPT) does not factor in such a way that the value of the open-loop optimal control problem (OL-OPT) appears. Instead, Bellman's principle of optimality [113] is employed. At time t , it is assumed that all decisions after that time are given by optimal feedback controllers, and the bound is arrived at through an information-theoretic comparison between the closed-loop optimal decision at t and the best open-loop decision at t . $\square$

At this point, however, it seems that this bound is not very useful. It's computation still requires designing $t_f - 1$ optimal feedback controllers. Moreover, for complicated sensors, computing the mutual information is difficult except in special cases (e.g., discrete or linear-Gaussian systems). The next section discuss algorithms to arrive at overestimates for each quantity, which yields a practical necessary condition on task-relevant performance.

7.4 Algorithms for Consistent Approximation of the TRIP Condition

The most complicated aspect of employing Theorem 3 is computing $R_t^{\perp \star}[\mathcal{U}_t](u_{0:t-1})$. This is handled by computing an upper bound through recursive application of the the single-step TRIP bound (with appropriate temporal scaling), which only requires optimizing a single open-loop action at each time step. The procedure, presented in Algorithm 4 yields an upper bound on Theorem 3:

Algorithm 4 Approximate Multi-Step Reward Bound

```

1:  $\hat{R}_{t_f+1}(u_{0:t_f}) \leftarrow 0$ 
2: for  $t = t_f, \dots, 0$  do
3:    $\hat{R}_t^{\perp*}(u_{0:t-1}) \leftarrow \sup_{u_t} \mathbb{E}[r_t(X_t, u_t) | U_{0:t-1} = u_{0:t-1}] + \hat{R}_{t+1}(u_{0:t})$ 
4:    $\hat{R}_t(u_{0:t-1}) \leftarrow (t_f - t + 1) \mathbb{D}^{-1} \left[ \frac{\hat{R}_t^{\perp*}(u_{0:t-1})}{(t_f - t + 1)} ; \mathbb{I}[Y_t; X_t] \right], \quad \forall u_{0:t-1} \in \mathbf{U}^t$ 
5: end for
6: return  $\hat{R}_0$  ▷ There are no previous inputs for  $\hat{R}_0$  to depend on

```

Theorem 4 ([11]). The result of Theorem 3 is an upper-bound on the the value of the optimal control problem (CL-OPT):

$$R^* \leq \mathbb{T}_0[\mathcal{Y}; r_{0:t_f}] \leq \hat{R}_0. \quad (7.12)$$

Numerically implementing Algorithm 4 requires some care. The inverse relative entropy, conditional expectation, supremum, and mutual information all require either exact computation or, due to Proposition 7, upper bounding numerical estimates.

First, the inverse of the relative entropy is done via convex programming. Specifically, the variational definition of this quantity is of the form of a geometric program. It is well known that, while not convex themselves, geometric programs can be solved in an efficient numerical manner through representation as an equivalent convex program [114].

The conditional expectation is estimated via sampling. Concentration inequalities, like the one in the following lemma, provide a means to compute a consistent over approximation with high probability:

Lemma 3 (Hoffding's Inequality [115]). Let $X \sim \mathcal{X} \in \Delta([0, 1])$ be a random variable following some distribution $\mathcal{X}$. Let $X_1, \dots, X_n \sim \mathcal{X}$ be samples from this distribution. With at least probability $1 - \delta$,

$$\mathbb{E}[X] \leq \underbrace{\frac{1}{N} \sum_{i=1}^N X_i}_{\text{Empirical Mean}} + \underbrace{\sqrt{\frac{\log(1/\delta)}{n}}}_{\text{Upper Confidence Interval}}. \quad (7.13)$$

Concentration inequalities can also be used to generate upper bounds on the mutual information via sampling when the sensor is not analytically tractable. They are applied

to variational approximations of the mutual information like the following “leave-one-out” approximation used in Section 7.5:

$$\mathbb{I}[Y_t; X_t] \leq \mathbb{E} \left[\frac{1}{N} \sum_{i=1}^N \frac{\mathcal{Y}(X_t^i) \{Y_t^i\}}{\frac{1}{N-1} \sum_{j \neq i} \mathcal{Y}(X_t^j) \{Y_t^j\}} \right]. \quad (7.14)$$

Here, $X_{0:t}^i, Y_{0:t}^i$ are samples conditioned on an open-loop sequence of inputs, $u_{0:t-1}$, and Lemma 3 is used to compute an upper bound on $\mathbb{I}[Y_t; X_t]$ with high-probability.

In Section 7.5, the supremum will be straight-forward to compute as all examples will consist of problems with finite input spaces (continuous state and output spaces are exhibited, however). Therefore all possible inputs can be tested and the one exhibiting the best reward is used. A possible method for implementing this supremum with continuous input spaces is to solve the *Lagrange dual problem*, which is a convex optimization problem that yields an upper bound on the supremum and is tight under certain conditions [114]. However, this method is not considered in the examples in this chapter.

7.5 Numerical Experiments

The full descriptions and parameters for all experiments are provided in Appendix C. Chosen experimental setups begin with discrete space problems for which Algorithm 4 is implemented exactly, and build in complexity to include continuous state and output spaces along with non-Gaussian sensor models. The latter examples demonstrate that the estimators described in Section 7.4 in conjunction with Lemma 3 provide useful estimates.

7.5.1 The Lava Problem

This example is a discretized version of the one considered in Appendix E.7.1. Again, the robot can move between five linearly connected states freely. On one end is the “lava” state, which is absorbing and incurs a penalty (in the form of receiving no reward). This example was selected as a demonstration because it both features an interesting optimal open-loop solution. It is also a small enough partially-observable decision problem (POMDP) that it can be solved exactly and thus quantifies the tightness of the bound provided by Theorem 4.

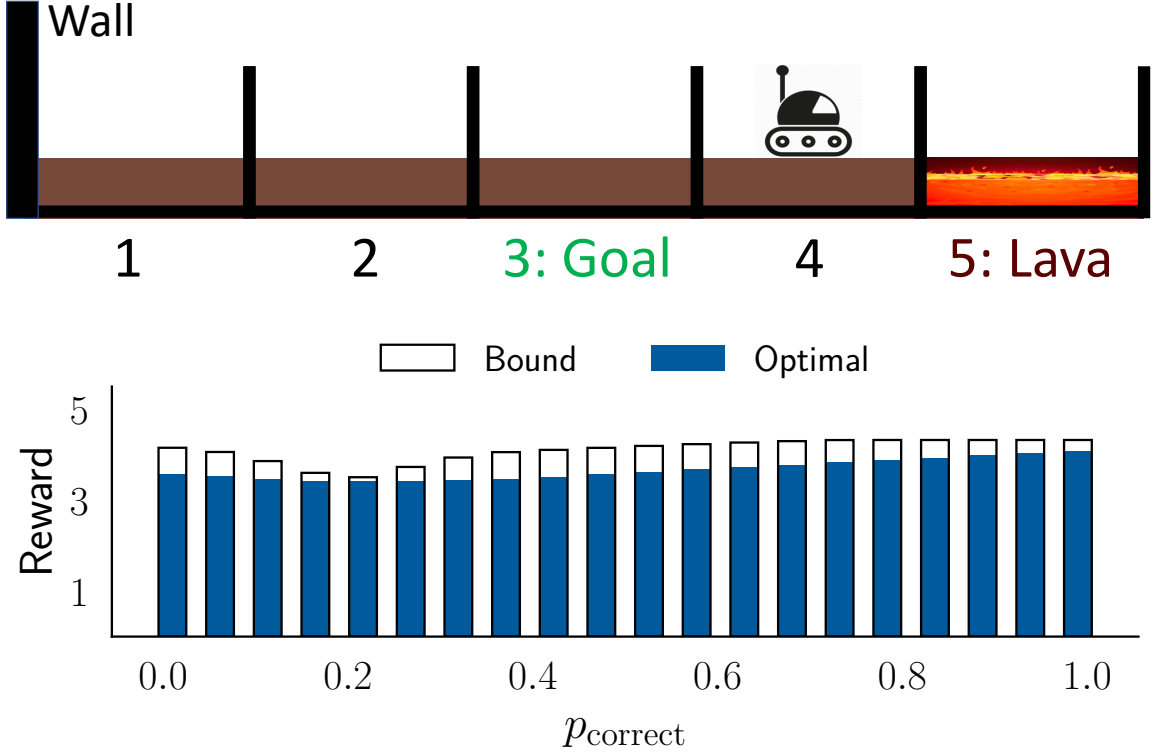


Figure 7.1: A schematic of the Lava Problem considered in Section 7.5.1 (a discretized version of the one described in Appendix E.7.1). The robot needs to navigate to a goal without falling into the lava (using a noisy sensor). We compare the upper bounds on achievable expected rewards computed by our approach with the optimal POMDP solution for different values of sensor noise.

The sensor provided to the robot is a noisy state estimator: with probability p_{correct} , the robot receives its actual state, otherwise it receives a random incorrect state. The bound computed using Algorithm 4 for different values of p_{correct} is presented in Figure 7.1 and compared with an exact solution computed using a Python POMDP solver [116]. The implementation of Algorithm 4 is also written in Python and provides its estimate approximately in $1/100$ of the time. Qualitatively, the bound is tight enough to provide a useful estimate of the actual optimal reward.

7.5.2 Ball-Catching

In this experiment, the ball-catching task is revisited. The dynamics of the robot and the ball are a time discretized double integrator and projectile motion respectively. The sensor estimates the states of the robot and the ball corrupted by additive anisotropic

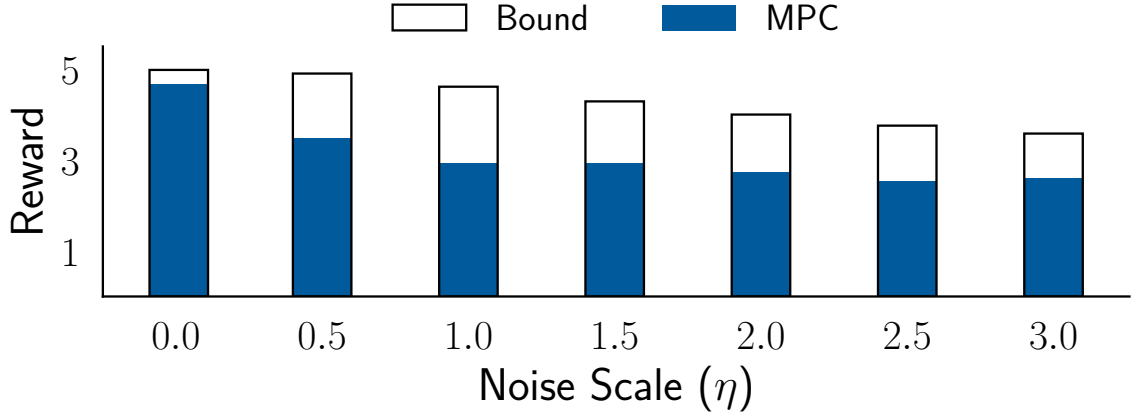


Figure 7.2: Results for the ball-catching example. We compare the upper bounds on achievable expected rewards with the expected rewards using MPC combined with Kalman filtering for different values of sensor noise (results for MPC are averaged across five evaluation seeds; the std. dev. across seeds is too small to visualize).

Gaussian noise, with the covariance of the Gaussian scaled by a parameter η . This sensor is chosen so that the mutual information is analytically available. The input space is a finite set of motion primitives. The reward is $r_t(x, u) = 1 - \max(d, 0)$, where d is the distance between the ball and the robot on the ground plane.

The results of Algorithm 4 compared against an MPC controller paired with a Kalman filter in Figure 7.2. Since the dynamical and sensor models are linear-Gaussian, the MPC approach is an exact solution to the problem. In this case, expected rewards cannot be computed analytically due to the form of the reward function. They are estimated using Lemma 3, which converged quickly enough that the confidence level was not distinguishable on the plot. The performance of the controller tracks the performance degradation predicted by the bound as η is increased. Moreover, it is clear that when, $\eta \geq 1$ the robot is fundamentally limited by the sensor noise as indicated by the decrease of the bound.

7.5.3 Obstacle Avoidance with a Depth Sensor

This last example is a motion planning problem, where the robot is placed in a rectangular workspace with circular obstacles visualized in Figure 7.3. The robot is modeled as a double integrator and control inputs are motion primitives, but the sensor is a relatively high-dimensional LIDAR model with ten rays and multiple kinds of non-Gaussian noise (see

[66]). The controller used for comparison is a learned neural network.

Experimental results are also shown in Figure 7.3 for different noise scales. The upper bound on R^* computed using Algorithm 4 with the estimators based on Lemma 3 described in Section 7.5 holds regardless of the size of the neural network, the network architecture, or algorithm used for policy learning. Meanwhile, the performance of the learned controller is very likely sub-optimal due to the network being of fixed size and the optimal control problem being non-convex. Therefore, in actuality, the bound may be tighter than was numerically computed. Nevertheless, the learned controller’s performance is comparatively close to that predicted by the bound for low noise scales. Moreover, a sensor with noise $\eta = 0.1$ achieves higher performance than the fundamental limit for a sensor with noise $\eta \geq 0.4$, thereby establishing that sensor noise of that scale fundamentally limits the capabilities of the robot on the task.

Additionally, the number N_{rays} of rays along which the LIDAR provides distance estimates was computed for two values at a fixed noise scale $\eta = 0.3$. We compare two sensors with $N_{\text{rays}} = 50$ (Sensor 1) and $N_{\text{rays}} = 5$ (Sensor 2) respectively. The upper bound on expected reward computed using our approach (with confidence $1 - \delta = 0.95$) for Sensor 2 is 0.79. A neural network policy for Sensor 1 achieves an expected reward of approximately 0.86, which surpasses the fundamental limit on performance for Sensor 2.

7.6 Tighter Bounds Using the Relative f -Entropy

This chapter concludes with a brief discussion of the generalization of its results to relative f -entropy [117, 118] (also known as the f -divergence) presented in the author’s work [11]. Specifically, the generalization of the relative entropy to the relative f -entropy is considered, which is defined for two distributions $\mathcal{P}, \mathcal{Q} \in \Delta(\mathbf{X})$ and a convex function $f : \mathbf{R}_+ \rightarrow \mathbf{R}_+$ satisfying $f(1) = 0$:

$$\mathbb{D}[\mathcal{P}; \mathcal{Q}] := \int dx \mathcal{Q}\{x\} f\left(\frac{\mathcal{P}\{x\}}{\mathcal{Q}\{x\}}\right). \quad (7.15)$$

Table 7.1: Examples of common f -entropies.

Name	Corresponding $f(x)$
Shannon	$x \mapsto x \log(x)$
Negative log	$x \mapsto -\log(x)$
Total variation	$x \mapsto \frac{1}{2} x - 1 $
Pearson χ^2	$x \mapsto (x - 1)^2$
Squared Hellinger	$x \mapsto (\sqrt{x} - 1)^2$
Neyman χ^2	$x \mapsto \frac{1}{x} - 1$

with equivalent definitions for Shannon’s other information-theoretic quantities. Shannon’s quantities correspond with the choice $f(x) = x \log(x)$. Other common choices for $f(x)$ are presented in Table 7.1. The theoretical and algorithmic developments in this chapter generalize by simply replacing the relative entropy with the relative f -entropy, although the proof of Proposition 7 uses a different technique [11].

The advantage to using the f -relative entropy is that $f(\cdot)$ is free to be chosen in a way that yields the tightest bound, i.e., $f(\cdot)$ is a decision variable minimized to find a *task-relevant measure of information*. This is similar to the discussion of identifying a task-relevant metric to yield a more accurate sufficient condition in Section 3.6.

Results repeating the Lava Problem described in Section 7.5.1 for different f -entropies are presented in Figure 7.4 and Figure 7.5. Both common choices of $f(\cdot)$ from Table 7.1 and optimized $f(\cdot)$ parameterized as a piecewise linear function are compared with the Shannon entropy results from Section 7.5.1. These results demonstrate that: (1) optimizing the f -entropy used can improve the tightness of Theorem 4, (2) the optimal choice of f -entropy is dependent on the noise level of the sensor.

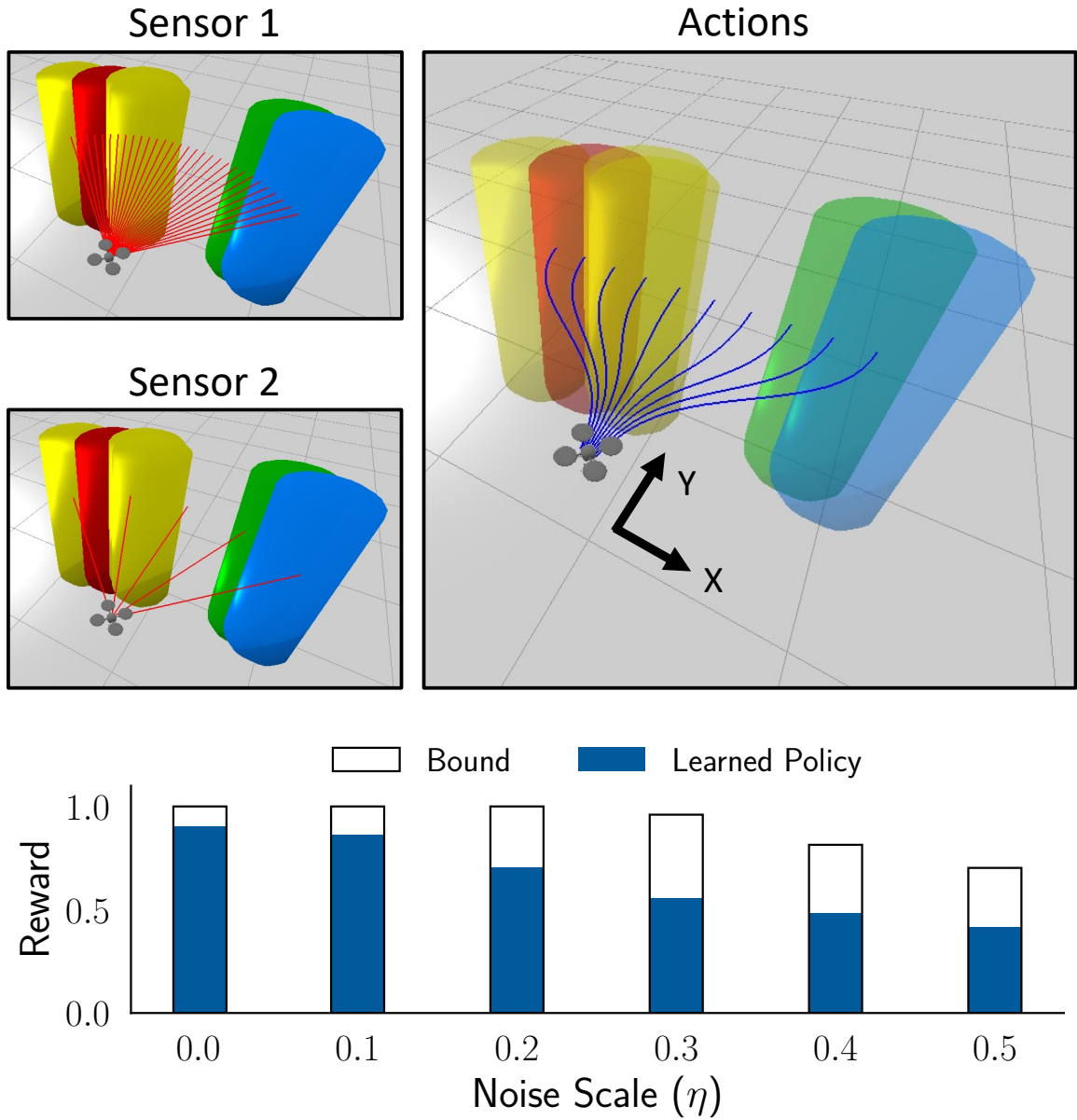


Figure 7.3: Top. This example establishes a fundamental limit on motion planning performance for a robot equipped with a noisy LIDAR. **Bot.** The upper bounds on achievable expected rewards with the expected rewards using a learned neural network policy for different values of sensor noise (results for the learned policy are averaged across five training seeds; the std. dev. is too small to visualize).

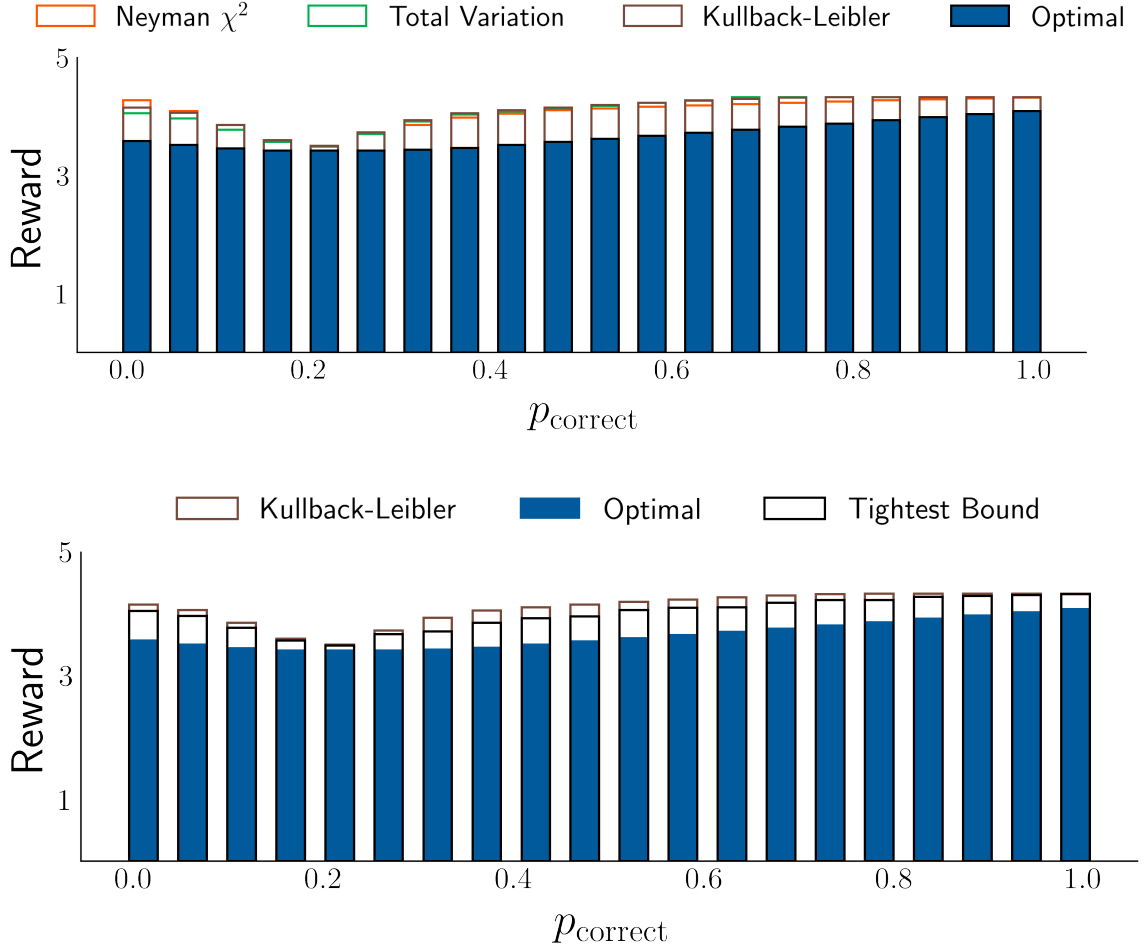


Figure 7.4: Top. The Lava Problem experiment described in Section 7.5.1 repeated with different f -relative entropies resulting in bounds of different tightness. **Bot.** The same experiment comparing the bound using Shannon’s relative entropy with an optimized f -relative entropy. Here, $f(\cdot)$ is parameterized as a piecewise linear convex function with the slopes of each segment serving as a decision variable. Optimal $f(\cdot)$ for each value of p_{correct} is shown in Figure 7.5.

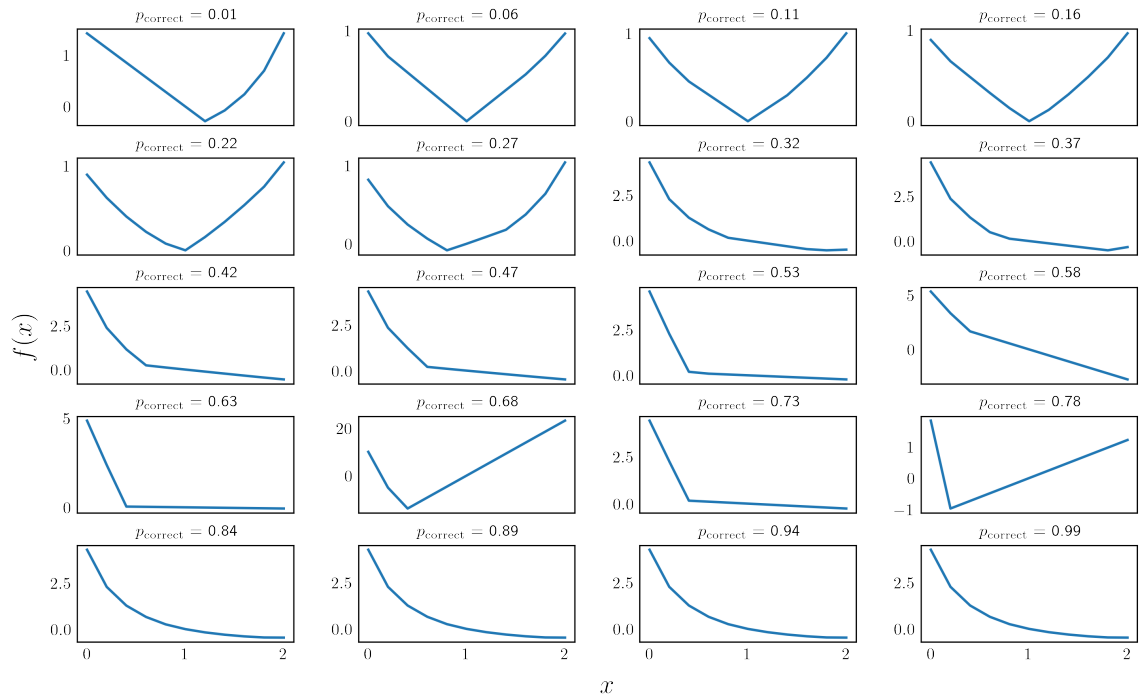


Figure 7.5: Optimized piecewise linear convex functions f that provide the tightest bounds for each corresponding sensor noise level on the Lava Problem described in Section 7.5.1.

Chapter 8

A Necessary Condition from Non-Equilibrium Thermodynamics

This short chapter describes a generalization of the necessary condition described in Chapter 7 that arose through application of Fano’s inequality and some related corollaries. While closely related to the generalization of Fano’s inequality discussed previously, the intuition and interpretation of the results are thermodynamic in origin as opposed to information-theoretic. Specifically, Landauer’s principle, described in Chapter 6, is generalized to the abstract setting of optimal control problems, leading to a necessary condition pertinent to problems in robotics.

This chapter is an early result of ongoing and previously unpublished research. As such, it does not include numerical experiments evaluating its potential application in robotics. . However, discussion of numerical feasibility is included as well as an example demonstrating that, for a particular choice of parameters, Landauer’s limit is recovered. Finally, the necessary condition is easily derived given the correct problem formulation. However, this formulation is very abstract, and more accurate bounds that account for properties of specialized dynamical systems appear in the statistical thermodynamics literature. Given the equivalence described in this chapter, it is likely that similar refinements tailored for robotics problems are achievable. Such future work is described in more detail in Chapter 9.

8.1 Background: Jaynes' Entropy

In order to facilitate the thermodynamical interpretation of this necessary condition, this section briefly describes the notion of entropy proposed by Jaynes [14] to correct well-known issues with Shannon's *differential entropy* for continuous distributions [13], which is defined:

$$\mathbb{S}[\mathcal{X}] := -\mathbb{E}[\log \mathcal{X}\{X\}], \quad X \sim \mathcal{X}. \quad (8.1)$$

The issues with this definition include: (1) does not corresponding to the infinitesimal limit of the Shannon entropy for discrete distributions, (2) the unit of the logarithm's argument having "units" $1/dx$ as opposed to a dimensionless probability in the discrete case, (3) it is not invariant under coordinate changes despite the intuition that bijections should not alter the information content of the distribution.

To correct these flaws, Jaynes' argues that the correct definition of entropy for continuous distributions should be defined relative to a prior distribution $\mathcal{X}^\perp$,

$$\mathbb{S}[\mathcal{X}; \mathcal{X}^\perp] := -\mathbb{D}[\mathcal{X}; \mathcal{X}^\perp], \quad (8.2)$$

where $\mathcal{X}^\perp$ is, in his parlance, the *invariant measure* since it confers invariance under coordinate changes to his entropy. Due to the non-negativity of the relative entropy, $\mathbb{S}[\mathcal{X}; \mathcal{X}^\perp] \leq 0$ and the distribution with the highest entropy is the invariant measure itself. Therefore, $\mathcal{X}^\perp$ specifies a reference point from which the entropy of $\mathcal{X}$ is measured. Common choices are the uniform distribution when the support of $\mathcal{X}$ is compact (in this case, Jaynes' and Shannon's entropies differ by the log-volume of the support set), the equilibrium distribution when studying a non-equilibrium thermal process, or another problem specific prior distribution. Finally, while the non-negativity of $\mathbb{S}[\mathcal{X}; \mathcal{X}^\perp]$ and generally unspecified nature of $\mathcal{X}^\perp$ may appear undesirable for measures of entropy, in the applications which include this chapter, the topic of study is how the entropy varies between distributions and its absolute value is not important.

8.2 Optimal Control Problem Formulation

The optimal control setting considered in this chapter is similar to previous chapters. Let $\mathbf{X} \subseteq \mathbf{R}^n$, $\mathbf{U} \subseteq \mathbf{R}^m$, $\mathbf{Y} \subseteq \mathbf{R}^p$ be the state space, input space, and output space respectively. The initial state is randomly selected according, i.e., $X_0 \sim \mathcal{X}_0$, the Markovian stochastic dynamics are denoted by the mechanism $\mathcal{F} : \mathbf{X} \times \mathbf{U} \rightarrow \Delta(\mathbf{X})$. The sensor model is given by a mechanism $\mathcal{Y} : \mathbf{X} \rightarrow \Delta(\mathbf{Y})$. Feedback is provided by a mechanism specified at each time step based on all available sensor outputs: $\mathcal{U}_t : \mathbf{Y}^{t+1} \rightarrow \mathbf{U}$. Thus, the dynamics are:

$$X_{t+1} \sim \mathcal{F}(X_t, U_t), \quad Y_t \sim \mathcal{Y}_t(X_t), \quad U_t \sim \mathcal{U}_t(Y_{0:t}). \quad (\text{CL})$$

Additionally, consider a hypothetical system with the same dynamics modified to have a potentially different initial distribution $X_0^\perp \sim \mathcal{X}_0^\perp$ and where the sensor outputs values independent of the state:

$$X_{t+1}^\perp \sim \mathcal{F}(X_t^\perp, U_t^\perp), \quad Y_t^\perp \sim \mathcal{Y}^\perp, \quad U_t^\perp \sim \mathcal{U}_t(Y_{0:t}^\perp). \quad (\text{OL})$$

The optimal control problem of interest is to minimize¹ some objective $J : \mathbf{X}^{t_f} \times \mathbf{U}^{t_f-1} \rightarrow \mathbf{R}_+$ under these dynamical constraints:

$$\underset{\mathcal{U}_{0:t_f}}{\text{minimize}} \quad \mathbb{E}[J(X_{0:t_f}, U_{0:t_f-1})] \quad \text{subject to} \quad (\text{CL}). \quad (\text{OPT})$$

8.3 A Necessary Condition from the Irreversibility of Feedback

Landauer's principle describes the *logical irreversibility* of a computational process. If a bit of information is erased, there is no way to invert the process to reacquire that information. This observation is generalized in a number of results in the statistical physics literature (see Chapter 6), including ones that make use of information consumption through feedback to resolve the philosophical difficulties of Maxwell's demon [119].

Similar intuition in this section is used to derive a thermodynamic necessary condition in the form of a lower bound on (OPT). Specifically, the measurement performed by $\mathcal{Y}_t(x)$ can

¹Minimizing a cost provides a better physical interpretation than maximizing a reward.

be used by $\mathcal{U}_t$ to reduce the entropy of the state distribution by causing it to concentrate — thereby removing information from the distribution of states. This is not true for open-loop observations provided by $\mathcal{Y}_t^\perp$, which can be reversed by sampling from the sequence in a reversed order. Therefore, the irreversibility of (CL) is quantified through comparison to (OL).

Specifically, this comparison is made via the relative entropy between trajectories of the two systems: $\mathbb{D}[\mathcal{T}; \mathcal{T}^\perp]$ where $\mathcal{T}$ and $\mathcal{T}^\perp$ are the joint distributions of $(X_{0:t_f}, Y_{0:t_f-1}, U_{0:t_f-1})$ and $(X_{0:t_f}^\perp, Y_{0:t_f-1}^\perp, U_{0:t_f-1}^\perp)$ respectively. Then, the following inequality holds due to a variational inequality describing the *free energy* of a thermal system (see Chapter 3 or Appendix D),

$$\mathbb{F}^\beta[J] := -\frac{1}{\beta} \log \mathbb{E} \left[\exp \left(-\beta J(X_{0:t_f}^\perp, U_{0:t_f-1}^\perp) \right) \right] + \frac{1}{\beta} \mathbb{S}[\mathcal{T}; \mathcal{T}^\perp] \leq \mathbb{E}[J(\mathcal{T})], \quad (8.3)$$

Critically, due to the structure of the two dynamical systems involved, the entropy factors:

$$\begin{aligned} \mathbb{S}[\mathcal{T}; \mathcal{T}^\perp] &= \mathbb{E} \left[\log \left(\frac{\mathcal{X}\{X_0\}}{\mathcal{X}\{X_0\}} \right) \right] - \sum_{t=0}^{t_f-1} \mathbb{E} \left[\log \left(\frac{\mathcal{F}(X_t, U_t)\{X_{t+1}\}}{\mathcal{F}(X_t, U_t)\{X_{t+1}\}} \cdot \frac{\mathcal{Y}(X_t)\{Y_t\}}{\mathcal{Y}^\perp\{Y_t\}} \cdot \frac{\mathcal{U}(Y_{0:t})\{U_t\}}{\mathcal{U}(Y_{0:t})\{U_t\}} \right) \right], \\ &= \mathbb{S}[\mathcal{X}_0; \mathcal{X}_0^\perp] + \sum_{t=0}^{t_f-1} \mathbb{S}[\mathcal{Y}_t(X_t); \mathcal{Y}_t^\perp]. \end{aligned} \quad (8.4)$$

This factorization leads to the main result of this brief chapter, which provides a more direct generalization of Theorem 2 than Theorem 3.

Theorem 5. Denote the optimum value of (OPT) by J^* . The following inequality holds for any feedback controller $\mathcal{U}_t$ and any $\mathcal{Y}_t(x)$ satisfying $\mathcal{Y}_t(x) \ll \mathcal{Y}_t^\perp$.

$$\mathbb{F}^\beta[J] + \frac{1}{\beta} \left(\mathbb{S}[\mathcal{X}_0; \mathcal{X}_0^\perp] - \sum_{t=0}^{t_f-1} \mathbb{S}[\mathcal{Y}_t(X_t); \mathcal{Y}_t^\perp] \right) \leq \mathbb{E}[J(X_{0:t_f}, U_{0:t_f-1})]. \quad (8.5)$$

Moreover, for any $\mathcal{Y}_t(x)$:

$$\inf_{u_{0:t_f-1}} \mathbb{F}^\beta[J(X_{0:t_f}, u_{0:t_f-1})] - \frac{1}{\beta} \left(\sum_{t=0}^{t_f-1} \mathbb{I}[X_t; Y_t] \right) \leq J^*. \quad (8.6)$$

Proof. The first inequality follows from the preceding discussion. The second follows from

extremizing each term of that inequality individually. First, choose $\mathcal{X}_0^\perp = \mathcal{X}_0$ to maximize $\mathbb{S}[\mathcal{X}_0; \mathcal{X}_0^\perp]$. the variational inequality described in (3.10) states that the infimum of the relative entropy over its second argument is the mutual information (see also [62] for a the derivation of that relationship). Since any choice of $\mathcal{Y}_t^\perp$ satisfying absolute continuity is admissible, the marginal distribution over sensor outputs at each time step is selected. Then, the free entropy is the only term containing decision variables.

Finally, $\mathcal{U}_t \circ \mathcal{Y}_{0:t}^\perp$ is a distribution over $\mathcal{U}$ independent of the state. Therefore, a lower bound on the free energy can be established by optimizing over both $(\mathcal{U}_t, \mathcal{Y}_t^\perp)$. Furthermore, without loss of generality, $\mathcal{U}_t$ can be chosen to be a delta distribution that outputs a constant u_t independent of the input y_t . Thus:

$$\inf_{\mathcal{Y}_t^\perp} \mathbb{F}^\beta[J] \geq \inf_{\mathcal{Y}_t^\perp, \mathcal{U}_t} \mathbb{F}^\beta[J] = \inf_{u_t \in \mathbf{U}} \mathbb{F}^\beta[J]. \quad (8.7)$$

□

The second inequality is similar to the bound derived in Chapter 7 using the TRIP. Like the TRIP-derived inequalities, this inequality is in terms of the open-loop performance measured by $\mathbb{F}^\beta[J]$ and the mutual information describing how much information is processed by the feedback controller. For the optimal controller to achieve better performance, either a tighter coupling between the state and input is required (i.e., $\mathbf{K}$ is modified so that U_t is less dependent on W_t) or the dynamics and set $\mathbf{K}^\perp$ needs to be modified in some manner to achieve a better open-loop performance.

The differences between these this necessary conditions and the TRIP-derived one are that the inverse relative entropy is not explicitly present, that costs were not assumed to be bounded, and open-loop performance is characterized using the free energy instead of the expected open-loop cost. The definition of the inverse relative entropy is due to the same variational relationship between the relative entropy, expectation, and free energy as that presented in (8.3). In Chapter 7, bounded costs were assumed, which allowed (8.3) in the special case of Bernoulli random variable to be used for continuous random variables to derive a bound. In this instance, that assumption was not made and (8.3) is used directly. The trade-off is that the free energy functional includes all moments of the distribution of

costs under (OL), as opposed to just the mean, however, no dynamic programming algorithm is necessary to compute the necessary condition. Presently, it has not been studied which of the two necessary conditions is tighter when both are applicable.

In terms of computation, sampling-based approaches in conjunction with concentration inequalities provide a means to estimate the terms of the necessary condition. The estimator (7.14) is applicable again, however, if an unbounded objective is used, a different estimator will be necessary to compute $\mathbb{F}^\beta[J]$. A Chernoff bound approach seems particularly well-suited due to similarities between the free energy and the form of the Chernoff bound.

To conclude this section, the following proposition gives the full-observability equivalent of Theorem 5, which is derived in the same manner by comparing processes controlled by a closed-loop state feedback controller $\mathcal{U}_t(x)$ and an open-loop controller $\mathcal{U}_t^\perp$.

Proposition 8. Denote the optimum value of (OPT) by $J^\star$. The following inequality holds for any $\mathcal{U}_t(x)$ satisfying for all $x \in \mathbf{X}$, $\mathcal{U}_t(x) \ll \mathcal{U}_t^\perp$.

$$\mathbb{F}^\beta[J] + \frac{1}{\beta} \left(\mathbb{S}[\mathcal{X}_0; \mathcal{X}_0^\perp] + \sum_{t=0}^{t_f-1} \mathbb{S}[\mathcal{U}_t(X_t); \mathcal{U}_t^\perp] \right) \leq \mathbb{E}[J(X_{0:t_f}, U_{0:t_f-1})]. \quad (8.8)$$

Moreover, for any $\mathcal{U}_t(x)$:

$$\inf_{u_{0:t_f-1}} \mathbb{F}^\beta[J(X_{0:t_f}, u_{0:t_f-1})] + \frac{1}{\beta} \left(\sum_{t=0}^{t_f-1} \mathbb{I}[X_t; U_t] \right) \leq J^\star. \quad (8.9)$$

8.4 Recovering the Landauer Limit

The inequalities in Proposition 8 can be shown to recover the form of Landauer's limit, in which cost plays the role of energy. To illustrate the analogy between the two, consider the dynamics and controllers,

$$\mathcal{F}(x, u) = -\partial_x H(x) + B(u + U_t) + \mathcal{N}(0, \Sigma), \quad \mathcal{U}_t(x) = k_t(x), \quad \mathcal{U}^\perp = 0, \quad (8.10)$$

where $B = [0, B_v]^\top$ limits control to only alter the velocity portion of the state and τ is the size of the time discretization. Assume that for these choices, the open-loop dynamics

admit a stationary distribution $\bar{\mathcal{X}}$. Without loss of generality, this distribution's density can be written in the form of a Gibbs measure,

$$\bar{\mathcal{X}}\{x\} \propto \exp(-\beta H_0(x)), \quad H_0(x) = -\frac{1}{\beta} \log(\bar{\mathcal{X}}\{x\}). \quad (8.11)$$

This representation does not serve a practical purpose but is used to provide physical intuition for this section. Next, consider a cost function of the form:

$$J(x_{0:t_f}, u_{0:t_f}) = H_{t_f}(x_{t_f}) + \sum_0^{t_f-1} \frac{1}{2} \|u\|_R^2, \quad \tau > 0. \quad (8.12)$$

The form of this cost function is common in robotics, where it encodes the task of searching for the shortest path, as measured by a squared norm of the controls, to reach a target region of the state space encoded by $C(x)$. Physically, x_t is the state of a mechanical system and u_t is acceleration. The quadratic input cost is interpreted as the Rayleigh dissipation function, which measures energy dissipated by the system due to friction, where τ^2 is the size of the time discretization. The two functions $H_0(x)$ and $H_{t_f}(x)$ are two Hamiltonians describing the internal energy of the system. Assume that the open-loop system begins at equilibrium, i.e., before time $t = 0$, the system has been in contact with a heat bath for long enough that it has equilibrated to a temperature β^{-1} under the dynamics governed by $H_0(x)$, thus $\mathcal{X}_0 = \mathcal{X}_0^\perp = \bar{\mathcal{X}}$. When $t = 0$, feedback control is applied in the form of $k(x, t)$ to cause the system about the concentrate around the minimum of $H_{t_f}(x)$. Note that because $\mathcal{X}_0^\perp$ is invariant to the open-loop dynamics,

$$\mathbb{F}^\beta[J] = \mathbb{F}^\beta[H_0]. \quad (8.13)$$

Let $k^*(x, t)$ be the optimal deterministic feedback policy. Inserting these relations into the necessary condition in Proposition 8:

$$\mathbb{F}^\beta[H_0] - \mathbb{E}[H_{t_f}(X_{t_f})] + \frac{1}{\beta} \left(\mathbb{S}[\mathcal{X}; \mathcal{X}^\perp] - \sum_{t=0}^{t_f-1} \mathbb{I}[X_t; U_t] \right) \leq \mathbb{E} \left[\sum_0^{t_f-1} \frac{1}{2} \|U_t\|_R^2 \right]. \quad (8.14)$$

Since $\mathcal{X}_0$ is a Gibbs measure, the variational representation of the free energy is an equality:

$$\mathbb{F}^\beta[H_0] = \mathbb{E}[H_0(X_0)] - \frac{1}{\beta} \mathbb{S}[\bar{\mathcal{X}}; \bar{\mathcal{X}}] = \mathbb{E}[H_0(X_0)]. \quad (8.15)$$

This relationship allows for the rearrangement of the necessary condition as:

$$\underbrace{\left(\mathbb{E}[H_0(X_0)] - \mathbb{E}[H_{t_f}(X_{t_f})] \right)}_{\text{Change in Internal Energy}} + \underbrace{\frac{1}{\beta} \left(\mathbb{S}[\mathcal{X}; \mathcal{X}^\perp] - \sum_{t=0}^{t_f-1} \mathbb{I}[X_t; U_t] \right)}_{\text{Bits Erased Through Feedback}} \leq \underbrace{\mathbb{E} \left[\sum_0^{t_f-1} \frac{1}{2} \|U_t\|_R^2 \right]}_{\text{Energy Dissipated}}. \quad (8.16)$$

If $H_0(x) = H_{t_f}(x)$ (i.e., the controller seeks to further concentrate the phase space distribution around the minimum of the same Hamiltonian), then Landauer's limit is recovered up to a choice of problem parameters. More broadly, only the means of the Hamiltonians have to match. This is the case considered in Landauer's original formulation, where he considered the stochastic evolution of a particle in a symmetric double-welled potential, and the erasure process altered the shape of the Hamiltonian to reallocate all of the probability density from being evenly distributed across the potential to being contained in a single well at the same average internal energy [12].

Chapter 9

Conclusion

9.1 Summary

This dissertation presents novel necessary and sufficient conditions for *task-driven* optimal control of robots (and other cyber-physical systems) subject to information constraints. At a basic level, conditions provide a means for roboticists to handle the complexities of working with high-dimensional sensors (e.g., cameras) that have become a staple of modern robotics. These sensors provide rich data that is general purpose and not tailored to any specific task and lack tractable analytical models, thereby making them difficult to analyze. Moreover, the data produced by these sensors is commonly input into general purpose algorithms to build detailed representations of the robot and its environment. As a result, working with such sensors and representations requires significant computation either online, or, in the case of many deep learning methods, offline during the training process to achieve performant control. Moreover, since no distinction is made in the control loop between task-relevant and task-irrelevant information, the robot’s decisions risk becoming tightly coupled with irrelevant features of its observations or environment representation. As a result, the robot’s performance will degrade rapidly and become potentially unsafe.

Necessary conditions quantify the limit of performance on a task achievable by a robot given a quantification of the information provided by its sensors and task specification in terms of the robots dynamics and cost function independent of the optimality or implementation of the robot’s controller. They provide a means to determine prior to the actual

design of the control system whether it is possible to achieve a desired level of performance with given this information. They are useful for establishing whether one sensor suite is more capable than another for that specific task (e.g., whether or not adding a LIDAR fundamentally improves the performance of a self-driving car already equipped with cameras). There are few such conditions established in robotics, with necessary conditions either developed on a case-by-case basis for different tasks, or presented in an abstract framework that is computationally intractable for non-trivial examples.

On the other hand, sufficient conditions provide a performance guarantee: if the controller and sensor satisfy particular criteria, then a certain level of performance is guaranteed. They provide a method to selectively decouple the robot’s control decisions from its sensing and actuation through limiting its dependence on task-irrelevant information. Such results are particularly useful when the robot is operating in a new environment that differs in a task-irrelevant manner from the environments for which the feedback loop was optimized, and it is unclear how the control loop will perform. Moreover, since the robot is only using a limited amount of task-relevant information, irrelevant variables in the sensing data can be discarded and the state estimator only needs to determine a few task-relevant components of the system state. As a result, these components of the feedback loop can be made much more computationally efficient.

From a technical perspective, the necessary and sufficient conditions derived in this dissertation all leverage variational relationships between the expected cost, the relative entropy between a closed-loop and open-loop controller, and the free energy — a quantity that captures higher moments of the cost distribution under the open-loop controller. In Chapter 7, a specialized version of this quantity for bounded rewards (equivalently, costs) is used to define the *task-relevant information potential* and the necessary condition is formed by deriving a dynamic programming algorithm to approximate it through recursive bounds. The intuition for doing so comes from a generalization of Fano’s inequality, a necessary condition describing information transfer through noisy channels, and treating the sensor as a noisy channel. In Chapter 8, the variational relationship is used in its general form to arrive at a necessary condition using *Landauer’s principle* — a physical relationship stating that reducing uncertainty is an irreversible process that fundamentally requires the

dissipation of energy — to provide intuition and equating control cost with the energy of a computational process. Finally, in Chapter 3, the variational relationship is used as a model of the decision-making process of a bounded-rationality agent. Such agents, namely humans, empirically demonstrate that relying on task-relevant, low-information heuristics is both computationally efficient and leads to robust decisions when a task is completed in new settings. Both the derived necessary and sufficient conditions are demonstrated to be empirically useful for non-trivial problems in robotics, including examples using rich sensors common in robotics like cameras and LIDAR (see Chapter 5 and Chapter 7).

9.2 Limitations and Future Work

The framework laid out in Chapter 3 features multiple opportunities for extension capitalizing on both the bounded-rationality and differential privacy frameworks. For the bounded-rationality framework, one algorithmic extension is to determine an algorithm to adapt the parameter β online based on the sensor or state estimator’s performance in an adaptive manner. If the robot is able to detect that the sensor is providing more or less task-relevant information than anticipated, it can adjust how much information it is using to achieve better performance. Such an algorithm will benefit from determining whether or not the bounds in Theorem 1 is convex in β (information use), or the weaker statement of whether increased estimation error causes the minimizing choice of β is monotone in the estimation error. In a similar vein, the bounded rationality framework can be used as a high-level planner that switches between different kinds of feedback policies to further improve their robustness (e.g., $\mathbf{H}_\infty$ controllers synthesized under different adversarial constraints). Another open question is how to optimize the metric used to quantify estimation error to yield a tighter bound by capturing only error in estimating the task-relevant variables, as described in Section 3.6. This problem is related to extending algorithms for the synthesis of task-relevant state representations beyond the two methods described in Chapter 4. A possible approach is to adapt the dimensionality reduction techniques used in statistical mechanics to derive equations of motion for task-relevant observables [15]. Finally, differential privacy lies at the heart of the sufficient conditions proved in Theorem 1.

The bounded-rationality controller is just one example of differentially private mechanisms present in the literature, and other algorithms demonstrating this property in the context of optimization have appeared [26]. These mechanisms can be used to design new differentially private feedback policies that confer similar benefits, or, using the computational capabilities of differential privacy, they can be composed with standard optimal control algorithms to increase their robustness.

Regarding the necessary conditions described in Chapter 7 and Chapter 8, there are a number of ways to extend these frameworks to yield improved bounds and new applications of these necessary conditions not yet considered. First, all sensory and dynamical models considered, both theoretically and experimentally, are assumed to be accurate. Two possibilities to handle this shortcoming include adversarial search into the framework and using differential privacy, specifically Lemma 1, to quantify performance on nearby sensor models. Additionally, the feasibility of using a Lagrangian relaxation of the infimum over open-loop controllers to allow for continuous input spaces needs to be explored. An interesting application not considered thus far is to convert these necessary conditions into *certificates of robustness* against an adversary. In more detail, if these conditions are applied to an adversarial decision-maker trying to limit the robot’s performance, they can be optimized during the control synthesis process to maximize how much sensor information such an adversary requires to do so — thereby reducing the class of adversaries capable of harming the robot. This approach is similar to approaches taken in cryptography to convert hardness or impossibility results into robust security protocols [120].

Finally, these bounds consider a fairly large class of decision-making problems. Appealing to the physical intuition provided in Chapter 8, they only consider the irreversibility of measuring the system state. In the physics literature, Landauer’s limit has been further tightened by considering broader factors impacting the entropy production or reduction of a process. Some are fairly general, like the irreversibility of the plant dynamics [101, 106, 107, 119], while others appear by when examining common structure of dynamical systems, including geometric and topological structures of system flows [121], quantization [122], interacting subsystems [123], sensing constraints [124], and nonlinear systems for which non-Shannon measures of information (e.g. f -divergences) are required [125]. Using

the physical analogy demonstrated in Chapter 8, pertinent results can be adapted from the physics literature for use in robotics. Moreover, quantities unique to robotics, including memory requirements, degrees of freedom and actuation, workspace geometry and topology, and obstacle density have the potential to impact entropy production and their incorporation can lead to tighter bounds and a more complete description on the fundamental limits of robotics.

Appendix A

Overview of Peer Reviewed Publications

This appendix contains reproductions of the four following publications (the author’s name is in **boldface**):

- **Vincent Pacelli** and Anirudha Majumdar, “Task-driven estimation and control via information bottlenecks,” in *Proceedings of the International Conference on Robotics and Automation*, IEEE, 2019. [8] (Appendix B)
- **Vincent Pacelli** and Anirudha Majumdar, “Learning task-driven control policies via information bottlenecks,” in the *Proceedings of Robotics: System and Science*, 2020. [9] (Appendix C)
- **Vincent Pacelli** and Anirudha Majumdar, “Robust control under uncertainty via bounded rationality and differential privacy,” in the *Proceedings International Conference on Robotics and Automation*, IEEE, 2022. [7] (Appendix D)
- Anirudha Majumdar and **Vincent Pacelli**, “Fundamental performance limits for sensor-based robot control and policy learning,” in the *Proceedings of Robotics: System and Science*, 2022. [10] (Appendix E)

Additionally, an extended version of the last article is presently under peer review:

- Anirudha Majumdar, Zhiting Mei, and **Vincent Pacelli**, “Fundamental limits for sensor-based robot control,” submitted to the International Journal of Robotics Research (Under Review; available on ArXiv). [11].

This extension primarily concerns the material in Section 7.6. The summary of each author’s contributions to these articles is described as follows for each appendix:

Appendix B. A. Majumdar proposed the abstract problem of synthesizing task-relevant controllers for robotics applications and their anticipated robustness and computational benefits. **V. Pacelli** formalized this problem mathematically, derived Theorem 8 and Theorem 9, implemented and produced all simulation results and figures, with A. Majumdar serving in an advisory role. Moreover, Theorem 6 was a joint work by both authors.

Appendix C. The primary contribution of this article was the implementation of the task-relevant framework presented in Appendix B via machine learning on a suite of more challenging robotics problems. All results in this paper are due to **V. Pacelli**, with A. Majumdar serving in an advisory role.

Appendix D. The connection between task-relevant control and differential privacy, proof of the robustness result Theorem 12 and its supporting theory, and all simulation results are due to **V. Pacelli**. Again, A. Majumdar served in an advisory role in the development of this work.

Appendix E. The problem of identifying a task-relevant necessary condition on the sensing information required for control was considered by both A. Majumdar and **V. Pacelli**, but two methods for formulating such a condition were pursued in a largely independent manner by the authors. A. Majumdar proposed and developed the methodology based on Fano’s inequality which appears in Chapter 7, and is responsible for proving the theory, including, Theorem 14 and Theorem 16 and the experimental results in Appendix E.7.1 and Appendix E.7.2. **V. Pacelli** pursued the thermodynamic approach described in Chapter 8, however its connection as a generalization to the publication reproduced in this appendix was only discovered shortly before the writing of

this dissertation and has not appeared in publication. **V. Pacelli** is also responsible for implementing Appendix E.7.3 and the creation of all figures summarizing empirical results. In the extended version of this publication [11],¹ Z. Mei developed the extension of this work to the f -entropy (see Section 7.6), with advisory input from **V. Pacelli** and A. Majumdar.

¹The extension is not reproduced in this dissertation as it is under review and contains a significant overlap with the existing peer-reviewed version.

Appendix B

Task-Driven Estimation and Control via Information Bottlenecks

B.1 Introduction

State-of-the-art techniques for controlling robotic systems typically rely heavily on accurately estimating the full state of the system and maintaining rich geometric representations of their environment. For example, a common approach to navigation is to build a dense occupancy map produced by scanning the environment and to use this map for planning and control. Similarly, control for walking or running robots typically involves estimating the full state of the robot (e.g., joint angles, velocities, and position relative to a goal location). However, such representations are often overly detailed when compared to a *task-driven representation*.

One example of a task-driven representation is the “gaze heuristic” from cognitive psychology [126–128]. When attempting to catch a ball, an agent can estimate the ball’s position and velocity, model how it will evolve in conjunction with environmental factors like wind, integrate the pertinent differential equations, and plan a trajectory in order to arrive at the ball’s final location. In contrast, cognitive psychology studies have shown

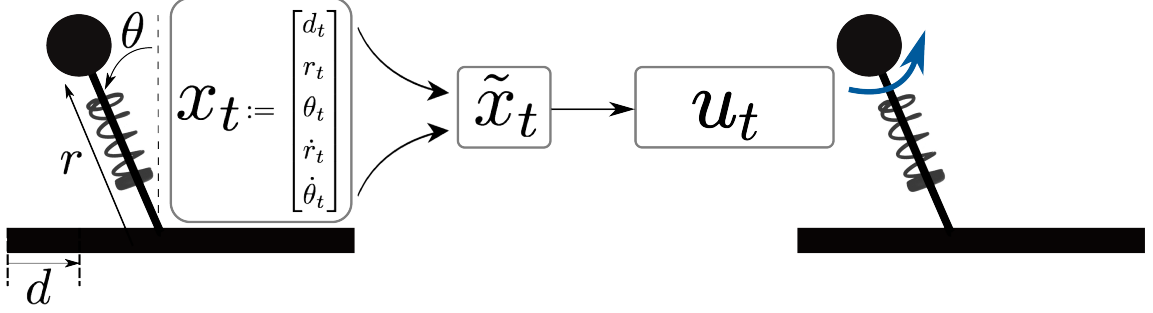


Figure B.1: A schematic of our technical approach. We seek to synthesize (offline) a minimalistic set of task-relevant variables (TRVs) $\tilde{x}_t$ that create a bottleneck between the full state x_t and the control input u_t . These TRVs are estimated online in order to apply the policy π_t . We demonstrate our approach on a spring-loaded inverted pendulum model whose goal is to run to a target location. Our approach automatically synthesizes a *one-dimensional* TRV $\tilde{x}_t$ sufficient for achieving this task.

that humans use a dramatically simpler strategy that entails maintaining the angle that the human’s gaze makes with the ball at a constant value. This method reduces a number of hard-to-monitor variables (e.g., wind speed) into a single easily-estimated variable. Modulating this variable alone results in accomplishing the task.

The gaze heuristic example highlights the two primary advantages of using a task-driven representation. First, a control policy that uses such a representation is more efficient to employ online since fewer variables need to be estimated. Second, since only a few prominent variables need to be estimated, fewer sources of measurement uncertainty result in a more robust policy. While one can sometimes manually design task-driven representations for a given task, we currently lack a principled theoretical and algorithmic framework for *automatically synthesizing* such representations. The goal of this paper is to develop precisely such an algorithmic approach.

Statement of Contributions. The main technical contribution of this paper is to formulate the synthesis of task-driven representations as an optimization problem using information bottleneck theory [129]. We present *offline* algorithms that encode the full state of the system into a set of *task-relevant variables (TRVs)* and simultaneously identify a performant policy (restricted to be a function of the TRVs) using novel iterative algorithms that exploit the structure of this optimization problem in a number of dynamical settings including discrete-state, linear-Gaussian, and nonlinear systems. We present *online* algo-

rithms for estimating the TRVs in order to apply the control policy. We demonstrate that our approach yields policies that are robust to *unmodeled* measurement uncertainty both theoretically (using results from the theory of risk metrics) and in a number of simulation experiments including running using a spring-loaded inverted pendulum (SLIP) model (Fig.B.1).

B.1.1 Related Work

By far the most common approach in practice for controlling robotic systems with nonlinear dynamics and partially observable state is to *independently* design an estimator for the *full* state (e.g., a Kalman filter [66]) and a controller that assumes perfect state information (e.g., designed using robust control techniques such as H_∞ control [130], sliding mode control [131, 132], passivity-based control [133, 134], or Lyapunov-based control [132, 134]). While this strategy is optimal for Linear-Quadratic-Gaussian (LQG) problems due to the *separation principle* [1], it can produce a brittle system due to errors in the state estimate for nonlinear systems (since the separation principle does not generally hold in this setting). We demonstrate that our task-driven approach affords significant robustness when compared to approaches that perform full state estimation and control assuming the separation principle (see Section B.5 for numerical examples). Moreover, in contrast to traditional robust estimation and control techniques, our approach does not rely on explicit models of measurement uncertainty. We demonstrate that robustness can be achieved implicitly as a *by-product* of task-driven representations.

Historically, the work on designing *information constrained* controllers has been pursued within the networked control theory literature [135–137]. Recently, the optimal co-design of data-efficient sensors and performant controllers has also been explored beyond network applications. One set of approaches — inspired by the cognitive psychology concept of *bounded rationality* [138, 139] — is to limit the information content of a control policy measured with respect to a default stochastic policy [32, 48, 49, 51, 140]. Another set of examples comes from the sensor selection problem in robotics, which involves selecting a minimal number of sensors or features to use for estimating the robot’s state [141–143]. While the work highlighted above shares our goal of designing “minimalistic” (i.e., infor-

mationally frugal) controllers, our approach here is fundamentally different. In particular, our goal is to design task-driven *representations* that form abstractions which are sufficient for the purpose of control. Online estimation and control is performed purely based on this representation without resorting to estimating the full state of the system (in contrast to the work highlighted above, which either assumes access to the full state or designs estimators for it).

A number of previous authors consider the construction of minimal-information representations. In information theory, this approach is typically referred to as the Information Bottleneck (IB) [129]. Recently, these ideas have been co-opted for designing control policies. In [144], a learning-based approach is suggested to find minimal-information state representations and control policies which use them. Our work differs in that we provide *analytic* (i.e., model-based) methods for finding such representations and policies and we explicitly characterize the resulting robustness. Another branch of work considers the construction of LQG policies that achieve a performance goal while minimizing an information-theoretic quantity such as the mutual information between inputs and outputs [33, 145] or Massey’s directed information [146, 147]. In contrast to these works, our derivation handles nonlinear systems and also presents robustness results for the resulting controllers, which have not been discussed to our knowledge in existing literature.

The work on *actionable information* in vision [148, 149] attempts to find invariant and task-relevant representations for visual tasks in the form of *minimal complete representations* (minimal sufficient statistics of a full or “complete” representation). While highly ambitious in scope, this work has largely been devoted to studying visual decision tasks (e.g., recognition, detection, categorization). The algorithmic approach taken in [148, 149] is thus tied to the specifics of visual problems (e.g., designing visual feature detectors that are invariant to nuisance factors such as contrast, scale, and translation). Our goals and technical approach here are complementary. We do not specifically target visual decision problems; instead, we seek to develop a general framework that is applicable to a broad range of robotic control problems and allows us to *automatically* synthesize task-driven representations (without manually designing feature detectors).

B.2 Task-Driven Representations and Controllers

Our goal in this section is to formally define the notion of a “task-driven representation” and formulate an optimization problem that automatically synthesizes such representations. We focus on control tasks in this paper and assume that a task is defined in terms of a (partially observable) Markov decision process ((PO)MDP). Let $x_t \in \mathcal{X}$, $u_t \in \mathcal{U}$, $y_t \in \mathcal{Y}$ denote the full state of the system, the control input, and the sensor output at time t respectively. Here the state space $\mathcal{X}$, input space $\mathcal{U}$, and output space $\mathcal{Y}$ may either be discrete or continuous. We assume that the dynamics and sensor model of the system are known and given by potentially time-varying conditional distributions $p_t(x_{t+1}|u_t, x_t)$, $\sigma_t(y_t|x_t)$ respectively. Let $c_0(x_0, u_0), c_1(x_1, u_1), \dots, c_{T-1}(x_{T-1}, u_{T-1}), c_T(x_T)$ be a sequence of cost functions that encode the robot’s desired behavior. The robot’s goal is to identify a control policy $\pi_t(u_t|y_t)$ that minimizes the expected value of these cost functions when the policy is executed *online*, i.e. minimize $\sum_{t=0}^T \mathbb{E}_{p_t} c_t(x_t, u_t) + \mathbb{E}_{p_T} c_T(x_T)$. In general, this optimization problem is infinite dimensional and challenging to solve.

The key idea behind our technical approach is to define a principled information-theoretic notion of *minimalism* of representations for tasks. Figure B.1 illustrates the main idea for doing this. Here $\tilde{x}_t \in \tilde{\mathcal{X}}$ are *task-relevant variables* (TRVs) that constitute a *representation*; one can think of the representation as a “sketch” of the full state that the robot uses for computing control inputs via $\pi_t(u_t|\tilde{x}_t)$. Ideally, such a representation filters out all the information from the state that is not relevant for computing control inputs — avoiding the introduction of unnecessary estimation error. A minimalist representation $\tilde{x}_t$ should thus create a *bottleneck* between the full state and the control input. We make this notion precise by leveraging the theory of information bottlenecks [129] and finding a stochastic mapping $q_t(\tilde{x}_t|x_t)$ that minimizes the *mutual information* between x_t and $\tilde{x}_t$

$$\mathbb{I}(x_t; \tilde{x}_t) := \mathbb{D}(p_t(x_t, \tilde{x}_t) \| p_t(x_t)p_t(\tilde{x}_t)), \quad (\text{B.1})$$

while ensuring the policy performs well (i.e., achieves low expected cost). Here, $\mathbb{D}(\cdot\|\cdot)$ represents the Kullback-Leibler (KL) divergence between two distributions. Intuitively,

minimizing the mutual information corresponds to designing TRVs $\tilde{x}_t$ that are as independent of the state x as possible. The map $q_t(\tilde{x}_t|x_t)$ is thus “squeezing out” all the irrelevant information from the state while maintaining enough information for choosing good control inputs. This representation formalizes our notion of a *task-driven representation*.

Formally, we pose the problem of finding task-driven representations as the following *offline* optimization problem, which we refer¹ to as (OPT):

$$\min_{p_t, q_t, \pi_t} \sum_{t=0}^T \left[\mathbb{E}_{p_t} c_t(x_t, u_t) + \frac{1}{\beta} \mathbb{I}(x_t; \tilde{x}_t) \right]. \quad (\text{OPT})$$

We note that the unconstrained problem (OPT) is equivalent to a constrained version where the mutual information is minimized subject to a constraint on the expected cost of the full-state MDP and β is the corresponding Lagrange multiplier. A heuristic approach for choosing an appropriate value for β is discussed in Appendix B.5.

So far, we have limited our discussion to the case where the robot has access to the full state of the system. However, the real benefit of the task-driven perspective is evident in the partially observable setting, where the robot only indirectly observes the state of the system via sensor measurements $y_0, \dots, y_t$, where $y_t \in \mathcal{Y}_t$. We denote the probability of observing a particular measurement in a given state by $\sigma_t(y_t|x_t)$. The prevalent approach for handling such settings is to design an estimator for the full state x_t . Our key idea here is to perform estimation for the TRVs $\tilde{x}_t$ instead of x_t . Specifically, our overall approach has two phases:

Offline. Synthesize the maps $\{q_t, \pi_t\}_{t=0}^{T-1}$ by solving (OPT).

Online. Estimate current TRV $\tilde{x}_t$ using sensor measurements and use this estimate to compute inputs via $\pi_t(u_t|\tilde{x}_t)$.

Perhaps the clearest benefit of our approach is the fact that the representation $\tilde{x}_t$ may be significantly lower-dimensional than the full state of the system; this can lead to significant reductions in online computations. Another advantage of the task-driven approach is robustness to estimation errors. To see this, let $p_t(x_t, \tilde{x}_t, u_t)$ denote the joint distribution

¹Technically, (OPT) is a kind of rate-distortion problem, not an information bottleneck problem, as the constraint is not specified using a divergence as a distortion function. However, $q_t(\tilde{x}_t|x_t)$ limits the flow of information from x_t to u_t so we use the term bottleneck as a conceptual aid.

over the state, representation, and inputs at time t that results when we apply the policy obtained by solving Problem (OPT) in the fully observable setting. Now, let $\tilde{p}_t(x_t, \tilde{x}_t, u_t)$ denote the distribution for the partially observable setting, i.e., when we estimate $\tilde{x}_t$ online using the robot's sensor measurements and use this estimate to compute control inputs.

Theorem 6. Let $\tilde{p}_t(x_t, \tilde{x}_t, u_t)$ be the distribution resulting from any estimator that satisfies the following condition:

$$\begin{aligned} \mathbb{D}(\tilde{p}_t(x_t, \tilde{x}_t, u_t) \| p_t(x_t, \tilde{x}_t, u_t)) \\ \leq \frac{1}{\beta} \mathbb{D}(p_t(x_t, \tilde{x}_t) \| p_t(x_t) p_t(\tilde{x}_t)). \end{aligned} \quad (\text{B.2})$$

Then, we have the following upper bound on the total expected cost:

$$\sum_{t=0}^T \mathbb{E}_{\tilde{p}_t} c_t(x_t, u_t) \leq \sum_{t=0}^T \left[\rho(c_t(x_t, u_t)) + \frac{1}{\beta} \mathbb{I}(x_t; \tilde{x}_t) \right], \quad (\text{B.3})$$

where ρ is the *entropic risk metric* [150, Example 6.20]:

$$\rho(c_t(x_t, u_t)) := \log \left[\mathbb{E}_{p_t} \exp(c_t(x_t, u_t)) \right]. \quad (\text{B.4})$$

Proof. By the well-known Donsker-Varadhan change of measure formula [151, Theorem 2.3.2], we have:

$$\begin{aligned} \mathbb{E}_{\tilde{p}_t} c_t(x_t, u_t) &\leq \log \left[\mathbb{E}_{p_t} \exp(c_t(x_t, u_t)) \right] \\ &\quad + \mathbb{D}(\tilde{p}_t(x_t, \tilde{x}_t, u_t) \| p_t(x_t, \tilde{x}_t, u_t)). \end{aligned} \quad (\text{B.5})$$

Then, using condition (B.2) and inequality (B.5), we obtain:

$$\begin{aligned} \mathbb{E}_{\tilde{p}_t} c_t(x_t, u_t) &\leq \log \left[\mathbb{E}_{p_t} \exp(c_t(x_t, u_t)) \right] \\ &\quad + \frac{1}{\beta} \mathbb{D}(p_t(x_t, \tilde{x}_t) \| p_t(x_t) p_t(\tilde{x}_t)) \end{aligned} \quad (\text{B.6})$$

$$= \rho(c_t(x_t, u_t)) + \frac{1}{\beta} \mathbb{I}(x_t; \tilde{x}_t). \quad (\text{B.7})$$

Summing over time gives us the desired result.

□

Intuitively, this theorem shows that *any* estimator for $\tilde{x}_t$ (in the partially observable setting) that results in a distribution $\tilde{p}_t(x_t, \tilde{x}_t, u_t)$ that is “close enough” to the distribution $p_t(x_t, \tilde{x}_t, u_t)$ in the fully observable case (i.e., when their KL divergence is less than $\frac{1}{\beta}$ times the KL divergence between $p_t(x_t, \tilde{x}_t)$ and the joint distribution $p_t(x_t)p_t(\tilde{x}_t)$ over x_t and $\tilde{x}_t$ that results when $\tilde{x}_t$ is assumed to be *independent* of x_t), the expected cost of the controller in the partially observable case is *guaranteed* to be bounded by the right hand side (RHS) of (B.3). Notice that this RHS is similar to the cost function of (OPT). In particular, the expected value operator is a *linearization* of the entropic risk metric² ρ . By solving (OPT), we are minimizing (a linear approximation of) an upper bound on the expected cost even when our state is only partially observable (as long as our estimator for $\tilde{x}_t$ ensures condition (B.2)). Once (OPT) is solved, we can use Theorem 6 to obtain a robustness bound by evaluating the RHS of (B.3).

B.3 Algorithms for Synthesizing Representations

In this section, we outline our approach for solving (OPT) offline. We note that (OPT) is non-convex in general. While one could potentially apply general non-convex optimization techniques such as gradient-based methods, computing gradients quickly becomes computationally expensive due to the large number of decision variables involved (even in the setting with finite state and action spaces, we have decision variables corresponding to $q_t(\tilde{x}_t|x_t)$ and $\pi_t(u_t|\tilde{x}_t)$ for every possible value of $x_t, \tilde{x}_t$ and u_t at every time step). Our key insight here is to exploit the structure of (OPT) to propose an efficient iterative algorithm in three different dynamical settings: discrete, linear-Gaussian, and nonlinear-Gaussian. These settings are particularly convenient to work with because they allow the objective of (OPT) to be *computed in closed-form*. In each setting, the algorithm iterates over the following three steps:

²This risk metric has a long history in robust control (including a close link to H_∞ control) [152–154]. We note, however, that it can sometimes be conservative in cases with rare, bad events.

1. Fix $\{q_t, \pi_t\}_{t=0}^{T-1}$ and solve for $\{p_t\}_{t=0}^T$ using the forward dynamical equations.
2. Fix $\{p_t\}_{t=0}^T, \{\pi_t\}_{t=0}^{T-1}$ and solve for $\{q_t\}_{t=0}^{T-1}$ by satisfying necessary conditions for optimality.
3. Fix $\{p_t\}_{t=0}^T, \{q_t\}_{t=0}^{T-1}$ and solve for $\{\pi_t\}_{t=0}^{T-1}$ by solving a convex optimization problem.

In our implementation, we iterate over these steps until convergence (or until an iteration limit is reached). This is a common strategy employed in solving similar kinds of MDPs with information-theoretic objectives [33, 155]. While we cannot currently guarantee convergence, our iterative procedure is extremely efficient (since all the computations above can be performed either in closed-form or via a convex optimization problem) and produces good solutions in practice (see Section B.5). We describe instantiations of each step for three different dynamical settings below.

B.3.1 Discrete Systems

In order to solve Step 1, note that the forward dynamics of the system for fixed $q_t(\tilde{x}_t|x_t)$ and $\pi_t(u_t|\tilde{x}_t)$ are given by:

$$\begin{aligned}
p_t(x_{t+1}|x_t) &= \sum_{u, \tilde{x}} p_t(x_{t+1}|x_t, u) \pi_t(u|\tilde{x}) q_t(\tilde{x}|x_t), \\
p_{t+1}(x_{t+1}) &= \sum_x p_t(x_{t+1}|x) p_t(x).
\end{aligned} \tag{B.8}$$

The Lagrangian functional for (OPT) is $\mathcal{L} = \sum_{t=0}^T \mathcal{L}_t$ where

$$\begin{aligned}
\mathcal{L}_t &= \sum_{\tilde{x}, u, x} c_t(x, u) \pi_t(u|\tilde{x}) q_t(\tilde{x}|x) p_t(x) \\
&\quad - \sum_{x'} \nu_{t+1}(x') \left(p_{t+1}(x') - \sum_{x, u, \tilde{x}} p_t(x'|x, u) \pi_t(u|\tilde{x}) q_t(\tilde{x}|x) p_t(x) \right) \\
&\quad + \frac{1}{\beta} \sum_{x, \tilde{x}} q_t(\tilde{x}|x) p_t(x) \log \left(\frac{q_t(\tilde{x}|x)}{q_t(\tilde{x})} \right).
\end{aligned}$$

where $\nu_t(x_t)$ are Lagrange multipliers. The Lagrange multipliers that normalize distribution variables are omitted since they do not contribute to the analysis. The following proposition demonstrates the structure of $q_t(\tilde{x}_t|x_t)$ using the first-order necessary condition (FONC) for

optimality [114].

Theorem 7. A necessary condition for $q_t(\tilde{x}|x)$ to be optimal for (OPT) is that

$$q_t(\tilde{x}_t|x_t) = \frac{q_t(\tilde{x}_t) \exp(-\beta \mathbb{E}(\nu_{t+1} + c_t|x_t, \tilde{x}_t))}{Z_t(x_t)}, \quad (\text{B.9})$$

where $\nu_T(x_T) = c_T(x_T)$ and

$$\nu_t(x_t) = \mathbb{E}(c_t + \nu_{t+1}|x_t) + \frac{1}{\beta} \mathbb{D}(q_t(\tilde{x}_t|x_t) || q_t(\tilde{x}_t)), \quad (\text{B.10})$$

$$Z_t(x_t) = \sum_{\tilde{x}} q_t(\tilde{x}) \exp(-\beta [\mathbb{E}(c_t + \nu_{t+1}|x_t, \tilde{x})]). \quad (\text{B.11})$$

Proof. See Appendix B.7.1. □

This proposition demonstrates that $q_t(\tilde{x}_t|x_t)$ is a *Boltzmann distribution* with $Z_t(x_t)$ as the *partition function* and β playing the role of inverse temperature. In order to solve Step 2, we simply evaluate the closed-form expression (B.9).

It is easily verified that the function $\nu_t(x_t)$ is the *cost-to-go function* for (OPT). Thus (OPT) can be written as a dynamic programming problem using $\nu_t(x_t)$:

$$\min_{q_t, \pi_t} \mathbb{E}_{p_t} \left(c_t + \nu_{t+1} + \frac{1}{\beta} \mathbb{D}(q_t(\tilde{x}_t|x_t) || q_t(\tilde{x}_t)) \right). \quad (\text{DP})$$

This allows us to solve Step 3. In particular, when $p_t(x_t), q_t(\tilde{x}_t|x_t)$ are fixed, (DP) is a linear programming problem in π_t and can thus be solved efficiently.

B.3.2 Linear-Gaussian Systems with Quadratic Costs

A discrete-time linear-Gaussian (LG) system is defined by the transition system

$$x_{t+1} = A_t x_t + B_t u_t + \epsilon_t, \quad \epsilon_t \sim N(0, \Sigma_{\epsilon_t}), \quad (\text{B.12})$$

where $\mathcal{X} = \mathbb{R}^n$, $\mathcal{U} = \mathbb{R}^m$ and $x_0 \sim N(\bar{x}_0, \Sigma_{x_0})$. We assume that the cost function is quadratic:

$$c_t(x, u) := \frac{1}{2} (x - g_t)^T Q_t (x - g_t) + \frac{1}{2} (u - w_t)^T R_t (u - w_t),$$

with $Q_t, R_t \succeq 0, R_T = 0$. We explicitly parameterize the TRVs and control policy as:

$$\tilde{x}_t = C_t x_t + a_t + \eta_t, \quad u_t = K_t \tilde{x}_t + h_t, \quad (\text{B.13})$$

where the random variable $\eta_t \sim N(0, \Sigma_{\eta_t})$ is additive process noise. This structure dictates that $p_t(x_t), q_t(\tilde{x}_t|x_t)$ are Gaussians $N(\bar{x}_t, \Sigma_{x_t}), N(\bar{\tilde{x}}_t, \Sigma_{\tilde{x}_t})$ respectively, with $\bar{\tilde{x}}_t = C_t \bar{x}_t + a_t, \Sigma_{\tilde{x}_t} = C_t \Sigma_{x_t} C_t^T + \Sigma_{\eta_t}$. This allows for both Steps 1 and 2 to be computed in closed form. The latter is presented in the following theorem.

Theorem 8. Define the notational shorthand $G_t := C_t^T (C_t \Sigma_{x_t} C_t^T + \Sigma_{\eta_t})^{-1} C_t, M_t := (A_t + B_t K_t C_t)$. For the LG system, the necessary condition (B.9) is equivalent to the conditions

$$\begin{aligned} C_t &= -\beta \Sigma_{\eta_t} K_t^T B_t^T P_{t+1} A_t, \\ a_t &= -\Sigma_{\eta_t} (\beta K_t^T B_t^T (b_{t+1} + P_{t+1} B_t h_t) \\ &\quad + \beta K_t^T R_t (h_t - w_t) - \Sigma_{\tilde{x}_t}^{-1} \bar{\tilde{x}}_t), \\ \Sigma_{\eta_t}^{-1} &= \Sigma_{\tilde{x}_t}^{-1} + \beta K_t^T (B_t^T P_{t+1} B_t + R_t) K_t, \end{aligned} \quad (\text{B.14})$$

where the cost-to-go function is the recursively defined quadratic function $\nu_t(x) = \frac{1}{2} x_t^T P_t x_t + b_t^T x_t + \text{constant}$ with values $P_T = Q_{T+1}, b_T = -Q_T g_T$ and

$$\begin{aligned} P_t &= Q_t + \beta^{-1} G_t + C_t^T K_t^T R_t K_t C_t + M_t^T P_{t+1} M_t, \\ b_t &= M_t^T P_{t+1} B_t (h_t + K_t a_t) - Q_t g_t - \beta^{-1} G_t \bar{x}_t \\ &\quad + C_t^T K_t^T R_t (K_t a_t + h_t - w_t) + M_t^T b_{t+1}. \end{aligned} \quad (\text{B.15})$$

Proof. See Appendix B.7.2. □

Finally, when q_t, p_t are fixed, (DP) is the unconstrained convex quadratic program

$\min_{K_t, h_t} J(K_t, h_t)$ where

$$\begin{aligned}
J(K_t, h_t) &:= \frac{1}{2}(K_t \bar{\bar{x}}_t + h_t - w_t)^T R_t (K_t \bar{\bar{x}}_t + h_t - w_t) \\
&+ \frac{1}{2} \text{tr} \left\{ \Sigma_{\bar{\bar{x}}_t} K_t^T R_t K_t \right\} + \frac{1}{2} \bar{x}_t^T A_t^T P_{t+1} (A_t \bar{x}_t + 2B_t h_t) \\
&+ \frac{1}{2} \bar{\bar{x}}_t^T K_t^T B_t^T P_{t+1} B_t (K_t \bar{\bar{x}}_t + 2h_t) + \frac{1}{2} f_t^T B_t^T P_{t+1} B_t h_t \\
&+ \frac{1}{2} \bar{x}_t^T (A_t^T P_{t+1} B_t K_t C_t + C_t^T K_t B_t P_{t+1} A_t) \bar{x}_t \\
&+ \bar{x}_t^T A_t^T P_{t+1} B_t K_t a_t + \frac{1}{2} \text{tr} \left\{ \Sigma_{\bar{\bar{x}}_t} K_t^T B_t^T P_{t+1} B_t K_t \right\} \\
&+ b_{t+1}^T (A_t \bar{x}_t + B_t K_t \bar{\bar{x}}_t + B_t h_t) \\
&+ \frac{1}{2} \text{tr} \left\{ \Sigma_{x_t} A_t^T P_{t+1} A_t \right\} + \frac{1}{2} \text{tr} \left\{ \Sigma_{x_t} C_t^T K_t^T B_t^T P_{t+1} A_t \right\} \\
&+ \frac{1}{2} \text{tr} \left\{ \Sigma_{x_t} A_t^T P_{t+1} B_t K_t C_t \right\}.
\end{aligned} \tag{B.16}$$

This program can be solved very efficiently (e.g., using active-set or interior point methods) [156].

B.3.3 Nonlinear-Gaussian Systems

When the dynamics are nonlinear-Gaussian (NLG), i.e. when (B.12) is changed to

$$x_{t+1} = f(x_t, u_t) + \epsilon_t, \quad \epsilon_t \sim N(0, \Sigma_{\epsilon_t}), \tag{B.17}$$

minimizing (OPT) is challenging due to $p_t(x_t)$ no longer being Gaussian. We tackle this challenge by leveraging our results for the LG setting and adapting the iterative Linear Quadratic Regulator (iLQR) algorithm [69, 157, 158].

Given an initial nominal trajectory $\{\hat{x}_t, \hat{u}_t\}_{t=0}^T$, the matrices $\{A_t, B_t\}_{t=0}^{T-1}$ are produced by linearizing $f(x_t, u_t)$ along the trajectory. The pair (A_t, B_t) describes the dynamics of a perturbation $\delta x_t = x_t - \hat{x}_t$ in the neighborhood of x_t for a perturbed input $\delta u_t = u_t - \hat{u}_t$ in the neighborhood of u_t :

$$\delta x_{t+1} = A_t \delta x_t + B_t \delta u_t + \epsilon_t, \quad \delta x_0 \sim N(0, \Sigma_{x_t}). \tag{B.18}$$

We compute (a quadratic approximation of) the perturbation costs $\delta c_t(\delta x, \delta u) := c_t(\hat{x}_t + \delta x, \hat{u}_t + \delta u)$ subject to (B.18). We can then apply the solution method outlined in Section B.3.2 to search for an optimal $\{\delta x_t, \delta u_t\}_{t=0}^T$. We then update the nominal state and input trajectories to $\{\hat{x}_t + \delta x_t, \hat{u}_t + \delta u_t\}_{t=0}^T$ and repeat the entire process until the nominal trajectory converges.

B.4 Online Estimation and Control

Once the task-driven representation and policy have been synthesized offline, we can leverage them for computationally-efficient and robust online control. The key idea behind our online approach is to use the robot’s sensor measurements $\{y_i\}_{i=1}^t$ to *only* estimate the TRVs $\tilde{x}_t$. Once $\tilde{x}_t$ has been estimated, the control policy $\pi_t(u_t|\tilde{x}_t)$ can be applied. This is in stark contrast to most prevalent approaches for controlling robotic systems, which aim to accurately estimate the full state x_t . We describe our online estimation approach below.

We maintain a belief distribution $\text{bel}(\tilde{x}_t)$ over the TRV-space $\tilde{\mathcal{X}}$ and update it at each time t using a Bayes filter [66]. Specifically, we perform two steps every t :

1. **Process Update.** The system model is used to update the belief-state to the current time step: $\overline{\text{bel}}(\tilde{x}_t) = \sum_{\tilde{x}_{t-1}} q_{t-1}(\tilde{x}_t|\tilde{x}_{t-1}, u_{t-1})\text{bel}(\tilde{x}_{t-1})$.
2. **Measurement Update.** The measurement model is used to integrate the observation y_t into the belief-state: $\text{bel}(\tilde{x}_t) \propto \sigma_t(y_t|\tilde{x}_t)\overline{\text{bel}}(\tilde{x}_t)$.

To apply this filter, the distributions $q_t(\tilde{x}_{t+1}|\tilde{x}_t, u_t)$ and $\sigma_t(y_t|\tilde{x}_t)$ are precomputed *offline*. Bayes’ theorem states $p_t(x_t|\tilde{x}_t) = q_t(\tilde{x}_t|x_t)p_t(x_t)/q_t(\tilde{x}_t)$. Consequently,

$$\begin{aligned} p_t(x_{t+1}|\tilde{x}_t, u_t) &= \sum_{x_t} p_t(x_{t+1}|u_t, x_t)p_t(x_t|\tilde{x}_t), \\ q_t(\tilde{x}_{t+1}|\tilde{x}_t, u_t) &= \sum_{x_{t+1}} q_t(\tilde{x}_{t+1}|x_{t+1})p_t(x_{t+1}|\tilde{x}_t, u_t), \\ \sigma_t(y_t|\tilde{x}_t) &= \sum_{x_t} \sigma_t(y_t|x_t)p_t(x_t|\tilde{x}_t). \end{aligned}$$

In the discrete case, the above equations can be evaluated directly. In the LG case, the measurement and update steps take the form of the traditional Kalman updates applied to

an LG system induced on the TRV-space by q_t and the system dynamics. This structure is elucidated in the following theorem.

Theorem 9. The measurement and process updates for a Bayesian filter on the TRVs $\{\tilde{x}_t\}_{t=0}^T$ in the LG case are the Kalman filter measurement and process updates for the LG system

$$\tilde{x}_{t+1} = \tilde{A}_t \tilde{x}_t + \tilde{B}_t u_t + \tilde{r}_t + \tilde{\epsilon}_t, \quad (\text{B.19})$$

$$\tilde{y}_t = \tilde{D}_t \tilde{x}_t + \tilde{\omega}_t, \quad (\text{B.20})$$

where

$$\begin{aligned} \tilde{A}_t &:= C_{t+1} A_t \Sigma_{x_t} C_t^T \Sigma_{\tilde{x}_t}^{-1}, & \tilde{C}_t &:= D_t \Sigma_{x_t} C_t^T \Sigma_{\tilde{x}_t}^{-1}, \\ \tilde{B}_t &:= C_{t+1} B_t, & \tilde{\omega}_t &\sim N(0, \Sigma_{y_t|\tilde{x}_t}), \\ \tilde{r}_t &:= -\tilde{A}_t \tilde{x}_t + C_{t+1} A_t \bar{x}_t + a_{t+1}, & \tilde{\epsilon}_t &\sim N(0, \Sigma_{\tilde{x}_t|\tilde{x}_t, u_t}). \end{aligned}$$

Proof. See Appendix B.7.3. □

In the NLG case, we use an extended Kalman filter over $\delta\tilde{x}_t$ (the perturbed TRV). Specifically, we use the linearized dynamics of $\delta\tilde{x}_t$ and apply the LG systems approach.

Given a belief $\text{bel}(\tilde{x}_t)$, we compute the control input by sampling $u_t \sim \pi_t(u_t|\tilde{x}_t^*)$, where $\tilde{x}_t^*$ is the maximum likelihood TRV $\tilde{x}_t^* := \max_{\tilde{x}_t} \text{bel}(\tilde{x}_t)$. Alternatively, one can sample the TRV from $q_t(\tilde{x}_t|x_t)$, but the MLE method is similar to how many Bayesian filters are implemented (e.g. Kalman filters) and allows for a more direct comparison.

B.5 Examples

In this section, we demonstrate the efficacy of our task-driven control approach on both a discrete-state scenario and a SLIP model. To select the value of β , the algorithm is run 10 times with β set to evenly spaced values in a listed interval. The controller used is the one with the lowest value of β subject to the expected cost of the controller being

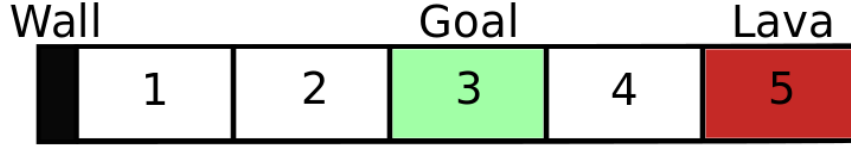


Figure B.2: The *lava scenario* [159] consists of five states connected in a line. The robot is allowed to move one step in either direction unless it enters the lava pit on the far right, which is absorbing. The robot receives a reward of 5 points upon entering the goal state and a penalty reward of -1 point otherwise. The terminal rewards are 10 points when in the goal and -10 points when in the lava.

below a particular value. This choice selects the performant controller with the least state information in the TRVs.

B.5.1 Lava Problem

The first example (Fig. B.2), adapted from [159] demonstrates a setting where the separation principle *does not hold*. If the robot’s belief distribution is $[0.3, 0.4, 0.0, 0.3, 0.0]^T$ while residing in state 4, the optimal action corresponding to the MLE of the state is to move right — not the optimal left.

Our algorithm was run with three TRVs and $\beta \in [0.001, 1]$ for 30 iterations. The value $\beta = 0.001$ was found to give a negative expected cost. The robot’s initial condition was sampled from the aforementioned belief distribution. Online, the robot was modeled to have a faulty sensor localizes to the correct state with a probability of 0.5, with a uniformly random incorrect state returned otherwise.

Fig. B.3 compares our algorithm’s performance with a separation principle approach, i.e. solving the MDP with perfect state information and then performs MLE for the state online. Interestingly, our algorithm produces a *deterministic policy* that moves the robot left three times — ensuring it is in state 1 — then moves right twice and remains in the goal. With this policy, it is impossible for the robot to enter the lava state, producing *a more performant, lower variance trajectory than the separation principle-based solution under measurement noise*. The solution is deterministic at low values of β because the distribution in (B.9) is almost uniform — thereby requiring the policy to be effectively open-loop.

B.5.2 SLIP Model Problems

Next, we apply the NLG variant of our algorithm to the SLIP model [160–162], which is depicted in Fig. B.1. The SLIP model is common in robotics for prototyping legged locomotion controllers. It consists of a single leg whose lateral motion is derived from a spring/piston combination. At touchdown, the state of the robot is given by $[d, \theta, \dot{r}, \dot{\theta}]^T$ where d is displacement of the head from the origin, θ is the touchdown angle, and $\dot{r}, \dot{\theta}$ are the radial and angular velocities. The system input is $\Delta\theta$, the change in the next touchdown angle. The parameters are the head mass, $M = 1$, the spring constant, $k = 300$, gravity $g = 9.8$, and leg length $r_{max} = 1$. Despite the model’s simplicity, the touchdown return map eludes a closed-form description, so MATLAB’s `ode45` is used to compute and linearize the return map.

The goal is to place the head of the robot at $d = 3.2$ after three hops. This experiment is based on a set of psychology experiments that examined the cognitive information used by humans for foot placement while running [163, 164]. Our NLG algorithm was run with $\beta \in [1, 200]$, control cost matrices $R_t = 10$ for all t , and a terminal state cost as the squared distance of the robot from $d = 3.2$. The initial distribution was Gaussian with mean $[0, 0.3927, -3.273, -6.788]^T$, which is in the vicinity of a fixed point of the return map, and covariance $10^{-3}I$. The process covariance was $\Sigma_{\epsilon_t} = 10^{-4}\text{diag}(1, 0.1, 0.5, 0.5)$. Here, $\beta = 23.11$.

The results for our simulation are shown in Fig. B.4. The algorithm is compared with iLQG solutions with correct and incorrect measurement models. The believed measurement model was a noisy version of the state with covariance $\Sigma_{\omega_t} = 10^{-4}I$ while the actual measurement model used $\Sigma_{\omega_t} = 10^{-3}S^T S$ where the entries of S sampled from a standard uniform distribution each trial. The correct iLQG solution is a locally optimal solution to the problem due to the separation principle. However, when modeling error is introduced, the iLQG solution’s performance degrades rapidly. Meanwhile, *the TRV-based control policy is a reliable (i.e. lower variance) and performant control strategy despite this modeling error.* In addition, the solution found by our algorithm satisfies $\text{rank}(C_t) = 1$ for all t . Therefore, *the online estimator needs to only track a single TRV corresponding to this subspace.*

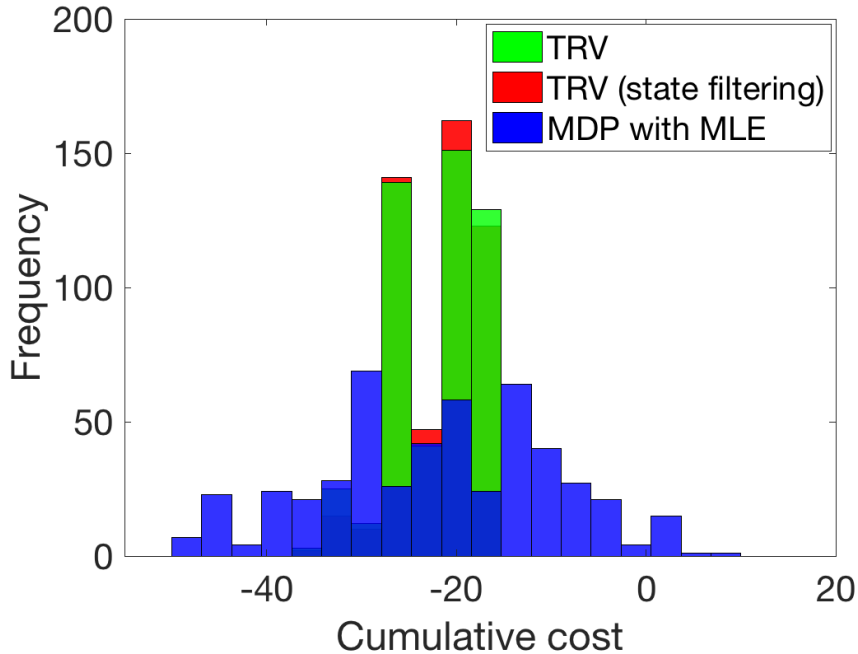


Figure B.3: This figure summarizes the outcome of 500 simulations of the Lava Problem with different control strategies. Each controller used a Bayesian filter to track the current belief distribution. The exact MDP solution (blue) applied the input corresponding to its MLE state. The TRV solutions sampled from the conditional distribution corresponding to their stochastic control policies given the MLE estimates of the state (red) or TRV (green).

B.6 Conclusion

We presented an algorithmic approach for task-driven estimation and control for robotic systems. In contrast to prevalent approaches for control that rely on accurate full-state estimation, our approach synthesizes a set of task-relevant variables (TRVs) that are sufficient for achieving the task. Our key insight is to pose the search for TRVs as an information bottleneck optimization problem. We solve this problem offline to identify TRVs and a control policy based on the TRVs. Online, we only estimate the TRVs to apply the policy. Our theoretical results suggest that this approach affords robustness to *unmodeled* measurement uncertainty. This is validated by thorough simulations, including a SLIP model running to a target location. Our simulations also demonstrate that our approach finds highly compressed TRVs (e.g. a *one-dimensional* TRV for the SLIP model).

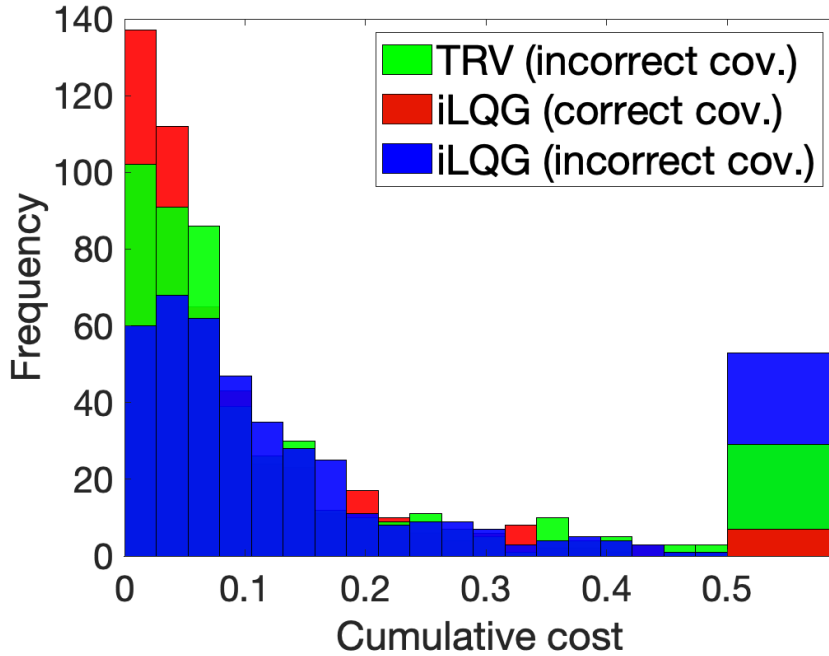


Figure B.4: This figure summarizes the outcome of 500 simulations of the SLIP Problem with different control strategies. Measurement covariance matrices were randomly sampled. For the iLQG control policy, a Kalman filter tracked the current state estimate, and the corresponding control was applied. For the TRV policy, a Kalman filter was maintained on the TRVs, and the control input corresponding to the MLE TRV was applied.

Challenges and Future Work. On the algorithmic front, we plan to develop approaches that directly minimize the RHS of (B.3) instead of a linear approximation of it. We expect that this may lead to improved robustness (as suggested by Theorem 6). On the practical front, we plan to implement our approach on a hardware platform that mimics the gaze heuristic and other examples including navigation problems (where the full state includes a description of the environment, e.g. in terms of an occupancy map). Perhaps the most exciting direction is to explore *active* versions of our approach where the control policy minimizes task-relevant uncertainty, in contrast to current approaches (e.g., belief space planning) that minimize full-state uncertainty.

We believe that the approach presented in this paper along with the indicated future directions represent an important step towards developing a principled, general framework for task-driven estimation and control.

B.7 Proofs

This appendix provides proofs for the main theorems stated in this article.

B.7.1 Proof of Theorem 7

The first step of this derivation is to determine the structure of the value function, $\nu_t(x_t)$.

The functional derivative of $\mathcal{L}$ with respect to $p_t(x_t)$ is

$$\begin{aligned} \frac{\delta \mathcal{L}}{\delta p_t(x_t)} &= \underbrace{\sum_{\tilde{x}, u} c_t(x_t, u) \pi_t(u|\tilde{x}_t) q_t(\tilde{x}|x_t)}_{\mathbb{E}(c_t|x_t)} \\ &\quad + \underbrace{\frac{1}{\beta} \sum_{\tilde{x}} q_t(\tilde{x}|x_t) \log \left(\frac{q_t(\tilde{x}|x_t)}{q_t(\tilde{x})} \right)}_{\mathbb{D}(q(\tilde{x}|x_t)||q_t(\tilde{x}))} - \nu_t(x_t) \\ &\quad + \underbrace{\sum_{u, \tilde{x}, x'} \nu_{t+1}(x') p_t(x'|x_t, u) \pi_t(u|\tilde{x}) q_t(\tilde{x}|x_t)}_{\mathbb{E}(\nu_{t+1}|x_t)}. \end{aligned} \quad (\text{B.21})$$

Invoking the FONC yields

$$\nu_t(x_t) = \mathbb{E}(c_t + \nu_{t+1}|x_t) + \frac{1}{\beta} \mathbb{D}(q(\tilde{x}|x_t)||q_t(\tilde{x})). \quad (\text{B.22})$$

Next, we repeat the process for the decision variable $q_t(\tilde{x}_t|x_t)$. The functional derivative of $\mathcal{L}$ is

$$\begin{aligned} \frac{\delta \mathcal{L}}{\delta q_t(\tilde{x}_t|x_t)} &= \left[\underbrace{\sum_u c_t(x_t, u) \pi_t(u|\tilde{x}_t)}_{\mathbb{E}(c_t|x_t, \tilde{x}_t)} - \frac{1}{\beta} \log \left(\frac{q_t(\tilde{x}_t|x_t)}{q(\tilde{x}_t)} \right) \right. \\ &\quad \left. + \underbrace{\sum_{u, x'} \nu_{t+1}(x') p_t(x'|x_t, u) \pi_t(u|\tilde{x}_t)}_{\mathbb{E}(\nu_{t+1}|x_t, \tilde{x}_t)} \right] p_t(x_t) \\ &\quad + \lambda_t(x_t), \end{aligned} \quad (\text{B.23})$$

where $\lambda_t(x_t)$ is the Lagrange multiplier corresponding to the normalization constraint of

$q_t(\tilde{x}_t|x_t)$. Invoking the FONC again and solving for $q_t(\tilde{x}_t|x_t)$ yields

$$q_t(\tilde{x}_t|x_t) = \frac{q_t(\tilde{x}_t) \exp(-\beta \mathbb{E}(\nu_{t+1} + c_t|x_t, \tilde{x}_t))}{\exp(\beta \lambda_t(x_t)/p_t(x_t))}. \quad (\text{B.24})$$

Since $q_t(\tilde{x}_t|x_t)$ is a probability distribution and must be normalized, it is the case that $\exp(\beta \lambda_t(x_t)/p_t(x_t)) = Z_t(x_t)$ where

$$Z_t(x_t) = \sum_{\tilde{x}} q_t(\tilde{x}) \exp(-\beta [\mathbb{E}(c_t + \nu_{t+1}|x_t, \tilde{x})]). \quad (\text{B.25})$$

B.7.2 Proof of Theorem 8

Again, the first step of this proof is to establish the structure of the value function $\nu_t(x_t)$.

By the previous theorem, the value function at the terminal time step is $\nu_T(x_T) = c_T(x_T)$.

This function is the quadratic $\nu_T(x_T) = x_T^T P_T x_T + b_T^T x_T + \text{const.}$ where

$$P_T = Q_{T+1}, \quad b_T = -Q_T g_T. \quad (\text{B.26})$$

Define $G_t := C_t^T (C_t \Sigma_{x_t} C_t + \Sigma_{\eta_t}^{-1}) C_t$. Through use of the fact that the KL-divergence is the quadratic

$$\mathbb{D}(q(\tilde{x}|x_t)||q_t(\tilde{x})) = \frac{1}{2}(\bar{x}_t - x_t)^T G_t (\bar{x}_t - x_t) + \text{const.},$$

recursively plugging in $\nu_{t+1}(x_{t+1})$ into (B.22) shows that $\nu_t(x_t) = x_t^T P_t x_t + b_t^T x_t + \text{const.}$ with

$$\begin{aligned} P_{t+1} &= Q_t + \beta^{-1} G_t + C_t^T K_t^T R_t K_t C_t \\ &\quad + (A_t + B_t K_t C_t)^T P_{t+1} (A_t + B_t K_t C_t), \end{aligned} \quad (\text{B.27})$$

$$\begin{aligned} b_t &= (A_t + B_t K_t C_t)^T P_{t+1} B_t K_t a_t - Q_t g_t - \beta^{-1} G_t \bar{x}_t \\ &\quad + C_t^T K_t^T R_t K_t a_t + (A_t + B_t K_t C_t)^T b_{t+1}. \end{aligned} \quad (\text{B.28})$$

Next, the logarithm of (B.25) is the quadratic

$$\begin{aligned} \log q_t(\tilde{x}_t|x_t) = & -\frac{1}{2}\tilde{x}_t^T \left(\Sigma_{\tilde{x}_t}^{-1} + \beta K_t^T (B_t^T P_{t+1} B_t + R_t) K_t \right) \tilde{x}_t \\ & - (\beta K_t^T B_t^T (b_{t+1} + P_{t+1} B_t h_t + P_{t+1} A_t x_t) \\ & - \Sigma_{\tilde{x}_t|x_t}^{-1} \bar{\tilde{x}}_t + \beta K_t^T R_t (h_t - w_t))^T \tilde{x}_t + \text{const.} \end{aligned} \quad (\text{B.29})$$

Since the logarithm of $q_t(\tilde{x}_t|x_t)$ is quadratic, $q_t(\tilde{x}_t|x_t)$ is a Gaussian distribution with mean and covariance

$$\begin{aligned} \mu_{\tilde{x}|x_t} = & \beta K_t^T B_t^T (b_{t+1} + P_{t+1} B_t h_t + P_{t+1} A_t x_t) \\ & - \Sigma_{\tilde{x}_t|x_t}^{-1} \bar{\tilde{x}}_t + \beta K_t^T R_t (h_t - w_t) \end{aligned} \quad (\text{B.30})$$

$$\Sigma_{\tilde{x}|x_t} = \Sigma_{\tilde{x}_t}^{-1} + \beta K_t^T (B_t^T P_{t+1} B_t + R_t) K_t \quad (\text{B.31})$$

Since $p_t(x_t)$ and $q_t(\tilde{x}_t|x_t)$ are related via the linear-Gaussian $\tilde{x}_t = C_t x_t + a_t + \eta_t$ where $\eta_t \sim N(0, \Sigma_{\eta_t})$, it is necessary that

$$C_t = -\beta \Sigma_{\eta_t} K_t^T B_t^T P_{t+1} A_t, \quad (\text{B.32})$$

$$\begin{aligned} a_t = & -\Sigma_{\eta_t} (\beta K_t^T B_t^T (b_{t+1} + P_{t+1} B_t h_t) \\ & + \beta K_t^T R_t (h_t - w_t) - \Sigma_{\tilde{x}_t}^{-1} (C_t \bar{x}_t + a_t)), \end{aligned}$$

$$\Sigma_{\eta_t} = \left(\Sigma_{\tilde{x}_t}^{-1} + \beta K_t^T (B_t^T P_{t+1} B_t + R_t) K_t \right)^{-1}.$$

B.7.3 Proof of Theorem 9

The distribution of the conditional random variable $x_t|\tilde{x}_t \sim N(\mu_{x_t|\tilde{x}_t}, \Sigma_{x_t|\tilde{x}})$ is the Gaussian distribution given by the minimum mean square error estimate (MMSE) estimate of x_t given $\tilde{x}_t$, i.e.

$$\mu_{x_t|\tilde{x}_t} = \bar{x}_t + \Sigma_{x_t} C_t^T \Sigma_{\tilde{x}_t}^{-1} (\tilde{x}_t - C_t \bar{x}_t - a_t), \quad (\text{B.33})$$

$$\Sigma_{x_t|\tilde{x}_t} = \Sigma_{x_t} - \Sigma_{x_t} C_t^T \Sigma_{\tilde{x}_t}^{-1} C_t \Sigma_{x_t}.$$

First, we compute the measurement model relating y_t and $\tilde{x}_t$. Since $y_t = D_t x_t + \omega_t$ where $\omega_t \sim N(0, \Sigma_{\omega_t})$, we have $y_t|\tilde{x}_t \sim N(\mu_{y_t|\tilde{x}_t}, \Sigma_{y_t|\tilde{x}_t})$ with

$$\begin{aligned}\mu_{y_t|\tilde{x}_t} &= D_t \bar{x}_t + D_t \Sigma_{x_t} C_t^T \Sigma_{\tilde{x}_t}^{-1} (\tilde{x}_t - \bar{\tilde{x}}_t), \\ \Sigma_{y_t|\tilde{x}_t} &= D_t \Sigma_{y_t|\tilde{x}_t} D_t^T - D_t \Sigma_{x_t} C_t^T \Sigma_{\tilde{x}_t}^{-1} C_t \Sigma_{x_t} D_t^T + \Sigma_{\omega_t},\end{aligned}\tag{B.34}$$

The affine change of variables

$$\tilde{D}_t := D_t \Sigma_{x_t} C_t^T \Sigma_{\tilde{x}_t}^{-1}, \quad \tilde{y}_t := y_t - D_t \bar{x}_t + \tilde{D}_t \tilde{x}_t,\tag{B.35}$$

defines a linear-Gaussian output in the common form:

$$\tilde{y}_t = \tilde{D}_t \tilde{x}_t + \tilde{\omega}_t, \quad \tilde{\omega}_t \sim N(0, \Sigma_{y_t|\tilde{x}_t}).\tag{B.36}$$

Now considering the process model, the distribution $x_{t+1}|\tilde{x}_t \sim N(\mu_{x_{t+1}|\tilde{x}_t}, \Sigma_{x_{t+1}|\tilde{x}_t})$ is given by

$$\begin{aligned}\mu_{x_{t+1}|\tilde{x}_t} &= A_t \mu_{x_t|\tilde{x}_t} + B_t u_t, \\ &= A_t \bar{x}_t + A_t \Sigma_{x_t} C_t^T \Sigma_{\tilde{x}_t}^{-1} (\tilde{x}_t - \bar{\tilde{x}}_t) + B_t u_t, \\ \Sigma_{x_{t+1}|\tilde{x}_t, u_t} &= A_t \Sigma_{x_t|\tilde{x}_t} A_t^T + \Sigma_{\epsilon_t},\end{aligned}\tag{B.37}$$

and

$$\begin{aligned}\mu_{\tilde{x}_{t+1}|\tilde{x}_t} &= C_{t+1} A_t \mu_{x_t|\tilde{x}_t} + C_{t+1} B_t u_t, \\ \Sigma_{\tilde{x}_{t+1}|\tilde{x}_t, u_t} &= C_{t+1} \Sigma_{x_{t+1}|\tilde{x}_t} C_{t+1}^T + \Sigma_{\eta_{t+1}}.\end{aligned}\tag{B.38}$$

This distribution can be written in the common form

$$\tilde{x}_{t+1} = \tilde{A}_t \tilde{x}_t + \tilde{B}_t u_t + \tilde{r}_t + \tilde{\epsilon}_t,\tag{B.39}$$

where

$$\begin{aligned}\tilde{A}_t &:= C_{t+1}A_t\Sigma_{x_t}C_t^T\Sigma_{\tilde{x}_t}^{-1}, & \tilde{B}_t &:= C_{t+1}B_t, \\ \tilde{r}_t &:= -\tilde{A}_t\tilde{\tilde{x}}_t + C_{t+1}A_t\bar{x}_t + a_{t+1}, & \tilde{\epsilon}_t &\sim N\left(0, \Sigma_{\tilde{x}_t|\tilde{x}_t, u_t}\right),\end{aligned}\tag{B.40}$$

The equations (B.36) and (B.39) constitute a LG system with $\tilde{x}_0 \sim N(C_0x_0+d_0, C_0\Sigma_{x_0}C_0^T + \Sigma_{\eta_0})$. Despite the statistical properties of this system's noise being linked through common parameters, e.g. $\Sigma_{\tilde{x}_t}$, they are independent random variables — given any subset of noise variables, the distribution of the remaining noise variables does not change. Since this LG system describes the evolution of $q_t(\tilde{x}_t)$ in time, the Bayesian updates for a filter on $\tilde{x}_t$ are given by the standard Kalman filter on this induced LG system.

Appendix C

Learning Task-Driven Control Policies via Information Bottlenecks

C.1 Introduction

The increasing availability of high-resolution sensors has significantly contributed to the recent explosion of robotics applications. For example, high-precision cameras and LIDARs now allow autonomous vehicles to perform tasks ranging from navigating busy city streets to mapping mines and buildings. By far the most common approach adopted by such systems is to utilize as much sensor information as is available in order to estimate the full state of these complex environments; the resulting state estimates are then used by the robot to choose control actions necessary to complete its task. However, this ubiquitous approach does not distinguish between the *task-relevant* and *task-irrelevant* portions of the system’s state. The result is an unnecessarily tight coupling between sensor observations and control actions that is often not robust to noise or uncertainty in irrelevant portions of the state. Moreover, these policies tend to have a large computational burden. This computational requirement can manifest either as a state estimator that needs to process large amounts of data in real time, or as a policy learned from sampling a large number

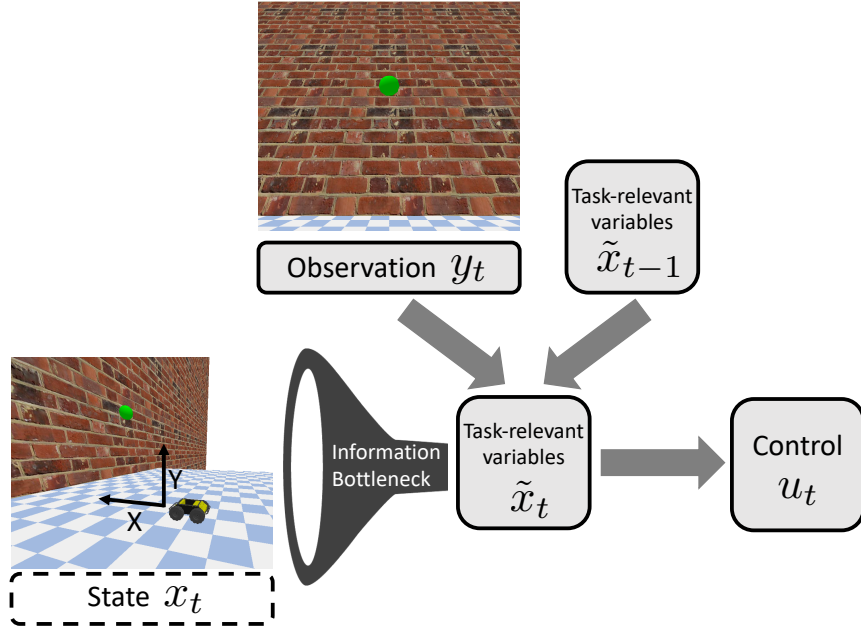


Figure C.1: A depiction of our approach applied to the ball-catching problem described in Appendix C.4.2. Our policy implements an information bottleneck that limits the amount of state information (in this case, the robot and ball position and velocities) extracted from the sensor observations (RGB images) to create a set of task-relevant variables (TRVs) on which the control action (robot velocity) depends. The result is a policy that is more robust to image noise and changes to the brick background texture.

of diverse operating environments. The goal of this paper is to address these challenges by synthesizing control policies that only depend on *task-driven representations* of the robot’s state. Doing so reduces the coupling between the states and control actions, improves robustness, and reduces computational requirements.

To illustrate the advantages of a task-driven policy, consider a ball-catching example. An agent equipped with a high-resolution camera is tasked with catching a ball (Figure C.1). Using its sensor, the agent can attempt to estimate the full state of the system (e.g., ball velocity, ego-motion, wind speed, etc.), feed this information into a physical model for the ball’s flight, integrate the model to find where the ball will land, and move to this location to catch it. This approach requires estimating every parameter involved in the ball’s motion and can easily be compromised when the ball is far away and difficult to see or when there are visual artifacts such as glare.

Instead, extensive cognitive psychology experiments [126–128] have demonstrated that

humans employ a task-driven strategy known as the gaze heuristic to catch projectiles. This strategy simply modulates the human’s speed in order to fix the position of the ball in their visual field. This policy, which only requires minimal sensor information and internal computation, naturally drives the human to the ball’s landing position. The gaze heuristic highlights that task-driven control policies are often robust control policies that only depend on small amounts of salient state information. Specifically, the heuristic is naturally *robust to distributional shifts* in irrelevant portions of the environment (e.g., the visual backdrop) while also being adaptable to perturbations like a gust of wind altering the ball’s course. Traditionally, task-driven policies have required hand-engineering for each control problem. This process can be difficult and time consuming. The goal of this paper is to propose a reinforcement learning framework that *automatically synthesizes* task-driven control policies for systems with nonlinear dynamics and high-dimensional observations (e.g., RGB or depth images).

Statement of Contributions. The main technical contribution of this paper is to formulate a reinforcement learning algorithm that synthesizes task-driven control policies. This synthesis is achieved by creating an *information bottleneck* [129] that limits how much state information the policy is allowed to use. We present a reinforcement learning algorithm — referred to as task-driven policy gradient (TDPG) — that leverages the recently-proposed mutual information neural estimator [70] to tractably search for an effective task-driven policy. Finally, we demonstrate that this formulation provides the key advantage of a task-driven approach — robustness to perturbations in task-irrelevant state and sensor variables. This benefit is demonstrated in three examples featuring nonlinear dynamics and high-dimensional sensor models: an adaption of the lava problem from the literature on partially observable Markov decision processes (POMDPs), ball catching using RGB images, and grasping an object using depth images.

C.1.1 Related Work

State Estimation and Differentiable Filtering. Classical approaches to controlling robotic systems typically involve two distinct pipelines: one that uses the robot’s sensors to estimate its state and another that uses this state estimate to choose control actions. Such

an architecture is motivated by the *separation principle* [1] from (linear) control theory and allows one to leverage powerful control-theoretic techniques for robust estimation and control [165, 166]. A recent line of work on *differentiable filtering* [167–169] has extended this traditional pipeline to elegantly handle rich sensor observations (e.g., images) by *learning* state estimators in an end-to-end manner via deep learning. However, as the gaze heuristic example from Section C.1 demonstrates, full state representations are often overly rich when viewed from the perspective of the task at hand. This is particularly true in settings where representing the full state requires capturing the state of the robot’s environment. Instead, our goal is to learn minimalistic *task-driven* representations that are sufficient for control. Such representations can be highly compact as compared to full state representations. Moreover, carefully-constructed task-driven representations have the potential to be robust to sensor noise and changes to irrelevant portions of the robot’s environment. Intuitively, this is because uncertainty or noise in irrelevant portions of the sensor observations are filtered out and thus no longer corrupt the robot’s actions. We present a theoretical result in Appendix C.2 along with simulation experiments in Appendix C.4 to support this intuition.

End-to-end Learning of Policies. Deep reinforcement learning approaches have the ability to learn control policies in an *end-to-end* manner [58, 170–175]. Such end-to-end approaches learn to create representations that are tuned to the task at hand by exploiting statistical regularities in the robot’s observations, dynamics, and environment. However, these methods do not explicitly attempt to learn representations that are task-driven (i.e., representations that filter out portions of the sensor observations that are irrelevant to the task). As a result, policies trained via standard deep RL techniques may be sensitive to changes in irrelevant portions of the robot’s environment (e.g., changes to the background color in a ball-catching task). In contrast, the deep RL-based approach we present seeks to explicitly learn task-driven representations that filter out irrelevant factors. Our simulation results in Section C.4 empirically demonstrate that our approach is robust to such *distributional shifts*.

Information Bottlenecks. Originally developed in the information theory literature, *information bottlenecks* [129, 176–178] allow one to formalize the notion of a “minimal-

information representation” that is sufficient for a given task (e.g., a prediction task in the context of supervised learning). Given an input random variable X and a target random variable Y , one seeks a representation $\tilde{X}$ that forms a Markovian structure $X \rightarrow \tilde{X} \rightarrow Y$. The representation is chosen to minimize the mutual information between X and $\tilde{X}$ while still maintaining enough information to predict Y from $\tilde{X}$. Recent work has sought to adapt this theory for synthesizing task-driven representations for control [8, 144]. In [8], a model-based approach for automatically synthesizing task-driven representations via information bottlenecks is presented. However, this approach is limited to settings with an explicit model of the robot’s sensor and dynamics. This prevents the approach from being applied to systems with rich sensing modalities (e.g. RGB or depth images), for which one cannot assume a model. In contrast, the approach we present here *learns* to create a task-driven representation via reinforcement learning and is directly applicable to settings with rich sensing. In [144], the authors use information bottlenecks to define minimal state representations for control tasks involving high-dimensional sensor observations (this approach is also related to the notion of *actionable information* [148, 149] in vision; see [144] for a discussion). However, [144] does not present concrete algorithms for learning task-driven representations. In contrast, we present a policy gradient algorithm based on *mutual information neural estimation* (MINE) [70]. Moreover, we note that our definition of a task-driven policy differs from that of [144]; the representations we learn seek to create a bottleneck between sensor observations and control actions as opposed to finding a minimal representation for predicting costs.

Exploration in RL via Mutual Information Regularization. The information bottleneck principle has also been used to improve sample efficiency in RL by encouraging exploration [179, 180]. More generally, there is a line of work on endowing RL agents with *intrinsic motivation* by maximizing the mutual information between the agent’s control actions and states [181–186]. Our focus instead is on improving robustness and generalization of policies; the information bottleneck-based approach we present is aimed at learning task-driven representations for control. This is achieved by minimizing the mutual information between the agent’s states and learned task-relevant variables.

C.2 Defining Task-Driven Policies

In this section, we formalize our notion of a task-driven control policy. Our definition is in terms of a reinforcement learning problem whose solution produces a set of task-relevant variables (TRVs) and a policy that depends only on these variables. We begin by formulating the problem as a finite-horizon partially-observable Markov decision process (POMDP) [187]. The robot’s states, control actions, and sensor observations at time $t \in \{0, \dots, T\}$ are denoted by $x_t \in \mathcal{X}_t$, $u_t \in \mathcal{U}_t$, and $y_t \in \mathcal{Y}_t$ respectively. The robot dynamics and sensor model are denoted by the conditional distributions $p(x_{t+1}|x_t, u_t)$ and $s(y_t|x_t)$ respectively. We do not assume knowledge of either of these distributions. The functions $c_0(x_0, u_0), \dots, c_{T-1}(x_{T-1}, u_{T-1})$ describe the cost at each time step with a terminal cost specified by $c_T(x_t, u_t) := c_T(x_T)$. Our goal is to find a policy $\pi_t(u_t|y_t)$ that solves

$$\underset{\pi_t(u_t|y_t)}{\text{minimize}} \mathbb{E}[c(\tau)] := \mathbb{E}\left[\sum_{t=0}^T c_t\right], \quad (\text{C.1})$$

when run online in a test environment. Throughout this paper, we use $\mathbb{E}[\cdot]$ to denote the expectation; it is subscripted with a distribution when necessary for clarity.

To achieve our goal of learning a task-driven policy, we choose the policy to have the recurrent structure illustrated in Figure C.1. We refer to $\tilde{x}_t \in \tilde{\mathcal{X}}_t$ as the *task-relevant variables* (TRVs) and search for two conditional distributions: $q(\tilde{x}_t|\tilde{x}_{t-1}, y_t), \pi(u_t|\tilde{x}_t)$. The first specifies a (stochastic) mapping from the previous TRVs and the current observation to the current TRVs. The second conditional distribution computes the control actions given the current TRVs. These will be parameterized by neural networks in the following sections. The overall policy can thus be expressed as:

$$\pi(u_t|y_t, \tilde{x}_{t-1}) = \int_{\tilde{\mathcal{X}}_t} \pi(u_t|\tilde{x}_t) q(\tilde{x}_t|\tilde{x}_{t-1}, y_t) d\tilde{x}_t.$$

Our goal is to learn TRVs that form a *compressed representation* of the state that is sufficient for the purpose of control. To formalize this, we leverage the theory of information bottlenecks [129] to limit how much information $\tilde{x}_t$ contains about x_t . This is quantified

using the mutual information

$$\mathbb{I}[x_t; \tilde{x}_t] := \mathbb{D}[p_t(x_t, \tilde{x}_t) \| p_t(x_t)q_t(\tilde{x}_t)] \quad (\text{C.2})$$

between the state x_t and TRVs $\tilde{x}_t$ at each time step. Here, $\mathbb{D}[\cdot \| \cdot]$ is the Kullback-Leibler (KL) divergence [188]. Intuitively, minimizing the mutual information corresponds to learning TRVs that are as independent of the state as possible. Thus, the TRVs “filter out” irrelevant information from the state. Thus, we would like to find a policy that solves:

$$\underset{\substack{q(\tilde{x}_t | \tilde{x}_{t-1}, y_t) \\ \pi_t(u_t | \tilde{x}_t)}}{\text{minimize}} \quad \mathcal{J} := \beta \mathbb{E}[c(\tau)] + \sum_{t=0}^T \mathbb{I}[x_t; \tilde{x}_t]. \quad (\text{C.3})$$

Here, β can be viewed as a Lagrange multiplier, and the above problem can be interpreted as minimizing the total information contained in $\tilde{x}_t$ subject to an upper bound on the expected cost over the time horizon.

In Section C.3, we develop a reinforcement learning algorithm for tackling (C.3). We refer to the resulting policies as *task-driven policies*. Note that the mutual information is invariant under bijective transforms of the random variables for which it is computed [70, 188]. As a result, the optimum value of $\mathcal{J}$ remains unchanged for equivalent representations of the robot and its environment.

Similar to the gaze heuristic (ref. Section C.1), we expect that a task-driven policy as defined above will be a robust policy. This benefit is conferred to our policy by minimizing the mutual information between states and TRVs. Intuitively, a policy is closer to being open-loop when less state information is present in $\tilde{x}_t$. The more open-loop the policy is, the less it is impacted by changes in the state or sensor distributions between environments. In [8], this intuition is formalized using the theory of *risk metrics* with the following theorem:

Theorem 10. Define the *entropic risk metric* [150, Example 6.20]:

$$\rho_\beta[c_t] := \frac{1}{\beta} \log \left[\mathbb{E}_{p_t} \exp(\beta c_t) \right]. \quad (\text{C.4})$$

Let $\check{p}_t(x_t, \tilde{x}_t, u_t)$ be any distribution satisfying:

$$\begin{aligned} \beta \mathbb{D}[\check{p}_t(x_t, \tilde{x}_t, u_t) \| p_t(x_t, \tilde{x}_t, u_t)] \\ \leq \mathbb{D}[p_t(x_t, \tilde{x}_t) \| p_t(x_t) q_t(\tilde{x}_t)]. \end{aligned} \quad (\text{C.5})$$

Then, the online expected cost is bounded by a combination of the entropic risk and mutual information:

$$\mathbb{E}_{\check{p}_t}[c(\tau)] \leq \sum_{t=0}^T \left(\rho_\beta[c_t] + \mathbb{I}[x_t; \tilde{x}_t] \right). \quad (\text{C.6})$$

The entropic risk is a functional that is similar to the expectation, but also accounts for higher moments of the distribution, e.g. its variance. The parameter β controls how much the metric weights the expected cost versus the higher moments, and $\lim_{\beta \rightarrow 0} \rho_\beta[c_t] = \mathbb{E}[c_t]$. Optimizing ρ_β can be difficult because computing its gradient requires computing $q(\tilde{x}_t)$ at each time step. Therefore, we optimize $\mathcal{J}$, a first-order approximation of (C.6). In section Appendix C.4, we demonstrate in multiple examples that minimizing our objective $\mathcal{J}$ produces a robust policy that generalizes beyond its training environment.

Finally, we remark that we can alternatively minimize $\mathbb{I}[\tilde{x}_t; y_t]$, instead of $\mathbb{I}[x_t; \tilde{x}_t]$. However, this is not consistent with Theorem 10. Moreover, as described in Appendix C.3, the mutual information will be estimated empirically and in many cases (see Appendix C.4), the dimension of the observation space is significantly larger than that of the state space. As a result, estimating $\mathbb{I}[\tilde{x}_t; y_t]$ is often more challenging.

C.3 Learning Task-Driven Policies

This section discusses finding a policy that approximately solves (C.3) within a reinforcement learning (RL) framework. If the mutual information term is removed from the objective $\mathcal{J}$, standard RL techniques such as policy gradient (PG) (e.g. [189, 190]) would be sufficient. However, the mutual information and its gradient are known to be difficult quantities to estimate. A number of tractable upper and lower bounds have been proposed recently to provide means for optimizing objectives containing a mutual information term [62]. We elected to use the recently-proposed mutual information neural estimator (MINE)

[70] due to its accuracy and ease of implementation. We then derive a PG-style algorithm in Appendix C.3.2.

C.3.1 Mutual Information Neural Estimator (MINE)

The MINE is based on the Donsker-Varadhan (DV) variational representation of the KL divergence [151, Theorem 2.3.2]. For any two distributions P, Q defined on sample space Ω , the KL divergence between them can be expressed as:

$$\mathbb{D}[P||Q] = \sup_{F:\Omega\rightarrow\mathbb{R}} \mathcal{J}_{dv} := \mathbb{E}_P F - \log \mathbb{E}_Q[\exp(F)]. \quad (\text{C.7})$$

The supremum is taken over all functions F such that the two expectations are finite. Let $\hat{\mathbb{E}}_P^N$ denote the empirical expectation computed with N i.i.d. samples from P . We can estimate the KL divergence by replacing the function class over which the supremum is taken by a family of neural networks F^θ parameterized by $\theta \in \Theta$:

$$\hat{\mathbb{D}}^N[P||Q] := \sup_{\theta \in \Theta} \underbrace{\hat{\mathbb{E}}_P^N F^\theta - \log \hat{\mathbb{E}}_Q^N [\exp(F^\theta)]}_{\hat{\mathcal{J}}_{dv}}. \quad (\text{C.8})$$

This approximation is a *strongly consistent* underestimate of (C.7) (see [70]). In short, this means that through choice of appropriate network structure and sample size, $\hat{\mathbb{D}}^N[P||Q]$ can approximate $\mathbb{D}[P||Q]$ arbitrarily closely.

Since the mutual information is a KL divergence, we can approximate (C.2) using the above KL approximation as:

$$\hat{\mathbb{I}}^N[x_t; \tilde{x}_t] := \hat{\mathbb{D}}^N[p_t(x_t, \tilde{x}_t)||p_t(x_t)q_t(\tilde{x}_t)]. \quad (\text{C.9})$$

This is the MINE. The algorithm for computing the MINE estimate is presented in Algorithm 5. At a high-level, the algorithm attempts to find neural network parameters θ in order to maximize $\hat{\mathcal{J}}_{dv}$ in (C.8) via stochastic gradient descent. We use the notation $\mathbb{E}_\perp[\cdot]$ to denote the expectation taken using $p_t(x_t)q_t(\tilde{x}_t)$. The expectation computed using the joint distribution $p_t(x_t, \tilde{x}_t)$ is unsubscripted. These expectations are approximated by sampling

Algorithm 5 Mutual Information Neural Estimator (MINE)

```
1: procedure TRAIN-MINE( $\theta, \{(x^n, \tilde{x}^n)\}_{n=1}^N$ )
2:   repeat
3:     Sample joint minibatch:  $\{(x^{j_b}, \tilde{x}^{j_b})\}_{b=1}^B$ .
4:     Sample marginal minibatch:  $\{\tilde{x}^{m_b}\}_{b=1}^B$ .
5:     Compute  $\hat{\mathbb{E}}^B[\nabla_{\theta} F^{\theta}]$  with  $\{(x^{j_b}, \tilde{x}^{j_b})\}_{b=1}^B$ .
6:     Compute  $\frac{\hat{\mathbb{E}}_{\perp}^B[\nabla_{\theta} F^{\theta}]}{\hat{\mathbb{E}}_{\perp}^B[\nabla_{\theta} \exp(F^{\theta})]}$  with  $\{(x^{j_b}, \tilde{x}^{m_b})\}_{b=1}^B$ .
7:     Update  $\theta$  using minibatches and (C.10).
8:   until Convergence of  $\hat{\mathcal{J}}_{dv}$ .
9:   return  $\theta$ 
10: end procedure
```

two minibatches of size $B < N$ uniformly from the training batch. The minibatches are denoted in Algorithm 5 by indices $j_{1:B}$ and $m_{1:B}$ respectively. The gradient of $\hat{\mathcal{J}}_{dv}$ is

$$\nabla_{\theta} \hat{\mathcal{J}}_{dv} = \hat{\mathbb{E}}^B[\nabla_{\theta} F^{\theta}] - \frac{\hat{\mathbb{E}}_{\perp}^B[\nabla_{\theta} F^{\theta} \exp(F^{\theta})]}{\hat{\mathbb{E}}_{\perp}^B[\nabla_{\theta} \exp(F^{\theta})]}. \quad (\text{C.10})$$

This gradient, which is computed using the minibatches, is used to update θ with stochastic gradient descent (or a similar optimizer like ADAM [85]). For additional details on training the MINE, see [70].

C.3.2 Task-Driven Policy Gradient Algorithm

We now describe our procedure for leveraging MINE within a policy gradient (PG) algorithm for tackling (C.3). As described in Section C.2, the policy is parameterized by two neural networks: a recurrent network $q^{\phi}(\tilde{x}_t | \tilde{x}_{t-1}, y_t)$ that outputs the TRVs and a feedforward network $\pi^{\psi}(u_t | \tilde{x}_t)$ that outputs the actual control action. Here $\phi \in \Phi, \psi \in \Psi$ represent the parameters for each network. In our implementation, both networks output the mean and diagonal covariance of multivariate Gaussian distributions. During training, we assume access to the states visited by the robot in order to compute the mutual information. Online execution, however, only requires access to the observations.

Computing the gradient of our objective (C.3) is difficult due to the presence of the mutual information, so the MINE is used to approximate this term.¹ This approximation

¹We note that, in order to employ the bound (C.6), we need to employ an overestimate of $\mathbb{I}[x_t; \tilde{x}_t]$. In Appendix C.4, we demonstrate that in practice, minimizing the MINE is a practical approximation that

allows us to derive a learning algorithm similar to policy gradient. Using the well-known identity

$$\nabla_{\psi} \pi^{\psi}(u_t | \tilde{x}_t) = \pi^{\psi}(u_t | \tilde{x}_t) \nabla_{\psi} \log \pi^{\psi}(u_t | \tilde{x}_t), \quad (\text{C.11})$$

the gradient of the expected cost with respect to ψ is given by

$$\nabla_{\psi} \mathbb{E}[c(\tau)] = \mathbb{E} \left[c(\tau) \sum_{t=0}^T \nabla_{\psi} \log \pi^{\psi}(u_t | \tilde{x}_t) \right]. \quad (\text{C.12})$$

Repeating this process for ϕ yields an analogous formula with $q^{\phi}(\tilde{x}_t | \tilde{x}_{t-1}, y_t)$ replacing $\pi^{\psi}(u_t | \tilde{x}_t)$:

$$\nabla_{\phi} \mathbb{E}[c(\tau)] = \mathbb{E} \left[c(\tau) \sum_{t=0}^T \nabla_{\phi} \log q^{\phi}(\tilde{x}_t | \tilde{x}_{t-1}, y_t) \right]. \quad (\text{C.13})$$

Finally, it remains to calculate $\nabla_{\phi, \psi} \hat{\mathbb{I}}^N[x_t; \tilde{x}_t]$. Though it is more tractable to optimize the MINE instead of the true mutual information, computing the gradient of the MINE with respect to the policy parameters is not entirely straightforward. The complexity lies in the fact that the gradient of the MINE with respect to these parameters depends on knowing or estimating the marginal distributions $p_t(x_t)$ and $q_t(\tilde{x}_t)$, both of which depend on ϕ and ψ . To approximate the MINE gradient we fix θ to the converged MINE parameters from Algorithm 5, which yields:

$$\hat{\mathbb{I}}^N[x_t; \tilde{x}_t] = \hat{\mathbb{E}}^N[F^{\theta}] - \log \hat{\mathbb{E}}_{\perp}^N[\exp(F^{\theta})]. \quad (\text{C.14})$$

Since the neural networks used to parameterize the policy produce Gaussian distributions, we can represent $\tilde{x}_t$ as

$$\tilde{x}_t = \mu^{\phi}(\tilde{x}_{t-1}, y_t) + A^{\phi}(\tilde{x}_{t-1}, y_t) \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I). \quad (\text{C.15})$$

This is similar to the reparameterization trick for variational autoencoders [191]. Here,

produces robust policies.

μ^ϕ, A^ϕ are computed using the mean and covariance output from the network $q^\phi(\tilde{x}_t|\tilde{x}_{t-1}, y_t)$. Since the dynamics and sensor model are unknown, the approximations that x_t and y_t are independent of ϕ and $x_t, y_t, \tilde{x}_t$ are independent of ψ are made. With these fixed, the gradient of (C.14) with respect to ϕ can be computed by storing $\epsilon_0, \dots, \epsilon_{t-1}$ as a part of each rollout and backpropogating using (C.15).

The task-driven policy gradient (TDPG) algorithm is outlined in Algorithm 6. Our learning process involves training $T - 1$ MINE networks, whose parameters are denoted $\theta_0, \dots, \theta_{T-1}$. For clarity in future sections, we refer to an iteration of the outer optimization loop as a policy epoch and an iteration of the optimization loop in Algorithm 5 as a MINE epoch. During each policy epoch, we rollout N trajectories using the current policy parameters ϕ, ψ . We then update each set of MINE parameters using Algorithm 5. Once the MINE networks converge, they are used to approximate $\mathcal{J}$ and optimize the policy. This is repeated until the empirical estimate of $\mathcal{J}$, which is given by $\hat{\mathcal{J}} := \beta \hat{\mathbb{E}}^N [c(\tau)] + \sum_{t=0}^T \hat{\mathbb{I}}^N [x_t; \tilde{x}_t]$ converges.

Implementation Details. It remains to specify how to select β in a principled manner. Returning to the perspective described in Appendix C.2, we treat $\mathcal{J}$ as the Lagrangian for minimizing the information shared between x_t and $\tilde{x}_t$ subject to an upper bound on the maximum expected cost the policy is allowed. Then, we sweep through a set of values for β and select the policy from the epoch with the lowest MINE estimate that also satisfies the specified limit on the empirical expected cost. This strategy produces the policy estimated to have the least state information present in the TRVs while satisfying our performance constraint.

It is likely that each MINE network is initialized to a poor estimate of the mutual information. In order to improve the initial estimate of the mutual information and its gradient, additional MINE epochs are used during the first policy epoch. In the following section, we will specify the number of additional epochs used. Moreover, as discussed in [70], using minibatches to estimate the MINE gradient in (C.10) leads to a biased estimate of the gradient. Replacing the denominator in (C.10) with the exponential moving average (EMA) of its value compensates for this bias.² This technique is used in some examples in

²The EMA is a filter defined on a sequence $f_0, \dots, f_t$ by the recursive relationship $\hat{f}_t = (1 - \alpha)a_t +$

Algorithm 6 Task-Driven Policy Gradient (TDPG)

```
1: repeat
2:   Rollout batch of  $N$  trajectories:  $\{(x_t^n, \tilde{x}_t^n, u_t^n)_{t=0}^T\}_{n=0}^N$ .
3:   for  $t = 0, \dots, T - 1$  do
4:      $\theta_t \leftarrow \text{TRAIN-MINE}(\theta_t, \{x_t^n, \tilde{x}_t^n\}_{n=1}^N)$ 
5:   end for
6:   Update  $\phi, \psi$  using rollout batch and (C.12), (C.13), (C.15).
7: until Convergence of  $\hat{J}$ .
```

the following section. We also limit the policy networks to output Gaussian distributions with diagonal covariances.

Finally, since we are solving a finite-horizon control problem, the mappings $q^\phi(\tilde{x}_t|\tilde{x}_{t-1}, y_t)$ and $\pi^\psi(u_t|\tilde{x}_t)$ may optionally be chosen to be time varying, i.e. use separate values of ϕ, ψ for each time step. With a rich enough network structure, this should have no impact on the policy since the network can always encode t in $\tilde{x}_t$. In practice, we found that this increased flexibility can help achieve a lower training cost in some cases at the cost of more computation per epoch.

C.4 Examples

In this section, we apply the algorithm described in Appendix C.3 to three problems: (i) a continuous state and action version of the “Lava Problem” from the POMDP literature, (ii) a vision-based ball-catching example, and (iii) a grasping problem with depth-image observations. For each of these problems, we present thorough simulation results demonstrating that the task-driven policy is robust to distributional shifts in the sensor model and testing environment. We compare our method against a policy with the same parameterization as ours, but trained using a standard policy gradient method in order to minimize the expected cost associated with the problem. The source code for these examples is available here: github.com/irom-lab/trc-nn

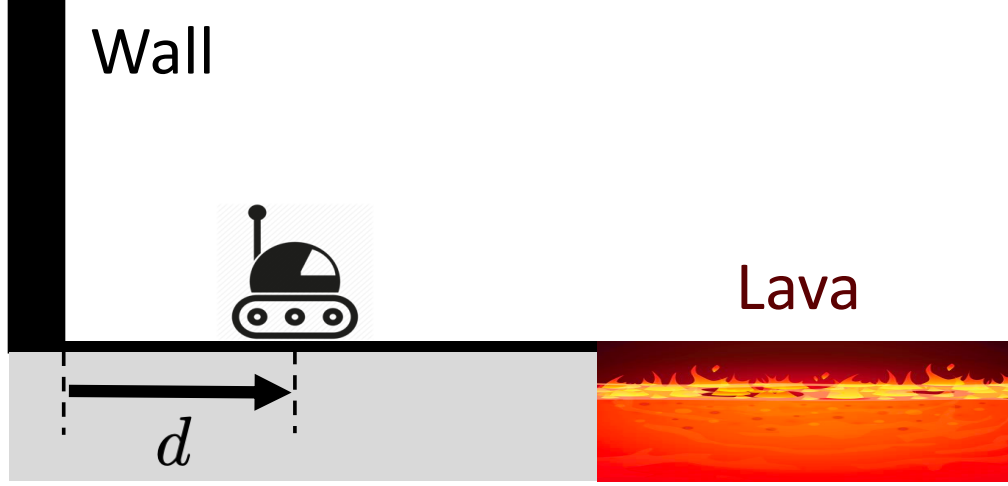


Figure C.2: An illustration of the Lava Problem described in Appendix C.4.1. The robot (a double integrator) needs to navigate to a target state without moving so far right that the robot falls into the lava.

C.4.1 Lava Problem

The first problem we consider is a continuous state and action version of the Lava problem (Figure C.2) [8, 76, 159, 192], which is a common example for evaluating robust solutions to POMDPs. This scenario involves a robot navigating to a goal location along a line segment between a wall and a lava pit. The robot is modeled as a time-discretized double integrator, i.e. its state $x_t = [d_t \ v_t]^T$ evolves with dynamics $x_{t+1} = [d_t + v_t \ v_t + u_t]^T$. Here d_t is the displacement from the wall (in meters) and v_t is the robot's velocity (in meters per second). The goal is to navigate to the state $g = [3 \ 0]^T$ within a time horizon of $T = 5$ steps. However, d_t is limited to the interval $[0, 5]$ m. If the robot collides with the wall located at $d = 0$, then its velocity is set to 0m/s as well. If the robot's position exceeds $d_t = 5$ m, then the robot falls into hot lava, where it is unable to move any further. At training time, the robot is provided with a high-quality estimate of its state. This is modeled by the choice $y_t \sim \mathcal{N}(x_t, \sigma^2 I)$, where $\sigma^2 = 0.0001$ and $\mathcal{N}$ denotes the Gaussian distribution. The cost function for this problem is $c_t(x_t, u_t) = \|x_t - g\|$ for $t = 0, \dots, T-1$ and $c_T = 100\|x_T - g\|$. The robot is initialized with $v_t = 0$ and d_t uniformly distributed between 0 and 5 meters.

Training Summary. Both $q^\phi(\tilde{x}_t|\tilde{x}_{t-1}, y_t)$ and $\pi^\psi(u_t|\tilde{x}_t)$ have two hidden layers with $\alpha \hat{f}_{t-1}, \hat{f}_0 = f_0$.

64 units and use exponential linear unit (ELU) nonlinearities [193]. We choose a two-dimensional space $\tilde{\mathcal{X}}_t$ of TRVs. In this example, we found the performance of all learned policies improved when the parameters ϕ and ψ were allowed to be time-varying. The MINE networks used in this example contain two hidden layers with 32 units each and ELU nonlinearities. A batch of 500 rollouts is used for each epoch, and a minibatch size of 50 is used for each MINE epoch, and an EMA with $\alpha = 5\text{E-}5$ was used to compute the

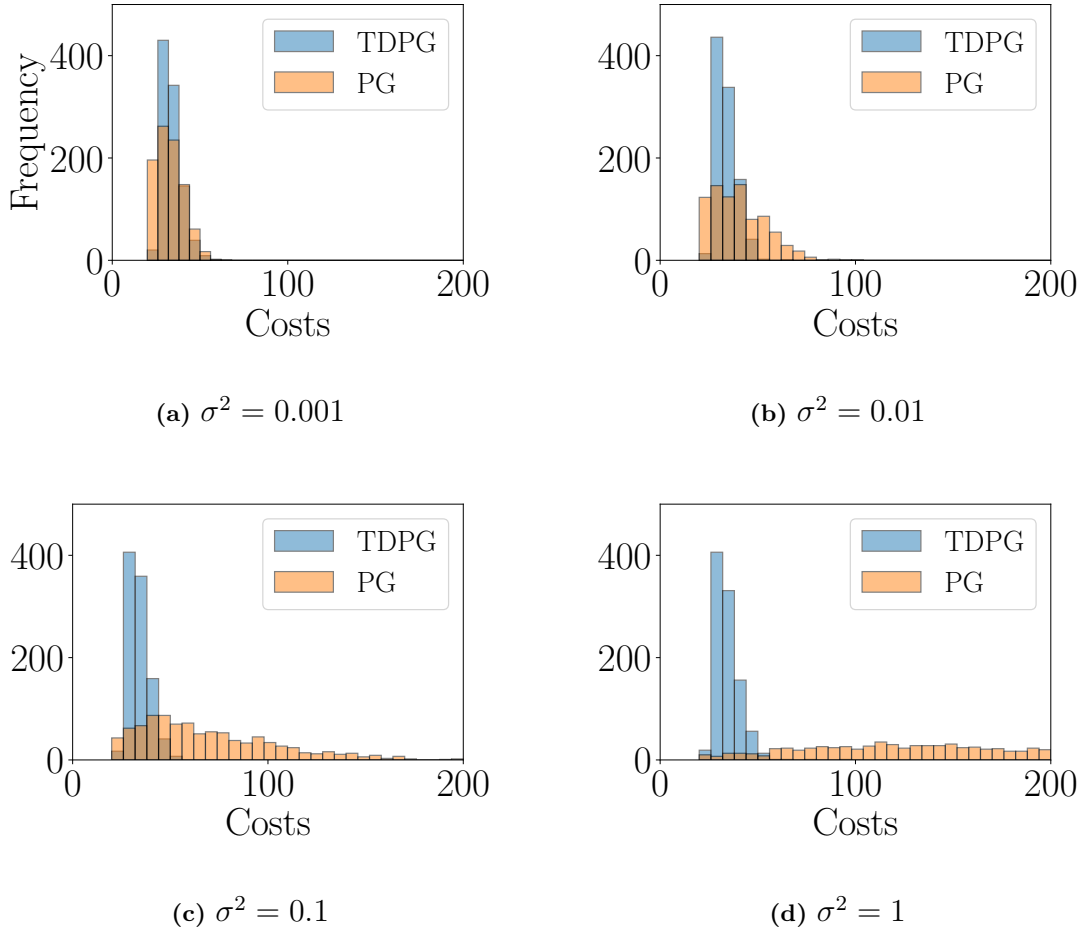


Figure C.3: These histograms compare our policy (TDPG) to the policy found by policy gradient (PG) in the Lava Problem described in Appendix C.4.1 by showing the frequencies at which each policy incurs different costs. The sensor noise of the test environment is increased from left to right. As the noise increases, the PG policy performance degrades, while the TDPG policy performance remains almost constant. This is because the TDPG algorithm found a task-relevant open-loop policy that is robust to this kind of disturbance.

Scenario	Policy Gradient		Task-Driven PG	
	Mean	Std.	Mean	Std.
Training	31.04	7.191	36.18	23.22
Sensor Noise $\sigma^2 = 1\text{E-}3$	31.58	8.632	36.55	25.02
Sensor Noise $\sigma^2 = 1\text{E-}2$	35.69	15.68	36.00	22.32
Sensor Noise $\sigma^2 = 1\text{E-}1$	66.29	36.48	36.20	21.85
Sensor Noise $\sigma^2 = 1\text{E-}0$	172.40	88.08	37.39	25.60

Table C.1: Performance of PG and TDPG Policies in Training and Testing Lava Environments

MINE gradient. The learning rates in this problem are $8\text{E-}4$ for the policy networks and $5\text{E-}5$ for the MINE network. All policies were trained for 300 epochs, with a computation time of about 10 seconds per epoch (about 50 minutes total for each policy). For this example, all computation was carried out on an Intel i9-7940X. Policies were trained with $\beta \in \{25^{-1}, 50^{-1}, 75^{-1}, 100^{-1}\}$ and the upper limit on the expected cost for selecting the policy was 40.

Policy Evaluation. We compare the resulting policy with one trained to minimize only the expected cost using policy gradient. The two policies are compared in Fig. C.3, and the statistics of their performance in different environments (i.e. different levels of sensor noise) are presented in Table C.1. Due to the presence of low sensor noise in the training environment, the PG policy finds it optimal to drive the robot directly towards the goal state. Interestingly, the TDPG algorithm finds a *qualitatively different* strategy. In particular, TDPG recovers the robust open-loop behavior described in [8, 76, 159]: regardless of initial position, the robot moves left until it collides with the wall, then moves right to the goal state. As the sensor noise σ^2 is increased, the performance of the PG policy degrades rapidly. For example, if the sensor reports the robot is to the left of the goal when it is really to the right of the goal, it can fall in the lava. In contrast, the TDPG is unaffected by the increased sensor noise.

C.4.2 Vision-Based Ball Catching

Next, we consider a ball-catching example inspired by the gaze heuristic discussed in Appendix C.1 (see Fig. C.1). We formalize this problem by considering a ball confined to a plane with x and y coordinates (b_t^x, b_t^y) . The robot is confined to the x -axis and must

navigate to b_t^x . The state of the system in this example is given by $x_t = [d_t \ b_t^x \ b_t^y \ v_t^x \ v_t^y]^\top$, where d_t represents the robot's displacement along the x -axis, v_t^x represents the ball's velocity along the x -axis, and v_t^y represents the ball's velocity along the y -axis (i.e., the ball's vertical velocity). The dynamics are given by:

$$\begin{bmatrix} d_{t+1} \\ b_{t+1}^x \end{bmatrix} = \begin{bmatrix} d_t + \Delta t \cdot u_t \\ b_t^x + \Delta t \cdot v_t^x \end{bmatrix}, \quad \begin{bmatrix} b_{t+1}^y \\ v_{t+1}^y \end{bmatrix} = \begin{bmatrix} b_t^y + \Delta t \cdot v_t^y \\ v_t^y - \Delta t \cdot g \end{bmatrix},$$

and $v_{t+1}^x = v_t^x$. Here, $g = 9.81\text{m/s}^2$ is gravity, and $\Delta t = \frac{1}{15}\text{s}$ is used to discretize the dynamics of the system in time. The robot's initial position is uniformly distributed over the interval $[-2, 2]\text{m}$. The ball is launched from $b_0^x = 8\text{m}$ and $b_0^y = 1\text{m}$ with fixed initial velocities $v_0^x = -4.5\text{m/s}$ and $v_0^y = 7.85\text{m/s}$. These initial conditions are chosen such that the ball always spends a fixed number of time steps above the x -axis. The time horizon $T = 25$ is chosen such that T is the last time step the ball remains above the x -axis. The cost function that we are trying to minimize is $c_t(x_t, u_t) = 0.01\|u_t\|$ for $t = 0, \dots, T-1$ and $c_T(x_T) = 100\|d_T - b_T^x\|$, which encourages the robot to be very close to the ball when

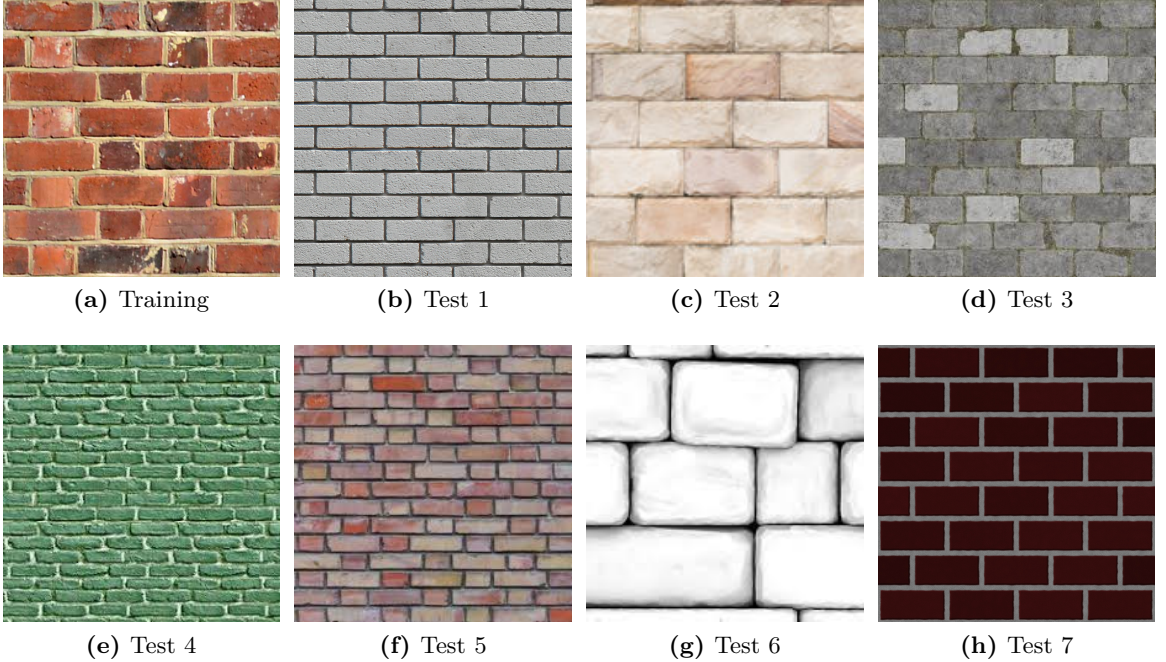


Figure C.4: Textures used as backdrops in different ball-catching environments as part of the example described in Appendix C.4.2.

it lands at the end of the time horizon. The sensor in this scenario is a camera mounted above the robot. This camera provides 64×64 RGB images with values scaled between 0 and 1. We also placed a wall with a red brick texture centered at 10m along the x -axis. All simulations are carried out using PyBullet [194]. A sample observation from the camera is presented in Fig. C.1, and a video depicting both policies operating in training and testing environments for this scenario is provided in the supplementary material.

Training Summary. In this example, $q^\phi(\tilde{x}_t|\tilde{x}_{t-1}, y_t)$ is parameterized by a network with 2 convolutional layers with 6 output channels, kernel size of 4, and stride of length 2, followed by two fully-connected layers using 32 units each. An ELU nonlinearity is applied between convolutional layers. After each fully-connected layer, tanh nonlinearities were used instead of ELU nonlinearities to prevent the values of $\tilde{x}_t$ from growing unbounded. The dimension of $\tilde{\mathcal{X}}_t$ is 8. The network $\pi^\psi(u_t|\tilde{x}_t)$ contains a single linear layer. The MINE networks used in this example contain two hidden layers with 64 units each and ELU nonlinearities. A batch of 200 rollouts is used for each epoch, and a minibatch size of 20 is used to train the MINE network. The learning rates in this problem are 1E-3 for the policy networks and 5E-5 for the MINE network. When training the TDPG policy, it was initialized with the PG solution and trained for 100 policy epochs. Each policy epoch contained 100 MINE epochs with 100,000 MINE epochs used on the first policy epoch, and an EMA with $\alpha = 5\text{E-}5$ was used to compute the MINE gradient. Policies with $\beta = 16^{-1}, 18^{-1}, \dots, 40^{-1}$ were evaluated, with the upper limit on the expected cost placed at 24. Rollouts were computed on an 3.7GHz i7-8700K CPU while all optimization was done using an Nvidia Titan Xp. Each policy epoch took about 45 seconds to compute.

Policy Evaluation. We consider two kinds of testing environments. All statistics for the test environments were computed using 1000 rollouts. In the first group of examples, we add random noise to each pixel; the noise is sampled from $\mathcal{N}(0, \sigma^2)$, where σ is varied between experiments. To ensure that the observed image is a valid RGB image after adding noise, we normalize the values to lie between 0 and 1. For each experiment, the mean cost and distance between the robot and the ball on the x -axis is presented in Table C.2. As the level of image noise increases, the performance of the PG policy deteriorates dramatically while the TDPG policy’s performance remains largely unchanged.

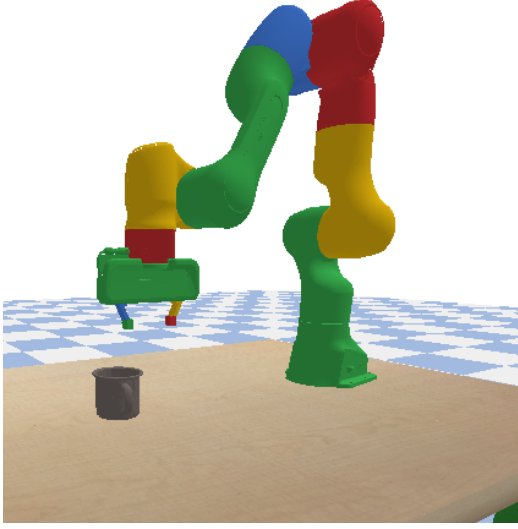
Scenario	Policy Gradient		Task-Driven PG	
	Cost	Dist. (m)	Cost	Dist. (m)
Training	8.702	0.085	18.577	0.184
Sensor Noise $\sigma = 0.10$	19.12	0.189	19.48	0.193
Sensor Noise $\sigma = 0.15$	46.68	0.464	20.12	0.199
Sensor Noise $\sigma = 0.20$	124.63	1.244	21.59	0.214
Sensor Noise $\sigma = 0.25$	187.53	1.874	27.11	0.274
Test Background 1	182.1	1.828	122.7	1.225
Test Background 2	214.3	2.141	79.90	0.797
Test Background 3	138.7	1.385	26.62	0.264
Test Background 4	95.38	0.952	28.11	0.279
Test Background 5	82.01	0.818	36.95	0.367
Test Background 6	208.5	2.083	166.4	0.166
Test Background 7	9.180	0.090	14.92	0.147

Table C.2: Mean Cost and Final Distance of PG and TDPG Policies in Training and Testing Ball-Catching Environments

In the second set of experiments, we change the texture of the backdrop (i.e. the wall in the background) for testing. This change is not task-relevant to catching the ball as long as the ball can be identified within the frame. Each texture is presented in Fig. C.4. The TDPG policy outperforms the PG policy in the seven testing environments except background 7. This texture has a similar hue (red) to the training environment texture allowing PG to perform well. The TDPG policy, however, outperforms PG in environments whose textures consist of hues that are different than the ones seen when training. Together with the previous experiments, this result suggests that the TDPG policy is more robust to observations that are qualitatively different in task-irrelevant ways than those provided to the policy during training. A video depicting the second set of experiments is available here: www.youtube.com/watch?v=Mwv0kkRveas

C.4.3 Grasping using a Depth Camera

In this final example, we consider the task of grasping and lifting an object (a mug) from a table using a Franka Emika Panda simulated with PyBullet (see Fig. C.5). This example is particularly interesting for studying task-driven policies due to the (approximate) radial symmetry of the mug about the vertical axis. Intuitively, this symmetry renders the orientation of the mug about the vertical axis largely irrelevant to the task of grasping the



(a) (a) Franka Emika Panda



(b) (b) Sensor Observation

Figure C.5: (a) A third-person perspective of the grasping scenario described in Appendix C.4.3 using PyBullet. (b) An upscaled example of the 128×128 depth image forming the sensor observation for this scenario.

mug. This is due to the fact that the robot does not need to know the precise orientation of the mug; it only needs to know enough about the orientation in order to estimate if the handle will directly interfere with the gripper’s location when the grasp is performed. A task-driven policy will remain largely unaffected by changes in the mug’s orientation and should therefore generalize to the full set of rotations.

In this grasping problem, the state of the system $x_t \in \mathbb{R}^4$ contains the position of the object and its rotation about the z -axis (which is oriented normal to the table surface) in radians. The control action $u_t \in \mathbb{R}^4$ specifies the position and orientation of the end effector of the arm when the grasp occurs. After grasping, the arm attempts to move the end effector vertically. A cost of 0 is awarded if the arm successfully lifts the object more than 0.05 meters above the table and a cost of 1 is assigned otherwise. The observation is a 128×128 depth image. An example observation is included in Fig. C.5. The initial state of the object is sampled uniformly from the set $[0.45 \text{ m}, 0.55 \text{ m}] \times [-0.05 \text{ m}, 0.05 \text{ m}] \times \{0 \text{ m}\} \times [-\frac{\pi}{4}, \frac{\pi}{4}]$.

Training Summary. We parameterized the policy in the following manner. The network $q^\phi(\tilde{x}_t | \tilde{x}_{t-1}, y_t)$ contains two convolutional layers with 6 output channels. The first uses a kernel size of 6 and the second uses a kernel size of 4; both use a stride length of 2.

The convolutional layers are followed by two fully-connected layers of sizes 128 and 64. An ELU linearity is used between convolutional layers and tanh nonlinearities are used after linear layers. The size of $\tilde{\mathcal{X}}_t$ is 16. The network $\pi(u_t|\tilde{x}_t)$ contains two fully connected layers with 64 units each and an ELU nonlinearity between them. The output of this network is then divided by 10 and added to a nominal control action of $[0.5, 0, 0.06, \frac{\pi}{6}]$. This scaling and translation is done to bias the policy to select grasping locations near the object to speed up learning early in the optimization process. Again, as in Appendix C.4.2, we initialize the TDPG solution to the PG solution at the start of training and learned policies with values of $\beta \in \{5^{-1}, 10^{-1}, 15^{-1}, 20^{-1}\}$ for 30 epochs.

In this example, the MINE proved particularly noisy and took longer to converge. To combat this, we increased the number of MINE epochs to 500,000 on the first policy epoch with 100,000 MINE epochs used on following policy epochs. To determine the value of the MINE, we applied an EMA with parameter $\alpha = 0.9$. No EMA was used for computing MINE gradients. The epoch with the lowest filtered MINE estimate with an expected cost below 0.15 was used for testing. Each epoch took approximately 5 minutes to compute. Rollouts were computed on an Intel 3.7GHz i7-8700K in parallel and optimization was done on a Titan Xp.

Policy Evaluation. The test results for this example are summarized in Table C.3. Again, all reported statistics are computed using 1000 trials. In the first testing environment, the set of angles the mug is placed at is expanded from $[-\frac{\pi}{4}, \frac{\pi}{4}]$ to $[0, 2\pi]$. The expected cost (i.e., grasp failure rate) of the PG policy increased twice as much as the TDPG policy in these new testing environments. In the second testing environment, the set of angles the object is placed at is again $[-\frac{\pi}{4}, \frac{\pi}{4}]$, but the x and y values were sampled from the larger set of $[0.425 \text{ m}, 0.575 \text{ m}] \times [-0.075 \text{ m}, 0.075 \text{ m}]$. In this setting, both policies perform equally poorly. This result supports our hypothesis that the rotation of the mug is largely unimportant for the task of lifting the mug because the TDPG policy was able to generalize to new mug angles, but not to the task-relevant translational coordinates of the mug. Meanwhile, the PG policy exhibits over fitting to task-irrelevant state information and is impacted poorly by its changes.

Scenario	Policy Gradient		Task-Driven PG	
	Mean	Std.	Mean	Std.
Training	0.107	0.309	0.094	0.292
Increased Rotation	0.182	0.386	0.132	0.339
Increased Translation	0.367	0.482	0.358	0.480

Table C.3: Performance of PG and TDPG Policies in Training and Testing Grasping Environments

C.5 Discussion and Conclusion

We presented a novel reinforcement learning algorithm for computing task-driven control policies for systems equipped with rich sensor observations (e.g., RGB or depth images). The key idea behind our approach is to learn a task-relevant representation that contains as little information as possible about the state of the system while being sufficient for achieving a low cost on the task at hand. Formally, this is achieved by using an information bottleneck criterion that minimizes the mutual information between the state of the system and a set of task-relevant variables (TRVs) used for computing control actions. We parameterize our policies using neural networks and present a novel policy gradient algorithm that leverages the recently-proposed mutual information neural estimator (MINE) for optimizing our objective. We refer to the resulting algorithm as task-driven policy gradient (TDPG).

We compare PG and TDPG policies in three experiments: an adaption of the canonical lava problem to continuous state spaces, a ball catching scenario inspired by the gaze heuristic from cognitive psychology, and a depth image-based grasping problem. In the lava example, the TDPG policy exploits the nonlinear dynamics to find a minimal information (open-loop) control policy that is robust to increased sensor noise. In the ball-catching example, the TDPG is also more robust to changes in the sensing model at test time than PG. These changes include both random noise corrupting the images and task-irrelevant structural changes, e.g. altering the textures in the robot’s environment. Finally, in the grasping scenario, the TDPG policy generalizes to rotated states of the object not seen during training on which the PG policy struggles. Together, these scenarios validate that our approach to designing task-driven control policies produces robust policies that can operate in environments unseen during training.

Future Work. There are a number of challenges and exciting directions for further exploration. First, we have observed that MINE often results in noisy estimates of the mutual information and can take many epochs to converge. This results in long training times for TDPG. We also employed an approximation of the gradient of MINE with respect to policy parameters (due to the challenges associated with estimating state distributions at each time step, as described in Section C.3). These observations motivate the exploration of other methods for minimizing the mutual information, eg., Stein variational gradient methods [55, 56]. Another direction for future work is to adapt more advanced on-policy methods (e.g., proximal policy optimization (PPO) [190]) to work with our approach, and potentially explore off-policy methods. We expect that these methods will be even more effective at learning task-driven policies. Finally, an exciting direction for future work is to explore the benefits that our approach affords in terms of sim-to-real transfer. The simulation results in this paper suggest that TDPG can be robust to task-irrelevant features system perturbations. Exploring whether this translates to more robust sim-to-real transfer is a promising future direction.

Appendix D

Robust Control Under Uncertainty via Bounded Rationality and Differential Privacy

D.1 Introduction

Despite the increasing availability of high-resolution sensors for robotic systems, *partial-observability* remains a challenge for controlling such systems. Sensing modalities such as vision and LIDAR are ultimately noisy and only provide partial information about the robot’s state and environment. In general, solving optimal control problems with partial observability is computationally intractable. One of the most common approaches to tackling this challenge is to assume the *separation principle* [1], i.e., to *independently* design (i) a state estimator, and (ii) a controller, e.g., one based on model-predictive control (MPC), that is optimal assuming perfect state estimation. The modularity afforded by such an approach coupled with the relative tractability of tackling the estimation and control problems independently make this framework appealing. However, the separation principle does not hold in general for robotic systems due to nonlinear dynamics / measurement models. A controller that assumes perfect state estimation can thus be highly sensitive to small errors in the state estimate, leading to significant brittleness of the overall control system. This

is particularly challenging in the increasingly common case where a (deep) learning model is used as part of the robot’s state estimation pipeline (due to potential over-fitting). As a result, robots often behave erratically when faced with unforeseen measurement errors despite possessing high-resolution sensors.

In contrast, the cognitive science literature on *bounded rationality* — a framework for analyzing decision-making under computation or information constraints — demonstrates that humans display impressive levels of robustness and generalization in dexterous tasks such as locomotion and ball-catching without relying on highly-accurate state estimates [3, 4]. For example, *gaze heuristics* are control laws used by humans to catch freely-falling objects by adjusting their running speed based on the motion of the object in their visual field [5, 195]. Unlike MPC strategies, gaze heuristics do not require an accurate estimate of the system state or other quantities such as the wind speed or object mass. In addition to the robustness afforded by such heuristics, they are often extremely computationally efficient. Interestingly, the dual benefits of robustness and computational efficiency are related to each other in the above heuristics — boundedness of online computation (i.e., bounded rationality) ensures that control actions are only loosely coupled to sensor measurements. Such a loose coupling can prevent uncertainty in measurements from significantly impacting controller performance.

Statement of Contributions. The key conceptual contribution of this work is to use the observation that bounded rationality is linked to robust decision-making as a guide and formalize it using the theory of *differential privacy* (DP) [17, 18, 21], a framework that uses an algorithm’s input-output sensitivity to quantify the information about an input gained by observing its output. Originally developed as a framework for ensuring the privacy of individuals’ data (e.g., in a database with published statistics), DP allows formalization of the idea that control inputs should not depend too tightly on state estimates. In particular, our approach preserves the modular structure of the separation principle by ensuring that the mapping from state estimates to control inputs is differentially private. Moreover, it exploits the interpretation of the differentially private *exponential mechanism* as synthesizing *information-constrained* (i.e., bounded-rationality) controllers.

The primary theoretical contribution of this paper is a novel bound on the expected per-

Motion Planning with Sensing Uncertainty

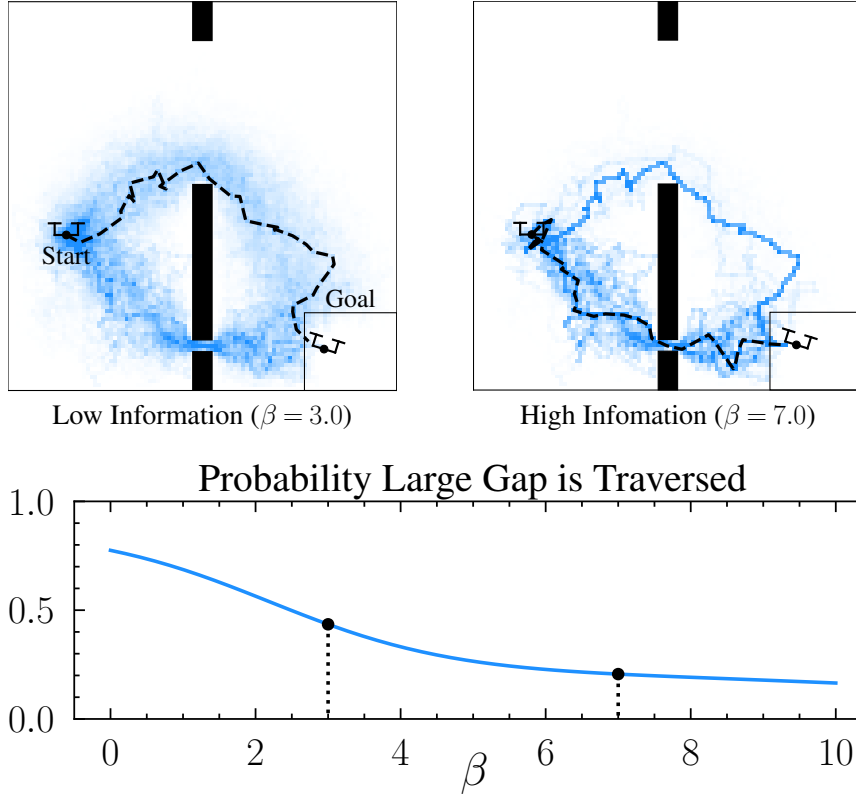


Figure D.1: The information-constrained motion-planning problem (Appendix D.6). The robot must navigate from the start to the goal using a limited amount of state information (correlated with β). The distribution of trajectories is visualized for two values of β . At low information, the robot prefers the longer path through the wider gap, which is more robust to positional uncertainty than the direct route preferred at higher information usage.

formance of such controllers in the presence of estimation error. The bound depends only on the cost incurred using a perfect state estimator, the magnitude of estimation errors, and the level of differential privacy. Moreover, due to the choice of the differentially-private mechanism on which our policy is based, our bound naturally applies to many popular inference-based control algorithms, such as optimal control methods based on importance sampling, Stein-variational gradient descent, and Q -learning. We are thus able to demonstrate the efficacy of the bounded-rationality approach we propose on a number of robotics problems of interest, such as stabilizing a nonlinear planar quadrotor and robust motion planning. To our knowledge, the approach presented in this paper is the first to utilize the framework of differential privacy to achieve robust and bounded-rational control of robotic systems.

D.2 Related Work

This section briefly reviews three areas of the literature that are typically considered independently from one another. However, the unifying theme is that they exploit various properties of the Gibbs measure.

D.2.1 Differential Privacy and Applications in Control Theory

Differential privacy (DP) [17, 18, 21, 22] is an algorithmic framework that arose to meet the conflicting needs of statisticians (who ultimately want to publish analyses of sensitive data sets) and participants (who want to keep their data private). DP offers an elegant solution based on the intuition that if the results published from a study are insensitive to the substitution of data from any given individual, then this offers privacy. The literature on DP is vast and many differentially-private *mechanisms* (i.e., stochastic algorithms) have been proposed. The most relevant for our purpose is the *exponential mechanism* [18] defined by the Gibbs measure; this mechanism provides a method for releasing the results of an optimal decision-making procedure based on data (e.g., Bayesian inference [23]). Additionally, DP enjoys several appealing theoretical properties, namely the preservation of privacy under composition of mechanisms, and the ability to rigorously analyze the trade-off between privacy and accuracy [18].

In the context of control theory, application of the DP formalism is largely limited to networked control and distributed systems [24–29]. These include privately solving distributed optimization problems, modifying control inputs to keep the system state private from an external observer, and aggregating measurements from privacy-concerned agents for filtering performed by a central entity. In contrast, we *do not* use DP for the sake of privacy; instead, DP formalizes the intuition described in Appendix D.1: limiting the sensitivity of control inputs to state estimates affords robustness to estimation error. To our knowledge, our work is the first to utilize DP for this purpose.

D.2.2 Bounded Rationality and Robust Decision-Making

Bounded rationality is a model of decision-making first introduced to address misalignment between economic theory based on rational agents and the reality of sub-optimal human decision-making [2]. In short, while agents often have an objective they are trying to optimize, their ability to optimize this objective is bounded by informational and computational constraints. The cognitive science literature has identified several *heuristics* used by humans to deal with such constraints [3, 4, 30, 31]. Empirical work in cognitive science and robotics suggests that in addition to efficiency, such heuristics can provide generalization to new environments [3–5, 8, 9].

While bounded rationality is formalized in a number of ways for artificial agents, the most relevant framework adds an *information-theoretic constraint* to an otherwise rational agent. More precisely, a bounded-rational agent is modeled as solving an *entropy-maximization* or *rate-distortion* problem [32–37]. Despite empirical evidence that such a bounded-rational agent can be robust to uncertainty or noise in sensor measurements and dynamics [8, 9, 38–40], formal analyses justifying such robustness benefits are limited. One approach is to use a variational representation of the (relative) entropy (e.g., the Donsker-Varadhan representation [41]) to derive a zero-sum game that the bounded-rational agent is implicitly playing against an adversary that chooses a cost function [35, 36, 42]. This approach is closely related to the *maximum-entropy principle* from Bayesian statistics [43] and leads to generalization results for some estimation problems. However, the payoff of the game is difficult to interpret (especially for the adversary) in the context of most control problems, which limits the usefulness of the analysis. Alternatively, an upper bound on the performance degradation of a bounded-rational agent under measurement error exists [8] using variational representations of the entropy, but its assumptions make the bound difficult to interpret practically. Instead, this paper demonstrates that the connection between DP and bounded rationality through the Gibbs measure allows for the derivation of a bound on control performance in the presence of estimation error through a large-deviation analysis. The trade-offs presented by this bound are easily understood and, importantly, the bound only depends on access to simulation or lab data without actually deploying the

robot in the target environment.

D.2.3 Optimal Control as Bayesian Inference

Another relevant vein of research at the intersection of information theory, Bayesian inference, and control theory is *linearly-solvable optimal control* (LSOC) [44–53]. These techniques exploit an identified equivalence between optimal control and Bayesian inference: an approximately optimal control sequence is found by sampling from a Gibbs measure over control inputs conditioned on the current state and the event that the sequence results in an optimal trajectory [47]. The result is that the nonlinear stochastic optimal control problem is solved as a linear, albeit infinite-dimensional, differential equation using a transformed cost function, and the solution is approximately computed using inference algorithms that include importance sampling [54], Stein-variational gradient descent (SVGD) [55], and Q -learning [56]. Despite the growing popularity of these algorithms for optimal control and empirical evidence for the generalization benefits they confer [57], theoretical knowledge of their robustness properties is limited [58]. This paper aims to fill this gap by providing a new, concrete analysis that these algorithms are provably robust to estimation error.

D.3 Notation

Random variables are denoted by uppercase letters (e.g., X), and realized quantities are denoted by lowercase letters (e.g., x). Deterministic functions appear in either case. Finite sequences are represented as $x_{i:j} = (x_k)_{k=i}^j$ for $i \leq j$. The indicator function for a set $\mathbf{A}$ is denoted $1_{\mathbf{A}}(\cdot)$. Functionals are double-struck, namely decorated varieties of the expectation $\mathbb{E}[\cdot]$, the relative entropy $\mathbb{D}[\cdot||\cdot]$, and the tightest Lipschitz constant of a scalar-valued function $\mathbb{L}[\cdot]$. Expectations are taken over the uppercase random variables (and mechanisms), e.g., in $\mathbb{E}[H(x, U)]$, x is fixed and integration is over U . Sets are in boldface and $\Delta(\mathbf{A})$ is the set of distributions with support $\mathbf{A}$. For brevity, it is assumed that all necessary moments of random variables exist, and spaces are measurable with their subsets coming from appropriate σ -algebras.

A *mechanism* (denoted by uppercase script) refers to a (randomized) algorithm (for-

mally, a transition kernel) between sets $\mathbf{X}$ and $\mathbf{Y}$. A mechanism $\mathcal{M} : \mathbf{X} \rightarrow \Delta(\mathbf{Y})$ defines a probability distribution on $\mathbf{Y}$ for each $x \in \mathbf{X}$. Denote by $\mathcal{M}(x)\{\cdot\}$ the density and measure of this distribution when applied to elements and measurable subsets of $\mathbf{Y}$ respectively. When clear from context, we will overload this notation by treating $\mathcal{M}(x)$ as a random variable with support $\mathbf{Y}$. Mechanisms may be composed, i.e., if $\mathcal{M}' : \mathbf{Y} \rightarrow \Delta(\mathbf{Z})$ is another mechanism, then $(\mathcal{M}' \circ \mathcal{M})(\cdot) := \mathcal{M}'(\mathcal{M}(\cdot))$.

D.4 Robust Single-Step Decision-Making

This section details the robustness properties that follow from applying a bounded-rationality approach to a single-step decision-making problem. Appendix D.5 utilizes composition properties of DP to extend the analysis to multi-step optimal control problems.

D.4.1 Problem Statement

Let the state of the robotic system be $x \in \mathbf{X}$. The goal of the agent (robot) is to process the information contained in this state and select a control input $u \in \mathbf{U}$ that minimizes a cost $H(x, u)$. The state $X \sim \mathcal{X}$ is random and the agent must find a feedback mechanism $\mathcal{U} : \mathbf{X} \rightarrow \Delta(\mathbf{X})$ that solves:

$$J^{\text{off}} := \min_{\mathcal{U}} J^{\text{off}}[\mathcal{U}] := \mathbb{E}[H(X, \mathcal{U}(X))]. \quad (\text{OFF})$$

Since X is made available to the agent, this problem is fully-observable. It is referred to as the *offline problem* since it corresponds to, e.g., a lab or simulation setting where the agent makes arbitrarily fine measurements.

In practice, the primary concern is performance of the feedback controller on the *online problem*, where the agent only has access to a noisy state estimate $\hat{X} \sim \hat{\mathcal{X}}(X)$.¹ The control

¹Typically $\hat{\mathcal{X}}(x)$ is the composition of a state estimator with a noisy sensor, but these mappings are implementation dependent and irrelevant to the analysis. It is simpler and without loss of generality to work only with their composition.

input $U \sim \mathcal{U}(\hat{X})$ is selected using this estimate, and the goal is to solve:

$$\min_{\mathcal{U}} \mathbb{J}^{\text{on}}[\mathcal{U}] := \mathbb{J}^{\text{off}}[\mathcal{U} \circ \hat{X}]. \quad (\text{ON})$$

This is a *partially-observable* decision problem and its general solution requires reformulating the problem into an intractably high-dimensional (often infinite-dimensional) optimization problem [196, 197]. Instead, we will demonstrate that a *bounded-rational* controller allows for the value of (ON) to be bounded in terms of (OFF). That is, *performance on the online problem can be guaranteed using only information available offline* due to a property of the bounded rationality controller known as *differential privacy*. The difference in performance between these two problems is defined to be $\Delta \mathbb{J}[\mathcal{U}] := \mathbb{J}^{\text{on}}[\mathcal{U}] - \mathbb{J}^{\text{off}}[\mathcal{U}]$, and the main contribution of this paper is to bound this quantity.

D.4.2 Differential Privacy

Differential privacy (DP) formalizes the observation that a mechanism does not reveal information about its input if the input may be replaced by a similar one without impacting the output distribution significantly. Specifically, *random metric DP* [22, 23, 198] encodes similarity via a metric² on the space of input data and is a natural fit for control applications where the input comes from a metric state space:

Definition 5 (Differential Privacy). Let $(\mathbf{X}, \rho)$ be a pseudometric space and $X, \hat{X}$ be two random variables supported on $\mathbf{X}$. A mechanism $\mathcal{U} : \mathbf{X} \rightarrow \Delta(\mathbf{U})$ is said to have (ρ, γ) -*random differential privacy* $((\rho, \gamma)\text{-DP})$ if, with probability $1 - \gamma$:

$$\forall u \in \mathbf{U}, \quad \log \mathcal{U}(X)\{u\} - \log \mathcal{U}(\hat{X})\{u\} \leq \rho(X, \hat{X}). \quad (\text{D.1})$$

Intuitively, metric DP describes a kind of Lipschitz continuity³ (in probability) and characterizes the sensitivity of mechanisms (stochastic controllers $\mathcal{U}$) to replacement of X with $\hat{X}$; specifically, the ratio of output densities is bounded by $\exp(\rho(X, \hat{X}))$. An

²A metric suited for the control problem at hand may present itself (see Appendix D.6 for problems with quadratic costs), or it may simply be taken to be a Euclidean metric.

³Specifically, it is Lipschitz continuity with a specific metric on probability measures [22].

important aspect of DP is that it is preserved under composition of mechanisms [18]. Two such composition properties are:

Proposition 9 (Post-Processing). If $\mathcal{M}_1 : \mathbf{X} \rightarrow \Delta(\mathbf{U})$ is (ρ, γ) -DP, then for any $\mathcal{M}_2$, $\mathcal{M}_2 \circ \mathcal{M}_1$ is (ρ, γ) -DP.

Proposition 10 (Composition). Let $\mathcal{M}_1, \mathcal{M}_2 : \mathbf{X} \rightarrow \Delta(\mathbf{U})$ be (ρ_1, γ_1) -DP and (ρ_2, γ_2) -DP mechanisms respectively. Then $\mathcal{M} = (\mathcal{M}_1, \mathcal{M}_2)$ is (ρ, γ) -DP where $\rho(x, \hat{x}) := \rho_1(x, \hat{x}) + \rho_2(x, \hat{x})$ and $\gamma := \gamma_1 + \gamma_2$.

Proof. See Appendix D.8. □

These results allow for the recursive composition of system dynamics and private controllers to yield a mechanism that computes costs in a private manner; this will allow us to extend the robustness theorem proven for single-step decision-making in the next subsection to multi-step problems (Appendix D.5).

D.4.3 The Exponential Mechanism and Bounded Rationality

A popular mechanism for DP is the *exponential mechanism* [18], which provides privacy in problems where the solution of an optimization problem is desired as output. Thus, it is a natural starting point for designing a *differentially-private optimal control algorithm*. Specifically, the exponential mechanism $\mathcal{U}^\beta : \mathbf{X} \rightarrow \Delta(\mathbf{U})$ is defined as the solution to the optimization problem:

$$\min_{\mathcal{U}} \mathbb{E} \left[H(X, U) + \beta^{-1} \mathbb{D}[\mathcal{U}(X) || \mathcal{U}^\perp] \right]. \quad (\text{BR-SS})$$

Here $\mathcal{U}^\perp \in \Delta(\mathbf{U})$ is a “prior” supported on $\mathbf{U}$ independent of X (emphasized by the superscript $\perp$) and β^{-1} is interpreted as the Lagrange multiplier of a relative entropy constraint. Specifically, for each value of β^{-1} , there exists $d > 0$ such that $\mathcal{U}^\beta$ solves the constrained problem:

$$\min_{\mathcal{U}} \mathbb{E} [H(X, U)] \quad \text{s.t.} \quad \mathbb{E} [\mathbb{D}[\mathcal{U}(X) || \mathcal{U}^\perp]] \leq d.$$

The values of β^{-1} and d are linked in that increasing β^{-1} corresponds to an increase in d , but in general an exact relationship is elusive.

Remark 10. This is an instance of a *maximum-entropy problem*⁴ from information theory [13, 60, 65], and corresponds to finding a mechanism that minimizes the expected cost without significant deviation from the prior (measured by the relative entropy). As $\beta \rightarrow \infty$, $\mathcal{U}^\beta(x)$ simply minimizes the expected cost. As $\beta \rightarrow 0$, the solution becomes the prior. Therefore, the mechanism can be viewed as an agent employing a *bounded-rational controller*, where it only uses a finite amount of information (due to computational or sensing constraints) about the state X to deviate from its default behavior specified by $\mathcal{U}^\perp$. Information usage (rationality) is directly controlled by β (inversely related to d).

Moreover, a prior that distributes weight evenly across its support may require a large value of β for $\mathcal{U}^\beta(x)$ to concentrate about the optimal control input. Similarly, a prior with a high concentration may require a large value of β and significant computational effort to produce solutions near the optimal control input. This latter case is a kind of exploration-exploitation trade-off due to the sampling schemes used by algorithmic implementations of the exponential mechanism.

The first-order optimality conditions for the problem (BR-SS) imply that the solution is a Gibbs measure [13, 60, 65]:

$$\mathcal{U}^\beta(x)\{u\} = \mathcal{U}^\perp\{u\} \exp(-\beta H(x, u)) / Z^\beta(x), \quad (\text{D.2})$$

$$Z^\beta(x) := \mathbb{E}^\perp[\exp(-\beta H(x, U))].$$

Here, $\mathbb{E}^\perp[\cdot]$ is the expectation computed using $\mathcal{U}^\perp$ as the controller. In the language from statistical mechanics, β is the *inverse temperature*, $H(x, u)$ is the *Hamiltonian* of the system, $Z^\beta(x)$ is the *partition function*, and,

$$F^\beta(x) := -\beta^{-1} \log Z^\beta(x), \quad (\text{D.3})$$

⁴The maximum-entropy problem (BR-SS) related to the *rate-distortion problems* common in bounded rationality models [8, 9, 32, 34] via a well-known variational relationship between the relative entropy and mutual information [99]. All subsequent analysis applies to these problems as well.

is known as the *free energy*. The latter is notably important for a number of reasons including it being a cumulant-generating function and its equality to the Lagrangian of (BR-SS) conditioned on $X = x$ (see Appendix D.8):

$$F^\beta(x) = \mathbb{E}[H(x, U)] + \beta^{-1} \mathbb{D}[\mathcal{U}^\beta(x) || \mathcal{U}^\perp]. \quad (\text{D.4})$$

Due to its popularity, the privacy properties of the exponential mechanism under different assumptions on $H(x, u)$ are well-studied. Only a simple assumption of Lipschitz continuity in x for each u is required [23]. However, this assumption will not hold for the applications of interest in Appendix D.6 due to $\mathbf{U}$ being non-compact. In these cases, random DP may still be achieved and is suitable for robust control:

Proposition 11. Consider the set of all $u \in \mathbf{U}$ for which $H(x, u)$ is at most l -Lipschitz in x , i.e. $\mathbf{U}(l) := \{u \in \mathbf{U} \mid \mathbb{L}[x \mapsto H(x, u)] < l\}$. Then, $\mathcal{U}^\beta(x)$ is $(2\beta l \rho, \gamma(l))$ -DP, where:

$$\gamma(l) := 1 - \mathbb{E} \left[1_{\mathbf{U}(l)}(U) \exp \left(-2\beta l \rho(X, \hat{\mathcal{X}}(X)) \right) \right].$$

Proof. See Appendix D.8. □

The proposition characterizes the trade-offs in selecting the free parameter l and prior $\mathcal{U}^\perp$. A larger value of l implies a larger set $\mathbf{U}(l)$ but a smaller region of integration in the definition of $\gamma(l)$. For very large values of l , the latter will dominate and the probability that privacy fails, which is $\gamma(l)$, becomes almost certain. Note that larger l implies a looser privacy constraint, since $2\beta l \rho(x, \hat{x})$ will grow for any fixed pair $x, \hat{x} \in \mathbf{X}$. The choice of prior $\mathcal{U}^\perp$ may also bias U toward regions that yield a smaller Lipschitz constant.

D.4.4 Quantifying the Robustness of Bounded Rationality

The key theoretical contribution of this paper is realizing that the proposed definition of random DP can quantify the performance of the bounded-rational agent (i.e., the private controller $\mathcal{U}^\beta$) when only a noisy estimate $\hat{X}$ of the state X is available — thereby approximating the separation principle. This idea is combined with a large deviations argument

that exploits the similarity between the large-deviation rate function [64] and the free energy (D.3) to derive the theorem:

Theorem 11. Define $\rho_\beta(x, \hat{x}) := 2\beta l \rho(x, \hat{x})$. With probability at least $1 - \gamma(l)$:⁵

$$\Delta \mathbb{J}[\mathcal{U}^\beta] \leq \beta^{-1} \mathbb{E} \left[\exp \left(\rho_\beta(X, \hat{X}) + \mathbb{D}[\mathcal{U}^\beta(X) || \mathcal{U}^\perp] \right) \right], \quad (\text{D.5})$$

Proof. See Appendix D.8. □

This theorem bounds the gap $\Delta \mathbb{J}[\mathcal{U}^\beta] := \mathbb{J}^{\text{on}}[\mathcal{U}^\beta] - \mathbb{J}^{\text{off}}[\mathcal{U}^\beta]$ in performance between the offline and online problems in terms of β , which describes the information usage of the controller. Importantly, it relates three separate quantities: the offline expected cost (which appears in $\Delta \mathbb{J}[\mathcal{U}^\beta]$), information usage (set by β), and the quality of the state estimator as measured by $\rho(x, \hat{x})$. Critically, all terms depend *only on offline information* — that is, the state estimator is not used for feedback in any of these terms but only to measure its error. Therefore, both elements of the feedback system may be designed and evaluated independently through this bound (similar to methodologies that adopt the separation principle). Moreover, the bound can be optimized to find β^* that yields the tightest bound on $\mathbb{J}^{\text{on}}[\mathcal{U}^\beta]$.

Remark 11. In cases where $\mathbb{D}[\mathcal{U}^\beta(X) || \mathcal{U}^\perp]$ cannot be evaluated, tractable bounds may be available depending on the context (e.g. Gibbs inequality [13], log-Sobolev inequalities [60, 65], and variational methods [62]).

D.5 Extension to Multi-Step Problems

In the multi-step optimal control problem, the agent attempts to minimize a sequence of non-negative cost functions $c_0, \dots, c_{t_f-1} : \mathbf{X} \times \mathbf{U} \rightarrow \mathbf{R}_+, c_{t_f} : \mathbf{X} \rightarrow \mathbf{R}_+$ over a time horizon t_f by selecting a feedback controller $U_t \sim \mathcal{U}_t(X_t)$ that accounts for the system dynamics, $X_{t+1} \sim \mathcal{F}_t(x_t, u_t)$. The state-input trajectories of the system generated by the

⁵Expectations in the theorem are conditioned on the event $U \in \mathbf{U}(l)$, which is why the consequent occurs with probability $\gamma(l)$.

choice of controller $\mathcal{U}_{0:t_f}$ is denoted $\mathcal{T}[\mathcal{U}_{0:t_f}]$ with $\mathcal{T}^\perp := \mathcal{T}[\mathcal{U}_{0:t_f}^\perp]$ being the prior trajectory distribution.

Introduce the shorthand for trajectory cost:

$$c_{0:t_f}(x_{0:t_f}, u_{0:t_f}) := c_0(x_0, u_0) + \cdots + c_{t_f}(x_{t_f}).$$

Adapting the single-step problem notation, the offline and online problems are written,

$$\begin{aligned} \min_{\mathcal{U}_{0:t_f}} \mathbb{J}^{\text{off}}[\mathcal{U}_{0:t_f}] &:= \mathbb{E}[c_{0:t_f}(X_{0:t_f}, U_{0:t_f})], \\ \min_{\mathcal{U}_{0:t_f}} \mathbb{J}^{\text{on}}[\mathcal{U}_{0:t_f}] &:= \mathbb{J}^{\text{off}}[(\mathcal{U} \circ \hat{\mathcal{X}})_{0:t_f}], \end{aligned}$$

where $\hat{\mathcal{X}}_{0:t_f}$ are mechanisms that introduce estimation error.

Bounded rationality is achieved in a similar manner to (BR-SS) by constraining the relative entropy between the sequence of closed-loop and open-loop controllers:

$$\min_{\mathcal{U}_{0:t_f}} \mathbb{J}^{\text{off}}[\mathcal{U}_{0:t_f}] \quad \text{s.t.} \quad \mathbb{D}[\mathcal{T}[\mathcal{U}_{0:t_f}] || \mathcal{T}^\perp] \leq d_{\text{traj}}. \quad (\text{BR-MS})$$

This optimal control problem admits a recursive solution,⁶

$$\mathcal{U}_t^\beta(x) := \arg \min_{\mathcal{U}_t} \mathbb{E}[H_t(X, U)] + \beta^{-1} \mathbb{D}[\mathcal{U}(x) || \mathcal{U}_t^\perp],$$

where β is the Lagrange multiplier corresponding to the entropy constraint. Then, $\mathcal{U}_t^\beta$ is an exponential mechanism and both the Hamiltonian and value function $V_t(x)$ are given by the recursive equations,

$$\begin{aligned} H_t(x, u) &:= c_t(x, u) + \mathbb{E}[V_{t+1}(\mathcal{F}_t(x, u))], \\ V_t(x) &:= \mathbb{E}[H_t(x, U_t)], \end{aligned}$$

where $V_{t_f}(x) := c_{t_f}(x)$. Notably, the free energy is equivalent to the value function: $F_t^\beta(x) = V_t(x)$. The aforementioned DP composition properties allow for extension of

⁶See, e.g. [8, 34, 37, 52, 58], for detailed solutions to similar problems.

Proposition 11, and subsequently Theorem 11, to the multi-step problem with changes made *mutatis mutandis*.

Proposition 12. Let $\mathbf{U}_t(l_t) := \{u \mid \mathbb{E}[x \mapsto H_t(x, u)] < l_t\}$. The mechanism,

$$\mathcal{M}(x_{0:t_f}) := (\mathcal{U}_1^\beta(x_1), \dots, \mathcal{U}_{t_f-1}^\beta(x_{t_f-1})),$$

is (ρ_β, γ) -DP where,

$$\rho_\beta(x_{0:t_f}, \hat{x}_{0:t_f}) := \sum_{t=0}^{t_f-1} 2\beta l_t \rho(x_t, \hat{x}_t), \quad \gamma := \sum_{t=0}^{t_f-1} \gamma_t,$$

and,

$$\gamma_t := 1 - \mathbb{E} \left[1_{\mathbf{U}_t(l_t)}(U_t) \exp \left(-2\beta l_t \rho(X_t, \hat{\mathcal{X}}_t(X_t)) \right) \right].$$

Theorem 12. Let $\rho_\beta(x, \hat{x})$ and γ be as in Proposition 12 and $\mathcal{T}^\beta := [\mathcal{U}_{0:t_f}]$. With probability at least $1 - \gamma(l_t)$,

$$\Delta \mathbb{J}[\mathcal{U}_{0:t_f}^\beta] \leq \frac{1}{\beta} \mathbb{E} \left[\exp \left(\rho_\beta(X_{0:t_f}, \hat{X}_{0:t_f}) + \mathbb{D}[\mathcal{T}^\beta \parallel \mathcal{T}^\perp] \right) \right].$$

Proof. See Appendix D.8. □

This result bounds the cumulative online cost in terms of the offline cost, level of information usage (set by β), and estimation error (measured by ρ) similarly to Theorem 11.

D.6 Numerical Examples

To demonstrate the efficacy of these robustness results, three numerical examples are presented in this section: a motion planning “double-slit” experiment and optimal control of both the linearized and nonlinear planar quadrotors.⁷ In each case, the dynamics of the

⁷The source code implementing these examples using JAX [199] and all experimental parameters are listed in a publicly available repository: <https://github.com/irom-lab/br-dp-robust>.

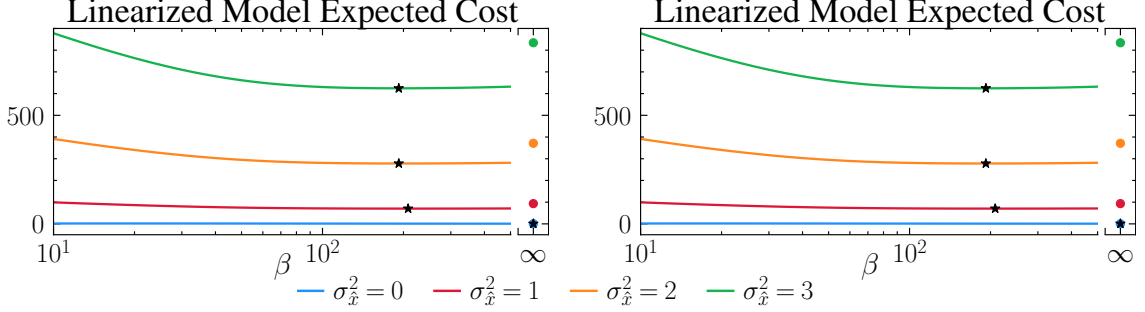


Figure D.2: Results of the numerical experiments described in Appendix D.6. Lines indicate the expected cost of stabilizing a linearized / nonlinear quadrotor model for a range of β and $\beta = \infty$. The latter case corresponds to the performance of the LQR / MPC controllers, which are performant given perfect state information. The $\star$ marker indicates $\beta^\star$ found for each value of σ_x^2 . For the nonlinear example, shading indicates standard deviations for 100 trials of SV-MPC. The bounded-rationality controllers outperform LQR / MPC in the presence of estimation error.

robot are chosen to be deterministic. While the developed theory permits stochastic dynamics, deterministic dynamics emphasizes the sources of randomness with which this article is focused: measurement error and bounded-rational control policies.

D.6.1 Motion Planning

Scenario. In this problem, the objective of the robot is to find the shortest possible path to some goal region $\mathbf{X}_g \subset \mathbf{X}$ while avoiding a set of obstacle configurations $\mathbf{X}_o \subset \mathbf{X}$ that is disjoint from $\mathbf{X}_g$. Both $\mathbf{X}_g$ and $\mathbf{X}_o$ are absorbing sets. For simplicity, the dynamics are the single integrator in the plane: $x_{t+1} = x_t + u_t$ with $\mathbf{X}, \mathbf{U} = [0, 1]^2$. The problem cost is the path length if the robot reaches the goal before intersecting with an obstacle and ∞ otherwise. The prior $\mathcal{U}_{0:t_f}^\perp$ is chosen to be a zero-mean Gaussian distribution.

The specific work space navigated by the robot is shown in Fig. D.1. The region $\mathbf{X}_o$ consists of the boundary of the shown space and a divider punctured with a large and small slit. A path through the large slit is more robust to measurement error while a path through the smaller one is less forgiving. The goal region is in the lower right. Similar experiments are present in the literature [40, 44–46].

Results. Importance sampling [54] is used to implement the bounded rationality controller. As shown in Fig. D.1, as the information constraint tightens ($\beta \rightarrow 0$), the agent shifts from navigating the direct-but-treacherous gap to the robust (indirect) route. This

demonstrates the robustness benefits of our differentially private control scheme.

D.6.2 Planar Quadrotor Stabilization Problems

Scenario. These examples focus on the stabilization of a planar quadrotor (constrained to the Y-Z plane and rotating about the X axis). The system state $x \in \mathbf{R}^6$ consists of the position and rotation of the robot and the time derivatives of these variables, i.e. $x = (y, z, \theta, \dot{y}, \dot{z}, \dot{\theta})$, while the control input $u \in \mathbf{R}^2$ is the thrust exerted by two opposing pairs of rotors (see [78, Section 6.6] for dynamics). The cost functions are the linear-quadratic regulator (LQR) cost functions [1]. By choosing the metric,

$$\rho(x, \hat{x}) = \frac{1}{2} \|xx^T - \hat{x}\hat{x}^T\|_F + \|x - \hat{x}\|_2,$$

the Hamiltonian $H_t(x, u)$ satisfies the conditions for the theory from Appendix D.5 to apply to both systems.

The system parameters are chosen to align with the Crazyflie 2.0 quadrotors: the mass is 0.03 kg and the moment of inertia is $1.43 \times 10^{-5} \text{ kg m}^2$ [79]. The system is temporally discretized with a time step of $\Delta t = 0.3 \text{ s}$ and $t_f = 13$. The initial condition is sampled from small Gaussian perturbations about the hover state given according to $\mathcal{N}(\bar{x}_0, \sigma_x^2)$ where $\bar{y}_0 = 1 \text{ m}$, $\bar{z}_0 = -1 \text{ m}$, the remaining mean entries are zero, and $\sigma_{x_0}^2 = (10^{-2}, 10^{-2}, 10^{-6}, 10^{-4}, 10^{-4}, 10^{-8})$. The open-loop prior is chosen by projecting the initial distribution through the LQR solution to find the marginal input. The estimation error is drawn from a stationary Gaussian distribution. Specifically, $\hat{\mathcal{X}}(x) = \mathcal{N}(x, \sigma_x^2 v)$, where $\sigma_x^2 \in \mathbf{R}_+$ is a scale parameter and $v = \text{diag}(0.25, 0.25, 0.1, 0.25, 0.25, 0.1)$.

Linearized Results. The system is linearized and the dynamic programming equations specifying the control policy are solved exactly for a feedback policy that is linear in the state with additive Gaussian noise. Details are included in Appendix D.8 The performance of the bounded-rationality controller is evaluated and compared with a controller that is optimal assuming perfect state estimation (LQR) paired with a sub-optimal state estimator. As shown in Fig. D.2, the bounded-rationality controller is more robust to estimation error than the LQR controller. The optimal information usage β^* decreases with the uncertainty

σ_x^2 .

Nonlinear Results. The experiments were then repeated using the nonlinear planar quadrotor dynamics. There is no closed-form solution to (BR-MS), but numerical methods for sampling from $\mathcal{U}_t^\beta(x)$ are available (see Appendix D.2.3). The SV-MPC algorithm [52] is chosen due to its increased efficiency compared to importance sampling and the fact that it reduces to gradient-based optimization of the trajectory with multiple random initializations in the $\beta \rightarrow \infty$ limit, which is a common MPC algorithm. The results are similar to the linear case: there is a prominent local minimum for β that outperforms MPC (which is optimal assuming perfect estimation) in the presence of estimation error when averaged over 100 trials. In this case, the monotonic relationship between β^* and σ_x^2 is not seen — possibly due to SV-MPC only approximating the controller’s distribution.

D.7 Conclusion

This paper proposes new theoretical justifications for the robustness of bounded-rational control policies using differential privacy. The stated performance guarantee for such policies has a modular structure reminiscent of the separation principle. Multiple numerical simulations demonstrate that using DP to create controllers provides robustness to estimation error.

Future Work. There remain a number of useful properties that need to be determined about the stated bounds that extend beyond the scope of this paper, e.g., the tightness of the bounds and whether the relationship between β^* and σ_x^2 is monotonic as suggested by the results for the linearized system in Fig. D.2. Developing tractable method to evaluate the robustness bounds is also of great interest. Opportunities to extend the experimental results of the article to new applications, such as sim-to-real transfer, and adapting other mechanisms from the DP literature to robust controller design may be considered.

D.8 Proofs

Proposition 9 (Post-Processing). If $\mathcal{M}_1 : \mathbf{X} \rightarrow \Delta(\mathbf{U})$ is (ρ, γ) -DP, then for any $\mathcal{M}_2$, $\mathcal{M}_2 \circ \mathcal{M}_1$ is (ρ, γ) -DP.

Proof. Let $\mathbf{Z}' \subset \mathbf{Z}$ be measurable and $\mathbf{Y}(\mathbf{Z}') := \{y \in \mathbf{Y} \mid \mathcal{M}_2(y)\{\mathbf{Z}'\} > 0\}$. Then for any $x, \hat{x} \in \mathbf{X}$,

$$\frac{(\mathcal{M}_2 \circ \mathcal{M}_1)(x)\{\mathbf{Z}'\}}{(\mathcal{M}_2 \circ \mathcal{M}_1)(\hat{x})\{\mathbf{Z}'\}} = \frac{\mathcal{M}_1(x)\{\mathbf{Y}(\mathbf{Z}')\}}{\mathcal{M}_1(\hat{x})\{\mathbf{Y}(\mathbf{Z}')\}} \leq \rho(x, \hat{x}). \quad (\text{D.6})$$

holds with precisely the probability that $\mathcal{M}_1(x)$ is private, i.e. probability $1 - \gamma$. $\square$

Proposition 10 (Composition). Let $\mathcal{M}_1, \mathcal{M}_2 : \mathbf{X} \rightarrow \Delta(\mathbf{U})$ be (ρ_1, γ_1) -DP and (ρ_2, γ_2) -DP mechanisms respectively. Then $\mathcal{M} = (\mathcal{M}_1, \mathcal{M}_2)$ is (ρ, γ) -DP where $\rho(x, \hat{x}) := \rho_1(x, \hat{x}) + \rho_2(x, \hat{x})$ and $\gamma := \gamma_1 + \gamma_2$.

Proof. Assume $\gamma_1, \gamma_2 = 1$. Let $\mathbf{V} := \mathbf{V}_1 \times \mathbf{V}_2 \subset \mathbf{V} \times \mathbf{V}$ be measurable and $x, \hat{x} \in \mathbf{X}$. Then:

$$\log \frac{\mathcal{M}(x)\{\mathbf{V}\}}{\mathcal{M}(\hat{x})\{\mathbf{V}\}} = \log \frac{\mathcal{M}_1(x)\{\mathbf{V}_1\}}{\mathcal{M}_1(\hat{x})\{\mathbf{V}_1\}} + \log \frac{\mathcal{M}_2(x)\{\mathbf{V}_2\}}{\mathcal{M}_2(\hat{x})\{\mathbf{V}_2\}} \leq \rho_1(x, \hat{x}) + \rho_2(x, \hat{x}). \quad (\text{D.7})$$

Now, when $\gamma_1, \gamma_2 < 1$, the above holds is estimated via the union bound:

$$\begin{aligned} \mathbb{P}\{\mathcal{M}_1 \text{ is private} \wedge \mathcal{M}_2 \text{ is private}\} &= 1 - \mathbb{P}\{\mathcal{M}_1 \text{ is not private} \vee \mathcal{M}_2 \text{ is not private}\}, \\ &\geq 1 - (\mathbb{P}\{\mathcal{M}_1 \text{ is not private}\} + \mathbb{P}\{\mathcal{M}_2 \text{ is not private}\}) \geq 1 - (\gamma_1 + \gamma_2). \end{aligned}$$

$\square$

Proposition 11. Consider the set of all $u \in \mathbf{U}$ for which $H(x, u)$ is at most l -Lipschitz in x , i.e. $\mathbf{U}(l) := \{u \in \mathbf{U} \mid \mathbb{L}[x \mapsto H(x, u)] < l\}$. Then, $\mathcal{U}^\beta(x)$ is $(2\beta l \rho, \gamma(l))$ -DP, where:

$$\gamma(l) := 1 - \mathbb{E} \left[1_{\mathbf{U}(l)}(U) \exp \left(-2\beta l \rho(X, \hat{X}(X)) \right) \right].$$

Proof. For all $u \in \mathbf{U}(l)$, (D.1) holds for the metric $2\beta l \rho(x, x')$ [23, Theorem 8]. All that is needed is to lower bound the probability of sampling $U \in \mathbf{U}(l)$, i.e. $U \in \mathcal{U}(\hat{X})\{\mathbf{U}(l)\}$. Since, again, (D.1) holds for all $u \in \mathbf{U}(l)$, it follows that for any $x, \hat{x} \in \mathbf{X}$:

$$\mathcal{U}^\beta(x)\{\mathbf{U}(l)\} \leq \exp(2\beta l \rho(x, \hat{x})) \mathcal{U}^\beta(\hat{x})\{\mathbf{U}(l)\} \quad (\text{D.8})$$

This inequality may be rewritten as:

$$\mathbb{E}^\perp \left[1_{\mathbf{U}(l)}(U) \exp(-2\beta l \rho(x, \hat{x})) \frac{\exp(-\beta H(x, U))}{Z^\beta(x)} \right] \leq \mathbb{E}^\perp \left[1_{\mathbf{U}(l)}(U) \frac{\exp(-\beta H(\hat{x}, U))}{Z^\beta(\hat{x})} \right]. \quad (\text{D.9})$$

Finally, the law of total expectation implies:

$$1 - \gamma(l) = \mathbb{E}^\perp \left[1_{\mathbf{U}(l)}(U) \exp(-2\beta l \rho(X, \hat{X})) \frac{\exp(-\beta H(X, U))}{Z^\beta(X)} \right] \quad (\text{D.10})$$

$$\leq \mathbb{E}^\perp \left[1_{\mathbf{U}(l)}(U) \frac{\exp(-\beta H(\hat{X}, U))}{Z^\beta(\hat{X})} \right]. \quad (\text{D.11})$$

The right hand side of this inequality is the probability that $U \in \mathbf{U}(l)$ as desired. $\square$

Property 1. For any $\mathcal{U}(x)$ such that $\mathcal{U}(x) \ll \mathcal{U}^\perp$ and any $x \in \mathbf{X}, u \in \mathbf{U}$, it holds that,

$$F^\beta(x) \leq \mathbb{E}[H(x, \mathcal{U}(x))] + \frac{1}{\beta} \mathbb{D}[\mathcal{U}(x) || \mathcal{U}^\perp]. \quad (\text{D.12})$$

The inequality is tight when $\mathcal{U} = \mathcal{U}^\beta$.

Proof. To begin, rearrange (D.2) into:

$$H(x, u) + \frac{1}{\beta} \log \frac{\mathcal{U}^\beta(x)\{u\}}{\mathcal{U}^\perp\{u\}} = F^\beta(x). \quad (\text{D.13})$$

Take the expectation with respect to $\mathcal{U}^\beta(x)$ of both sides ($F^\beta(x)$ is constant in u) to achieve the equality half of the consequent (D.12). Now, since $\mathcal{U}^\beta$ is the solution to the minimization problem (BR-SS), any other feasible choice of $\mathcal{U}$ (i.e. any choice such that $\mathcal{U} \ll \mathcal{U}^\perp$) will produce a value for the problem greater than or equal the optimal value. Therefore,

$$F^\beta(x) = \mathbb{E}[H(x, \mathcal{U}^\beta(x))] + \beta^{-1} \mathbb{D}[\mathcal{U}^\beta(x) || \mathcal{U}^\perp] \quad (\text{D.14})$$

$$\leq \mathbb{E}[H(x, \mathcal{U}(x))] + \frac{1}{\beta} \mathbb{D}[\mathcal{U}(x) || \mathcal{U}^\perp]. \quad (\text{D.15})$$

$\square$

Property 2. With respect to β , the free energy $F_\beta(x)$ has derivative,

$$\frac{\partial F^\beta(x)}{\partial \beta} = -\frac{\mathbb{D}[\mathcal{E}^\beta(x) \|\mathcal{E}^\perp]}{\beta^2} \leq 0, \quad (\text{D.16})$$

and limits,

$$\lim_{\beta \rightarrow \infty} F^\beta(x) = \inf_{u \in \mathbf{U}} H(x, u), \quad \lim_{\beta \rightarrow 0} F^\beta(x) = \mathbb{E}^\perp[H(x, U)]. \quad (\text{D.17})$$

Proof. The first property is directly from differentiation. The second follows from the fact that finite solutions to (BR-SS) require $\forall x \in \mathbf{X}, \mathcal{E}^\beta(x) \ll \mathcal{E}^\perp$, and increasing β causes the density to concentrate on minimizing members in the support of $\mathcal{E}^\perp$. The last property follows from L'Hôpital's rule. $\square$

Property 3. Define $l(u) := \mathbb{L}[x \mapsto H(x, u)]$ is l -Lipschitz continuous, then $F^\beta(x)$ is $\mathbb{E}[l(U)]$ -Lipschitz continuous.

Proof. Since $x \mapsto H(x, u)$ is $l(u)$ -Lipschitz continuous,

$$\left\| \frac{\partial H(x, u)}{\partial x} \right\|_\infty \leq l(u). \quad (\text{D.18})$$

Differentiating $F^\beta(x) = -\beta^{-1} \log Z^\beta(x)$:

$$\frac{\partial F^\beta(x)}{\partial x} = \frac{\mathbb{E}^\perp \left[\frac{\partial H(x, U)}{\partial x} \exp(-\beta H(x, U)) \right]}{Z^\beta(x)}. \quad (\text{D.19})$$

Jensen's inequality implies:

$$\begin{aligned} \left\| \frac{\partial F^\beta(x)}{\partial x} \right\|_\infty &= \left\| \frac{\mathbb{E}^\perp \left[\frac{\partial H(x, U)}{\partial x} \exp(-\beta H(x, U)) \right]}{Z^\beta(x)} \right\|_\infty, \\ &\leq \frac{\mathbb{E}^\perp \left[\left\| \frac{\partial H(x, U)}{\partial x} \right\|_\infty \exp(-\beta H(x, U)) \right]}{Z^\beta(x)}, \\ &\leq \frac{\mathbb{E}^\perp [l(U) \exp(-\beta H(x, U))]}{Z^\beta(x)} = \mathbb{E}[l(U)]. \end{aligned} \quad (\text{D.20})$$

$\square$

Theorem 11. Define $\rho_\beta(x, \hat{x}) := 2\beta l \rho(x, \hat{x})$. With probability at least $1 - \gamma(l)$:⁸

$$\Delta \mathbb{J}[\mathcal{U}^\beta] \leq \beta^{-1} \mathbb{E} \left[\exp \left(\rho_\beta(X, \hat{X}) + \mathbb{D}[\mathcal{U}^\beta(X) | \mathcal{U}^\perp] \right) \right], \quad (\text{D.5})$$

Proof. Define the standard notation $\mathbb{P}\{\mathbf{A}\} := \mathbb{E}[1_{\mathbf{A}}(\mathbf{A})]$ for $\mathbf{A}$ is a subset of the space over which integration is performed, i.e. $\mathbf{A} \subseteq \mathbf{X} \times \mathbf{X} \times \mathbf{U}$. The Chernoff bound [65] applied to the mechanism $\hat{\mathcal{H}}(x) := H(x, (\mathcal{U}^\beta \circ \hat{\mathcal{X}})(x))$ yields for any $a, \zeta \in \mathbf{R}_+$:

$$\mathbb{P} \left\{ \hat{\mathcal{H}}(x) \geq a \right\} \leq \mathbb{E} \left[\exp \left(\zeta \hat{\mathcal{H}}(x) - \zeta a \right) \right]. \quad (\text{D.21})$$

Since $\mathcal{U}^\beta(x)$ is $(\rho_\beta, \gamma(l))$ -private, the mechanism $\hat{\mathcal{H}}(x)$ can be switched with its fully-observable version $\mathcal{H}(x) := H(x, \mathcal{U}^\beta(x))$ when the event $U \in \mathbf{U}(l)$ occurs. This event transpires with probability $\gamma(l)$. Conditioned on this occurrence, the bound becomes:

$$\mathbb{P} \left\{ \hat{\mathcal{H}}(x) \geq a \right\} \leq \mathbb{E} \left[\exp \left(\rho_\beta(x, \hat{X}) + \zeta \mathcal{H}(x) - \zeta a \right) \right]. \quad (\text{D.22})$$

Since $\rho_\beta(x, \hat{X})$ is independent from $\mathcal{H}(x)$, the expectation factors and the right-hand side may be rewritten,

$$\mathbb{P} \left\{ \hat{\mathcal{H}}(x) \geq a \right\} \leq \mathbb{E} \left[\exp \left(\rho_\beta(x, \hat{X}) - \zeta a \right) \right] \mathbb{E} \left[\exp \left(\zeta \mathcal{H}(x) \right) \right], \quad (\text{D.23})$$

$$= \mathbb{E} \left[\exp \left(\rho_\beta(x, \hat{X}) + I(a; \zeta) \right) \right], \quad (\text{D.24})$$

where:

$$I(a; \zeta) := \log \mathbb{E} \left[\exp \left(\zeta \mathcal{H}(x) \right) \right] - \zeta a. \quad (\text{D.25})$$

When the infimum of $I(a; \zeta)$ is taken with respect to the ζ , this function is known as the *rate function* in large-deviation theory [64]. However, such extremization will not be performed. Instead, the values of a and ζ will be judiciously chosen based on the properties of $\mathcal{U}^\beta(x)$ so that β may be optimized over to jointly minimize all terms. In addition to be

⁸Expectations in the theorem are conditioned on the event $U \in \mathbf{U}(l)$, which is why the consequent occurs with probability $\gamma(l)$.

analytically tractable, it prevents the optimal value of β being tied to the optimal value of ζ , which is dependent on x — the latter is unknown to the controller in the online scenario and therefore not useful for selecting β . Specifically, choose $\zeta = \beta$ and $a = \mathbb{E}[\mathcal{H}(x)] + b$. In this case, as a result of Property 1,

$$\begin{aligned}
I(\mathbb{E}[\mathcal{H}(x)] + b; \beta) &:= \log \mathbb{E} [\exp (\beta H(x, U))] - \beta(\mathbb{E}[\mathcal{H}(x)] + b), \\
&= \log \mathbb{E}^\perp \left[\frac{\exp (\beta H(x, U)) \exp (-\beta H(x, U))}{Z^\beta(x)} \right] - \beta(\mathbb{E}[\mathcal{H}(x)] + b), \\
&= -\log Z^\beta(x) - \beta(\mathbb{E}[\mathcal{H}(x)] + b), \\
&= \beta F^\beta(x) - \beta(\mathbb{E}[\mathcal{H}(x)] + b), \tag{D.26}
\end{aligned}$$

$$= \mathbb{D}[\mathcal{U}^\beta(x) || \mathcal{U}^\perp] - \beta b. \tag{D.27}$$

Plugging these values into the concentration inequality yields:

$$\mathbb{P} \left\{ \hat{\mathcal{H}}(x) - \mathbb{E}[\mathcal{H}(x)] \geq b \right\} \leq \mathbb{E} \left[\exp \left(\rho_\beta(x, \hat{X}) + \mathbb{D}[\mathcal{U}^\beta(x) || \mathcal{U}^\perp] - \beta b \right) \right]. \tag{D.28}$$

After integrating b from the realizable range of the quantity $\hat{\mathcal{H}}(x) - \mathbb{E}[\mathcal{H}(x)]$, namely from $b = -\mathbb{E}[\mathcal{H}(x)]$ to the limit $b \rightarrow \infty$ to find,

$$\mathbb{E}[\hat{\mathcal{H}}(x)] - \mathbb{E}[\mathcal{H}(x)] \leq \beta^{-1} \mathbb{E} \left[\exp \left(\rho_\beta(x, \hat{X}) + \mathbb{D}[\mathcal{U}^\beta(x) || \mathcal{U}^\perp] \right) \right], \tag{D.29}$$

apply the law of total expectation to get the s:

$$\mathbb{J}^{\text{on}}[\mathcal{U}^\beta] - \mathbb{J}^\star \leq \beta^{-1} \mathbb{E} \left[\exp \left(\rho_\beta(X, \hat{X}) + \mathbb{D}[\mathcal{U}^\beta(X) || \mathcal{U}^\perp] \right) \right]. \tag{D.30}$$

The other inequality is derived in the same manner by choosing $a = J^\star + b$, and noting that J^{off} is the lowest achievable cost for either the online or offline control problem, therefore the limits of integration over b should be from $b = 0$ to the limit $b \rightarrow \infty$. $\square$

Theorem 12. Let $\rho_\beta(x, \hat{x})$ and γ be as in Proposition 12 and $\mathcal{T}^\beta := [\mathcal{U}_{0:t_f}]$. With probability

at least $1 - \gamma(l_t)$,

$$\Delta \mathbb{J}[\mathcal{U}_{0:t_f}^\beta] \leq \frac{1}{\beta} \mathbb{E} \left[\exp \left(\rho_\beta(X_{0:t_f}, \hat{X}_{0:t_f}) + \mathbb{D}[\mathcal{T}^\beta || \mathcal{T}^\perp] \right) \right].$$

Proof. Let $U_t \sim \mathcal{U}_t^\beta(X_t)$, $\hat{U}_t \sim (\mathcal{U}_t^\beta \circ \hat{X}_t)(X_t)$, and let $\mathbb{P}$ be the same as in the proof of Theorem 11 except now defined over trajectories of the pertinent variables. Apply Proposition 12 so that the bound:

$$\mathbb{P}\{c_{0:T}(X_{0:t_f}, \hat{U}_{0:t_f}) \geq a\} \leq \exp\left(\rho_\beta(X_{0:t_f}, \hat{X}_{0:t_f})\right) \mathbb{P}\{c_{0:t_f}(X_{0:t_f}, U_{0:t_f}) \geq a\}. \quad (\text{D.31})$$

Now, as in Theorem 11, all that remains is a large-deviation analysis of $\mathbb{P}\{c_{0:t_f}(X_{0:t_f}, U_{0:t_f}) \geq a\}$. This analysis is achieved by comparing the solution of (BR-MS) with a relaxation of the optimization problem. First, for clarity, (BR-MS) is rewritten in the following explicit form:

$$\begin{aligned} & \underset{\mathcal{T}, \mathcal{U}_{0:t_f}}{\text{minimize}} && \mathbb{E} \left[c_{0:t_f}(X_{0:t_f}, U_{0:t_f}) \right] \\ & \text{subject to} && X_0 \sim \mathcal{X}_0, && X_{t+1} \sim \mathcal{F}_t(X_t, U_t), \\ & && U_t \sim \mathcal{U}_t(X_t), && \mathcal{T}_t \ll \mathcal{T}_t^\perp, \\ & && \mathcal{T}(x_0) = (x_0, X_{1:t_f}, U_{0:t_f}), && \mathbb{D}[\mathcal{T}(x_0) || \mathcal{T}^\perp] \leq d_{\text{traj}}. \end{aligned} \quad (\text{BR})$$

Here, the trajectory distribution, conditioned on the initial condition x_0 , is the mechanism $\mathcal{T}(x_0)$ and appears as a decision variable. Through the equality constraint, it is entirely defined by the choice of $\mathcal{U}_{0:t_f}(x_{0:t_f})$.

The problem (BR) is recognized as a maximum-entropy problem like (BR-SS). However, it has an additional dynamical constraint: $X_{t+1} \sim \mathcal{F}_t(X_t, U_t)$. Therefore, from Property 1, it follows that:

$$\begin{aligned} \log \mathbb{E} \left[c_{0:t_f}(X_{0:t_f}, U_{0:t_f}) \right] &= -\log \mathbb{E}^\perp \left[\exp \left(c_{0:t_f}(X_{0:t_f}, U_{0:t_f}) \right) \right] \\ &\leq \beta \mathbb{E} \left[c_{0:t_f}(X_{0:t_f}, U_{0:t_f}) \right] + \mathbb{D}[\mathcal{U}^\beta(x) || \mathcal{U}^\perp]. \end{aligned}$$

By defining,

$$\begin{aligned}\mathcal{H}(x_0) &:= c_0(x_0, \mathcal{U}_0^\beta(x_0)) + \sum_{t=1}^{t_f-1} c_t(X_t, \mathcal{U}_t^\beta(X_t)) + c_{t_f}(X_{t_f}), \\ \hat{\mathcal{H}} &:= c_0(x_0, (\mathcal{U}_0^\beta \circ \hat{\mathcal{X}}_0)(x_0)) + \sum_{t=1}^{t_f-1} c_t(X_t, (\mathcal{U}_t^\beta \circ \hat{\mathcal{X}}_t)(X_t)) + c_{t_f}(X_{t_f}),\end{aligned}$$

where $\mathcal{H}(x_0)$ uses the feedback policy $\mathcal{U}_t^\beta(x_t)$ and $\hat{\mathcal{H}}(x_0)$ uses the feedback policy $(\mathcal{U}_t^\beta \circ \hat{\mathcal{X}}_t)(x_t)$, the steps of the proof of Theorem 11 may now be repeated with the caveat that transitioning from (D.26) to (D.27) is now done using the preceding inequality. Since $I(a; \zeta)$ appears as a positive term in the exponent of the Chernoff bound, this alternation does not change the proof steps meaningfully. The inequality is also tight in the notable case that the dynamics are deterministic, since the distribution over feasible state-input trajectories is now fully determined by the control policy. $\square$

D.9 Explicit LQG Solution of the Bounded-Rationality Policy

This section derives the linear-quadratic-Gaussian (LQG) solution for the bounded-rationality optimal control problem (BR-MS). The strict notation the document's body is relaxed for one more suited for linear algebra. For completeness the model is restated with the addition of Gaussian process noise. The dynamics are,

$$X_{t+1} = AX_t + BU_t + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, \Sigma_{\epsilon_t}), \quad (\text{D.32})$$

and the cost functions are,

$$c_t(x, u) = x^\top Qx + u^\top Ru, \quad c_{t_f}(x) = x^\top Q_f x. \quad (\text{D.33})$$

Here, $\mathbf{X} = \mathbf{R}^n$, $\mathbf{U} = \mathbf{R}^m$ and $Q, Q_f, \Sigma_{\epsilon_t} \in \mathbf{S}_{++}^n, R \in \mathbf{S}_{++}^m$ are positive-definite matrices, and X_t, U_t are the random states at time $0 \leq t \leq t_f$.

The goal is to find a policy that minimizes the total of the above cost functions over

the horizon t_f . To begin the dynamic programming argument, assume a quadratic form for $V_{t+1}(x)$:

$$V_{t+1}(x) = \frac{1}{2}x^T P_{t+1}x + b_{t+1}^T x + d_{t+1}. \quad (\text{D.34})$$

Then, $H_t(x, u)$ is expressed as,

$$H_t(x, u) = \frac{1}{2}(u - v_t)^T S_t^{-1}(u - v_t) + k_t(x), \quad (\text{D.35})$$

where:

$$S_t := (B^T P_{t+1} B + R)^{-1}, \quad v_t := -S_t B^T (P_{t+1} A x + b_{t+1}), \quad (\text{D.36})$$

Since $H_t(x, u)$ is quadratic, choosing $\mathcal{U}_t^\perp$ to be Gaussian ensures (D.2) is Gaussian: $\pi_t(u_t|x_t) = \mathcal{N}(\bar{u}_t(x_t), \Sigma_{u_t|x_t})$. Plugging in the prior and completing the square yields:

$$\bar{u}(x) = -\Sigma_{u_t|x_t}(\beta B^T P_{t+1} A x + \beta B^T b_{t+1} + \Sigma_{u_t^\perp}^{-1} \bar{u}_t^\perp), \quad \Sigma_{u_t|x_t}^{-1} = \beta B^T P_{t+1} B + \beta R - \Sigma_{u_t^\perp}^{-1}. \quad (\text{D.37})$$

More practically, the distribution may be expressed as the affine relationship,

$$u_t = K_t x_t + \eta_t, \quad \eta_t \sim \mathcal{N}(\bar{\eta}_t, \Sigma_{\eta_t}). \quad (\text{D.38})$$

where:

$$\Sigma_{\eta_t}^{-1} = \beta B^T P_{t+1} B + \beta R + \Sigma_{u_t^\perp}^{-1}, \quad \bar{\eta}_t = -\Sigma_{\eta_t}(\beta B^T b_{t+1} - \Sigma_{u_t^\perp}^{-1} \bar{u}_t^\perp), \quad K_t = -\beta \Sigma_{\eta_t} B^T P_{t+1} A. \quad (\text{D.39})$$

Finally, the value function at time t can be computed as,

$$V_t(x) = \mathbb{E}[H_t(x, U_t)] = \frac{1}{2}x^T P_t x + b_t^T x + d_t. \quad (\text{D.40})$$

where

$$\begin{aligned}
P_t &= Q + K_t^T R K_t + A^T P_{t+1} A + K_t^T B^T P_{t+1} B K_t + \frac{1}{2} (K_t^T B^T P_{t+1} A + A^T P_{t+1} B K_t), \\
b_t^T &= \bar{\eta}_t^T R K_t + \bar{\eta}_t^T B^T P_{t+1} A + \bar{\eta}_t^T B^T P_{t+1} B K_t + b_{t+1}^T (A + B K_t), \\
d_t &= d_{t+1} + \frac{1}{2} \bar{\eta}_t^T (R + B^T P_{t+1} B) \bar{\eta}_t + b_{t+1}^T B \bar{\eta}_t + \frac{1}{2} \text{tr} \left(\Sigma_{\eta_t} (R + B^T P_{t+1} B) \right) + \frac{1}{2} \text{tr} (\Sigma_{\epsilon_t} P_{t+1}).
\end{aligned}$$

The recursive description is completed by observing that the terminal cost function implies

$$P_{t_f} = Q_f, b_{t_f} = 0, d_{t_f} = 0.$$

Remark 12. As $\beta \rightarrow \infty$, the LQR solution is recognized (and is the same as the linear-quadratic Gaussian problem solution). Similarly, as $\beta \rightarrow 0$, it is clear that the statistics of U_t match those of $\pi_t^\perp$.

Appendix E

Fundamental Performance Limits for Sensor-Based Robot Control and Policy Learning

E.1 Introduction

Robotics is often characterized as the problem of transforming “pixels to torques” [200]: how can an embodied agent convert raw sensor inputs into actions in order to accomplish a given task? In this paper, we seek to understand the *fundamental limits* of this process by studying the following question: is there an *upper bound* on performance imposed by the sensors that a robot is equipped with?

As a motivating example, consider the recent debate around the “camera-only” approach to autonomous driving favored by Tesla versus the “sensor-rich” philosophy pursued by Waymo [86]. Is an autonomous vehicle equipped only with cameras *fundamentally limited* in terms of the performance or safety it can achieve? By “fundamental limit”, we mean a bound on performance (or safety) on a given task that holds *regardless* of the form of control policy one utilizes (e.g., a neural network with billions of parameters, a nonlinear model predictive control scheme combined with a particle filter, etc.), how the policy is synthesized (e.g., via model-free reinforcement learning, model-based control, etc.), or how

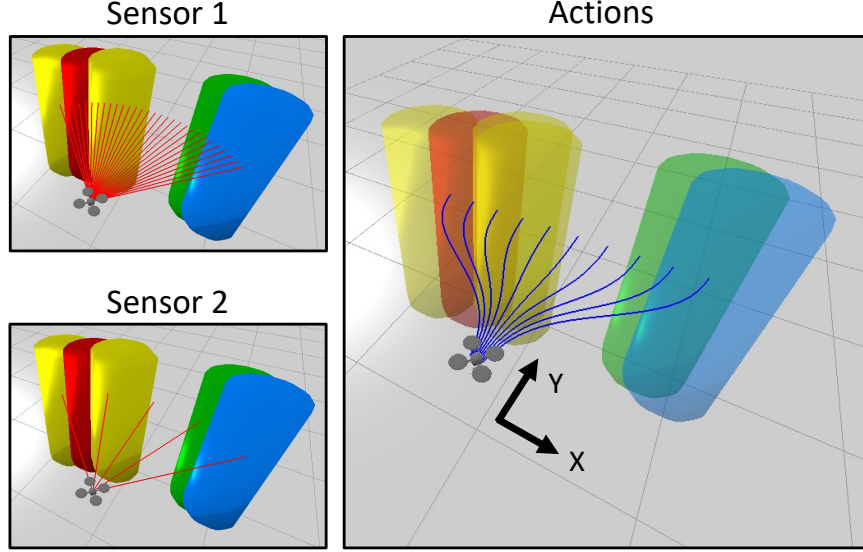


Figure E.1: Our goal is to establish fundamental limits on performance for a given task imposed by a robot’s sensors. We demonstrate our approach on examples that include obstacle avoidance with a noisy depth sensor (fig. left) using motion primitives (fig. right). We also show how we can use our approach to establish the superiority of one sensor (Sensor 1: a dense depth sensor) over another (Sensor 2: a sparse depth sensor) for a given task.

much computation is available to the robot or software designer.

While there have been tremendous algorithmic advancements in robotics over decades, we currently lack a “science” for understanding such fundamental limits [88]. Current practice in robotics is often *empirical* in nature (e.g., trying different perception and control architectures with neural networks of growing size and varying architectures). Techniques for establishing fundamental limits imposed by a sensor would potentially allow us to glean important design insights (e.g., realizing that a particular sensor is not sufficient for a task and must be replaced). Further, such techniques could allow us to establish the superiority of one suite of sensors over another from the perspective of a given task, e.g., by synthesizing a control policy for one sensor suite that achieves better performance than the fundamental bound for another suite.

In this paper, we take a step towards this goal. We first observe that any technique for establishing fundamental bounds on performance imposed by a given sensor must take into account two factors: (i) the *quality* of the sensor (e.g., the amount of information about the state of the robot and its environment provided by the sensor), and, importantly, (ii) the

task that the robot is meant to accomplish. As an example, consider a drone equipped with a (noisy) depth sensor (Fig. E.1). Depending on the nature of the task, the robot may need more or less information from its sensors. For example, suppose that the obstacle locations are highly constrained such that a particular sequence of actions always succeeds in avoiding them (i.e., there is a purely open-loop policy that achieves good performance on the task); in this case, even an extremely noisy or sparse depth sensor allows the robot to perform well. However, if the distribution of obstacles is such that there is no pre-defined gap in the obstacles, then a noisy or sparse depth sensor may fundamentally limit the achievable performance on the task. The achievable performance is thus intuitively influenced by the amount of *task-relevant information* provided by the robot’s sensors.

Statement of contributions. Our primary contribution is to develop theory and algorithms for establishing fundamental bounds on performance imposed by a robot’s sensors for a given task. Our key insight is to define a quantity that captures the *task-relevant information* provided by the robot’s sensors. Using a novel version of the *generalized Fano inequality* from information theory, we demonstrate that this quantity provides a fundamental upper bound on expected reward for one-step decision making problems. We then extend this bound to multi-step settings via a dynamic programming approach and propose algorithms for computing the resulting bounds for systems with potentially continuous state and observation spaces, nonlinear and stochastic dynamics, and non-Gaussian sensor models (but with discretized action spaces). We demonstrate our approach on three examples: (i) the lava problem from the literature on partially observable Markov decision processes (POMDPs), (ii) a robot catching a freely-falling object, and (iii) obstacle avoidance using a depth sensor (Fig. E.1). We demonstrate the strength of our bounds on these examples by comparing them against the performance achieved by concrete control policies: the optimal POMDP solution for the lava problem, a model-predictive control (MPC) scheme for the catching example, and a learned neural network policy for the obstacle avoidance problem. We also present applications of our approach for establishing the superiority of one sensor over another (from the perspective of a given task). To our knowledge, the results in this paper are the first to provide general-purpose techniques for establishing fundamental bounds on performance for sensor-based control of robots.

E.1.1 Related Work

Domain-specific performance bounds. Prior work in robotics has established fundamental bounds on performance for particular problems. For example, [93, 94] consider high-speed navigation through an (ergodic) forest consisting of randomly-placed obstacles. Results from percolation theory [95] are used to establish a critical speed beyond which there does not exist (with probability one) an infinite collision-free trajectory. The work in [201] establishes limits on the speed at which a robot can navigate through unknown environments in terms of perceptual latency. Classical techniques from robust control [202] have also been utilized to establish fundamental limits on performance for control tasks (e.g., pole balancing) involving linear output-feedback control and sensor noise or delays [96]. The results in [97] demonstrate empirical correlation of the complexity metrics presented in [96] with sample-efficiency and performance of learned perception-based controllers on a pole-balancing task. The approaches mentioned above consider specific tasks (e.g., navigation in ergodic forests) or relatively narrow classes of problems (e.g., linear output-feedback control). In contrast, our goal is to develop a general and broadly applicable theoretical and algorithmic framework for establishing fundamental bounds on performance imposed by a sensor for a given task.

Comparing sensors. The notion of a *sensor lattice* was introduced in [90, 203] for comparing the power of different sensors (see [91] for a similar approach for comparing robots). The sensor lattice provides a partial ordering on different sensors based on the ability of one sensor to simulate another. However, most pairs of sensors are *incomparable* using such a scheme. Moreover, the sensor lattice does not establish the superiority of one sensor over another from the perspective of a given task; instead, the partial ordering is based on the ability of one sensor to perform as well as another in terms of filtering (i.e., state estimation). In this paper, we also demonstrate the applicability of our approach for comparing different sensors. However, this comparison is *task-driven*; we demonstrate how one sensor can be proved to be fundamentally better than another from the perspective of a given task.

Fano’s inequality and its extensions. In its original form, *Fano’s inequality* [13]

relates the lowest achievable error of estimating a signal x from an observation y in terms of the noise in the channel that produces observations from signals. In recent years, Fano’s inequality has been significantly extended and applied for establishing fundamental limits for various statistical estimation problems, e.g., lower bounding the Bayes and minimax risks for different learning problems [99, 100, 204]. In this paper, we build on generalized versions of Fano’s inequality [99, 100] in order to obtain fundamental bounds on performance for robotic systems with noisy sensors. On the technical front, we contribute by deriving a stronger version of the generalized Fano inequalities presented in [99, 100] by utilizing the *inverse of the KL divergence* (Sec. E.3) and computing it using *geometric programming* [114, Ch. 4.5]. The resulting inequality, which may be of independent interest, allows us to derive fundamental upper bounds on performance for one-step decision making problems. We then develop a dynamic programming approach for recursively applying the generalized Fano inequality in order to derive bounds on performance for multi-step problems.

E.2 Problem Formulation

E.2.1 Notation

We denote sequences by $x_{i:j} := (x_k)_{k=i}^j$ for $i \leq j$. Expectations are typically denoted as $\mathbb{E}[\cdot]$ with the variable of integration or its measure appearing below it for contextual emphasis, e.g.: $\mathbb{E}_x[\cdot]$, $\mathbb{E}_{p(x)}[\cdot]$.

E.2.2 Problem Statement

We denote the state of the robot and its environment at time-step t by $s_t \in \mathcal{S}$. Let p_0 denote the initial state distribution. Let the robot’s sensor observation and control action at time-step t be denoted by $o_t \in \mathcal{O}$ and $a_t \in \mathcal{A}$ respectively. Denote the (stochastic) dynamics of the state by $p_t(s_t|s_{t-1}, a_{t-1})$ and suppose that the robot’s sensor is described by $\sigma_t(o_t|s_t)$. The robot’s task is prescribed using reward functions $r_0, r_1, \dots, r_{T-1} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ at each time-step (up to a finite horizon).

Assumption 2 (Bounded rewards). We assume that rewards are bounded, and without

further loss of generality we assume that $r_t(s_t, a_t) \in [0, 1]$, $\forall s_t \in \mathcal{S}, a_t \in \mathcal{A}, t \in \{0, \dots, T-1\}$.

The robot’s goal is to find a (potentially time-varying and history-dependent) control policy $\pi_t : \mathcal{O}^{t+1} \rightarrow \mathcal{A}$ that maximizes the total expected reward:

$$R^* := \sup_{\pi_{0:T-1}} \mathbb{E}_{s_{0:T-1}, o_{0:T-1}} \left[\sum_{t=0}^{T-1} r_t(s_t, \pi_t(o_{0:t})) \right]. \quad (\text{E.1})$$

Goal: Our goal is to *upper bound* the best achievable expected reward R^* for a given sensor $\sigma_{0:T-1}$. We note that we are allowing for completely general policies that are arbitrary time-varying functions of the entire history of observations received up to time t (as long as the functions satisfy measurability conditions that ensure the existence of the expectation in (E.1)). An upper bound on R^* thus provides a fundamental bound on achievable performance that holds regardless of how the policy is parameterized (e.g., via neural networks, receding-horizon control architectures, etc.) or synthesized (e.g., via reinforcement learning, optimal control techniques, etc.).

E.3 Background

In this section, we briefly introduce some background material that will be useful throughout the paper.

E.3.1 KL Divergence and Mutual Information

The Kullback-Leibler (KL) divergence between two distributions is defined as:

$$\mathbb{D}(p(x) \| q(x)) := \mathbb{E}_{p(x)} \left[\log \frac{p(x)}{q(x)} \right]. \quad (\text{E.2})$$

The mutual information between two random variables x and y is defined as:

$$\mathbb{I}(x; y) := \mathbb{D}(p(x, y) \| p(x)p(y)), \quad (\text{E.3})$$

where $p(x, y)$ is the joint distribution, and $p(x)$ and $p(y)$ are the resulting marginal distributions. This quantity captures the amount of information one obtains about one random

variable (e.g., the state s_t) by observing another random variable (e.g., sensor observations o_t).

E.3.2 Inverting Bounds on the KL Divergence

Let $\mathcal{B}_p$ and $\mathcal{B}_q$ be Bernoulli distributions on $\{0, 1\}$ with mean p and q respectively. For $p, q \in [0, 1]$, we define¹:

$$\mathbb{D}_{\mathcal{B}}(p||q) := \mathbb{D}(\mathcal{B}_p||\mathcal{B}_q) = p \log \frac{p}{q} + (1-p) \log \frac{1-p}{1-q}.$$

In subsequent sections, we will obtain bounds on a quantity $p^* \in [0, 1]$ given by $\mathbb{D}_{\mathcal{B}}(p^*||q) \leq c$ for some $q \in [0, 1]$ and $c \geq 0$. In order to upper bound p^* , we will use the *KL inverse*:

$$\mathbb{D}^{-1}(q|c) := \sup \{p \in [0, 1] \mid \mathbb{D}_{\mathcal{B}}(p||q) \leq c\}. \quad (\text{E.4})$$

It is then easy to see that $p^* \leq \mathbb{D}^{-1}(q|c)$.

Since $\mathbb{D}_{\mathcal{B}}(\cdot||\cdot)$ is (jointly) convex in both arguments, the optimization problem in (E.4) is a convex problem. In particular, one can compute the KL inverse efficiently using a *geometric program* [114, Ch. 4.5] with a single decision variable p .

E.4 Performance Bound for Single-Step Problems

In this section, we will derive an upper bound on the best achievable reward R^* in the single time-step decision-making setting. This bound will then be extended to the multi-step setting in Sec. E.5.

When $T = 1$, our goal is to upper bound the following quantity:

$$R^*(\sigma_0; r_0) := \sup_{\pi_0} \mathbb{E}_{s_0, o_0} [r_0(s_0, \pi_0(o_0))] \quad (\text{E.5})$$

$$= \sup_{\pi_0} \mathbb{E}_{p_0(s_0)} \mathbb{E}_{\sigma_0(o_0|s_0)} [r_0(s_0, \pi_0(o_0))]. \quad (\text{E.6})$$

¹We adopt the commonly used convention when working with the KL divergence of taking $0 \cdot \log(0/a) := 0, \forall a$, and $a \cdot \log(a/0) := \infty, \forall a > 0$.

The notation $R^*(\sigma_0; r_0)$ highlights the dependence of the best achievable reward in terms of the robot’s sensor and task (as specified by the reward function). As highlighted in Sec. E.1, the amount of information that the robot requires from its sensors in order to obtain high expected reward depends on its task; certain tasks may admit purely open-loop policies that obtain high rewards, while other tasks may require high-precision sensing of the state. We formally define a quantity that captures this intuition and quantifies the *task-relevant information* provided by the robot’s sensors. We then demonstrate that this quantity provides an upper bound on $R^*(\sigma_0; r_0)$.

Definition 6 (Task-relevant information potential). Let $\mathbb{I}(o_0; s_0)$ be the mutual information between the robot’s sensor observation and state. Define:

$$R_0^\perp := \sup_{a_0} \mathbb{E}_{s_0}[r_0(s_0, a_0)] \quad (\text{E.7})$$

as the highest achievable reward using an *open-loop* policy. Then define the *task-relevant information potential (TRIP)* of a sensor σ_0 for a task specified by reward function r_0 as:

$$\tau(\sigma_0; r_0) := \mathbb{D}^{-1}(R_0^\perp | \mathbb{I}(o_0; s_0)). \quad (\text{E.8})$$

In order to interpret the TRIP, we state two useful properties of the KL inverse.

Proposition 13 (Monotonicity of KL inverse). The KL inverse $\mathbb{D}^{-1}(q|c)$ is:

1. monotonically non-decreasing in $c \geq 0$ for fixed $q \in [0, 1]$,
2. monotonically non-decreasing in $q \in [0, 1]$ for fixed $c \geq 0$.

Proof. The first property follows from the fact that increasing c loosens the KL divergence constraint in the optimization problem in (E.4). The proof of the second property utilizes the envelope theorem [112, Corollary 5] and is provided in Appendix E.9 (Lemma 4). $\square$

The TRIP $\tau(\sigma_0; r_0)$ depends on two factors: the mutual information $\mathbb{I}(o_0; s_0)$ (which depends on the robot’s sensor) and the best reward $R_0^\perp$ achievable by an open-loop policy (which depends on the robot’s task). Using Proposition 13, we see that as the sensor

provides more information about the state (i.e., as $\mathbb{I}(o_0; s_0)$ increases for fixed $R_0^\perp$), the TRIP is monotonically non-decreasing. Moreover, the TRIP is a monotonically non-decreasing function of $R_0^\perp$ for fixed $\mathbb{I}(o_0; s_0)$. This qualitative dependence is intuitively appealing: if there is a good open-loop policy (i.e., one that achieves high reward), then the robot’s sensor can provide a small amount of information about the state and still lead to good overall performance. The specific form of the definition of TRIP is motivated by the result below, which demonstrates that the TRIP upper bounds the best achievable expected reward $R^*(\sigma_0; r_0)$ in Eq. (E.5).

Theorem 13 (Single-step performance bound). The best achievable reward is upper bounded by the task-relevant information potential (TRIP) of a sensor:

$$\tau(\sigma_0; r_0) \geq R^*(\sigma_0; r_0) := \sup_{\pi_0} \mathbb{E}_{s_0, o_0} [r_0(s_0, \pi_0(o_0))]. \quad (\text{E.9})$$

Proof. The proof is provided in Appendix E.9 and is inspired by the proof of the generalized Fano inequality presented in [100, Proposition 14]. The bound (E.9) tightens the generalized Fano inequality [99, 100] by utilizing the KL inverse (in contrast to the methods in [99, 100], which may be interpreted as indirectly bounding the KL inverse). The result presented here may thus be of independent interest. $\square$

Theorem 13 provides a *fundamental bound* on performance (in the sense of Sec. E.1) imposed by the sensor for a given single-step task. This bound holds for *any* policy, independent of its complexity or how it is synthesized or learned.

E.5 Performance Bound for Multi-Step Problems: Fano’s Inequality with Feedback

In this section, we derive an upper bound on the best achievable reward R^* defined in (E.1) for the general multi time-step setting. The key idea is to extend the single-step bound from Theorem 13 using a dynamic programming argument.

Let $\pi_k^t : \mathcal{O}^{k-t+1} \rightarrow \mathcal{A}$ denote a policy that takes as input the sequence of observations $o_{t:k}$ from time-step t to k (for $k \geq t$). Thus, a policy π_k^0 at time-step k utilizes all observations

received up to time-step k . Given an initial state distribution p_0 and an open-loop action sequence $a_{0:t-1}$, define the reward-to-go from time $t \in \{0, \dots, T-1\}$ given $a_{0:t-1}$ as:

$$R_t := \mathbb{E}_{\substack{s_{t:T-1}, o_{t:T-1} \\ a_{0:t-1}}} \left[\sum_{k=t}^{T-1} r_k(s_k, \pi_k^t(o_{t:k})) \right], \quad (\text{E.10})$$

where the expectation,

$$\mathbb{E}_{\substack{s_{t:T-1}, o_{t:T-1} \\ a_{0:t-1}}} [\cdot] \quad (\text{E.11})$$

is taken with respect to the distribution of states $s_{t:T-1}$ and observations $o_{t:T-1}$ one receives if one propagates p_0 using the open-loop sequence of actions from time-steps² 0 to $t-1$, and then applies the closed-loop policies $\pi_t^t, \pi_{t+1}^t, \dots, \pi_{T-1}^t$ from time-steps t to $T-1$. We further define $R_T := 0$.

Now, for $t \in \{0, \dots, T-1\}$, define:

$$R_t^\perp := \sup_{a_t} \left[\mathbb{E}_{s_t | a_{0:t-1}} [r_t(s_t, a_t)] + R_{t+1} \right], \quad (\text{E.13})$$

and

$$R_t^{\perp \star} := \sup_{\pi_{t+1}^{t+1}, \dots, \pi_{T-1}^{t+1}} R_t^\perp. \quad (\text{E.14})$$

The following result then leads to a recursive structure for computing an upper bound on $R^\star$.

Proposition 14 (Recursive bound). For any $t = 0, \dots, T-1$, the following inequality holds for any open-loop sequence of actions $a_{0:t-1}$:

$$\sup_{\pi_t^t, \dots, \pi_{T-1}^t} R_t \leq \underbrace{(T-t) \cdot \mathbb{D}^{-1} \left(\frac{R_t^{\perp \star}}{T-t} \mid \mathbb{I}(o_t; s_t) \right)}_{=: \tau_t(\sigma_{t:T-1}; r_{t:T-1})}. \quad (\text{E.15})$$

²For $t = 0$, we use the convention that $a_{0:-1}$ is the empty sequence and:

$$\mathbb{E}_{\substack{s_{0:T-1}, o_{0:T-1} \\ a_{0:-1}}} [\cdot] := \mathbb{E}_{s_{0:T-1}, o_{0:T-1}} [\cdot]. \quad (\text{E.12})$$

Proof. The proof follows a similar structure to Theorem 13 and is presented in Appendix E.9. $\square$

To see how we can use Proposition 14, we first use (E.1) and (E.10) to note that the LHS of (E.15) for $t = 0$ is equal to R^* :

$$R^* = \sup_{\pi_0^0, \dots, \pi_{T-1}^0} R_0 \leq \tau_0(\sigma_{0:T-1}; r_{0:T-1}) . \quad (\text{E.16})$$

The quantity $\tau_t(\sigma_{t:T-1}; r_{t:T-1})$ may be interpreted as a multi-step version of the TRIP from Definition 6. This quantity depends on the mutual information $\mathbb{I}(o_t; s_t)$, which is computed using the distribution $p_t(s_t|a_{0:t-1})$ over s_t that one obtains by propagating p_0 using the open-loop sequence of actions $a_{0:t-1}$:

$$\mathbb{I}(o_t; s_t) = \mathbb{D} \left(p_t(s_t|a_{0:t-1})\sigma_t(o_t|s_t) \parallel p_t(s_t|a_{0:t-1})\sigma_t(o_t) \right). \quad (\text{E.17})$$

In addition, $\tau_t(\sigma_{t:T-1}; r_{t:T-1})$ depends on $R_t^{\perp*}$, which is then divided by $(T - t)$ to ensure boundedness between $[0, 1]$ (see Assumption 2). The quantity $R_t^{\perp*}$ can itself be upper bounded using (E.15) with $t + 1$, as we demonstrate below. Such an upper bound on $R_t^{\perp*}$ for $t = 0$ leads to an upper bound on R^* using (E.16) and the monotonicity of the KL inverse (Proposition 13). Applying this argument recursively leads to Algorithm 7, which computes an upper bound on R^* .

Algorithm 7 Multi-Step Performance Bound

- 1: Initialize $\bar{R}_T(a_{0:T-1}) = 0, \forall a_{0:T-1}$.
 - 2: **for** $t = T - 1, T - 2, \dots, 0$ **do**
 - 3: $\bar{R}_t(a_{0:t-1}) = (T - t) \cdot \mathbb{D}^{-1} \left(\frac{\bar{R}_t^{\perp*}}{T - t} \parallel \mathbb{I}(o_t; s_t) \right), \forall a_{0:t-1},$
 - 4: where $\bar{R}_t^{\perp*} := \sup_{a_t} \mathbb{E}_{s_t|a_{0:t-1}} \left[r_t(s_t, a_t) \right] + \bar{R}_{t+1}(a_{0:t})$.
 - 5: **end for**
 - 6: **return** $\bar{R}_0$ (bound on achievable expected reward).
-

Theorem 14 (Multi-step performance bound). Algorithm 7 returns an upper bound on the best achievable reward R^* .

Proof. We provide a sketch of the proof here, which uses (backwards) induction. In particular, Proposition 14 leads to the inductive step. See Appendix E.9 for the complete proof.

We prove that for all $t = T - 1, \dots, 0$,

$$\sup_{\pi_t^t, \dots, \pi_{T-1}^t} R_t \leq \bar{R}_t(a_{0:t-1}), \quad \forall a_{0:t-1}. \quad (\text{E.18})$$

Thus, in particular,

$$R^* = \sup_{\pi_0^0, \dots, \pi_{T-1}^0} R_0 \leq \bar{R}_0. \quad (\text{E.19})$$

We prove (E.18) by backwards induction starting from $t = T - 1$. We first prove the base step. Using (E.15), we obtain:

$$\sup_{\pi_{T-1}^{T-1}} R_{T-1} \leq \mathbb{D}^{-1}\left(R_{T-1}^{\perp*} \mid \mathbb{I}(o_{T-1}; s_{T-1})\right). \quad (\text{E.20})$$

Using the fact that $R_T = 0$, we can show that $R_{T-1}^{\perp*} = \bar{R}_{T-1}^{\perp*}$. Combining this with (E.20) and the monotonicity of the KL inverse (Proposition 13), we see:

$$\sup_{\pi_{T-1}^{T-1}} R_{T-1} \leq \mathbb{D}^{-1}\left(\bar{R}_{T-1}^{\perp*} \mid \mathbb{I}(o_{T-1}; s_{T-1})\right) \quad (\text{E.21})$$

$$= \bar{R}_{T-1}(a_{0:T-2}). \quad (\text{E.22})$$

In order to prove the induction step, suppose that for $t \in \{0, \dots, T - 2\}$, we have

$$\sup_{\pi_{t+1}^{t+1}, \dots, \pi_{T-1}^{t+1}} R_{t+1} \leq \bar{R}_{t+1}(a_{0:t}). \quad (\text{E.23})$$

We then need to show that

$$\sup_{\pi_t^t, \dots, \pi_{T-1}^t} R_t \leq \bar{R}_t(a_{0:t-1}). \quad (\text{E.24})$$

We can use the induction hypothesis (E.23) to show that $R_t^{\perp*} \leq \bar{R}_t^{\perp*}$. Combining this with (E.15) and the monotonicity of the KL inverse (Proposition 13), we obtain the desired

result (E.24):

$$\sup_{\pi_t^t, \dots, \pi_{T-1}^t} R_t \leq (T-t) \cdot \mathbb{D}^{-1} \left(\frac{\bar{R}_t^{\perp \star}}{T-t} \mid \mathbb{I}(o_t; s_t) \right) \quad (\text{E.25})$$

$$= \bar{R}_t(a_{0:t-1}). \quad (\text{E.26})$$

□

E.6 Numerical Implementation

In order to compute the single-step bound using Theorem 13 or the multi-step bound using Algorithm 7, we require the ability to compute (or bound) three quantities: (i) the KL inverse, (ii) the mutual information $\mathbb{I}(o_t; s_t)$, and (iii) the quantity $\bar{R}_t^{\perp \star}$. As described in Sec. E.3.2, we can compute the KL inverse efficiently using a geometric program (GP) [114, Ch. 4.5] with a single decision variable. There are a multitude of solvers for GPs including Mosek [205] and the open-source solver SCS [206]. Next, we describe the computation of $\mathbb{I}(o_t; s_t)$ and $\bar{R}_t^{\perp \star}$ in different settings.

E.6.1 Analytic Computation

In certain settings, one can compute $\mathbb{I}(o_t; s_t)$ and $\bar{R}_t^{\perp \star}$ exactly. We discuss two such settings of interest below.

Discrete POMDPs. In cases where the state space $\mathcal{S}$, action space $\mathcal{A}$, and observation space $\mathcal{O}$ are finite, one can compute $\mathbb{I}(o_t; s_t)$ exactly by propagating the initial state distribution p_0 forward using open-loop action sequences $a_{0:t-1}$ and using the expression (E.17) (which can be evaluated exactly since we have discrete probability distributions). The expectation term in $\bar{R}_t^{\perp \star}$ can be computed similarly. In addition, the supremum over actions can be evaluated exactly via enumeration.

Linear-Gaussian systems with finite action spaces. One can also perform exact computations in cases where (i) the state space $\mathcal{S}$ is continuous and the dynamics $p_t(s_t | s_{t-1}, a_{t-1})$ are given by a linear dynamical system with additive Gaussian uncertainty, (ii) the observation space

operO is continuous and the sensor model $\sigma_t(o_t|s_t)$ is such that the observations are linear (in the state) with additive Gaussian uncertainty, (iii) the initial state distribution p_0 is Gaussian, and (iv) the action space $\mathcal{A}$ is finite. In such settings, one can analytically propagate p_0 forward through open-loop action sequences $a_{0:t-1}$ using the fact that Gaussian distributions are preserved when propagated through linear-Gaussian systems (similar to Kalman filtering [66]). One can then compute $\mathbb{I}(o_t; s_t)$ using (E.17) by leveraging the fact that all the distributions involved are Gaussian, for which KL divergences can be computed in closed form [207]. One can also compute $\bar{R}_t^{\perp\star}$ exactly for any reward function that permits the analytic computation of the expectation term using a Gaussian (e.g., quadratic reward functions); the supremum over actions can be evaluated exactly since $\mathcal{A}$ is finite.

E.6.2 Computation via Sampling and Concentration Inequalities

General settings. Next, we consider more general settings with: (i) continuous state and observation spaces, (ii) arbitrary (e.g., non-Gaussian/nonlinear) dynamics $p_t(s_t|s_{t-1}, a_{t-1})$, which are potentially not known analytically, but can be sampled from (e.g., as in a simulator), (iii) arbitrary (e.g., non-Gaussian/nonlinear) sensor $\sigma_t(o_t|s_t)$, but with a probability density function that can be numerically evaluated given any particular state-observation pair, (iv) an arbitrary initial state distribution p_0 that can be sampled from, and (v) a finite action space. Our bound is thus broadly applicable, with the primary restriction being the finiteness of $\mathcal{A}$; we leave extensions to continuous action spaces for future work (see Sec. E.8).

We first discuss the computation of $\bar{R}_t^{\perp\star}$. Since the supremization over actions can be performed exactly (due to the finiteness of $\mathcal{A}$), the primary challenge here is to evaluate the expectation:

$$\mathbb{E}_{s_t|a_{0:t-1}} \left[r_t(s_t, a_t) \right]. \quad (\text{E.27})$$

We note that any upper bound on this expectation leads to an upper bound on $\bar{R}_t^{\perp\star}$, and thus a valid upper bound on $R^\star$ (due to the monotonicity of the KL inverse; Proposition 13). One can thus obtain a high-confidence upper bound on (E.27) by sampling states $s_t|a_{0:t-1}$, and using any concentration inequality [115]. In particular, since we assume boundedness

of rewards (Assumption 2), we can use Hoeffding’s inequality.

Theorem 15 (Hoeffding’s inequality [115]). Let z be a random variable bounded within $[0, 1]$, and let $z_1, \dots, z_n$ denote i.i.d. samples. Then, with probability at least $1 - \delta$ (over the sampling of $z_1, \dots, z_n$), the following bound holds with probability at least $1 - \delta$:

$$\mathbb{E}[z] \leq \frac{1}{n} \sum_{i=1}^n z_i + \sqrt{\frac{\log(1/\delta)}{2n}}. \quad (\text{E.28})$$

In our numerical examples (Sec. E.7), we utilize a slightly tighter version of Hoeffding’s inequality (see Appendix E.10).

Next, we discuss the computation of $\mathbb{I}(o_t; s_t)$. Again, we note that any upper bound on $\mathbb{I}(o_t; s_t)$ yields a valid upper bound on R^* due to the monotonicity of the KL inverse. In this work, we utilize *variational bounds* on mutual information; in particular, we use the “leave-one-out bound” [62]:

$$\mathbb{I}(o_t; s_t) \leq \mathbb{E} \left[\frac{1}{K} \sum_{i=1}^K \left[\log \frac{\sigma_t(o_t^{[i]} | s_t^{[i]})}{\frac{1}{K-1} \sum_{j \neq i} \sigma_t(o_t^{[i]} | s_t^{[j]})} \right] \right], \quad (\text{E.29})$$

where the expectation is over size- K batches $\{(s_t^{[i]}, o_t^{[i]})\}_{i=1}^K$ of sampled states $s_t | a_{0:t-1}$ and observations sampled using $\sigma_t(o_t | s_t)$. The quantity $\sigma_t(o_t^{[i]} | s_t^{[i]})$ denotes (with slight abuse of notation) the evaluation of the density function corresponding to the sensor model. Since the bound (E.29) is in terms of an expectation, one can again obtain a high-confidence upper bound by sampling state-observation batches and applying a concentration inequality (e.g., Hoeffding’s inequality if the quantity inside the expectation is bounded).

We note that the overall implementation of Algorithm 7 may involve the application of multiple concentration inequalities (each of which holds with some confidence $1 - \delta_i$). One can obtain the overall confidence of the upper bound on R^* by using a union bound: $1 - \delta = 1 - \sum_i \delta_i$.

E.6.3 Tightening the Bound with Multiple Horizons

We end this section by discussing a final implementation detail. Let T denote the time horizon of interest (as in Sec. E.2). For any $H \in \{1, \dots, T\}$, one can define

$$R_H^* := \sup_{\pi_{0:H-1}} \mathbb{E}_{\substack{s_{0:H-1} \\ o_{0:H-1}}} \left[\sum_{t=0}^{H-1} r_t(s_t, \pi_t(o_{0:t})) \right] \quad (\text{E.30})$$

as the best achievable reward for a problem with horizon H (instead of T). One can then apply Algorithm 7 to compute an upper bound on R_H^* . Since rewards are assumed to be bounded within $[0, 1]$ (Assumption 2), we can observe that $R^* = R_T^* \leq R_H^* + (T - H)$. In practice, we sometimes find that this bound provides a tighter bound on R^* for some $H < T$ (as compared to directly applying Algorithm 7 with a horizon of T). For our numerical examples, we thus sweep through different values for the horizon H and report the lowest upper bound $R_H^* + (T - H)$.

E.7 Examples

We demonstrate our approach on three examples: (i) the lava problem from the POMDP literature, (ii) an example with continuous state and observation spaces corresponding to a robot catching a freely-falling object, and (iii) obstacle avoidance using a depth sensor with non-Gaussian noise. We illustrate the strength of our upper bounds on these examples by comparing them against the performance achieved by concrete control policies (i.e., lower bounds on achievable performance). We also demonstrate the applicability of our approach for establishing the superiority of one sensor over another (from the perspective of a given task). Code for all examples can be found at: <https://github.com/irom-lab/performance-limits>.

E.7.1 Lava Problem

The first example we consider is the lava problem (Fig. E.2) [9, 76, 77] from the POMDP literature.

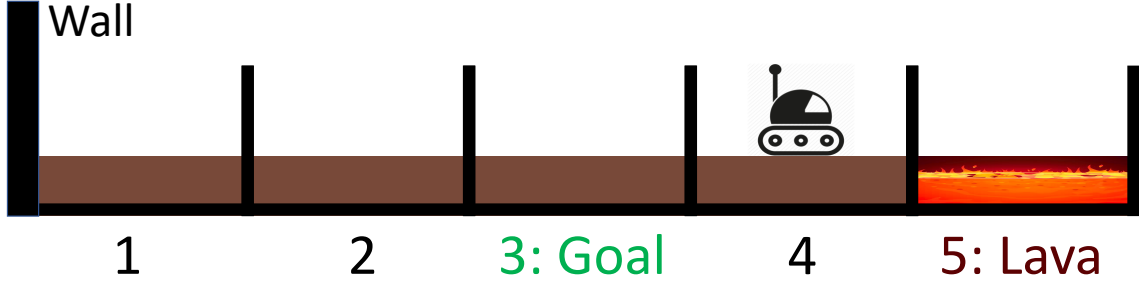


Figure E.2: An illustration of the lava problem. The robot needs to navigate to a goal without falling into the lava (using a noisy sensor).

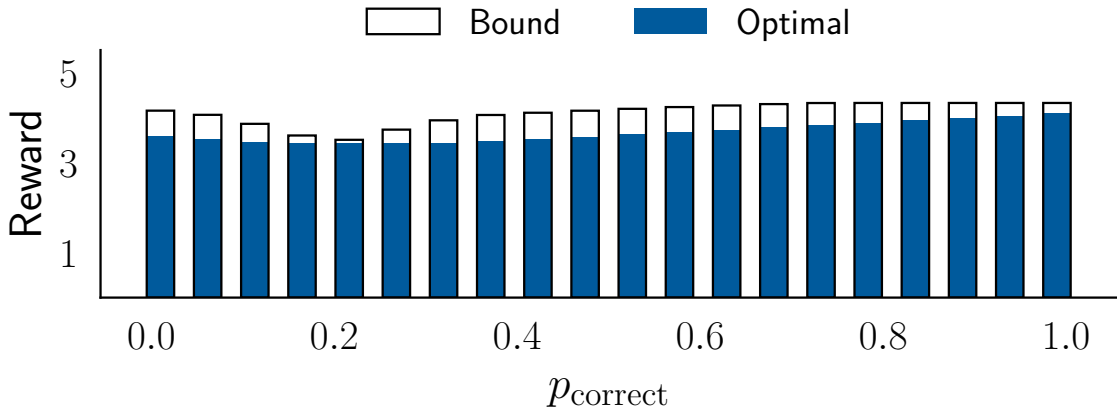


Figure E.3: Results for the lava problem. We compare the upper bounds on achievable expected rewards computed by our approach with the optimal POMDP solution for different values of sensor noise.

Dynamics. The setting consists of five discrete states (Fig. E.2) and two actions (**left** and **right**). If the robot falls into the lava state, it remains there (i.e., the lava state is absorbing). If the robot attempts to go left from state 1, it remains at state 1. The initial state distribution p_0 is chosen to be uniform over the non-lava states.

Sensor. The robot is equipped with a sensor that provides a (noisy) state estimate. The sensor reports the correct state (i.e., $o_t = s_t$) with probability p_{correct} (and a uniformly randomly chosen incorrect state with probability $1 - p_{\text{correct}}$).

Rewards. The robot’s objective is to navigate to the goal state (which is an absorbing state), within a time horizon of $T = 5$. This objective is encoded via a reward function $r_t(s_t, a_t)$, which is purely state-dependent. The reward associated with being in the lava is 0; the reward associated with being at the goal is 1; the reward at all other states is 0.1.

Results. An interesting feature of this problem is that it admits a purely *open-loop* policy that achieves a high expected reward. In particular, consider the following sequence of actions: `left`, `right`, `right`. No matter which state the robot starts from, this sequence of actions will steer the robot to the goal state (recall that the goal is absorbing). Given the initial distribution and rewards above, this open-loop policy achieves an expected reward of 3.425. Suppose we set $p_{\text{correct}} = 1/5$ (i.e., the sensor just uniformly randomly returns a state estimate and thus provides no information regarding the state). In this case, Algorithm 7 returns an upper bound: $3.5 \geq R^*$.

Next, we plot the upper bounds provided by Algorithm 7 for different values of sensor noise by varying p_{correct} . The resulting bounds are shown in Fig. E.3. Since the lava problem is a finite POMDP, one can compute R^* *exactly* using a POMDP solver. Fig. E.3 compares the upper bounds on R^* returned by Algorithm 7 with R^* computed using the `pomdp.py` package [116]. The figure illustrates that our approach provides strong bounds on the best achievable reward for this problem. We also note that computation of the POMDP solution (for each value of p_{correct}) takes ~ 20 s, while the computation of the bound takes ~ 0.2 s (i.e., $\sim 100\times$ faster).

E.7.2 Catching a Falling Object

Next, we consider a problem with continuous state and observation spaces. The goal of the robot is to catch a freely-falling object such as a ball (Fig. E.4).

Dynamics. We describe the four-dimensional state of the robot-ball system by $s_t := [x_t^{\text{rel}}, y_t^{\text{rel}}, v_t^{\text{x,ball}}, v_t^{\text{y,ball}}] \in \mathbb{R}^4$, where $[x_t^{\text{rel}}, y_t^{\text{rel}}]$ is the relative position of the ball with respect to the robot, and $[v_t^{\text{x,ball}}, v_t^{\text{y,ball}}]$ corresponds to the ball’s velocity. The action a_t is the horizontal speed of the robot and can be chosen within the range $[-0.4, 0.4]$ m/s (discretized in increments of 0.1m/s). The dynamics of the system are given by:

$$s_{t+1} = \begin{bmatrix} x_{t+1}^{\text{rel}} \\ y_{t+1}^{\text{rel}} \\ v_{t+1}^{\text{x,ball}} \\ v_{t+1}^{\text{y,ball}} \end{bmatrix} = \begin{bmatrix} x_t^{\text{rel}} + (v_t^{\text{x,ball}} - a_t)\Delta t \\ y_t^{\text{rel}} + \Delta t v_t^{\text{y,ball}} \\ v_t^{\text{x,ball}} \\ v_t^{\text{y,ball}} - g\Delta t \end{bmatrix}, \quad (\text{E.31})$$

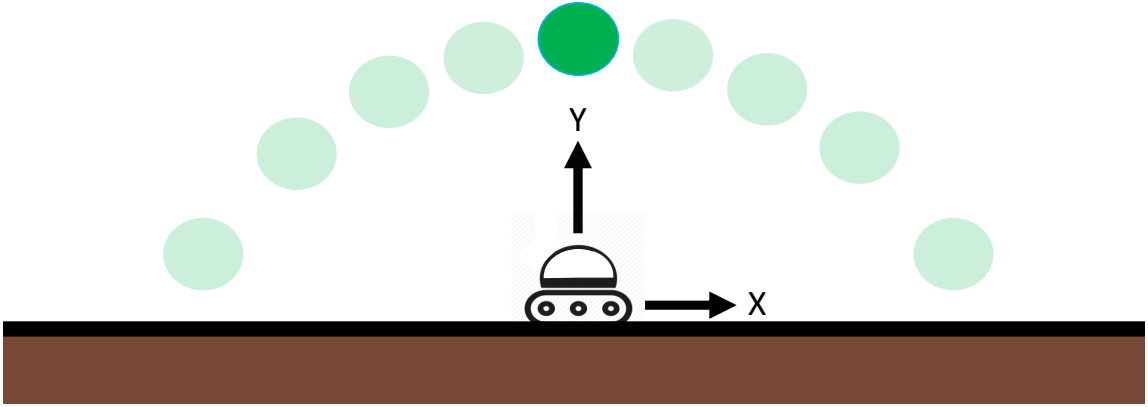


Figure E.4: An illustration of the ball-catching example with continuous state and observation spaces. The robot is constrained to move horizontally along the ground and can control its speed. Its goal is to track the position of the falling ball using a noisy estimate of the ball’s state.

where $\Delta t = 1$ is the time-step and $g = 0.1\text{m/s}^2$ is chosen such that the ball reaches the ground within a time horizon of $T = 5$. The initial state distribution p_0 is chosen to be a Gaussian with mean $[0.0\text{m}, 1.05\text{m}, 0.0\text{m/s}, 0.05\text{m/s}]$ and diagonal covariance matrix $\text{diag}([0.01^2, 0.1^2, 0.2^2, 0.1^2])$.

Sensor. The robot’s sensor provides a noisy state estimate $o_t = s_t + \epsilon_t$, where ϵ_t is drawn from a Gaussian distribution with zero mean and diagonal covariance matrix $\eta \cdot \text{diag}([0.5^2, 1.0^2, 0.75^2, 1.0^2])$. Here, η is a noise scale that we will vary in our experiments.

Rewards. The reward function $r_t(s_t, a_t)$ is chosen to encourage the robot to track the ball’s motion. In particular, we choose $r_t(s_t, a_t) = \max(1 - 2|x_t^{\text{rel}}|, 0)$. The reward is large when x_{rel} is close to 0 (with a maximum reward of 1 when $x_t^{\text{rel}} = 0$); the robot receives no reward if $|x_t^{\text{rel}}| \geq 0.5$. The robot’s goal is to maximize the expected cumulative reward over a time horizon of $T = 5$.

Results. Unlike the lava problem, this problem does not admit a good open-loop policy since the initial distribution on $v_0^{\text{x,ball}}$ is symmetric about 0; thus, the robot does not have *a priori* knowledge of the ball’s x-velocity direction (as illustrated in Fig. E.4). Fig. E.5 plots the upper bound on the expected cumulative reward obtained using Algorithm 7 for different settings of the observation noise scale η . Since the dynamics (E.31) are affine, the

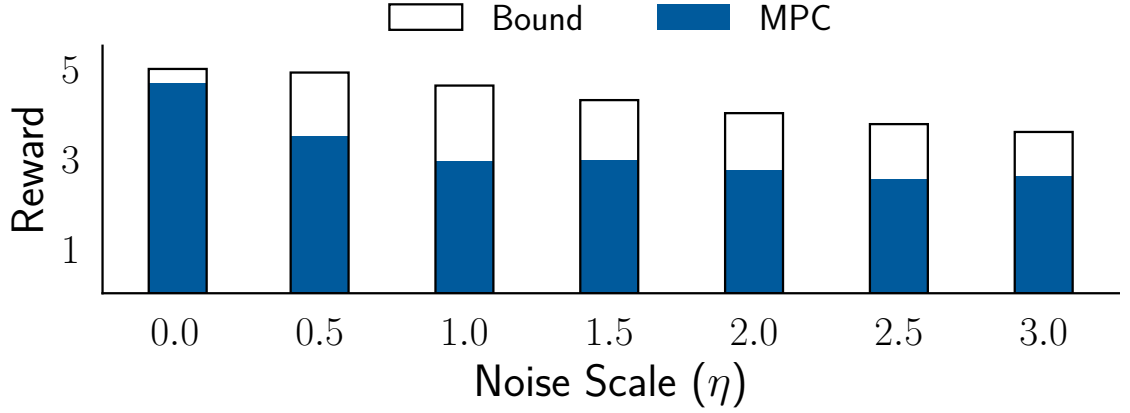


Figure E.5: Results for the ball-catching example. We compare the upper bounds on achievable expected rewards with the expected rewards using MPC combined with Kalman filtering for different values of sensor noise (results for MPC are averaged across five evaluation seeds; the std. dev. across seeds is too small to visualize).

sensor model is Gaussian, and the initial state distribution is also Gaussian, we can apply the techniques described in Sec. E.6.1 for *analytically* computing the quantities of interest in Algorithm 7.

Fig. E.5 also compares the upper bounds on R^* with achievable *lower bounds* by applying a model-predictive control (MPC) scheme combined with a Kalman filter for state estimation. We estimate the expected reward achieved by the MPC controller using 100 initial conditions sampled from p_0 . Fig. E.5 plots the average of these expected rewards across five random seeds (the resulting std. dev. is too small to visualize). As the figure illustrates, the MPC controller obeys the fundamental bound on reward computed by our approach. Moreover, the performance of the controller qualitatively tracks the degradation of achievable performance predicted by the bound as η is increased. Finally, we observe that sensors with noise scales $\eta = 1$ and higher are *fundamentally limited* as compared to a noiseless sensor. This is demonstrated by the fact that the MPC controller for $\eta = 0$ achieves higher performance than the fundamental limit on performance for $\eta = 1$.

E.7.3 Obstacle Avoidance with a Depth Sensor

For our final example, we consider the problem of obstacle avoidance using a depth sensor (Fig. E.1). This is a more challenging problem with higher-dimensional (continuous) state and observation spaces, and non-Gaussian sensor models.

State and action spaces. The robot is initialized at the origin with six cylindrical obstacles of fixed radius placed randomly in front of it. The state $s_t \in \mathbb{R}^{12}$ of this system describes the locations of these obstacles in the environment. In addition, we also place “walls” enclosing a workspace $[-1, 1]\text{m} \times [-0.1, 1.2]\text{m}$ (these are not visualized in the figure to avoid clutter). The initial state distribution p_0 corresponds to uniformly randomly choosing the x-y locations of the cylindrical obstacles from the set $[-1, 1]\text{m} \times [0.9, 1.1]\text{m}$. The robot’s goal is to navigate to the end of the workspace by choosing a motion primitive to execute (based on a noisy depth sensor described below). Fig. E.1 illustrates the set of ten motion primitives the robot can choose from; this set corresponds to the action space.

Rewards. We treat this problem as a one-step decision making problem (Sec. E.4). Once the robot chooses a motion primitive to execute based on its sensor measurements, it receives a reward of 0 if the motion primitive results in a collision with an obstacle; if the motion primitive results in collision-free motion, the robot receives a reward of 1. The expected reward for this problem thus corresponds to the probability of safe (i.e., collision-free) motion.

Sensor. The robot is equipped with a depth sensor which provides distances along $n_{\text{rays}} = 10$ rays. The sensor has a field of view of 90° and a maximum range of 1.5m. We use the noise model for range finders described in [66, Ch. 6.3] and consider two kinds of measurement errors: (i) errors due to failures to detect obstacles, and (ii) random noise in the reported distances. For each ray, there is a probability $p_{\text{miss}} = 0.1$ that the sensor misses an obstacle and reports the maximum range (1.5m) instead. In the case that an obstacle is not missed, the distance reported along a given ray is sampled from a Gaussian with mean equal to the true distance along that ray and std. dev. equal to η . The noise for each ray is sampled independently. Overall, this is a non-Gaussian sensor model due to the combination of the two kinds of errors.

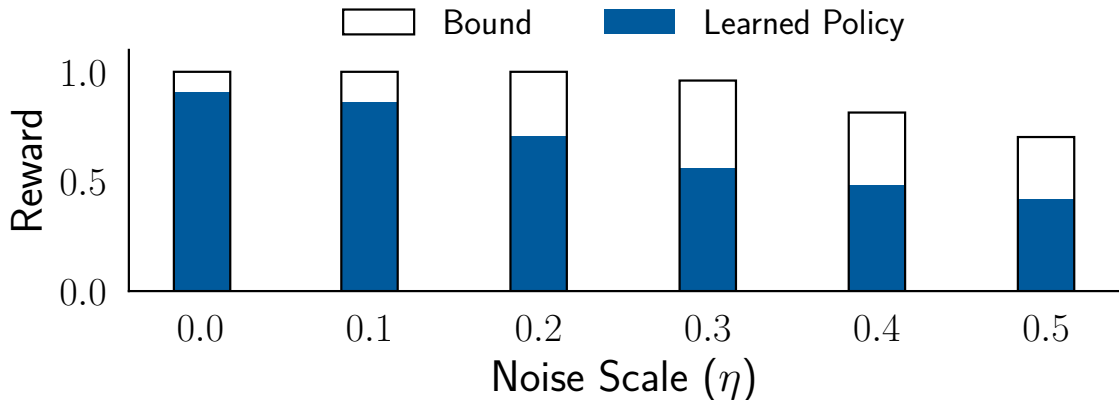


Figure E.6: Results for the obstacle avoidance example. We compare the upper bounds on achievable expected rewards with the expected rewards using a learned neural network policy for different values of sensor noise (results for the learned policy are averaged across five training seeds; the std. dev. is too small to visualize).

Results. We implement Algorithm 7 using the sampling-based techniques described in Sec. E.6.2. We sample 20K obstacle environments for upper bounding the open-loop rewards associated with each action. We also utilize 20K batches (each of size $K = 1000$) for upper bounding the mutual information using (E.29). We utilize a version of Hoeffding’s inequality (Theorem 15) presented in Appendix E.10 to obtain an upper bound on R^* that holds with probability $1 - \delta = 0.95$.

Fig. E.6 shows the resulting upper bounds for different values of the observation noise std. dev. η . We compare these with rewards achieved by neural network policies trained to perform this task. For each η , we sample 5000 training environments and corresponding sensor observations. For each environment, we generate a ten-dimensional training label by recording the minimum distance to the obstacles achieved by each motion primitive and passing the vector of distances through an (element-wise) softmax transformation. We use a cross-entropy loss to train a neural network that predicts the label of distances for each primitive given a sensor observation as input. We use two fully-connected layers with a ReLU nonlinearity; the output is passed through a softmax layer. At test-time, a given sensor observation is passed as input to the trained neural network; the motion primitive corresponding to the highest predicted distance is then executed. We estimate the expected reward achieved by the trained policy using 5000 test environments (unseen during training).

Fig. E.6 plots the average of these expected rewards across five training seeds for each value of η (the std. dev. across seeds is too small to visualize).

We emphasize that the upper bound on R^* computed using our approach is a fundamental limit on performance that holds regardless of the size of the neural network, the network architecture, or algorithm used for policy learning. We also highlight the fact that a sensor with noise $\eta = 0.1$ achieves higher performance than the fundamental limit for a sensor with noise $\eta = 0.4$ or 0.5 .

Finally, we apply our approach in order to compare two sensors with varying number n_{rays} of rays along which the depth sensor provides distance estimates (Fig. E.1). For this comparison, we fix $\eta = 0.3$ and $p_{\text{miss}} = 0.05$. We compare two sensors with $n_{\text{rays}} = 50$ (Sensor 1) and $n_{\text{rays}} = 5$ (Sensor 2) respectively. The upper bound on expected reward computed using our approach (with confidence $1 - \delta = 0.95$) for Sensor 2 is 0.79. A neural network policy for Sensor 1 achieves an expected reward of approximately 0.86, which surpasses the fundamental limit on performance for Sensor 2.

E.8 Discussion and Conclusions

We have presented an approach for establishing fundamental limits on performance for sensor-based robot control and policy learning. We defined a quantity that captures the amount of task-relevant information provided by a sensor; using a novel version of the generalized Fano inequality, we demonstrated that this quantity upper bounds the expected reward for one-step decision making problems. We developed a dynamic programming approach for extending this bound to multi-step problems. The resulting framework has potentially broad applicability to robotic systems with continuous state and observation spaces, nonlinear and stochastic dynamics, and non-Gaussian sensor models. Our numerical experiments demonstrate the ability of our approach to establish strong bounds on performance for such settings. In addition, we provided an application of our approach for comparing different sensors and establishing the superiority of one sensor over another for a given task.

Challenges and future work. There are a number of challenges and directions for future work associated with our approach. On the theoretical front, we expect that our bounds in Theorems 13 and 14 can be extended to utilize general f-divergences (instead of the KL divergence); this could potentially lead to tighter bounds. In addition, it would be interesting to handle settings where the sensor model is inaccurate (in contrast to this paper, where we have focused on establishing fundamental limits given a particular sensor model). For example, one could potentially perform an adversarial search over a family of sensor models in order to find the model that results in the lowest bound on achievable performance.

On the algorithmic front, the primary challenges are: (i) efficient computation of bounds for longer time horizons, and (ii) extensions to continuous action spaces. As presented, Algorithm 7 requires an enumeration over action sequences. Finding more computationally efficient versions of Algorithm 7 is thus an important direction for future work. The primary bottleneck in extending our approach to continuous action spaces is the need to perform a supremization over actions when computing $\bar{R}_t^{\perp\star}$ in Algorithm 7. However, we note that any upper bound on $\bar{R}_t^{\perp\star}$ also leads to a valid upper bound on $R^\star$. Thus, one possibility is to use a Lagrangian relaxation [114] to upper bound $\bar{R}_t^{\perp\star}$ in settings with continuous action spaces.

Our work also opens up a number of exciting directions for longer-term research. While we have focused on establishing fundamental limits imposed by imperfect sensing in this work, one could envision a broader research agenda that seeks to establish bounds on performance due to other limited resources (e.g., onboard computation or memory). One concrete direction is to combine the techniques presented here with information-theoretic notions of bounded rationality [7, 32, 36]. Finally, another exciting direction is to turn the impossibility results provided by our approach into certificates of *robustness* against an adversary. Specifically, consider an adversary that can observe our robot’s actions; if one could establish fundamental limits on performance *for the adversary* due to its inability to infer the robot’s internal state (and hence its future behavior) using past observations, this provides a certificate of robustness against *any* adversary. This is reminiscent of a long tradition in cryptography of turning impossibility or hardness results into robust protocols

for security [120, Ch. 9].

Overall, we believe that the ability to establish fundamental limits on performance for robotic systems is crucial for establishing a science of robotics. We hope that the work presented here along with the indicated directions for future work represent a step in this direction.

E.9 Proofs

Lemma 4 (Monotonicity of KL inverse). $\mathbb{D}^{-1}(q|c)$ is monotonically non-decreasing in $q \in [0, 1]$ for $c \geq 0$.

Proof. Recall that the KL inverse is defined using the following optimization problem with decision variable p :

$$\mathbb{D}^{-1}(q|c) := \sup \{p \in [0, 1] \mid \mathbb{D}_{\mathcal{B}}(p||q) \leq c\}. \quad (\text{E.32})$$

We first prove that:

$$\mathbb{D}^{-1}(q|c) \geq q. \quad (\text{E.33})$$

For contradiction, suppose that $\mathbb{D}^{-1}(q|c) < q$. Then, let $p' := q$. Notice that p' is a feasible point for the optimization problem (E.32) since $\mathbb{D}_{\mathcal{B}}(p'||q) = \mathbb{D}_{\mathcal{B}}(q||q) = 0$, and $p' = q \in [0, 1]$. But, by assumption, $p' = q > \mathbb{D}^{-1}(q|c)$; this contradicts the statement that $\mathbb{D}^{-1}(q|c)$ is the optimal solution to (E.32).

This allows us to demonstrate the monotonicity of $\mathbb{D}^{-1}(q|c)$ in q for the case when $c = 0$. In particular, when $c = 0$, we have that $\mathbb{D}^{-1}(q|c) = q$. Thus, monotonicity of $\mathbb{D}^{-1}(q|c)$ in q follows immediately.

Next, we demonstrate the monotonicity of $\mathbb{D}^{-1}(q|c)$ in q for the case when $c > 0$. We note that:

- The objective function of (E.32) is continuous and concave (indeed, linear).
- The constraints of (E.32) can be written in the form $g(p, q) \geq 0$ with g continuous and concave (since the KL divergence is jointly convex in both arguments, and the other constraints on p are linear).

- There is a point (in particular, the point $p = q$) that is strictly feasible (i.e., $g(p, q) > 0$) for (E.32) for $c > 0$ and $q \in [0, 1]$.

The optimization problem (E.32) thus satisfies the conditions for the envelope theorem [112, Corollary 5], which allows us to characterize the derivative of (E.32) with respect to the parameter q :

$$\frac{\partial[\mathbb{D}^{-1}(q|c)]}{\partial q} = \sup_{p \in X^*(q)} \inf_{\lambda \in Y^*(q)} \frac{\partial L(p, q, \lambda)}{\partial q}, \quad (\text{E.34})$$

where $L(p, q, \lambda)$ is the Lagrangian for (E.32), $X^*(q)$ are the set of optimizers for (E.32), $Y^*(q)$ are the set of optimizers for the Lagrange dual, and $\lambda = [\lambda_1, \lambda_2, \lambda_3]$ are the Lagrange multipliers corresponding to the three constraints:

$$c - \mathbb{D}_{\mathcal{B}}(p||q) \geq 0, \quad (\text{E.35})$$

$$p \geq 0, \quad (\text{E.36})$$

$$1 - p \geq 0. \quad (\text{E.37})$$

The Lagrangian is given by:

$$L(p, q, \lambda) = p + \lambda_1(c - \mathbb{D}_{\mathcal{B}}(p||q)) + \lambda_2 p + \lambda_3(1 - p). \quad (\text{E.38})$$

Computing the partial derivative with respect to q :

$$\frac{\partial L(p, q, \lambda)}{\partial q} = -\lambda_1 \frac{\partial \mathbb{D}_{\mathcal{B}}(p||q)}{\partial q} \quad (\text{E.39})$$

$$= -\lambda_1 \frac{\partial}{\partial q} \left[p \log \frac{p}{q} + (1 - p) \log \frac{1 - p}{1 - q} \right] \quad (\text{E.40})$$

$$= -\lambda_1 \left[-\frac{p}{q} + \frac{1 - p}{1 - q} \right]. \quad (\text{E.41})$$

We note that the Lagrange multiplier λ_1 must be nonnegative. In addition, we know from (E.33) that the optimal value p^* of the optimization problem (E.32) must be greater than

or equal to q . Thus:

$$\frac{p^*}{q} \geq 1, \text{ and } \frac{1-p^*}{1-q} \leq 1 \quad (\text{E.42})$$

$$\implies \frac{p^*}{q} \geq \frac{1-p^*}{1-q} \quad (\text{E.43})$$

$$\implies \frac{\partial L(p, q, \lambda)}{\partial q} \geq 0. \quad (\text{E.44})$$

Hence, the envelope theorem (E.34) demonstrates that $\mathbb{D}^{-1}(q|c)$ is monotonically non-decreasing in q . $\square$

Theorem 13 (Single-step performance bound). The best achievable reward is upper bounded by the task-relevant information potential (TRIP) of a sensor:

$$\tau(\sigma_0; r_0) \geq R^*(\sigma_0; r_0) := \sup_{\pi_0} \mathbb{E}_{s_0, o_0} [r_0(s_0, \pi_0(o_0))]. \quad (\text{E.9})$$

Proof. For a given policy π_0 , define:

$$R := \mathbb{E}_{p_0(s_0)} \mathbb{E}_{\sigma_0(o_0|s_0)} [r_0(s_0, \pi_0(o_0))], \quad (\text{E.45})$$

and

$$\tilde{R} := \mathbb{E}_{p_0(s_0)} \mathbb{E}_{q(o_0)} [r_0(s_0, \pi_0(o_0))]. \quad (\text{E.46})$$

The only difference between R and $\tilde{R}$ is that the observations o_0 in $\tilde{R}$ are drawn from a state-*independent* distribution q .

Now, assuming bounded rewards (Assumption 2), we have:

$$\mathbb{D}_{\mathcal{B}}(R \| \tilde{R}) = \mathbb{D}_{\mathcal{B}} \left(\mathbb{E}_{p_0(s_0)} \mathbb{E}_{\sigma_0(o_0|s_0)} [r_0(s_0, \pi_0(o_0))] \parallel \mathbb{E}_{p_0(s_0)} \mathbb{E}_{q(o_0)} [r_0(s_0, \pi_0(o_0))] \right) \quad (\text{E.47})$$

$$\leq \mathbb{E}_{p_0(s_0)} \mathbb{D}_{\mathcal{B}} \left(\mathbb{E}_{\sigma_0(o_0|s_0)} [r_0(s_0, \pi_0(o_0))] \parallel \mathbb{E}_{q(o_0)} [r_0(s_0, \pi_0(o_0))] \right) \quad (\text{E.48})$$

$$\leq \mathbb{E}_{p_0(s_0)} \mathbb{D}(\sigma_0(o_0|s_0) \parallel q(o_0)). \quad (\text{E.49})$$

The first inequality above follows from Jensen's inequality, while the second follows from the data processing inequality (see [100, Corollary 2] for the specific version). From the well-known variational representation of the mutual information (see, e.g., [99]), we note that:

$$\mathbb{I}(o_0; s_0) = \inf_q \mathbb{E}_{p_0(s_0)} \mathbb{D}\left(\sigma_0(o_0|s_0) \parallel q(o_0)\right). \quad (\text{E.50})$$

Thus, taking the infimum of (E.49) with respect to q , we see:

$$\mathbb{D}_B(R \parallel \tilde{R}) \leq \mathbb{I}(o_0; s_0). \quad (\text{E.51})$$

We can then upper bound R using the KL inverse:

$$R \leq \mathbb{D}^{-1}\left(\tilde{R} \mid \mathbb{I}(o_0; s_0)\right). \quad (\text{E.52})$$

Thus,

$$\sup_{\pi_0} R \leq \sup_{\pi_0} \mathbb{D}^{-1}\left(\tilde{R} \mid \mathbb{I}(o_0; s_0)\right). \quad (\text{E.53})$$

From the monotonicity of the KL inverse (Lemma 4), we have:

$$\sup_{\pi_0} R \leq \mathbb{D}^{-1}\left(\sup_{\pi_0} \tilde{R} \mid \mathbb{I}(o_0; s_0)\right). \quad (\text{E.54})$$

Since by definition,

$$R^*(\sigma_0; r_0) = \sup_{\pi_0} R, \quad (\text{E.55})$$

we have:

$$R^*(\sigma_0; r_0) \leq \mathbb{D}^{-1}\left(\sup_{\pi_0} \tilde{R} \mid \mathbb{I}(o_0; s_0)\right). \quad (\text{E.56})$$

Now, using the Fubini-Tonelli theorem, we see:

$$\sup_{\pi_0} \tilde{R} = \sup_{\pi_0} \mathbb{E}_{p_0(s_0)} \mathbb{E}_{q(o_0)} [r_0(s_0, \pi_0(o_0))] \quad (\text{E.57})$$

$$= \sup_{\pi_0} \mathbb{E}_{q(o_0)} \mathbb{E}_{p_0(s_0)} [r_0(s_0, \pi_0(o_0))] \quad (\text{E.58})$$

$$\leq \mathbb{E}_{q(o_0)} \sup_{\pi_0} \mathbb{E}_{p_0(s_0)} [r_0(s_0, \pi_0(o_0))] \quad (\text{E.59})$$

$$= \mathbb{E}_{q(o_0)} \sup_{a_0} \mathbb{E}_{p_0(s_0)} [r_0(s_0, a_0)] \quad (\text{E.60})$$

$$= \sup_{a_0} \mathbb{E}_{p_0(s_0)} [r_0(s_0, a_0)]. \quad (\text{E.61})$$

Since open-loop actions are special cases of policies, we also have:

$$\sup_{\pi_0} \tilde{R} = \sup_{\pi_0} \mathbb{E}_{p_0(s_0)} \mathbb{E}_{q(o_0)} [r_0(s_0, \pi_0(o_0))] \geq \sup_{a_0} \mathbb{E}_{p_0(s_0)} [r_0(s_0, a_0)]. \quad (\text{E.62})$$

As a result, we see that:

$$\sup_{\pi_0} \tilde{R} = \sup_{a_0} \mathbb{E}_{p_0(s_0)} [r_0(s_0, a_0)] = R_0^\perp, \quad (\text{E.63})$$

where $R_0^\perp$ is as defined in (E.7). Combining this with (E.56), we obtain the desired result:

$$R^*(\sigma_0; r_0) \leq \mathbb{D}^{-1} \left(R_0^\perp \mid \mathbb{I}(o_0; s_0) \right) =: \tau(\sigma_0; r_0). \quad (\text{E.64})$$

□

Proposition 14 (Recursive bound). For any $t = 0, \dots, T-1$, the following inequality holds for any open-loop sequence of actions $a_{0:t-1}$:

$$\sup_{\pi_t^t, \dots, \pi_{T-1}^t} R_t \leq \underbrace{(T-t) \cdot \mathbb{D}^{-1} \left(\frac{R_t^{\perp*}}{T-t} \mid \mathbb{I}(o_t; s_t) \right)}_{=: \tau_t(\sigma_{t:T-1}; r_{t:T-1})}. \quad (\text{E.15})$$

Proof. The proof follows a similar structure to that of Theorem 13. First, note that R_t

defined in (E.10) can be written as:

$$R_t = \mathbb{E}_{\substack{s_t| \\ a_{0:t-1}}} \mathbb{E}_{o_t|s_t} \mathbb{E}_{\substack{s_{t+1:T-1}, o_{t+1:T-1} \\ s_t, o_t}} \left[\sum_{k=t}^{T-1} r_k(s_k, \pi_k^t(o_{t:k})) \right].$$

Define:

$$\tilde{R}_t := \mathbb{E}_{\substack{s_t| \\ a_{0:t-1}}} \mathbb{E}_{q(o_t)} \mathbb{E}_{\substack{s_{t+1:T-1}, o_{t+1:T-1} \\ s_t, o_t}} \left[\sum_{k=t}^{T-1} r_k(s_k, \pi_k^t(o_{t:k})) \right].$$

The only difference between $\tilde{R}_t$ and R_t is that the observations o_t in $\tilde{R}_t$ are drawn from a state-independent distribution q .

For the sake of notational simplicity, we will assume that R_t and $\tilde{R}_t$ have been normalized to be within $[0, 1]$ by scaling with $1/(T-t)$. The desired result (E.15) then follows from the bound we prove below by simply rescaling with $(T-t)$.

Now,

$$\mathbb{D}_{\mathcal{B}}(R_t \| \tilde{R}_t) = \mathbb{D}_{\mathcal{B}} \left(\mathbb{E}_{\substack{s_t| \\ a_{0:t-1}}} \mathbb{E}_{o_t|s_t} \mathbb{E}_{\substack{s_{t+1:T-1}, o_{t+1:T-1} \\ s_t, o_t}} \left[\sum_{k=t}^{T-1} r_k(s_k, \pi_k^t(o_{t:k})) \right] \right. \quad (\text{E.65})$$

$$\left. \left\| \mathbb{E}_{\substack{s_t| \\ a_{0:t-1}}} \mathbb{E}_{q(o_t)} \mathbb{E}_{\substack{s_{t+1:T-1}, o_{t+1:T-1} \\ s_t, o_t}} \left[\sum_{k=t}^{T-1} r_k(s_k, \pi_k^t(o_{t:k})) \right] \right\| \right)$$

$$\leq \mathbb{E}_{\substack{s_t| \\ a_{0:t-1}}} \mathbb{D}_{\mathcal{B}} \left(\mathbb{E}_{o_t|s_t} \mathbb{E}_{\substack{s_{t+1:T-1}, o_{t+1:T-1} \\ s_t, o_t}} \left[\sum_{k=t}^{T-1} r_k(s_k, \pi_k^t(o_{t:k})) \right] \right. \quad (\text{E.66})$$

$$\left. \left\| \mathbb{E}_{q(o_t)} \mathbb{E}_{\substack{s_{t+1:T-1}, o_{t+1:T-1} \\ s_t, o_t}} \left[\sum_{k=t}^{T-1} r_k(s_k, \pi_k^t(o_{t:k})) \right] \right\| \right)$$

$$\leq \mathbb{E}_{\substack{s_t| \\ a_{0:t-1}}} \mathbb{D}(\sigma_t(o_t|s_t) \| q(o_t)). \quad (\text{E.67})$$

The first inequality above follows from Jensen's inequality, while the second follows from the data processing inequality (see [100, Corollary 2] for the specific version).

From the well-known variational representation of the mutual information [99], we note that:

$$\mathbb{I}(o_t; s_t) = \inf_q \mathbb{E}_{\substack{s_t| \\ a_{0:t-1}}} \mathbb{D}(\sigma_t(o_t|s_t) \| q(o_t)). \quad (\text{E.68})$$

Thus, taking the infimum of (E.67) with respect to q , we see:

$$\mathbb{D}_{\mathcal{B}}(R_t \| \tilde{R}_t) \leq \mathbb{I}(o_t; s_t). \quad (\text{E.69})$$

We can then upper bound R_t using the KL inverse:

$$R_t \leq \mathbb{D}^{-1}(\tilde{R}_t \mid \mathbb{I}(o_t; s_t)). \quad (\text{E.70})$$

Thus,

$$\sup_{\pi_t^t, \dots, \pi_{T-1}^t} R_t \leq \sup_{\pi_t^t, \dots, \pi_{T-1}^t} \mathbb{D}^{-1}(\tilde{R}_t \mid \mathbb{I}(o_t; s_t)). \quad (\text{E.71})$$

Notice that the LHS is precisely the quantity we are interested in upper bounding in Proposition 14. From the monotonicity of the KL inverse (Lemma 4), we have:

$$\sup_{\pi_t^t, \dots, \pi_{T-1}^t} R_t \leq \mathbb{D}^{-1}(\tilde{R}_t^* \mid \mathbb{I}(o_t; s_t)), \quad (\text{E.72})$$

where

$$\tilde{R}_t^* := \sup_{\pi_t^t, \dots, \pi_{T-1}^t} \tilde{R}_t = \sup_{\pi_t^t, \dots, \pi_{T-1}^t} \mathbb{E}_{s_t | q(o_t)} \mathbb{E}_{a_{0:t-1}} \mathbb{E}_{s_{t+1:T-1}, o_{t+1:T-1} | s_t, o_t} \mathbb{E} \left[\sum_{k=t}^{T-1} r_k(s_k, \pi_k^t(o_{t:k})) \right]. \quad (\text{E.73})$$

Now, using the Fubini-Tonelli theorem:

$$\sup_{\pi_t^t, \dots, \pi_{T-1}^t} \mathbb{E}_{s_t | q(o_t)} \mathbb{E}_{a_{0:t-1}} \mathbb{E}_{s_{t+1:T-1}, o_{t+1:T-1} | s_t, o_t} \mathbb{E} \left[\sum_{k=t}^{T-1} r_k(s_k, \pi_k^t(o_{t:k})) \right] \quad (\text{E.74})$$

$$= \sup_{\pi_t^t, \dots, \pi_{T-1}^t} \mathbb{E}_{q(o_t)} \mathbb{E}_{s_t | a_{0:t-1}} \mathbb{E}_{s_{t+1:T-1}, o_{t+1:T-1} | s_t, o_t} \mathbb{E} \left[\sum_{k=t}^{T-1} r_k(s_k, \pi_k^t(o_{t:k})) \right] \quad (\text{E.75})$$

$$= \sup_{\pi_{t+1}^t, \dots, \pi_{T-1}^t} \left[\sup_{\pi_t^t} \mathbb{E}_{q(o_t)} \mathbb{E}_{s_t | a_{0:t-1}} \mathbb{E}_{s_{t+1:T-1}, o_{t+1:T-1} | s_t, o_t} \mathbb{E} \left[\sum_{k=t}^{T-1} r_k(s_k, \pi_k^t(o_{t:k})) \right] \right] \quad (\text{E.76})$$

$$\leq \sup_{\pi_{t+1}^t, \dots, \pi_{T-1}^t} \left[\mathbb{E}_{q(o_t)} \sup_{\pi_t^t} \mathbb{E}_{s_t | a_{0:t-1}} \mathbb{E}_{s_{t+1:T-1}, o_{t+1:T-1} | s_t, o_t} \mathbb{E} \left[\sum_{k=t}^{T-1} r_k(s_k, \pi_k^t(o_{t:k})) \right] \right]. \quad (\text{E.77})$$

Notice that:

$$\mathbb{E}_{q(o_t)} \sup_{\pi_t^t} \mathbb{E}_{s_t | a_{0:t-1}} \mathbb{E}_{s_{t+1:T-1}, o_{t+1:T-1} | s_t, o_t} \left[\sum_{k=t}^{T-1} r_k(s_k, \pi_k^t(o_{t:k})) \right] \quad (\text{E.78})$$

$$= \mathbb{E}_{q(o_t)} \sup_{\pi_t^t} \mathbb{E}_{s_t | a_{0:t-1}} \left[r_t(s_t, \pi_t^t(o_t)) + \mathbb{E}_{s_{t+1}, o_{t+1} | s_t, \pi_t^t(o_t)} \left[r_{t+1}(s_{t+1}, \pi_{t+1}^t(o_{t:t+1})) + \dots \right] \dots \right] \quad (\text{E.79})$$

$$= \mathbb{E}_{q(o_t)} \sup_{a_t} \mathbb{E}_{s_t | a_{0:t-1}} \underbrace{\left[r_t(s_t, a_t) + \mathbb{E}_{s_{t+1}, o_{t+1} | s_t, a_t} \left[r_{t+1}(s_{t+1}, \pi_{t+1}^{t+1}(o_{t+1})) + \dots \right] \dots \right]}_{\text{Does not depend on } o_t} \quad (\text{E.80})$$

$$= \sup_{a_t} \mathbb{E}_{s_t | a_{0:t-1}} \left[r_t(s_t, a_t) + \mathbb{E}_{s_{t+1}, o_{t+1} | s_t, a_t} \left[r_{t+1}(s_{t+1}, \pi_{t+1}^{t+1}(o_{t+1})) + \dots \right] \dots \right]. \quad (\text{E.81})$$

Here, (E.80) follows (E.79) since q is a fixed distribution that does not depend on the state.

We thus see that (E.77) equals:

$$\sup_{\pi_{t+1}^t, \dots, \pi_{T-1}^t} \underbrace{\left[\sup_{a_t} \mathbb{E}_{s_t | a_{0:t-1}} \left[r_t(s_t, a_t) + \mathbb{E}_{s_{t+1}, o_{t+1} | s_t, a_t} \left[r_{t+1}(s_{t+1}, \pi_{t+1}^{t+1}(o_{t+1})) + \dots \right] \dots \right] \right]}_{\text{Does not depend on } o_t} \quad (\text{E.82})$$

$$= \sup_{\pi_{t+1}^{t+1}, \dots, \pi_{T-1}^{t+1}} \left[\sup_{a_t} \mathbb{E}_{s_t | a_{0:t-1}} \left[r_t(s_t, a_t) + \mathbb{E}_{s_{t+1}, o_{t+1} | s_t, a_t} \left[r_{t+1}(s_{t+1}, \pi_{t+1}^{t+1}(o_{t+1})) + \dots \right] \dots \right] \right] \quad (\text{E.83})$$

$$= \sup_{\pi_{t+1}^{t+1}, \dots, \pi_{T-1}^{t+1}} \left[\sup_{a_t} \mathbb{E}_{s_t | a_{0:t-1}} \left[r_t(s_t, a_t) \right] + \mathbb{E}_{s_{t+1:T-1}, o_{t+1:T-1} | a_{0:t}} \left[\sum_{k=t+1}^{T-1} r_k(s_k, \pi_k^{t+1}(o_{t+1:k})) \right] \right] \quad (\text{E.84})$$

$$= \sup_{\pi_{t+1}^{t+1}, \dots, \pi_{T-1}^{t+1}} R_t^\perp \quad (\text{E.85})$$

$$= R_t^{\perp*}. \quad (\text{E.86})$$

We have thus proved that $\tilde{R}_t^* \leq R_t^{\perp*}$ (indeed, since open-loop policies are special cases of feedback policies, we also have $\tilde{R}_t^* \geq R_t^{\perp*}$ and thus $\tilde{R}_t^* = R_t^{\perp*}$). Since the RHS of (E.72) is a monotonically non-decreasing function of $\tilde{R}_t^*$ (Lemma 4), the observation that $\tilde{R}_t^* \leq R_t^{\perp*}$ implies:

$$\sup_{\pi_t^t, \dots, \pi_{T-1}^t} R_t \leq \mathbb{D}^{-1} \left(R_t^{\perp*} \mid \mathbb{I}(o_t; s_t) \right). \quad (\text{E.87})$$

□

Theorem 14 (Multi-step performance bound). Algorithm 7 returns an upper bound on the best achievable reward R^* .

Proof. Using (backwards) induction, we prove that for all $t = T - 1, \dots, 0$,

$$\sup_{\pi_t^t, \dots, \pi_{T-1}^t} R_t \leq \bar{R}_t(a_{0:t-1}), \quad \forall a_{0:t-1}. \quad (\text{E.88})$$

Thus, in particular,

$$R^* = \sup_{\pi_0^0, \dots, \pi_{T-1}^0} R_0 \leq \bar{R}_0. \quad (\text{E.89})$$

We prove (E.88) by backwards induction starting from $t = T - 1$. In particular, Proposition 14 leads to the inductive step. We first prove the base step of induction using $t = T - 1$. Using (E.15), we see:

$$\sup_{\pi_{T-1}^{T-1}} R_{T-1} \leq \mathbb{D}^{-1}\left(R_{T-1}^{\perp*} \mid \mathbb{I}(o_{T-1}; s_{T-1})\right). \quad (\text{E.90})$$

By definition (see (E.14)),

$$\begin{aligned} R_{T-1}^{\perp*} &= \sup_{a_{T-1}} \mathbb{E}_{s_{T-1} \mid a_{0:T-2}} \left[r_{T-1}(s_{T-1}, a_{T-1}) \right] + \underbrace{R_T}_{=0} \\ &= \sup_{a_{T-1}} \mathbb{E}_{s_{T-1} \mid a_{0:T-2}} \left[r_{T-1}(s_{T-1}, a_{T-1}) \right] \\ &= \bar{R}_{T-1}^{\perp*}. \end{aligned}$$

Combining this with (E.90) and the monotonicity of the KL inverse (Proposition 13), we see:

$$\sup_{\pi_{T-1}^{T-1}} R_{T-1} \leq \mathbb{D}^{-1}\left(\bar{R}_{T-1}^{\perp*} \mid \mathbb{I}(o_{T-1}; s_{T-1})\right) \quad (\text{E.91})$$

$$= \bar{R}_{T-1}(a_{0:T-2}). \quad (\text{E.92})$$

Next, we prove the induction step. Suppose it is the case that for $t \in \{0, \dots, T - 2\}$, we

have

$$\sup_{\pi_{t+1}^{t+1}, \dots, \pi_{T-1}^{t+1}} R_{t+1} \leq \bar{R}_{t+1}(a_{0:t}). \quad (\text{E.93})$$

We then need to show that

$$\sup_{\pi_t^t, \dots, \pi_{T-1}^t} R_t \leq \bar{R}_t(a_{0:t-1}). \quad (\text{E.94})$$

To prove this, we first observe that

$$\begin{aligned} R_t^{\perp\star} &:= \sup_{\pi_{t+1}^{t+1}, \dots, \pi_{T-1}^{t+1}} \sup_{a_t} \left[\mathbb{E}_{s_t|a_{0:t-1}} [r_t(s_t, a_t)] + R_{t+1} \right] \\ &= \sup_{a_t} \left[\mathbb{E}_{s_t|a_{0:t-1}} [r_t(s_t, a_t)] + \sup_{\pi_{t+1}^{t+1}, \dots, \pi_{T-1}^{t+1}} R_{t+1} \right]. \end{aligned}$$

Combining this with the induction hypothesis (E.93), we see

$$R_t^{\perp\star} \leq \sup_{a_t} \left[\mathbb{E}_{s_t|a_{0:t-1}} [r_t(s_t, a_t)] + \bar{R}_{t+1}(a_{0:t}) \right] \quad (\text{E.95})$$

$$=: \bar{R}_t^{\perp\star}. \quad (\text{E.96})$$

Finally, combining this with (E.15) and the monotonicity of the KL inverse (Prop. 13), we obtain the desired result (E.94):

$$\sup_{\pi_t^t, \dots, \pi_{T-1}^t} R_t \leq (T-t) \cdot \mathbb{D}^{-1} \left(\frac{\bar{R}_t^{\perp\star}}{T-t} \mid \mathbb{I}(o_t; s_t) \right) \quad (\text{E.97})$$

$$= \bar{R}_t(a_{0:t-1}). \quad (\text{E.98})$$

□

E.10 Chernoff-Hoeffding Bound

In our numerical examples (Sec. E.7), we utilize a slightly tighter version of Hoeffding's inequality than the one presented in Theorem 15. In particular, we use the following Chernoff-Hoeffding inequality (see [208, Theorem 5.1]).

Theorem 16 (Chernoff-Hoeffding inequality [208]). Let z be a random variable bounded

within $[0, 1]$, and let $z_1, \dots, z_n$ denote i.i.d. samples. Then, with probability at least $1 - \delta$ (over the sampling of $z_1, \dots, z_n$), the following bound holds with probability at least $1 - \delta$:

$$\mathbb{D}_{\mathcal{B}}\left(\frac{1}{n} \sum_{i=1}^n z_i \parallel \mathbb{E}[z]\right) \leq \frac{\log(2/\delta)}{n}. \quad (\text{E.99})$$

We can obtain an upper bound on $\mathbb{E}[z]$ using (E.99) as follows:

$$\mathbb{E}[z] \leq \sup \left\{ p \in [0, 1) \mid \mathbb{D}_{\mathcal{B}}\left(\frac{1}{n} \sum_{i=1}^n z_i \parallel p\right) \leq \frac{\log(2/\delta)}{n} \right\}. \quad (\text{E.100})$$

The optimization problem in the RHS of (E.100) is analogous to the KL inverse defined in Sec. E.3.2, and can be thought of as a “right” KL inverse (instead of a “left” KL inverse). Similar to the KL inverse in Sec. E.3.2, we can solve the optimization problem in (E.100) using geometric programming.

Bibliography

- [1] B. Anderson and J. Moore, *Optimal Control: Linear Quadratic Methods*. Courier Corporation, 2007.
- [2] H. A. Simon, “A behavioral model of rational choice,” *The Quarterly Journal of Economics*, vol. 69, no. 1, pp. 99–118, 1955.
- [3] G. Gigerenzer and H. Brighton, “Homo heuristics: Why biased minds make better inferences,” *Topics in Cognitive Science*, vol. 1, no. 1, pp. 107–143, 2009.
- [4] G. Gigerenzer and W. Gaissmaier, “Heuristic decision making,” *Annual Review of Psychology*, vol. 62, pp. 451–482, 2011.
- [5] S. Höfer, J. Raisch, M. Toussaint, and O. Brock, “No free lunch in ball catching: A comparison of cartesian and angular representations for control,” *PloS One*, vol. 13, no. 6, 2018.
- [6] S. Chapman, “Catching a baseball,” *American Journal of Physics*, vol. 36, no. 10, pp. 868–870, 1968.
- [7] V. Pacelli and A. Majumdar, “Robust control under uncertainty via bounded rationality and differential privacy,” in *International Conference on Robotics and Automation*, IEEE, 2022, pp. 3467–3474.
- [8] V. Pacelli and A. Majumdar, “Task-driven estimation and control via information bottlenecks,” in *Proceedings of the International Conference on Robotics and Automation*, IEEE, 2019, pp. 2061–2067.
- [9] V. Pacelli and A. Majumdar, “Learning task-driven control policies via information bottlenecks,” in *Proceedings of Robotics: System and Science (RSS)*, 2020.

- [10] A. Majumdar and V. Pacelli, “Fundamental performance limits for sensor-based robot control and policy learning,” in *Proceedings of Robotics: System and Science (RSS)*, 2022.
- [11] A. Majumdar, Z. Mei, and V. Pacelli, “Fundamental limits for sensor-based robot control,” *arXiv preprint arXiv:2202.00129*, 2023.
- [12] R. Landauer, “Irreversibility and heat generation in the computing process,” *IBM Journal of Research and Development*, vol. 5, no. 3, pp. 183–191, 1961.
- [13] T. M. Cover, *Elements of Information Theory*. John Wiley & Sons, 1999.
- [14] E. T. Jaynes, “Information theory and statistical mechanics,” *Physical Review*, vol. 106, no. 4, p. 620, 1957.
- [15] R. Zwanzig, *Nonequilibrium Statistical Mechanics*. Oxford University Press, 2001.
- [16] M. D. Donsker and S. R. S. Varadhan, “Asymptotic evaluation of certain Markov process expectations for large time,” *Communications on Pure and Applied Mathematics*, vol. 28, pp. 1–47, 1975.
- [17] C. Dwork, F. McSherry, K. Nissim, and A. Smith, “Calibrating noise to sensitivity in private data analysis,” in *Proceedings of the Theory of Cryptography Conference*, Springer, 2006, pp. 265–284.
- [18] C. Dwork and A. Roth, “The algorithmic foundations of differential privacy,” *Foundations and Trends in Theoretical Computer Science*, vol. 9, no. 3-4, pp. 211–407, 2014.
- [19] V. Mnih *et al.*, “Playing atari with deep reinforcement learning,” *arXiv preprint arXiv:1312.5602*, 2013.
- [20] X. Guo, S. Singh, H. Lee, R. L. Lewis, and X. Wang, “Deep learning for real-time Atari game play using offline Monte-Carlo tree search planning,” *Advances in Neural Information Processing Systems*, vol. 27, 2014.
- [21] C. Dwork, “Differential privacy,” *Encyclopedia of Cryptography and Security*, pp. 338–340, 2011.

- [22] K. Chatzikokolakis, M. E. Andrés, N. E. Bordenabe, and C. Palamidessi, “Broadening the scope of differential privacy using metrics,” in *Proceedings of the International Symposium on Privacy Enhancing Technologies*, Springer, 2013, pp. 82–102.
- [23] C. Dimitrakakis, B. Nelson, Z. Zhang, A. Mitrokotsa, and B. Rubinstein, “Differential privacy for Bayesian inference through posterior sampling,” *Journal of Machine Learning Research*, vol. 18, no. 11, pp. 1–39, 2017.
- [24] J. Le Ny and G. J. Pappas, “Differentially private filtering,” *IEEE Transactions on Automatic Control*, vol. 59, no. 2, pp. 341–354, 2013.
- [25] J. Cortés, G. E. Dullerud, S. Han, J. Le Ny, S. Mitra, and G. J. Pappas, “Differential privacy in control and network systems,” in *Proceedings of the Conference on Decision and Control*, IEEE, 2016, pp. 4252–4272.
- [26] E. Nozari, P. Tallapragada, and J. Cortés, “Differentially private distributed convex optimization via objective perturbation,” in *Proceedings of the American Control Conference*, IEEE, 2016, pp. 2061–2066.
- [27] S. Han, U. Topcu, and G. J. Pappas, “Differentially private distributed constrained optimization,” *IEEE Transactions on Automatic Control*, vol. 62, no. 1, pp. 50–64, 2016.
- [28] F. Koufogiannis and G. J. Pappas, “Differential privacy for dynamical sensitive data,” in *Proceedings of the Conference on Decision and Control*, IEEE, 2017, pp. 1118–1125.
- [29] S. Han and G. J. Pappas, “Privacy in control and dynamical systems,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 1, pp. 309–332, 2018.
- [30] W. H. Warren Jr., D. S. Young, and D. N. Lee, “Visual control of step length during running over irregular terrain,” *Journal of Experimental Psychology: Human Perception and Performance*, vol. 12, no. 3, p. 259, 1986.
- [31] D. N. Lee, “A theory of visual control of braking based on information about time-to-collision,” *Perception*, vol. 5, no. 4, pp. 437–459, 1976.

- [32] N. Tishby and D. Polani, “Information theory of decisions and actions,” in *Perception-Action Cycle: Models, Architectures, and Hardware*, Springer, 2011, pp. 601–636.
- [33] R. Fox and N. Tishby, “Minimum-information lqg control part I: Memoryless controllers,” in *Proceedings of the Conference on Decision and Control*, IEEE, 2016, pp. 5610–5616.
- [34] R. Fox and N. Tishby, “Bounded planning in passive POMDPs,” in *Proceedings of the International Conference on Machine Learning*, 2012.
- [35] P. A. Ortega and D. D. Lee, “An adversarial interpretation of information-theoretic bounded rationality,” in *Proceedings of the Conference on Artificial Intelligence*, 2014.
- [36] P. A. Ortega, D. A. Braun, J. Dyer, K.-E. Kim, and N. Tishby, “Information-theoretic bounded rationality,” *arXiv preprint arXiv:1512.06789*, 2015.
- [37] E. Shafieepoorfard, M. Raginsky, and S. P. Meyn, “Rationally inattentive control of Markov processes,” *SIAM Journal on Control and Optimization*, vol. 54, no. 2, pp. 987–1016, 2016.
- [38] M. Igl *et al.*, “Generalization in reinforcement learning with selective noise injection and information bottleneck,” *Advances in Neural Information Processing Systems*, vol. 32, pp. 13 978–13 990, 2019.
- [39] X. Lu, K. Lee, P. Abbeel, and S. Tiomkin, “Dynamics generalization via information bottleneck in deep reinforcement learning,” *arXiv preprint arXiv:2008.00614*, 2020.
- [40] A. R. Pedram, J. Stefarr, R. Funada, and T. Tanaka, “Rationally inattentive path-planning via RRT,” in *Proceedings of the American Control Conference*, IEEE, 2021, pp. 3440–3446.
- [41] M. D. Donsker and S. R. S. Varadhan, “Asymptotic evaluation of certain Markov process expectations for large time. IV,” *Communications on Pure and Applied Mathematics*, vol. 36, no. 2, pp. 183–212, 1983.
- [42] B. Eysenbach and S. Levine, “Maximum entropy RL (provably) solves some robust RL problems,” *arXiv preprint arXiv:2103.06257*, 2021.

- [43] P. D. Grünwald and A. P. Dawid, “Game theory, maximum entropy, minimum discrepancy and robust Bayesian decision theory,” *Annals of Statistics*, vol. 32, no. 4, pp. 1367–1433, 2004.
- [44] H. J. Kappen, “Linear theory for control of nonlinear stochastic systems,” *Physical Review Letters*, vol. 95, no. 20, p. 200 201, 2005.
- [45] M. B. Horowitz and J. W. Burdick, “Optimal navigation functions for nonlinear stochastic systems,” in *Proceedings of the International Conference on Intelligent Robots and Systems*, IEEE, 2014, pp. 224–231.
- [46] H. J. Kappen, “Path integrals and symmetry breaking for optimal control theory,” *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2005, no. 11, 2005.
- [47] E. Todorov, “General duality between optimal control and estimation,” in *Proceedings of the Conference on Decision and Control*, IEEE, 2008, pp. 4286–4292.
- [48] E. Todorov, “Efficient computation of optimal actions,” *Proceedings of the National Academy of Sciences*, vol. 106, no. 28, pp. 11 478–11 483, 2009.
- [49] G. Williams *et al.*, “Information theoretic MPC for model-based reinforcement learning,” in *Proceedings of the International Conference on Robotics and Automation*, IEEE, 2017, pp. 1714–1721.
- [50] E. A. Theodorou and E. Todorov, “Relative entropy and free energy dualities: Connections to path integral and KL control,” in *Proceedings of the Conference on Decision and Control*, IEEE, 2012, pp. 1466–1473.
- [51] D. A. Braun, P. A. Ortega, E. Theodorou, and S. Schaal, “Path integral control and bounded rationality,” in *Proceedings of the Symposium on Adaptive Dynamic Programming and Reinforcement Learning*, IEEE, 2011, pp. 202–209.
- [52] A. Lambert, A. Fishman, D. Fox, B. Boots, and F. Ramos, “Stein variational model predictive control,” in *Proceedings of the Conference on Robot Learning*, 2020.
- [53] L. Barcelos, A. Lambert, R. Oliveira, P. Borges, B. Boots, and F. Ramos, “Dual on-line Stein variational inference for control and dynamics,” in *Proceedings of Robotics: System and Science (RSS)*, 2021.

- [54] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012.
- [55] Q. Liu and D. Wang, “Stein variational gradient descent: A general purpose Bayesian inference algorithm,” in *Advances in Neural Information Processing Systems*, 2016.
- [56] T. Haarnoja, H. Tang, P. Abbeel, and S. Levine, “Reinforcement learning with deep energy-based policies,” in *Proceedings of the International Conference on Machine Learning*, 2017, pp. 1352–1361.
- [57] T. Haarnoja, V. Pong, A. Zhou, M. Dalal, P. Abbeel, and S. Levine, “Composable deep reinforcement learning for robotic manipulation,” in *Proceedings of the International Conference on Robotics and Automation*, IEEE, 2018, pp. 6244–6251.
- [58] S. Levine, “Reinforcement learning and control as probabilistic inference: Tutorial and review,” *arXiv preprint arXiv:1805.00909*, 2018.
- [59] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, Mar. 1960, ISSN: 0021-9223. DOI: 10.1115/1.3662552.
- [60] A. Maurer, “Thermodynamics and concentration,” *Bernoulli*, vol. 18, no. 2, pp. 434–454, 2012.
- [61] N. Tishby, F. C. Pereira, and W. Bialek, “The information bottleneck method,” *arXiv preprint physics/0004057*, 2000.
- [62] B. Poole, S. Ozair, A. Van Den Oord, A. Alemi, and G. Tucker, “On variational bounds of mutual information,” in *Proceedings of the International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, PMLR, 2019, pp. 5171–5180.
- [63] H. Chernoff, “A measure of asymptotic efficiency for tests of a hypothesis based on the sum of observations,” *The Annals of Mathematical Statistics*, pp. 493–507, 1952.
- [64] H. Touchette, “A basic introduction to large deviations: Theory, applications, simulations,” *arXiv preprint arXiv:1106.4146*, 2011.

- [65] M. Raginsky and I. Sason, “Concentration of measure inequalities in information theory, communications, and coding,” *Foundations and Trends in Communications and Information Theory*, vol. 10, no. 1-2, pp. 1–247, 2013.
- [66] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.
- [67] A. Gelman, J. B. Carlin, H. S. Stern, and D. B. Rubin, *Bayesian Data Analysis*, 3rd. Chapman and Hall, 2020.
- [68] Ø. Skare, E. Bølviken, and L. Holden, “Improved sampling-importance resampling and reduced bias importance sampling,” *Scandinavian Journal of Statistics*, vol. 30, no. 4, pp. 719–737, 2003.
- [69] W. Li and E. Todorov, “Iterative linear quadratic regulator design for nonlinear biological movement systems,” in *International Conference on Informatics in Control, Automation and Robotics*, Citeseer, 2004, pp. 222–229.
- [70] M. I. Belghazi *et al.*, “Mutual information neural estimation,” in *International Conference on Machine Learning*, PMLR, 2018, pp. 531–540.
- [71] D. Wang, Z. Tang, C. Bajaj, and Q. Liu, “Stein-variational gradient descent with matrix-valued kernels,” *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [72] R. T. M’Closkey and J. W. Burdick, “Periodic motions of a hopping robot with vertical and forward motion,” *International Journal of Robotics Research*, vol. 12, no. 3, pp. 197–218, 1993.
- [73] W. J. Schwind and D. E. Koditschek, “Control of forward velocity for a simplified planar hopping robot,” in *Proceedings of the International Conference on Robotics and Automation*, IEEE, vol. 1, 1995, pp. 691–696.
- [74] H. Geyer, “Simple models of legged locomotion based on compliant limb behavior,” Ph.D. dissertation, 2005.
- [75] W. H. Warren, “Action-scaled information for the visual control of locomotion,” in *Closing the Gap*, Psychology Press, 2007, pp. 261–296.

- [76] A. R. Cassandra, L. P. Kaelbling, and M. L. Littman, “Acting optimally in partially observable stochastic domains,” in *AAAI*, vol. 94, 1994, pp. 1023–1028.
- [77] P. R. Florence, “Integrated perception and control at high speed,” Ph.D. dissertation, Massachusetts Institute of Technology, 2017.
- [78] J. Steinhardt and R. Tedrake, “Finite-time regional verification of stochastic nonlinear systems,” *International Journal of Robotics Research*, vol. 31, no. 7, pp. 901–923, 2012.
- [79] J. Förster, “System identification of the Crazyflie 2.0 nano quadcopter,” B.S. Thesis, ETH Zurich, 2015.
- [80] M. Fliess, J. Lévine, P. Martin, and P. Rouchon, “On differentially flat nonlinear systems,” in *Nonlinear Control Systems Design*, Elsevier, 1993, pp. 159–163.
- [81] M. van Nieuwstadt, M. Rathinam, and R. M. Murray, “Differential flatness and absolute equivalence of nonlinear control systems,” *Journal on Control and Optimization*, vol. 36, no. 4, pp. 1225–1239, 1998.
- [82] R. M. Murray, M. Rathinam, and W. Sluis, “Differential flatness of mechanical control systems: A catalog of prototype systems,” in *International Mechanical Engineering Congress and Exposition*, ASME, 1995.
- [83] D. Mellinger and V. Kumar, “Minimum snap trajectory generation and control for quadrotors,” in *IEEE International Conference on Robotics and Automation*, 2011, pp. 2520–2525.
- [84] C. K. I. Williams and C. E. Rasmussen, *Gaussian Processes for Machine Learning*. MIT Press, 2006, vol. 2.
- [85] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
- [86] J. Morris, “Can Tesla really do without radar for full self-driving?” *Forbes*, Jul. 2021.
- [87] H. A. Simon, *The Sciences of the Artificial*, 3rd. MIT press, 2019.

- [88] D. E. Koditschek, “What is robotics? Why do we need it and how can we get it?” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, pp. 1–33, 2021.
- [89] G. N. Nair, R. J. Evans, I. M. Y. Mareels, and W. Moran, “Topological feedback entropy and nonlinear stabilization,” *Transactions on Automatic Control*, vol. 49, no. 9, pp. 1585–1597, 2004.
- [90] S. M. LaValle, *Sensing and filtering: A fresh perspective based on preimages and information spaces*. Citeseer, 2012.
- [91] J. M. O’Kane and S. M. LaValle, “Comparing the power of robots,” *The International Journal of Robotics Research*, vol. 27, no. 1, pp. 5–23, 2008.
- [92] F. Z. Saberifar, S. Ghasemlou, D. A. Shell, and J. M. O’Kane, “Toward a language-theoretic foundation for planning and filtering,” en, *International Journal of Robotics Research*, vol. 38, no. 2-3, pp. 236–259, Mar. 2019, SAGE Publications.
- [93] S. Karaman and E. Frazzoli, “High-speed flight in an ergodic forest,” in *Proceedings of the International Conference on Robotics and Automation*, 2012, pp. 2899–2906.
- [94] S. Choudhury, S. Scherer, and J. A. Bagnell, “Theoretical limits of speed and resolution for kinodynamic planning in a Poisson forest,” in *Proceedings of Robotics: System and Science (RSS)*, 2015.
- [95] B. Bollobás and O. Riordan, *Percolation*. Cambridge University Press, 2006.
- [96] Y. P. Leong and J. C. Doyle, “Understanding robust control theory via stick balancing,” in *Proceedings of the Conference on Decision and Control*, 2016, pp. 1508–1514.
- [97] J. Xu, B. Lee, N. Matni, and D. Jayaraman, “How are learned perception-based controllers impacted by the limits of robust control?” In *Learning for Dynamics and Control*, PMLR, 2021, pp. 954–966.
- [98] R. Fano, *Transmission of Information: A Statistical Theory of Communication*. MIT press, 1961.

- [99] X. Chen, A. Guntuboyina, and Y. Zhang, “On Bayes risk lower bounds,” *The Journal of Machine Learning Research*, vol. 17, no. 1, pp. 7687–7744, 2016.
- [100] S. Gerchinovitz, P. Ménard, and G. Stoltz, “Fano’s inequality for random variables,” *Statistical Science*, vol. 35, no. 2, pp. 178–201, 2020.
- [101] U. Seifert, “From stochastic thermodynamics to thermodynamic inference,” *Annual Review of Condensed Matter Physics*, vol. 10, pp. 171–192, 2019.
- [102] D. J. Evans, E. G. D. Cohen, and G. P. Morriss, “Probability of second law violations in shearing steady states,” *Physical review letters*, vol. 71, no. 15, p. 2401, 1993.
- [103] D. J. Evans and D. J. Searles, “The Fluctuation Theorem,” *Advances in Physics*, vol. 51, no. 7, pp. 1529–1585, 2002.
- [104] C. Jarzynski, “Nonequilibrium equality for free energy differences,” *Phys. Rev. Lett.*, vol. 78, pp. 2690–2693, 14 Apr. 1997. DOI: 10.1103/PhysRevLett.78.2690.
- [105] G. E. Crooks, “Entropy production fluctuation theorem and the nonequilibrium work relation for free energy differences,” *Physical Review E*, vol. 60, no. 3, p. 2721, 1999.
- [106] T. Sagawa and M. Ueda, “Nonequilibrium thermodynamics of feedback control,” *Physical Review E*, vol. 85, no. 2, p. 021 104, 2012.
- [107] T. Sagawa, “Thermodynamic and logical reversibilities revisited,” *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2014, no. 3, P03025, 2014.
- [108] J. M. R. Parrondo, J. M. Horowitz, and T. Sagawa, “Thermodynamics of information,” *Nature Physics*, vol. 11, no. 2, pp. 131–139, 2015.
- [109] J. Grau-Moya, M. Krüger, and D. A. Braun, “Non-equilibrium relations for bounded rational decision-making in changing environments,” *Entropy*, vol. 20, no. 1, p. 1, 2018.
- [110] Y. Chen, T. T. Georgiou, and M. Pavon, “Optimal transport in systems and control,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, no. 1, pp. 89–113, 2021. DOI: 10.1146/annurev-control-070220-100858.

- [111] Y. Chen, T. T. Georgiou, and A. Tannenbaum, “Stochastic control and nonequilibrium thermodynamics: Fundamental limits,” *IEEE transactions on automatic control*, vol. 65, no. 7, pp. 2979–2991, 2019.
- [112] P. Milgrom and I. Segal, “Envelope theorems for arbitrary choice sets,” *Econometrica*, vol. 70, no. 2, pp. 583–601, 2002.
- [113] R. Bellman, *Dynamic Programming*. Princeton University Press, 1957.
- [114] S. Boyd, S. P. Boyd, and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004.
- [115] S. Boucheron, G. Lugosi, and P. Massart, *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press, 2013.
- [116] K. Zheng and S. Tellex, “Pomdp-py: A framework to build and solve POMDP problems,” *arXiv preprint arXiv:2004.10099*, 2020.
- [117] A. Rényi, “On measures of entropy and information,” in *Proceedings of the Fourth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1: Contributions to the Theory of Statistics*, University of California Press, vol. 4, 1961, pp. 547–562.
- [118] I. Csiszár, “A class of measures of informativity of observation channels,” *Periodica Mathematica Hungarica*, vol. 2, no. 1-4, pp. 191–213, Mar. 1972.
- [119] T. Sagawa and M. Ueda, “Information thermodynamics: Maxwell’s demon in nonequilibrium dynamics,” *Nonequilibrium Statistical Physics of Small Systems: Fluctuation Relations and Beyond*, pp. 181–211, 2013.
- [120] J.-P. Aumasson, *Serious cryptography: a practical introduction to modern encryption*. No Starch Press, 2017.
- [121] B. Mahault, E. Tang, and R. Golestanian, “A topological fluctuation theorem,” *Nature Communications*, vol. 13, no. 1, p. 3036, 2022.
- [122] R. Kawai, J. M. R. Parrondo, and C. Van den Broeck, “Dissipation: The phase-space perspective,” *Physical Review Letters*, vol. 98, no. 8, p. 080 602, 2007.

- [123] D. H. Wolpert, “Minimal entropy production rate of interacting systems,” *New Journal of Physics*, vol. 22, no. 11, p. 113 013, 2020.
- [124] A. Kolchinsky and D. H. Wolpert, “Work, entropy production, and thermodynamics of information under protocol constraints,” *Physical Review X*, vol. 11, no. 4, p. 041 024, 2021.
- [125] J. Korbel and D. H. Wolpert, “Stochastic thermodynamics and fluctuation theorems for non-linear systems,” *New Journal of Physics*, vol. 23, no. 3, p. 033 049, 2021.
- [126] G. Gigerenzer, *Gut Feelings: The Intelligence of the Unconscious*. Penguin, 2007.
- [127] P. McLeod, N. Reed, and Z. Dienes, “Psychophysics: How fielders arrive in time to catch the ball,” *Nature*, vol. 426, no. 6964, p. 244, 2003.
- [128] D. M. Shaffer, S. M. Krauchunas, M. Eddy, and M. K. McBeath, “How dogs navigate to catch frisbees,” *Psychological Science*, vol. 15, no. 7, pp. 437–441, 2004.
- [129] N. Tishby, F. C. Pereira, and W. Bialek, “The information bottleneck method,” in *Proceedings of the 37th Allerton Conference on Communication, Control, and Computing*, (Urbana-Champaign, Illinois), 1999.
- [130] B. A. Francis, *A Course in H-Infinity Control Theory*. Springer-Verlag, 1987.
- [131] C. Edwards and S. Spurgeon, *Sliding mode control: theory and applications*. CRC Press, 1998.
- [132] J.-J. E. Slotine, W. Li, *et al.*, *Applied nonlinear control*. Prentice hall Englewood Cliffs, NJ, 1991, vol. 199.
- [133] R. Ortega, J. A. L. Perez, P. J. Nicklasson, and H. J. Sira-Ramirez, *Passivity-based control of Euler-Lagrange systems: mechanical, electrical and electromechanical applications*. Springer Science & Business Media, 2013.
- [134] H. K. Khalil, “Nonlinear systems,” *Prentice-Hall, New Jersey*, vol. 2, no. 5, pp. 5–1, 1996.
- [135] G. N. Nair, F. Fagnani, S. Zampieri, and R. J. Evans, “Feedback control under data rate constraints: An overview,” *Proc. of the IEEE*, vol. 95, no. 1, pp. 108–137, 2007.

- [136] A. S. Matveev and A. V. Savkin, *Estimation and control over communication networks*. Springer Science & Business Media, 2009.
- [137] T. Tanaka, K.-K. K. Kim, P. A. Parrilo, and S. K. Mitter, “Semidefinite programming approach to Gaussian sequential rate-distortion trade-offs,” *IEEE Trans. Automat. Control*, vol. 62, no. 4, pp. 1896–1910, 2017.
- [138] H. A. Simon, “Theories of bounded rationality,” *Decision and organization*, vol. 1, no. 1, pp. 161–176, 1972.
- [139] G. Gigerenzer and R. Selten, *Bounded rationality: The adaptive toolbox*. MIT press, 2002.
- [140] J. Grau-Moya, F. Leibfried, T. Genewein, and D. A. Braun, “Planning with information-processing constraints and model uncertainty in Markov decision processes,” in *Joint European Conf. Machine Learning and Knowledge Discovery in Databases*, Springer, 2016, pp. 475–491.
- [141] J. L. Williams, J. W. Fisher III, and A. S. Willsky, “Performance guarantees for information theoretic active inference,” in *Artificial Intelligence and Statistics*, 2007, pp. 620–627.
- [142] V. Tzoumas, L. Carlone, G. J. Pappas, and A. Jadbabaie, “Control and sensing co-design,” *arXiv preprint arXiv:1802.08376*, 2018.
- [143] L. Carlone and S. Karaman, “Attention and anticipation in fast visual-inertial navigation,” in *IEEE Int. Conf. Robotics and Automation*, IEEE, 2017, pp. 3886–3893.
- [144] A. Achille and S. Soatto, “A separation principle for control in the age of deep learning,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 1, pp. 287–307, 2018.
- [145] R. Fox and N. Tishby, “Minimum-information LQG control part ii: Retentive controllers,” in *Proc. 55th Conf. Decision and Control*, IEEE, 2016, pp. 5603–5609.
- [146] J. L. Massey, “Causality, feedback and directed information,” in *Proc. Int. Symp. Inf. Theory Applic.(ISITA-90)*, 1990, pp. 303–305.

- [147] T. Tanaka, P. M. Esfahani, and S. K. Mitter, “LQG control with minimum directed information: Semidefinite programming approach,” *IEEE Trans. Automat. Control*, vol. 63, no. 1, pp. 37–52, 2018.
- [148] S. Soatto, “Actionable information in vision,” in *Machine Learning for Computer Vision*, Springer, 2013, pp. 17–48.
- [149] S. Soatto, “Steps towards a theory of visual information: Active perception, signal-to-symbol conversion and the interplay between sensing and control,” *arXiv preprint arXiv:1110.2053*, 2011.
- [150] A. Shapiro, D. Dentcheva, and A. Ruszczyński, *Lectures on Stochastic Programming: Modeling and Theory*. SIAM, 2009.
- [151] R. M. Gray, *Entropy and Information Theory*, 2nd. Springer Science & Business Media, 2011.
- [152] P. Whittle, “Risk-sensitive linear/quadratic/Gaussian control,” *Advances in Applied Probability*, vol. 13, no. 4, pp. 764–777, 1981.
- [153] P. Whittle, “Risk sensitivity, a strangely pervasive concept,” *Macroeconomic Dynamics*, vol. 6, no. 1, p. 5, 2002.
- [154] K. Glover and J. C. Doyle, “State-space formulae for all stabilizing controllers that satisfy an H_∞ -norm bound and relations to relations to risk sensitivity,” *Systems & Control Letters*, vol. 11, no. 3, pp. 167–172, 1988.
- [155] R. Fox and N. Tishby, “Optimal selective attention in reactive agents,” *arXiv preprint arXiv:1512.08575*, 2015.
- [156] S. Wright and J. Nocedal, “Numerical optimization,” *Springer Science*, vol. 35, no. 67-68, p. 7, 1999.
- [157] E. Todorov and W. Li, “A generalized iterative lqg method for locally-optimal feedback control of constrained nonlinear stochastic systems,” in *Proc. American Control Conf.*, IEEE, 2005, pp. 300–306.
- [158] D. H. Jacobson and D. Q. Mayne, “Differential dynamic programming,” 1970.

- [159] P. R. Florence, “Integrated perception and control at high speed,” M.S. thesis, Massachusetts Institute of Technology, 2017.
- [160] R. T. M’Closkey and J. W. Burdick, “Periodic motions of a hopping robot with vertical and forward motion,” *Int. J. of Robotics Research*, vol. 12, no. 3, pp. 197–218, 1993.
- [161] W. J. Schwind and D. E. Koditschek, “Control of forward velocity for a simplified planar hopping robot,” in *Proc. Int. Conf. Robotics and Automation, 1995. Proceedings.*, IEEE, vol. 1, 1995, pp. 691–696.
- [162] H. Geyer, “Simple models of legged locomotion based on compliant limb behavior,” Ph.D. dissertation, Verlag nicht ermittelbar, 2005.
- [163] W. H. Warren Jr., D. S. Young, and D. N. Lee, “Visual control of step length during running over irregular terrain.,” *J. of Experimental Psychology: Human Perception and Performance*, vol. 12, no. 3, p. 259, 1986.
- [164] W. H. Warren, “Action-scaled information for the visual control of locomotion,” in *Closing the Gap*, Psychology Press, 2012, pp. 261–296.
- [165] G. E. Dullerud and F. Paganini, *A Course in Robust Control Theory: A Convex Approach*. Springer Science & Business Media, 2013, vol. 36.
- [166] K. Zhou and J. C. Doyle, *Essentials of Robust Control*. Prentice hall Upper Saddle River, NJ, 1998, vol. 104.
- [167] T. Haarnoja, A. Ajay, S. Levine, and P. Abbeel, “Backprop KF: Learning discriminative deterministic state estimators,” in *Advances in Neural Information Processing Systems*, 2016, pp. 4376–4384.
- [168] R. Jonschkowski, D. Rastogi, and O. Brock, “Differentiable particle filters: End-to-end learning with algorithmic priors,” in *Proceedings of Robotics: Science and Systems*, Pittsburgh, Pennsylvania, Jun. 2018.
- [169] P. Karkus, D. Hsu, and W. S. Lee, “Particle filter networks: End-to-end probabilistic localization from visual observations,” *arXiv preprint arXiv:1805.08975*, 2018.

- [170] S. Gupta, J. Davidson, S. Levine, R. Sukthankar, and J. Malik, “Cognitive mapping and planning for visual navigation,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 2616–2625.
- [171] S. Gupta, D. Fouhey, S. Levine, and J. Malik, “Unifying map and landmark based representations for visual navigation,” *arXiv preprint arXiv:1712.08125*, 2017.
- [172] P. Karkus, X. Ma, D. Hsu, L. P. Kaelbling, W. S. Lee, and T. Lozano-Pérez, “Differentiable algorithm networks for composable robot learning,” *arXiv preprint arXiv:1905.11602*, 2019.
- [173] S. Levine, C. Finn, T. Darrell, and P. Abbeel, “End-to-end training of deep visuomotor policies,” *The Journal of Machine Learning Research*, vol. 17, no. 1, pp. 1334–1373, 2016.
- [174] N. Sünderhauf *et al.*, “The limits and potentials of deep learning for robotics,” *The International Journal of Robotics Research (IJRR)*, vol. 37, no. 4-5, pp. 405–420, 2018.
- [175] Y. Zhu *et al.*, “Target-driven visual navigation in indoor scenes using deep reinforcement learning,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2017, pp. 3357–3364.
- [176] A. A. Alemi, I. Fischer, J. V. Dillon, and K. Murphy, “Deep variational information bottleneck,” *arXiv preprint arXiv:1612.00410*, 2016.
- [177] A. Kolchinsky, B. D. Tracey, and D. H. Wolpert, “Nonlinear information bottleneck,” *Entropy*, vol. 21, no. 12, p. 1181, 2019.
- [178] N. Tishby and N. Zaslavsky, “Deep learning and the information bottleneck principle,” in *Proceedings of the IEEE Information Theory Workshop*, IEEE, 2015, pp. 1–5.
- [179] A. Goyal *et al.*, “InfoBot: Transfer and exploration via the information bottleneck,” in *Proceedings of the International Conference on Learning Representations*, 2019.
- [180] P. Yingjun and H. Xinwen, “Learning representations in reinforcement learning: An information bottleneck approach,” *arXiv preprint arXiv:1911.05695*, 2019.

- [181] A. S. Klyubin, D. Polani, and C. L. Nehaniv, “Empowerment: A universal agent-centric measure of control,” in *IEEE Congress on Evolutionary Computation*, IEEE, vol. 1, 2005, pp. 128–135.
- [182] C. Salge, C. Glackin, and D. Polani, “Empowerment and state-dependent noise-an intrinsic motivation for avoiding unpredictable agents,” in *Proceedings of the Artificial Life Conference*, MIT Press, 2013, pp. 118–125.
- [183] C. Salge, C. Glackin, and D. Polani, “Empowerment—an introduction,” in *Guided Self-Organization: Inception*, Springer, 2014, pp. 67–114.
- [184] S. Tiomkin, D. Polani, and N. Tishby, “Control capacity of partially observable dynamic systems in continuous time,” *arXiv preprint arXiv:1701.04984*, 2017.
- [185] T. Yu, G. Shevchuk, D. Sadigh, and C. Finn, “Unsupervised visuomotor control through distributional planning networks,” *arXiv preprint arXiv:1902.05542*, 2019.
- [186] R. Zhao, S. Tiomkin, and P. Abbeel, “Learning efficient representation for intrinsic motivation,” *arXiv preprint arXiv:1912.02624*, 2019.
- [187] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT press Cambridge, 1998, vol. 1.
- [188] T. M. Cover and J. A. Thomas, *Elements of Information Theory*. John Wiley & Sons, 2012.
- [189] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, “Trust region policy optimization,” in *Proceedings of the International Conference on Machine Learning*, 2015, pp. 1889–1897.
- [190] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [191] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” *arXiv preprint arXiv:1312.6114*, 2013.
- [192] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, “Planning and acting in partially observable stochastic domains,” *Artificial Intelligence*, vol. 101, no. 1-2, pp. 99–134, 1998.

- [193] D.-A. Clevert, T. Unterthiner, and S. Hochreiter, “Fast and accurate deep network learning by exponential linear units (ELUs),” *arXiv preprint arXiv:1511.07289*, 2015.
- [194] E. Coumans and Y. Bai, *Pybullet, a python module for physics simulation for games, robotics and machine learning*, 2018.
- [195] B. Belousov, G. Neumann, C. A. Rothkopf, and J. Peters, “Catching heuristics are optimal control policies,” in *Advances in Neural Information Processing Systems*, 2016, pp. 1426–1434.
- [196] D. P. Bertsekas, *Dynamic Programming and Optimal Control*. Athena Scientific, 2017.
- [197] M. R. James and J. S. Baras, “Partially observed differential games, infinite-dimensional Hamilton-Jacobi-Isaacs equations, and nonlinear H_∞ control,” *SIAM Journal on Control and Optimization*, vol. 34, no. 4, pp. 1342–1364, 1996.
- [198] R. Hall, A. Rinaldo, and L. Wasserman, “Random differential privacy,” *Journal of Privacy and Confidentiality*, vol. 4, no. 2, pp. 43–59, 2012.
- [199] J. Bradbury *et al.*, *JAX: Composable transformations of Python+NumPy programs*, 2018.
- [200] O. Brock, “Is robotics in need of a paradigm shift?” In *Berlin Summit on Robotics: Conference Report*, 2011, pp. 1–10.
- [201] D. Falanga, S. Kim, and D. Scaramuzza, “How fast is too fast? The role of perception latency in high-speed sense and avoid,” *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 1884–1891, 2019.
- [202] J. C. Doyle, B. A. Francis, and A. R. Tannenbaum, *Feedback Control Theory*. Courier Corporation, 2013.
- [203] S. M. LaValle, “Sensor lattices: Structures for comparing information feedback,” in *International Workshop on Robot Motion and Control (RoMoCo)*, 2019, pp. 239–246.
- [204] J. C. Duchi and M. J. Wainwright, “Distance-based and continuum Fano inequalities with applications to statistical estimation,” *arXiv preprint arXiv:1311.2669*, 2013.
- [205] MOSEK ApS, *Mosek fusion api for python 9.0.84(beta)*, 2019.

- [206] B. O’Donoghue, E. Chu, N. Parikh, and S. Boyd, *SCS: Splitting conic solver, version 2.0.2*, <https://github.com/cvxgrp/scs>, Nov. 2017.
- [207] J. C. Duchi, “Derivations for linear algebra and optimization,” 2016.
- [208] W. Mulzer, “Five proofs of Chernoff’s bound with applications,” *arXiv preprint arXiv:1801.03365*, 2018.

ProQuest Number: 30491369

INFORMATION TO ALL USERS

The quality and completeness of this reproduction is dependent on the quality and completeness of the copy made available to ProQuest.



Distributed by ProQuest LLC (2023).

Copyright of the Dissertation is held by the Author unless otherwise noted.

This work may be used in accordance with the terms of the Creative Commons license or other rights statement, as indicated in the copyright statement or in the metadata associated with this work. Unless otherwise specified in the copyright statement or the metadata, all rights are reserved by the copyright holder.

This work is protected against unauthorized copying under Title 17,
United States Code and other applicable copyright laws.

Microform Edition where available © ProQuest LLC. No reproduction or digitization of the Microform Edition is authorized without permission of ProQuest LLC.

ProQuest LLC
789 East Eisenhower Parkway
P.O. Box 1346
Ann Arbor, MI 48106 - 1346 USA